

Казанцев В. В., Гнайберг А. В.

C# и Godot

РАЗРАБОТКА ЧИСТЫХ
И МАСШТАБИРУЕМЫХ
ИГРОВЫХ СИСТЕМ



БИБЛИОТЕКА
ГЕЙМДИЗАЙНА

АМК
ИЗДАТЕЛЬСТВО

В. В. Казанцев, А. В. Гнайберг

C# и Godot.

Разработка чистых и масштабируемых игровых систем



Москва, 2026

УДК 794:004.42Godot

ББК 77.563

K14

Казанцев В. В., Гнайберг А. В.

K14 **C# и Godot. Разработка чистых и масштабируемых игровых систем. – М.: ДМК Пресс, 2026. – 314 с.: ил.**

ISBN 978-5-93700-483-3

Книга посвящена архитектуре игровых проектов на Godot и C#. Она показывает, как превратить хаотичный набор скриптов в управляемую и расширяемую систему, опираясь на принципы разделения ответственности, слабой связанности и направления зависимостей. На реальных задачах геймдева разбираются доменная логика, событийная модель, машины состояний, сохранения и подготовка к релизу – всё, что позволяет проекту жить долго и меняться без боли.

Издание адресовано разработчикам, уже знакомым с основами C# и желающим освоить системное проектирование на Godot. Предварительное знание движка не требуется, но базовое понимание языка C# необходимо.

УДК 794:004.42Godot

ББК 77.563

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-5-93700-483-3

© Казанцев В. В., Гнайберг А. В., 2026

© Перевод, оформление, издание,
ДМК Пресс, 2026

Содержание

От издательства	9
Предисловие	10
 Глава 1. Почему C в Godot	12
1.1. Роль C# в современной разработке игр	12
1.2. Обзор архитектуры Godot	17
1.3. Управляемая и неуправляемая среда.....	20
1.4. Когда выбирать C# вместо GDScript	29
1.5. Установка и настройка окружения	33
1.6. Первый минимальный C#-скрипт	36
 Глава 2. Понимание движка	39
2.1. Знакомство с Nodes и Scenes	39
2.2. Дерево сцены	42
2.3. Жизненный цикл Node.....	47
2.3.1. Контракт на исполнение: фазы вместо хаоса.....	47
2.3.2. Ключевые callback'и и их роли	49
2.3.3. Типичные ошибки и как они убивают архитектуру	50
2.4. Сигналы и поток событий.....	52
2.4.1. От прямых вызовов к модели «факт – реакция».....	52
2.4.2. Как проектировать сигнал: контракт, имя, данные и локальность	53
2.4.3. Типичные ошибки и дисциплина подписок.....	55
2.5. Память и владение объектами	57
2.5.1. Два времени жизни, которые нужно подружить	57
2.5.2. Дерево сцен как основа владения	57
2.5.3. Висячие ссылки, забытые подписки и утечки – как это лечить	58
2.5.4. Стратегия чистого владения: кто создаёт, тот и уничтожает	59
2.6. Основы отладки.....	60
2.6.1. Цикл гипотез: от наблюдения к первопричине	61
2.6.2. Инструменты наблюдаемости: как заставить систему заговорить.....	61

2.6.3. Карта ошибок: связываем симптомы с архитектурными нарушениями	62
2.6.4. Ловушки мышления и профилактическая архитектура	63

Глава 3. C# как инструмент системного дизайна 66

3.1. Классы и ответственность.....	66
3.1.1. Класс как архитектурный рубеж.....	66
3.1.2. Связность и цена размытых границ	67
3.1.3. Проектируем классы: от хаоса к ролям	69
3.2. Инкапсуляция	71
3.2.1. Инкапсуляция как архитектурная защита и язык дизайна.....	71
3.2.2. Как построить надёжный контракт: свойства, методы и инварианты	72
3.2.3. Инкапсуляция на практике: отладка, масштабирование и работа в команде	73
3.3. События и делегаты	75
3.3.1. Делегаты: ссылки на методы как значения	75
3.3.2. События: контролируемая рассылка уведомлений.....	76
3.3.3. Жизненный цикл подписки и утечки памяти.....	78
3.3.4. События, прямые вызовы и сигналы Godot: когда что использовать	79
3.4. Интерфейсы: как договариваться о возможностях, не привязываясь к деталям	80
3.5. Асинхронность: как не заморозить игру и не потерять контроль	84
3.5.1. Task и async/await: как работают обещания и паузы	84
3.5.2. Асинхронность в мире Godot: где она уместна, а где опасна.....	85
3.5.3. Состояние между мирами: главная ловушка await.....	87
3.5.4. Где асинхронность действительно нужна, а где лучше без неё	88
3.6. Как не превратить проект в тыкву: типичные архитектурные ошибки и их профилактика.....	88
Шаг 2: прямые зависимости и скрытое знание о структуре сцены	91

Глава 4. Структурирование игровой логики..... 96

4.1. Разделение ответственности (Separation of Concerns)	96
4.1.1. Concern как единица архитектурной устойчивости.....	96
4.1.2. Три слоя архитектуры: представление, правила, инфраструктура	97
4.1.3. Цена смешения: как невинные shortcuts превращаются в системный долг	98
4.1.4. От хаоса к границам: рефакторинг на примере инвентаря.....	99
4.1.5. Модульное мышление и отладка в разделённой системе	101
4.2. Композиция поведения: от монолитных сущностей к набору ролей.....	101

4.2.1. Роль как архитектурная единица: контракт, состояние и границы.....	102
4.2.2. Контейнер сущности: сборка, оркестрация и связывание	104
4.2.3. Гранулярность компонентов и предотвращение архитектурного шума.....	106
4.2.4. Компонентное мышление в контексте Godot: две плоскости композиции	106
4.3. Наследование и композиция: два способа выразить вариативность	107
4.3.1. Два отношения: «является» и «содержит»	108
4.3.2. Ловушки наследования: неявные контракты и комбинаторный рост	108
4.3.3. Композиция как инструмент вариативности и динамической замены.....	110
4.3.4. Реалистичный гибрид и практические ориентиры.....	111
4.4. Организация больших проектов: от стихийного кода к управляемой структуре	112
4.4.1. От прототипа к сотням файлов: три эффекта разрушения порядка.....	112
4.4.2. Слои, фиши и вертикальные срезы: поиск осей организации	113
4.4.3. Границы модулей, неймспейсы и конвенции: удержание порядка в долгой перспективе	114
4.4.4. Когда и как перестраивать структуру: триггеры роста и рефакторинг без катастрофы.....	115
4.5. Направление зависимостей: как защитить правила игры от деталей реализации.....	117
4.5.1. Почему ядро не должно знать о периферии: пример с растущей энтропией	118
4.5.2. Инверсия зависимостей через контракты и адаптеры.....	119
4.5.3. Профилактика циклов и сохранение дисциплины в долгосрочной перспективе	121

Глава 5. Игрок как система

5.1. Абстракция ввода: учим персонажа слушаться не кнопок, а намерений.....	123
5.1.1. Главная ошибка новичка: рождение «божественного метода»	124
5.1.2. Проектирование намерения: контракт, понятный и кнопке, и ИИ... ..	124
5.1.3. Контроллер как потребитель намерений: логика без привязки к железу	126
5.1.4. Магия буферизации: как простить игроку ошибку на 100 миллисекунд.....	128
5.1.5. Реплеи, AI и мультиплеер: один интерфейс, чтобы править всеми.....	129

5.1.6. Подводные камни: о чём нельзя забывать	130
5.2. Архитектура движения: конвейер от намерения к позиции	130
5.2.1. Конвейер движения: три шага, которые нельзя смешивать	131
5.2.2. Владение скоростью: почему так важно, чтобы у каждой величины был хозяин	133
5.2.3. Детерминизм и отладка: превращаем ощущения в науку	133
5.2.4. Жёсткая граница: почему анимация не имеет права двигать персонажа	134
5.2.5. Собираем всё воедино: как это выглядит в Godot-сцене.....	135
5.3. PhysicsBody и CharacterBody: выбор физической основы для движения.....	136
5.3.1. Две модели движения: симуляция физики и контроллер.....	136
5.3.2. Архитектурные последствия: управляемость, отладка и предсказуемость	137
5.3.3. Практические сценарии и критерии выбора	138
5.4. Реализация машины состояний для движения персонажа	140
5.4.1. От неявных флагов к явным состояниям	141
5.4.2. Минимальный каркас: состояния, переходы и фазы жизни.....	143
5.4.3. Интеграция с моделью движения и обработка прерываний	145
5.4.4. Проектирование переходов и предотвращение хаоса.....	147
5.5. Разделение поведения и анимации: односторонний поток данных.....	148
5.5.1. Ловушка обратной связи: когда анимация начинает управлять поведением.....	148
5.5.2. Контракт визуального представления: факты вместо команд.....	149
5.5.3. Синхронизация, тайминги и проблема root motion.....	151
5.5.4. Влияние на командную работу, отладку и мультиплеер	152
Глава 6. Мир и взаимодействие	154
6.1. Коллизионные слои и маски: проектирование политики взаимодействия.....	154
6.1.1. Декларация ролей и политики обнаружения	154
6.1.2. Разделение каналов: блокировка и обнаружение	156
6.1.3. Проектирование политики через константы и владение	157
6.1.4. Архитектурная дисциплина: избавляемся от хаоса в коде	158
6.2. Системы взаимодействия: контрактная модель вместо прямых вызовов.....	159
6.2.1. Протокол взаимодействия: инициатор, цель и контекст	159
6.2.2. Разделение обнаружения и исполнения	161
6.2.3. Отказы как часть контракта и расширяемость системы	163
6.3. Архитектура камеры: наблюдатель, а не контроллер	164
6.3.1. Разделение обязанностей игрока и наблюдателя.....	164
6.3.2. Слоистый пайплайн: от цели до финального кадра	165

6.3.3. Управление режимами и модификаторами	167
6.4. Событийно-ориентированный геймплей: факты вместо опроса	168
6.4.1. От постоянного опроса к подписке на изменения	168
6.4.2. Область видимости событий и предотвращение хаоса	170
6.4.3. Синхронный вызов против отложенной обработки	171
6.5. Базовая структура AI: конвейер восприятия, решения и действия	172

Глава 7. State Machine и проектирование поведения..... 179

7.1. Конечный автомат как фундамент игровой логики	179
7.2. Иерархические конечные автоматы: управление глубиной поведения.....	184
7.3. Стратегия как механизм вариативности алгоритмов	188
7.4. Связывание модулей через уведомления: Наблюдатель	193

Глава 8. Event-Driven архитектура..... 196

8.1. Сигналы Godot и события C# – два инструмента для уведомлений	196
8.2. Глобальная шина событий: централизованная коммуникация в масштабах всей игры.....	199
8.3. Техники ослабления связанности: контракты, события и границы модулей.....	202
8.4. Тестирование систем в изоляции как архитектурный инструмент	206

Глава 9. UI как отдельный слой..... 210

9.1. Архитектура UI как слоя представления	210
9.2. Привязка данных в UI: от ручного хаоса к реактивной синхронизации.....	212
9.3. Удержание UI в роли слоя представления	216

Глава 10. Системы сохранения и загрузки..... 221

10.1. Сериализация как контракт данных	221
10.2. Версионирование сохранений: эволюция данных без потери прогресса	224
10.3. Безопасность сохранений: защита данных в недоверенной среде.....	227
10.4. Асинхронные системы сохранения	230

Глава 11. Производительность и оптимизация..... 234

11.1. Профилирование как инженерная дисциплина.....	234
11.2. Снижение аллокаций: управление памятью в горячих путях	237
11.3. Объектные пулы: переиспользование вместо создания.....	240
11.4. Динамическое управление сценами: жизненный цикл мира	243

Глава 12. Подготовка к релизу и сопровождение проекта	247
12.1. Экспорт как снятие маски с неявных зависимостей	248
12.2. Конфигурация как инструмент управления окружениями	252
12.3. Debug и Release: два разных продукта	256
12.4. От прототипа к архитектуре: реконструкция и рефакторинг	260
12.5. Код-ревью, дрейф и долгосрочное сопровождение	264
Глава 13. Las Vegas Digital Hub: архитектура уровня продакшен на практике	268
13.1. Архитектурный манифест: уход от стандартных скриптов	269
13.2. Система состояний: State Machine для игрового автомата	274
13.3. Data-Driven подход: Resource-пресеты	285
13.4. Событийная модель: Event Bus для децентрализованной синхронизации	291
13.5. Многопоточность и RNG: безопасные вычисления вероятностей	297
13.6. Оптимизация в условиях Лас-Вегаса: проверенные практики	303
Заключение	312
Предметный указатель	313

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Предисловие

Люди начинают делать игры, чтобы увидеть движение на экране. Первый же персонаж, дёрнувшийся в ответ на нажатие клавиши, приносит ни с чем не сравнимое ощущение: мир оживает, и вы – его создатель. Но между этим детским восторгом и способностью построить игру, которая не развалится через месяц разработки, лежит пропасть. Большинство прыгает в неё, не оглядываясь, и разбивается о скалы хаотичного кода. Перед вами книга для тех, кто хочет не просто перепрыгнуть, а построить мост. Мы живём в эпоху быстрых решений. Короткие видео, пошаговые tutorиалы, готовые шаблоны убеждают нас, что главное – результат. Но результат без понимания – картонный домик. Вы можете за десять минут заставить персонажа прыгнуть, следуя инструкции. Вы даже можете собрать целый прототип, удивляющий друзей. Но когда проект начнёт расти, а требования – множиться, хаотичный код перестанет работать. Система станет хрупкой, изменения начнут ломать логику в самых неожиданных местах, и вы поймёте: проблема не в языке и не в движке, а в отсутствии архитектуры.

Книга адресована тем, кто хочет начать с нуля, но не желает оставаться на уровне случайных решений. Новичкам, только знакомящимся с C# и Godot. Разработчикам, переходящим с других движков, например Unity или Unreal. Инженерам, которым важны структура и предсказуемость системы.

Почему C#? За пределами игровой индустрии это язык, на котором строят распределённые системы, облачные сервисы и корпоративные приложения. Его статическая типизация не враг, а союзник: компилятор ловит ошибки до того, как они добираются до игрока. Чёткая структура классов, управляемая среда выполнения, десятилетия проверенных инженерных практик – всё это создаёт фундамент, на котором можно возводить здание любой сложности. Godot с его открытостью и гибкостью идеально ложится на этот фундамент, а связка двух инструментов становится мощнее каждого по отдельности.

Почему архитектура? Игра – не сцена и не набор спрайтов. Это совокупность взаимодействующих систем. Игрок, камера, ввод, физика, интерфейс, сохранения – все эти части должны работать согласованно, как оркестр, а не как толпа солистов. С правильной структурой даже небольшой проект остаётся устойчивым и готовым к расширению. Если же структурой пренебречь, даже простая игра быстро превратится в хаос, где каждая правка вызывает лавину новых ошибок.

Книга начинается с нуля, но ноль здесь – не отсутствие знаний, а чистая точка старта. Мы будем разбирать не только «как сделать», но и «почему

устроено именно так». Каждая строчка кода, каждый архитектурный принцип появляются здесь по определённой причине, и задача читателя – эту причину понять.

Для кого эта книга

Издание рассчитано на несколько категорий читателей. Начинающие разработчики, которые хотят стартовать с правильной основы. Если у вас нет опыта в Godot или C#, книга проведёт от базовых принципов к системному пониманию движка. Разработчики, переходящие с других движков, например Unity или Unreal. Если вы уже знакомы с геймдевом, но хотите освоить Godot и использовать C# как основной инструмент, книга поможет быстро адаптироваться к архитектуре движка. Инженеры и программисты, которым важна структура. Если вы предпочитаете продуманную организацию кода, разделение ответственности и масштабируемые решения, материал будет соответствовать вашему подходу. Книга может **не подойти**, если вы ищете быстрый результат без понимания системы, хотите лишь повторять шаги за автором или вам достаточно минимального прототипа без перспективы дальнейшего развития.

Как работать с книгой

Материал построен последовательно, каждая часть опирается на предыдущую. Рекомендуются читать главы по порядку, особенно при старте с нуля. Даже знакомые на первый взгляд темы закладывают принципы, которые будут использоваться дальше.

Пишите код вручную. Не копируйте примеры автоматически, набирайте их самостоятельно. Тактильная память и необходимость обдумывать каждую строку дают более глубокое понимание структуры. Вы заметите детали, которые легко пропустить при копировании, и сформируете привычку аккуратного проектирования. Код в книге не является «магией» – каждая строка появляется по определённой причине.

Не пропускайте архитектурные разделы. В книге есть главы с меньшим объёмом практики и более глубоким анализом структуры – именно они формируют системное мышление. Если пропустить эти разделы, проект может работать, но понимание останется поверхностным.

Работайте в своём темпе. Материал рассчитан на постепенное освоение. Некоторые главы можно перечитывать после завершения книги – по мере роста опыта многие принципы начнут восприниматься иначе. Цель издания – не только дать инструменты, но и научить мыслить категориями систем. При осознанном подходе к чтению результат окажется значительно глубже, чем просто готовый проект.

1

Почему C в Godot

1.1. Роль C# в современной разработке игр

Когда говорят о языках в геймдеве, спор обычно сводится к двум полюсам: скорость прототипирования и производительность исполнения. В реальных проектах язык гораздо чаще определяет не быстроту обчёта кадров, а скорость и безопасность изменения системы. А это уже характеристика архитектурная – она диктует, насколько предсказуемо можно добавлять механики, переносить ответственность между подсистемами, заменять реализации и удерживать проект в состоянии развития, а не набора исключений. C# занял в индустрии нишу языка, позволяющего строить игровые системы на уровне продукта. Его положение сложилось на пересечении трёх факторов: управляемая среда выполнения, статическая типизация и зрелая инженерная экосистема. Как следствие, C# выбирают для задач, в которых важны не единичные трюки, а дисциплина изменений – контролируемые зависимости, минимизация регрессий, обозримая сложность и способность поддерживать кодовую базу годами.

Выбор языка как фундамент архитектуры

Выбор языка всегда означает выбор набора ограничений и инструментов, которые будут подталкивать архитектуру в одни стороны и делать болезненными другие. Два языка, способных выразить одинаковую логику, могут радикально различаться стоимостью ошибок и ценой улучшений. Архитектурное измерение проявляется здесь в нескольких конкретных вещах.

Первое – контракты и границы. Чем проще выражать интерфейсы, типы данных и ограничения на входы и выходы, тем легче фиксировать границы подсистем и удерживать направление зависимостей. Второе – обратная

связь: компиляция, анализаторы, подсказки IDE и тестовые фреймворки определяют, насколько быстро разработчик получает сигнал «ты сломал систему» и насколько точно этот сигнал локализован. Третье – рефакторинг как нормальная операция. В растущем проекте переписывание частей системы не исключение, а рутина, и язык либо поддерживает безопасные переименования, поиск использований и типовые преобразования, либо превращает изменения в серию ручных проверок. Далее идёт модель исполнения и цена абстракций: управляемая память, сборка мусора, JIT/AOT, исключения и асинхронность формируют своего рода «физику» кода, которую архитектура обязана учитывать. Наконец, интеграция с инструментами и процессами – код-ревью, CI, статический анализ, профилирование, форматирование и генерация документации – является не украшением, а частью производственного контура, и игнорировать её при выборе языка нельзя. За выбором языка по принципу «на нём удобнее писать» фактически стоит выбор архитектуры по умолчанию: неявные правила композиции кода, стиль выражения зависимостей и типовые паттерны, которые будут воспроизводиться десятки раз. В небольшом прототипе такой выбор почти незаметен. В системе из десятков тысяч строк он превращается в главный источник либо порядка, либо хаоса.

Статическая типизация как инструмент фиксации договорённостей

Статическая типизация в C# полезна не сама по себе, а как механизм фиксации договорённостей. Типы здесь играют три взаимосвязанные роли. Прежде всего они дают сжатое описание данных: тип задаёт форму, с которой должна работать подсистема, и тем самым снижает число плавающих соглашений, существующих только в голове разработчика или в комментариях. Кроме того, типы служат инструментом локализации ошибок – компилятор и анализаторы находят несоответствия там, где динамический код часто падает далеко от источника проблемы. Архитектурно это означает более короткую цепочку причинно-следственных связей. Наконец, они создают основание для безопасного рефакторинга: когда типы и сигнатуры отражают реальные границы, IDE может проводить изменения механически корректно, и чем больше проект, тем сильнее это влияние на стоимость развития. Из этого следует важный вывод: типы не служат ни украшением, ни формальной строгостью; они выступают частью архитектурного языка проекта. Если подсистемы общаются через размытые структуры вроде «объекта со случайными полями» или «словаря строк», любая попытка модульности будет постоянно протыкаться исключениями и скрытыми зависимостями. Если же границы выражены явными типами, меняется сама динамика работы: спор о том, что передаётся в подсистему, переходит из неформального общения в проверяемое условие. Допустим, вы вводите отдельный тип `PlayerId` вместо `string`. Компилятор немедленно подсветит все места, где идентификатор игрока пытались передать как обычную строку или, хуже того, как целое число. Десятки потенциальных

багов устраняются ещё до первого запуска. Статическая типизация не гарантирует хорошую архитектуру автоматически – она лишь делает видимыми противоречия. Можно выстроить иерархию типов, которая создаёт иллюзию структуры, но на деле усиливает связанность. Поэтому полезный подход состоит в том, чтобы начинать не с классов, а с ответственностей и потоков данных, и лишь затем выбирать минимально достаточные типы для фиксации границ. Современная эволюция C# – nullable-аннотации, record-типы, pattern matching, улучшения в generics – расширяет набор средств для выражения инвариантов, но суть остаётся прежней: сделать недопустимые состояния труднее выразимыми, а допустимые – проще проверяемыми.

В игровой разработке множество состояний, которые почти всегда должны быть валидными: здоровье не уходит ниже нуля, идентификатор сущности не пуст, ссылка на ресурс указывает на существующий контент, команда содержит полный набор параметров. В прототипе такие требования держат в голове, но в системе их необходимо фиксировать. C# позволяет закреплять инварианты на нескольких уровнях строгости. На уровне API сигнатура метода и типы параметров задают, что именно ожидается – отдельный тип для идентификатора, отдельный для времени, отдельный для расстояния. Всё это не избыточная формальность, а способ уменьшить число неявных соглашений. На уровне структуры данных необязательные поля, явные значения по умолчанию и различие между отсутствием значения и нулевым значением делают такие контракты проверяемыми. На уровне поведения к ним добавляются предусловия, постусловия, ограничение способов создания объекта через factory-методы и закрытие сеттеров там, где состояние не должно меняться произвольно.

Системный эффект возникает тогда, когда язык проекта начинает совпадать с языком предметной области. Если в коде есть понятия «команда взаимодействия», «результат симуляции», «снимок состояния», «правило начисления очков», обсуждение архитектуры становится конкретным: можно указать, какие подсистемы имеют право создавать эти сущности, кто их читает, кто трансформирует и где заканчивается ответственность. C# хорошо подходит для такого словаря за счёт выразительных типов и средств организации кода.

Среда исполнения, экосистема и масштабирование

Языковая среда выполнения – такая же часть архитектуры, как и система типов. Управляемая память и сборка мусора создают характерный профиль рисков, приводя не к классической утечке памяти, а к непредсказуемым паузам, росту аллокаций, фрагментации управляемой кучи, ошибкам из-за захвата ссылок и долгоживущих объектов. Архитектурный вывод прост: горячие пути должны проектироваться и измеряться. Если логика распределена случайно, невозможно понять, где и почему происходят аллокации и растёт давление на GC. При ясных границах подсистем, напротив, легче ограничить создание временных объектов в критических местах, заранее

выбрать между value types и reference types там, где это действительно важно, и локализовать оптимизации, не переписывая весь проект. Другая сторона модели исполнения – публикация под разные платформы. У проектов могут быть различные требования: быстрые итерации на ПК, жёсткие ограничения на платформах с АОТ-компиляцией, лимиты на размер сборки, запрет динамической генерации кода и reflection. Эти требования не решаются «в конце», а влияют на выбор подходов к сериализации, конфигурации, инструментам и способам расширения системы. В современном производственном геймдеве язык представляет собой связку из синтаксиса, компилятора, IDE, экосистемы анализаторов, тестового контура, сборки и публикации. C# превосходит многие альтернативы именно зрелостью и хорошей интеграцией этой связки. Ключевые свойства, которые её отличают: навигация и понимание кода через быстрый переход к определению, поиск использований и граф вызовов; рефакторинги как операции первого класса – извлечение метода, изменение сигнатур, безопасное переименование; статический анализ, включающий предупреждения о null, неиспользуемых переменных и сомнительных преобразованиях; развитая инфраструктура тестирования – unit-тесты, моки, измерение покрытия. Практическим критерием зрелости инструментов служит способность поддерживать массовые изменения. Если инструментальный контур позволяет выполнять их последовательно, с компиляцией и тестами, команда получает возможность улучшать архитектуру по мере роста. Если контур слаб, появляется стратегия «не трогать то, что работает», и архитектура закрепляется в раннем, случайном состоянии.

Экосистема C# – это прежде всего NuGet и культурная привычка оформлять повторно используемый код как пакет с версионированием. Лёгкость подключения зависимостей может как ускорить разработку, так и увеличить связанность. На сторонние библиотеки полезно смотреть как на экономический выбор: каждая добавляет поверхность обновлений и потенциальных конфликтов, приносит свою модель данных и философию API. Для прототипа быстрое подключение пакета часто рационально. Для системы важнее, чтобы зависимость закрепляла границу, а не размывала её. Если библиотека заставляет протаскивать свои типы через весь проект, она становится частью архитектуры независимо от намерений. Если же она локализована в одном слое и скрыта за собственным интерфейсом, проект сохраняет управляемость. В одиночных проектах архитектура часто подменяется личной памятью автора: он знает, где что лежит, и держит в голове неявные зависимости. С ростом проекта и числа участников эта опора исчезает. Возникает потребность в формализованных правилах: что считается публичным API подсистемы, какие данные служат входом и выходом, где заканчивается ответственность одного слоя и начинается другого. C# помогает такому масштабированию. Он делает выразимыми границы через интерфейсы, абстрактные классы, generics и модульность через сборки, а также позволяет частично перевести архитектурные правила из устных соглашений в машинную валидацию через анализаторы, форматирование и CI.

Прототипный код против системного: разница в стоимости изменений

Прототипный код решает задачу «проверить идею». Он допускает временные зависимости с пометкой «потом разберёмся», жёстко прошитые параметры, дублирование логики и смешение слоёв – UI, геймплей, сохранения и отладка в одном месте. Такой код полезен ровно до момента, когда проект перестаёт быть экспериментом. Переход заметен по симптомам: добавление механики требует трогать всё сразу, исправление бага ломает другое, любое изменение вызывает цепочку неожиданных последствий. Системный код решает иную задачу: делать изменения предсказуемыми. Его ключевая единица – не функция и не класс, а граница: что служит входом для подсистемы (данные, события, команды), что служит выходом (состояние, события, результаты вычислений), что скрыто внутри и может меняться без каскада правок. Разница между прототипом и системой заключается в стоимости изменения. Прототип даёт выигрыш в скорости первого результата, а система – в скорости десятого изменения. Архитектурный смысл выбора C# состоит в том, что он поддерживает системный режим работы: позволяет фиксировать границы, проверять их и безопасно перерабатывать. Важно не путать структуру со сложностью. Плохая архитектура проявляется не в отсутствии слоёв, а в отсутствии ясных причин, по которым эти слои существуют. В C# легко создать инфраструктуру, которая выглядит солидно, но не помогает. Поэтому ориентиром должны быть не паттерны как таковые, а наблюдаемая цель: уменьшение связанности, локализация изменений, ясные контракты и управляемое состояние. Игровой проект – набор взаимодействующих процессов: ввод, симуляция, визуализация, аудио, интерфейс, сохранения, загрузка контента, сетевое взаимодействие. Даже в одиночной игре эти процессы неизбежно пересекаются. Если не определить, как именно они связаны, связи возникают случайно – там, где оказалось удобно прямо сейчас. *Структурное мышление* – это привычка отвечать на вопросы до написания кода. Какая часть системы владеет состоянием? Где проходит граница между доменной логикой и представлением? Какие зависимости должны быть направлены внутрь, к домену, а какие – наружу, к инфраструктуре? C# предоставляет для этого выразительные средства, а также инструментальную поддержку, чтобы ограничения не забывались со временем. Полезно различать два режима архитектурных решений. Решения уровня домена – что такое действие игрока, правило, событие, результат – должны быть стабильными и понятными, даже если реализация меняется. Решения уровня исполнения – как часто обновляется подсистема, где хранятся кешы, как формируются данные для отрисовки – должны быть измеряемыми и заменяемыми без изменения доменных контрактов. Если эти два уровня перепутаны, язык уже не спасает: оптимизации расползаются по коду, а логика превращается в набор условий «для производительности». Хорошая архитектура, напротив, позволяет в одном месте говорить о правилах игры, а в другом – о стоимости их вычисления. C# удобен тем, что поддерживает оба слоя: выразительные доменные типы и практич-

ные инструменты для измерения и оптимизации. Современная разработка игр часто включает несколько дисциплин: геймдизайн, программирование, инструменты, пайплайн контента, тестирование, аналитику. Язык, на котором описываются системы, становится общей точкой соприкосновения. C# играет роль инженерного языка общего назначения, который остаётся достаточно высокоуровневым для быстрой разработки и достаточно строгим, чтобы удерживать большие проекты. Он не отменяет необходимости думать о структуре, но снижает трение там, где структура должна быть закреплена технически. Переход от кода, который просто делает что-то на экране, к коду, описывающему систему, начинается с осознания того, что язык является частью архитектуры, а не просто сменным синтаксисом.

1.2. Обзор архитектуры Godot

Любой игровой движок удобнее всего рассматривать как архитектурную платформу, которая чётко разделяет обязанности между универсальной, высокопроизводительной инфраструктурой и уникальным кодом конкретной игры. Инфраструктура забирает себе повторяющиеся, сложные и требовательные к ресурсам задачи: взаимодействие с графическим API, расчёт физики, управление памятью, обработку ввода с разных устройств. Игровой код, в свою очередь, сосредоточен на доменных правилах, уникальном состоянии и поведении, которые, собственно, и отличают симулятор фермы от космического шутера. Это разделение является фундаментальной причиной существования движков. Они избавляют разработчика от необходимости переписывать технический фундамент для каждого нового проекта и позволяют строить систему на основе уже отлаженных и оптимизированных подсистем. Godot в этой рамке предстаёт как многослойная система с ясной иерархией. Нижние слои реализуют «физику исполнения»: управление потоками, памятью, ресурсами, математические библиотеки и низкоуровневые API. Над ними надстраиваются профильные подсистемы – рендеринг, физика, аудио. Верхние слои предоставляют модель сцены и среду для скриптинга, где и живёт пользовательская логика. У каждого слоя свои инварианты, своя зона ответственности и своя цена ошибок. Чем яснее разработчик представляет себе эти границы, тем проще ему принимать верные архитектурные решения: где хранить состояние, как организовать обмен событиями, какие части системы можно безболезненно заменить, а какие образуют её несущий каркас.

Ядро, подсистемы и их роль в формировании игрового мира

С точки зрения архитектуры движок выполняет четыре ключевые роли. Он предоставляет набор сервисов времени выполнения, задавая главный цикл обновления, точки входа, очереди событий, модель ресурсов и правила взаи-

модействия с операционной системой. Он содержит профильные подсистемы с чёткими контрактами: рендеринг ожидает данные определённой формы, физика обрабатывает коллайдеры и параметры среды, аудио работает с потоками и источниками. Движок также предлагает модель композиции мира – способ описывать существующие в игре объекты и их связи через иерархии, сцены и ресурсы. Наконец, он включает мощный инструментальный контур: редактор, импорт ассетов, сериализацию и средства отладки. Для архитектуры критически важно понимать, что инструменты задают формат данных и влияют на процессы изменений, поэтому считать их чем-то второстепенным было бы опасной ошибкой.

Ядро Godot написано на C++. Это не идеологическое, а сугубо практическое решение, гарантирующее максимальную близость к платформе и полный контроль над расходами процессора и памяти. На этом уровне решаются задачи, для которых цена абстракций и накладных расходов критически высока: прямое взаимодействие с графическими API, управление ресурсами, базовая математика, работа с файловой системой и сетевые примитивы. Принципиально, что C++ служит здесь не языком для описания геймплея, а средством создания надёжной и быстрой инфраструктуры. Ядро предоставляет верхним слоям стабильные сервисы, и пользовательский код на C# живёт над этими сервисами, используя их через публичные контракты, а не подменяя их. Внутри движка удобно мыслить категориями подсистем, каждая из которых отвечает за свою часть работы с собственными типами данных и временными ограничениями. Рендеринг преобразует состояние мира в изображение, выступая в роли потребителя данных, почти никогда не являясь источником истины. Физика, напротив, нередко сама становится частью этой истины, определяя итоговое положение объектов и характер их столкновений. Поэтому игровая логика обязана уважать временную модель физики и границы её ответственности. Аудио, как и рендеринг, в основном потребляет состояние, реагируя на события и параметры, но требует неразрывной буферизации, из-за чего внезапные задержки становятся сразу заметными игроку. Общая закономерность такова: все подсистемы движка – это специализированные вычислительные машины с собственными шкалами времени, и задача архитектуры игры – грамотно формировать для них данные и читать результаты, не позволяя подсистемам превращаться в «глобальный мозг» игры.

Сцены, ресурсы и время как архитектурные абстракции

Над ядром и подсистемами располагается слой скриптинга. Здесь, в окружении C#, размещается логика конкретной игры: её правила, состояния, реакции на события и координация подсистем. В Godot этот слой полностью отделён от ядра и общается с ним через публичный API. Архитектурно это крайне важная граница, означающая, что игра не должна опираться на внутренние детали движка. Иначе любое обновление Godot или смена платформы станет огромным риском. Скриптовый слой служит адаптером между уникальным

доменом игры и универсальной инфраструктурой. Движок предоставляет общие механизмы вроде сцен, ресурсов и событий, а игра с помощью этих механизмов создаёт свои собственные правила.

Верхние слои Godot включают систему сцен. В контексте этого архитектурного обзора достаточно зафиксировать её главное назначение: сцены задают композицию мира и способ повторного использования его частей. Сцена в Godot – это единица композиции и сериализации, позволяющая собирать сложное поведение из более простых элементов, хранить структуру объектов и их параметры не в коде, а в данных, и переиспользовать фрагменты как шаблоны. Предположим, вы создаёте платформер с десятью типами врагов. Без сцен пришлось бы либо дублировать описание каждого врага, либо городить гигантский God-класс с кучей условий. Сцены позволяют описать врага как независимый шаблон со своей геометрией и параметрами здоровья, а затем расставлять экземпляры этих шаблонов на уровне. Дизайнер сможет менять скорость конкретного врага правкой параметра в сцене, не заходя в код и не рискуя сломать остальных. Для движка жизненно важно чётко разделять «код» и «данные». Если код содержит только правила и связи между подсистемами, а данные описывают конкретные экземпляры (уровни, параметры, звуки), проект становится редактируемым. Значительную часть изменений можно вносить без перекомпиляции и правки логики. Godot системно поддерживает этот подход через собственную систему ресурсов и импорт-пайплайн. Ресурс в Godot – это объект данных, который можно хранить как самостоятельную сущность, переиспользовать в разных местах без копирования и загружать во время выполнения. Архитектурно ресурсы образуют граф зависимостей контента, превращая игру из набора объектов, порождённых кодом, в структуру данных, которую код интерпретирует.

Такой подход даёт проекту две важнейшие характеристики – воспроизводимость и управляемость изменений. Если состояние сцены и её зависимости выражены в данных, загрузка уровня становится детерминированной операцией: один и тот же набор ресурсов даёт один и тот же результат, резко снижая число магических эффектов, зависящих от порядка вызовов. Когда изменяется контент, меняются лишь данные, а не логика. Дизайнер меняет параметры, программист – правила, а движок обеспечивает корректное превращение параметров в состояние и представление.

Любой движок задаёт и модель времени – порядок, в котором части системы оживают. Godot выступает в роли оркестратора, обеспечивая повторяющийся цикл сбора входов, обновления состояния, формирования данных для подсистем и выдачи итогового кадра. Здесь архитектору важно различать два слоя времени. Время симуляции – это последовательность изменений состояния мира по правилам игры и законам физики, определяющая, когда и как можно менять состояние без противоречий. Время представления – формирование изображения, звука, UI и анимаций, следующих за состоянием и имеющих право на собственные интерполяции.

Типичная и очень дорогая ошибка – смешение этих слоёв. Например, здоровье игрока должно уменьшаться в момент получения урона (время симу-

ляции), а полоска здоровья в UI – плавно анимироваться к новому значению в течение нескольких кадров (время представления). Если же код уменьшает здоровье внутри метода, который одновременно обновляет полоску, ориентируясь на delta кадра, то при любых просадках FPS механика урона начинает вести себя по-разному. Правильное решение выглядит иначе: игровая логика атомарно уменьшает здоровье и выставляет факт «здоровье изменилось», а UI-компонент получает этот факт и самостоятельно решает, с какой скоростью двигать полоску.

Место C# в общей картине

Если рассматривать Godot как платформу, то C# находится на том же уровне, где пишется пользовательская логика и инструменты проекта. Он взаимодействует с движком через публичные абстракции: объекты сцены, ресурсы, события и API подсистем. Это значит, что архитектура игрового кода должна исходить из модели, где движок является нижним, неизменяемым слоем, который предоставляет возможности и ограничения, но не принимает доменных решений. C# в данном контексте служит инструментом для описания именно этих вот уникальных правил, состояний и поведения.

Разделение обязанностей между движком и игрой – базовая проверка любой архитектуры. Движок предоставляет цикл исполнения и точки интеграции, управляет ресурсами и их жизненным циклом, обеспечивает синхронизацию подсистем рендера, физики и аудио, а также предлагает модель композиции мира через сцены. Игра, в свою очередь, определяет свои сущности и их состояния, задаёт правила изменения этих состояний, интерпретирует низкоуровневые входы и результаты симуляции как события своего домена и формирует представления для визуализации. Смешение этих обязанностей обычно начинается с безобидных вещей, ускоряющих прототипирование, но быстро стирает границы и делает проект хрупким. Godot предоставляет все механизмы для удержания этих границ, но работают они лишь при осознанном применении: отделять состояние от представления, отделять логику от инфраструктуры и фиксировать точки интеграции.

1.3. Управляемая и неуправляемая среда

Термины `managed` и `unmanaged` описывают не скорость языка, а режим исполнения кода и распределение обязанностей между программой и средой выполнения. Для архитектуры игры это важно по простой причине: режим исполнения определяет, как устроены память, владение объектами, границы безопасности и цена некоторых операций. Проявляются эти свойства не в отдельных строках, а в форме всей системы: как она хранит состояние, как часто создаёт временные объекты, как взаимодействует с движком, как переживает длительную эволюцию. Если сейчас тема кажется абстрактной,

держите в голове практическую цель: позже обсуждаемые понятия проявятся как конкретные причины багов и роста стоимости изменений – особенно на границе движка, в жизненном цикле узлов, в подписках и во владении объектами.

Неуправляемая среда: нативный код и прямая ответственность

Unmanaged-среда – это режим, в котором код выполняется как нативная программа без обязательной прослойки, управляющей памятью и безопасностью. Типичный представитель – C++-код движка. В таком режиме программа работает напрямую с адресным пространством процесса, память выделяется и освобождается по правилам, заданным самим кодом (явно или через идиомы языка), а корректность доступа к памяти в значительной степени лежит на разработчике. Ошибки на уровне памяти способны приводить не к исключениям, а к неопределённому поведению: повреждению данных, редким крашам и призрачным багам, которые то проявляются, то исчезают при малейшем изменении окружения. Неформально `unmanaged` означает, что границы системы не защищены средой выполнения. Любая ошибка владения или времени жизни объекта способна проявиться далеко от места возникновения. В обмен разработчик получает максимально прямой контроль: над формой данных в памяти, над моментом освобождения ресурсов, над взаимодействием с системными API. Это не означает, что `unmanaged`-код обязательно «ручной во всём». В зрелых кодовых базах значительная часть управления ресурсами выражается через строгие правила владения и автоматизацию на уровне языка – например, детерминированное освобождение ресурсов при выходе из области видимости (вспомните идиому `RAII` в C++). Но принцип остаётся прежним: ответственность за корректность лежит на коде проекта и его внутренних соглашениях, а не на внешнем рантайме.

Управляемая среда: runtime, GC и контур безопасности

Managed-среда – это режим, в котором код выполняется под управлением среды выполнения, или `runtime`. Для C# это означает наличие рантайма, который управляет размещением объектов в памяти и их временем жизни, отслеживает достижимость объектов и автоматически освобождает память (`garbage collection`), а также обеспечивает набор гарантий безопасности на уровне памяти: в обычном коде невозможно получить произвольный доступ к чужим адресам. Кроме того, рантайм предоставляет стандартные механизмы обработки ошибок, метаданные типов и инфраструктуру загрузки и исполнения кода.

Полезно различать два вида безопасности. Первый – безопасность памяти: рантайм делает так, чтобы обычный код не мог случайно прочитать или записать произвольный участок памяти. Ошибки здесь чаще выражаются как исключения либо как логические дефекты, а не как повреждение памяти.

ти процесса. Второй – логическая корректность: рантайм ничего не знает о доменных правилах игры. Он не предотвращает неправильную модель состояния, не гарантирует правильный порядок событий и не защищает от архитектурной связанности. Он лишь делает класс низкоуровневых ошибок менее вероятным и более диагностируемым. По сути, *managed* – это контракт с инфраструктурой: часть обязанностей по управлению временем жизни и диагностике берёт на себя среда выполнения, а код платит за это некоторой стоимостью абстракций и некоторыми ограничениями.

Что такое runtime (концептуально)

Рантайм – это компонент, обеспечивающий исполнение кода как системы, а не как набора разрозненных инструкций. Концептуально он предоставляет несколько взаимосвязанных сервисов. Модель типов и метаданные дают описание классов, интерфейсов, методов и полей, а также проверяемые контракты на уровне типов. Управление памятью включает аллокацию объектов, отслеживание достижимости и освобождение памяти. Обработка исключений предлагает единый механизм сигнализации ошибок и восстановления контроля. Загрузка и связывание кода обеспечивают подключение модулей, разрешение зависимостей и версионирование. Наконец, оптимизация исполнения преобразует промежуточное представление кода в машинные инструкции под конкретную платформу. Архитектурно рантайм – это операционная система для кода уровня приложения. Он задаёт правила жизни объектов и делает их предсказуемыми в рамках собственных гарантий. Поэтому обсуждение *managed* и *unmanaged* в играх – это прежде всего обсуждение того, где именно требуется жёсткий контроль, а где достаточно гарантий и удобства.

Различия модели памяти: stack vs heap (концептуально)

Для практического понимания достаточно развести два понятия: стек и кучу. Это не исчерпывает реальную картину, но задаёт правильные вопросы. Стек – область памяти, связанная с вызовами функций и областью видимости. Данные на стеке обычно живут вместе с текущим потоком исполнения и исчезают автоматически при выходе из области. Это естественная модель для временных значений и короткоживущих данных. Куча – область, где размещаются объекты, живущие дольше текущего вызова. В *unmanaged*-мире код должен сам обеспечить освобождение таких объектов и связанных с ними ресурсов. В *managed*-мире объекты в куче живут до тех пор, пока остаются достижимыми, а освобождение памяти выполняет сборщик мусора.

Разница между ручным и автоматическим управлением проявляется в том, как разработчик проектирует систему. В *unmanaged*-коде критически важно заранее формулировать владение – «кто освобождает», – иначе система становится неустойчивой. В *managed*-коде владение в смысле памяти упрощается, но появляется новая ответственность: контролировать профиль

аллокаций и давление на GC, чтобы избежать непредсказуемых пауз. Архитектурный вывод: managed-код позволяет проще и безопаснее оперировать долгоживущими объектами, но требует дисциплины в обращении с временными выделениями памяти в часто выполняемых участках.

Память vs другие ресурсы: детерминизм всё ещё важен

Даже в managed-среде важно различать память и ресурсы, не сводящиеся к байтам в куче: файловые дескрипторы, сетевые соединения, нативные буферы, объекты GPU, аудиопотоки. Эти ресурсы существуют в слоях ниже рантайма и обычно требуют детерминированного освобождения – иначе система упирается в ограничения платформы или подсистем движка. В unmanaged-коде детерминизм освобождения ресурсов – часть базовой модели: ресурсы освобождаются в момент, когда код считает это правильным (часто автоматически, по правилам владения). В managed-коде освобождение памяти автоматизировано, но освобождение внешних ресурсов остаётся обязанностью архитектуры: нужно явно задавать жизненный цикл и границы владения. Типичный пример – объект `PackedScene`, который вы загрузили в C#-коде. GC может собрать управляемую обёртку когда угодно, но нативная сцена в памяти движка останется, пока вы явно не выгрузите её. Рассчитывать, что сборщик мусора наведёт порядок в мире движка, – прямая дорога к утечкам. Это важный момент для игровых проектов, потому что большая часть тяжёлых сущностей находится не в managed-куче, а в нативном мире движка или платформы. Архитектурная ошибка – подменять управление ресурсами надеждой на GC. Сборщик мусора освобождает managed-объекты, но не знает, когда игре нужно освободить текстуру, остановить поток, закрыть файл или выгрузить уровень. Корректная слоистая архитектура делает ресурсы явными: существует слой, отвечающий за их создание, учёт и своевременное освобождение. Managed-код может быть удобной оболочкой для таких решений, но не их заменой.

Ручное vs автоматическое управление: что меняется на практике

Автоматическое управление памятью часто описывают фразой «вам не нужно думать о памяти». Для игр это неверно. Правильная формулировка: вам не нужно вручную освобождать память обычных объектов, но вы обязаны проектировать так, чтобы поведение системы по памяти было предсказуемым. В managed-среде ключевые практические аспекты таковы. Сборка мусора реагирует на аллокации: чем больше временных объектов создаётся в цикле обновления, тем чаще рантайм вынужден выполнять сборку. Паузы – проблема не скорости, а латентности: даже при высокой средней производительности редкие паузы способны ломать ощущение управления и воспроизведение аудио, а также усложнять отладку спорадических рывков. Память – такой же архитектурный ресурс, как и время CPU. Её нужно плани-

ровать: где создаются объекты, какие из них долгоживущие, какие короткоживущие, где аллокации допустимы, а где нежелательны. В `unmanaged`-среде профиль рисков иной: высокая предсказуемость освобождения ресурсов при строго сформулированном владении и высокая цена ошибок при плохо сформулированном или нарушенном владении (утечки, `use-after-free`, повреждение памяти). Поэтому движки обычно строят `unmanaged`-ядро как максимально контролируемое и устойчивое, а поверх него предоставляют API, избавляющий игровой код от повторения низкоуровневых решений.

Указатели, `undefined behavior` и режимы отказа

Чтобы различие между `managed` и `unmanaged` не выглядело как абстрактная теория, полезно назвать главные режимы отказа для каждого подхода.

В `unmanaged`-коде ключевая опасность связана с тем, что память описывается адресами, а корректность доступа не гарантирована автоматически. Указатель – это значение, интерпретируемое как адрес. Если адрес неправильный или объект по нему уже не существует, последствия могут быть любыми: от немедленного краша до тихого повреждения состояния, которое проявится через полчаса и в совершенно другом месте. Именно поэтому `unmanaged`-код требует дисциплины владения и строгих правил времени жизни: иначе отладка превращается в поиск следствий без надёжной причины. В `managed`-коде типичный режим отказа иной. Вместо повреждения памяти чаще возникают исключения из-за некорректных предположений (`null`-ссылки, выход за границы массива), логические ошибки в состоянии из-за неправильной модели ответственности и деградация производительности из-за чрезмерных аллокаций или слишком частых переходов через границы интеграции.

С архитектурной точки зрения `managed`-среда сдвигает класс проблем вверх по уровню абстракции. Она не избавляет от ошибок, но делает их более локализуемыми и менее разрушительными для процесса. Именно поэтому `managed`-код часто выбирают для слоя правил игры: там важно быстро находить и исправлять дефекты поведения, не рискуя разрушить фундамент.

Мифы о производительности versus архитектурная реальность

Вокруг `managed`-языков в играх существует набор устойчивых мифов. Они мешают, потому что подменяют архитектурные причины общими лозунгами. Разберём три самых распространённых.

Миф первый: `managed` всегда медленнее. Реальность такова, что большая часть времени в игре тратится на подсистемы движка – рендер, физику, загрузку ресурсов, – и на алгоритмы, выбранные архитектором. Стоимость `managed`-кода становится критичной лишь в определённых классах задач: очень частые вызовы, плотные структуры данных, высокая интенсивность

аллокаций, чрезмерно болтливое взаимодействие с нативным ядром. Во всех остальных случаях узким местом оказывается дизайн данных и частота обновлений, а не факт наличия рантайма.

Миф второй: GC неизбежно делает игру дёрганой. Сборщик мусора реагирует на поведение программы. Если архитектура создаёт много мусора в горячих местах – паузы вероятны. Если архитектура минимизирует аллокации в критических циклах и локализует временные объекты – проблема становится управляемой. Это вопрос дисциплины данных и границ, а не «плохой технологии».

Миф третий: *unmanaged* автоматически означает быстро. *Unmanaged* даёт возможность быть быстрым и предсказуемым, но не гарантирует этого. Плохая локальность данных, лишние копии, неправильные алгоритмы и чрезмерная связанность подсистем одинаково разрушают производительность в любом режиме исполнения.

Отдельно стоит различать *throughput* и *latency*. *Throughput* отвечает на вопрос, сколько работы система делает в среднем за секунду. *Latency* – насколько равномерно распределена работа во времени. Для игры *latency* обычно критичнее: игрок замечает неровности кадра, даже если средний FPS высокий. Поэтому обсуждение *managed*-среды не должно сводиться к спору «быстрее или медленнее». Важнее понимать, какие архитектурные решения приводят к всплескам работы: массовые аллокации, синхронные загрузки, лавины событий, чрезмерная связанность подсистем. Полезно также помнить, что рантайм – не только GC. Это ещё и правила оптимизации и исполнения кода. *Managed*-код может иметь фазы разогрева, может вести себя по-разному на разных платформах и режимах сборки, а некоторые классы операций становятся дороже из-за проверок безопасности. Эти эффекты редко ломают архитектуру сами по себе, но подсказывают правильный стиль проектирования: держать горячие участки простыми, ограничивать аллокации, измерять, а не предполагать.

Архитектурно полезная позиция такова: вы выбираете модель системы, которая делает поведение измеряемым и корректируемым. Фактор *managed/unmanaged* влияет на стоимость некоторых решений, но не заменяет инженерный подход к данным, частоте обновления и границам ответственности.

Почему движки обычно неуправляемые

Причины, по которым ядра движков пишут на *unmanaged*-языках, не сводятся к скорости. Это целый набор требований инфраструктуры. Движку нужен прямой доступ к платформенным API – графическим и аудиоинтерфейсам, системным вызовам, драйверам. Ему требуется полный контроль над памятью и размещением данных: для рендера, физики и стриминга контента критично, как именно данные лежат в памяти и как перемещаются между подсистемами. Необходима предсказуемость латентности – движок отвечает за тайминг кадра и синхронизацию подсистем, и непредсказуемые паузы в базовом слое особенно опасны. Требуется интеграция с нативными библио-

теками: физические движки, аудиообъекты, кодеки, декодеры – значительная часть инфраструктуры существует как нативный код. Наконец, нужна стабильность ABI и долгоживущие контуры: движок живёт в разнообразных сборках, на разных платформах и с разными требованиями. Все эти факторы формируют unmanaged-ядро как машину, обеспечивающую устойчивую базу. Движок обязан быть платформой, а платформа почти всегда требует нативного уровня.

Почему игровая логика часто управляемая

Логика игры имеет другой профиль требований. Здесь характерны высокая частота изменений в процессе разработки, большое число мелких правил и исключений предметной области, необходимость поддерживать читаемость и предсказуемость в быстро растущем коде и желание уменьшить класс низкоуровневых ошибок, которые плохо диагностируются и дорого исправляются. Managed-языки хорошо подходят для этого слоя, потому что снижают цену итераций и повышают качество обратной связи. При этом логика игры редко требует тотального контроля над памятью на уровне байтов – ей важнее стабильные контракты, ясные границы подсистем и корректная работа с состоянием и событиями. Это не универсальное правило и не единственно верная архитектура. Но для большинства проектов разделение «движок – unmanaged, игра – managed» оказывается устойчивым компромиссом: нижний слой даёт контроль и производительность там, где это необходимо, а верхний – скорость разработки и безопасность там, где это рационально.

Как C# интегрируется в неуправляемое ядро Godot

В Godot ядро движка остаётся unmanaged (C++), а C# выполняется как managed-код поверх рантайма. Между ними существует связующий слой, делающий объекты движка доступными в C# и позволяющий вызывать API движка из управляемого кода. Архитектурно здесь важны три свойства границы. Первое – граница владения: объекты движка имеют собственный жизненный цикл, который не совпадает с жизненным циклом управляемых объектов. В managed-коде вы обычно работаете с представлениями-обёртками, вызывающими на нативные сущности, а значит, сборщик мусора не является единственным фактором времени жизни – существуют правила движка. Второе – граница представления данных: managed- и unmanaged-миры могут иметь разную форму данных, и при переходе через границу иногда требуется преобразование (упаковка, распаковка, копирование, проверка типов). Это не катастрофа, но цена интерфейса, и если архитектура строится на очень частых мелких вызовах через границу, стоимость начинает суммироваться. Третье – граница диагностики: в managed-коде ошибки выражаются как исключения и диагностируются инструментами рантайма, а в unmanaged-коде классы ошибок другие. При пересечении границы важно понимать, где

именно возникла проблема – в логике игры, в вызове API или в нативной подсистеме. Чёткие границы модулей и аккуратные контракты уменьшают площадь неопределённости. Сам слой интеграции не должен становиться центральной темой архитектуры игры. Его задача – быть прозрачным мостом. Но архитектура обязана учитывать существование этого моста и проектировать взаимодействие так, чтобы данные и события передавались крупнее и осмысленнее, а не как поток мелких импульсов.

Практическое следствие – вопрос гранулярности. Если игровая логика строится как множество маленьких вызовов к движку, каждый из которых несёт мало смысла, система становится чувствительной к стоимости моста и к неявным преобразованиям данных. Если же взаимодействие формулируется как более крупные операции – «подготовить данные», «сделать шаг симуляции», «сформировать команды для подсистем», – появляется два преимущества: уменьшается количество пересечений границы и проще контролировать поток данных. Это не призыв делать всё батчами, а критерий архитектурного качества: связь между слоями должна быть осмысленной и ограниченной.

Рассмотрим конкретный пример. Допустим, персонаж получает урон, и нужно проиграть звук, анимацию и эффект частиц.

```
public void TakeDamage(int damage)
{
    Health -= damage;
    GetNode<AudioStreamPlayer>("HitSound").Play();
    GetNode<AnimationPlayer>("Anim").Play("HitReaction");
    GetNode<CPUParticles2D>("BloodParticles").Emitting = true;
}
```

Такой код дёргает движок трижды ради одной доменной операции, и каждый вызов `GetNode` – это пересечение границы. Если вы захотите заменить звуковой движок или переименовать узел `BloodParticles`, править придётся логику получения урона, которая не должна ничего знать о частицах.

```
// Лучше: доменная операция порождает событие, подсистемы реагируют сами.
public event Action<int> DamageTaken;
public void TakeDamage(int damage)
{
    Health -= damage;
    DamageTaken?.Invoke(damage);
}
```

Здесь взаимодействие с движком – один вызов события. Звук, анимация и эффекты подписываются на `DamageTaken` независимо, и логика урона остаётся чистой.

Ещё одно следствие – необходимость уважать жизненные циклы обоих миров. В игре существуют объекты домена (состояния, правила), а в движке – объекты инфраструктуры (ресурсы, узлы, представления). Связь между ними должна проектироваться: доменные сущности не зависят от того, как

именно движок хранит внутренние указатели, а инфраструктурные объекты не становятся единственным местом хранения смысла игры. Такая дисциплина делает проект устойчивым к изменениям и уменьшает число ошибок «владелец уже умер», характерных для границы managed/unmanaged.

Архитектурное разделение: слой движка versus слой геймплея

Разделение на engine-слой и gameplay-слой – не абстрактная схема, а способ удерживать ответственность и контролировать стоимость изменений. Engine-слой (unmanaged) отвечает за универсальные механизмы: время исполнения, подсистемы, ресурсы, взаимодействие с платформой. Gameplay-слой (managed) отвечает за доменные правила, состояние, интерпретацию событий и композицию поведения на уровне игры. С практической точки зрения это означает, что игровой код должен формулировать собственную модель состояния и взаимодействий, а движок использовать как сервисный слой. Если game-код начинает прятать доменные решения внутри объектов движка без явных контрактов, границы размываются, и проект начинает зависеть от случайных деталей инфраструктуры. Если же доменная логика живёт в управляемом слое и общается с движком через ограниченный набор точек интеграции, система становится более устойчивой: изменения локализуются, а ответственность остаётся ясной. Полезный критерий корректного разделения – способность описать обмен между слоями как поток: какие факты приходят из движка (ввод, результаты симуляции, события), какие решения принимаются в игре (правила, переходы состояния) и какие команды возвращаются в движок (обновление представления, запуск эффектов, запросы к подсистемам). Если этот поток получается описать без оговорок «и ещё мы где-то меняем состояние напрямую» – архитектура близка к устойчивой.

Что помнить перед выбором языка

Смысл обсуждения managed и unmanaged в контексте Godot с C# сводится к трём практическим выводам. Unmanaged-среда (ядро движка) даёт полный контроль над памятью и ресурсами, но требует строгой дисциплины владения: ошибка здесь – это не исключение с читаемым стектрейсом, а повреждение данных, которое может аукнуться через полчаса. Managed-среда (C#) снимает с разработчика низкоуровневую ответственность за память и делает ошибки наблюдаемыми, однако взамен требует контролировать профиль аллокаций и аккуратно проектировать взаимодействие с нативным слоем. Рантайм – не волшебная палочка, а инфраструктура: часть проблем он делает невозможными, часть – более заметными, но архитектурные решения о границах, состоянии и потоке данных по-прежнему остаются за разработчиком. Различия между этими средами становятся архитектурно значимыми тогда, когда проект выходит за пределы эксперимента: появляется плотный поток

событий, много объектов и слоёв, долгий жизненный цикл кода. В таком режиме важнее не идеологическая чистота, а ясное разделение ответственности и понятная модель данных.

Теперь, когда граница между управляемым и неуправляемым кодами очерчена, можно перейти к следующему практическому вопросу: в каких случаях игровая логика на C# оправдана, а когда разумнее остаться в пределах GDScript. Ответ строится не на противопоставлении «лучше/хуже», а на конкретных характеристиках проекта.

1.4. Когда выбирать C# вместо GDScript

Выбор между C# и GDScript в Godot не является вопросом вкуса или спором о том, какой язык лучше. Это сугубо архитектурное решение, затрагивающее ожидаемый профиль изменений, предполагаемый срок жизни кодовой базы, её итоговый объём, устройство процесса разработки и круг лиц, которые будут сопровождать проект. Оба языка представляют собой валидные инструменты, способные сосуществовать в рамках одного проекта. Разница заключается в том, какие свойства системы они делают дешёвыми, а какие – дорогими. Цель раздела – предложить рамку, в которой решение принимается по требованиям к проекту: масштабу, профилю рисков, глубине интеграции с редактором и характеру производственного процесса.

Первый шаг к осознанному выбору состоит в различении двух категорий языковых сред. GDScript исторически и концептуально остаётся языком скриптинга, ориентированным на быстрое описание поведения внутри движка и тесную интеграцию с редактором. Он выступает частью платформы Godot и формирует естественный путь, при котором логика пишется быстро, прямо рядом со сценами, параметрами и ресурсами. C#, напротив, является структурированным языком общего назначения, который привносит иной стиль организации кода: более выраженные границы модулей, опору на явные типы и развитый инструментальный контур вне редактора. В Godot C# занимает ту же роль языка пользовательской логики, но адаптирован скорее для проектирования систем, чем для быстрой скриптовой обвязки. Разница между этими категориями проявляется не в синтаксисе, а в том, как проект эволюционирует со временем. Скриптовый слой хорошо чувствует себя в режимах, где код близок к контенту и изменения часты, но локальны. Структурированный слой выигрывает, когда изменения становятся системными: пересматриваются границы подсистем, усложняется модель данных, перераспределяется ответственность. Представьте себе небольшой платформер с пятью типами врагов. Каждый враг оформлен отдельной сценой с парой десятков строк поведения. Здесь GDScript идеален: вы открываете сцену, правите скрипт, запускаете, и весь цикл занимает секунды. Теперь вообразите, что проект вырос в метроидванию с сорока типами врагов, системой инвентаря, деревом прокачки и сохранениями. Поведение врагов начинает

пересекаться: одним нужна реакция на стихийный урон, другим – общая логика патрулирования, третьим – общий поиск пути. В этот момент дублирование кода в локальных скриптах становится проблемой, и те же задачи естественнее решаются выделением общих компонентов, интерфейсов и сервисов, что составляет сильную сторону C#.

Чтобы нейтрально сравнивать инструменты, полезно взглянуть на то, какие дефекты более вероятны и как быстро они становятся видимыми. В проектах, где значительная часть кода представляет собой локальные реакции на события сцены, ошибки чаще проявляются как «поведение не то» и обнаруживаются непосредственным тестированием. Здесь критично быстро менять и проверять, и скриптовый подход поддерживает такую динамику. В проектах же, где логика разрастается в сеть подсистем, состояний и сервисов данных, ошибки часто выглядят как несоответствие контрактов: одна подсистема ожидает одно, другая отдаёт другое; часть данных стала обязательной, но не обновилась во всех местах; рефакторинг оставил неочевидный разрыв. Классический пример – метод `ApplyDamage(int amount)`. В GDScript вы меняете сигнатуру на `ApplyDamage(int amount, DamageType type)`, запускаете игру и получаете ошибку, только когда конкретный враг попытается получить урон, возможно, через полчаса игры. В C# компилятор укажет каждое место расхождения немедленно. Речь идёт именно о профиле обнаружения: где вы хотите получить сигнал об ошибке – при компиляции и анализе или в рантайме через сценарии и ручную проверку. Архитектурное решение состоит в том, чтобы согласовать этот профиль с масштабом и стоимостью регрессий.

GDScript рационально выбирать, когда ценность даёт скорость локальных итераций и близость к редактору, а масштаб системы пока невелик или контролируем. Эта ценность раскрывается в нескольких типичных сценариях. Первый – быстрое прототипирование механик, где темп экспериментов важнее формальной структуры, а скриптинг рядом со сценой снижает накладные расходы. Второй сценарий – небольшие проекты с ограниченным масштабом и простой доменной моделью, где стоимость профессиональной модульности может не окупиться. Третий – плотная интеграция с редактором там, где поведение тесно связано с настройкой сцен и ресурсов: UI-логика, инструментальные скрипты внутри редактора, мелкие компоненты поведения, живущие рядом с контентом. Четвёртый – скриптинг как связующий слой, когда задача сводится к тому, чтобы склеить существующие подсистемы, а не строить новую доменную модель. Во всех этих случаях архитектурная рекомендация одинакова: если выбирается GDScript, стоит осознанно ограничивать область его применения. Полезно заранее определить точку выхода из чисто скриптового режима, ориентируясь не на количество строк, а на симптомы связанности: появление общих глобальных состояний, каскадные изменения от простой правки, рост числа скрытых зависимостей между сценами, потребность повторно использовать одну и ту же логику без дублирования. При появлении таких симптомов имеет смысл либо усилить дисциплину границ в скриптовом коде, либо перенести системный слой в более строгую структуру.

C# рационально выбирать, когда проект начинает вести себя как система: в нём кристаллизуются подсистемы, границы ответственности, длительная жизнь кода и необходимость безопасно менять структуру. В средних и больших системах, где логика игры представляет собой сеть взаимодействующих модулей, ценность явных контрактов возрастает, а типы и интерфейсы помогают закрепить границы и удерживать направление зависимостей. Для долгоживущих проектов, в которых происходят частые структурные изменения (переименование понятий, переразбиение модулей, миграции данных, замена реализаций), IDE-рефакторинги и компиляция становятся механизмом контроля рисков. Безопасность рефакторинга оказывается критичной: когда изменение сигнатуры или типа данных затрагивает сотни мест, динамическая проверка делается неприемлемо дорогой, а статическая проверка и анализаторы уменьшают класс тихих ошибок и ускоряют код-ревью. Инструменты вне редактора – CI, автоматические проверки, линтеры, тесты, генерация артефактов – хорошо поддерживаются экосистемой C#, и архитектура проекта легче привязывается к процессу. Наконец, в командной работе C# способствует читаемости и предсказуемости кода через более явные структуры модулей и инструменты навигации, что снижает стоимость вхождения новых участников и уменьшает зависимость проекта от личной памяти отдельных разработчиков. Здесь важно уточнение: C# не служит магическим средством «сделать архитектуру». Он лишь делает архитектурные решения более выраженными и проверяемыми. Если проект не формулирует границы и ответственность, типы не спасут. Но если границы формулируются, C# помогает удерживать их в живом коде.

Выбор между удобством интеграции с редактором и целостностью кодовой базы также является частым источником ошибок. Эти цели совместимы, но требуют ясной границы. Редактор и сцены – сильная сторона Godot, они задают форму данных проекта и упрощают создание контента. Однако если доменные правила начинают растворяться в настройках сцен и локальных скриптах без явного системного слоя, целостность ухудшается: правила становится сложно найти, переиспользовать и менять без побочных эффектов. Обратная крайность – строить систему так, будто редактор не существует, – обычно приводит к потере управляемости контента и утяжелению итераций. Практичная архитектура делит ответственность: редактор и сцены задают данные и композицию, а системный слой – правила интерпретации и поведения. Язык выбирается под тот уровень, где находится основной риск: в данных и контенте или в структуре правил. Масштабируемость проекта – не рост количества файлов, а способность удерживать систему в предсказуемом состоянии при росте требований. В этой способности участвуют модель данных, направление зависимостей и процесс изменений. GDScript часто удобен, когда модель данных близка к структуре сцен, а изменения локальны. Но по мере роста возникают риски: соглашения о данных существуют неявно и проверяются только в рантайме; границы между подсистемами размываются из-за возможности «достать что-то по ссылке»; рефакторинг дорожает, так как часть ошибок обнаруживается лишь через сценарии игры.

C# чаще выбирают, когда эти риски начинают доминировать над скоростью локальных итераций. Он делает контракты более явными, а изменения – механически проверяемыми, что не отменяет архитектурных ошибок, но снижает их частоту и делает заметнее.

С точки зрения сопровождения полезно различать две категории кода. Код поведения, привязанный к контенту (локальные реакции, UI-связки, сценарные элементы), может быть комфортен в скриптовом режиме. Код систем, которые переживут множество изменений (правила, состояние, взаимодействие подсистем, сервисы данных), обычно выигрывает от языка и инструментов, поддерживающих долгую жизнь и строгие контракты. В крупных проектах эти категории часто разделяют даже организационно: скриптинг в верхних слоях, структурированный код в системном слое. Статическая типизация полезна не как строгость ради строгости, а как ускорение обратной связи. Она помогает в трёх ключевых точках: стабильность интерфейсов (изменения публичного контракта немедленно подсвечиваются компиляцией), снижение доли неявных соглашений (явные структуры с полями и типами оставляют меньше пространства для ошибок) и рефакторинг как часть процесса (позволяет менять структуру без ручной проверки каждого пути исполнения). GDScript даёт другую обратную связь: быстрый запуск, быстрый эксперимент, непосредственное наблюдение поведения на сцене. Архитектурный выбор сводится к тому, какая обратная связь важнее для текущего масштаба: мгновенная проверка поведения или механическая проверка структурных изменений.

Вопрос производительности часто используют как аргумент за или против языка, но для архитектуры полезнее смотреть на источники стоимости. Язык действительно может иметь значение в плотных вычислительных задачах, выполняемых каждый кадр на большом объёме данных, в местах с обильными аллокациями и временными объектами, в частых пересечениях границы между пользовательским кодом и ядром движка, а также в системах, где критична равномерность времени кадра. Язык обычно вторичен, когда узкое место находится в подсистемах движка или когда архитектура создаёт лишнюю работу из-за неправильной модели данных. Практическое правило таково: язык выбирают под архитектуру и процесс, а производительность затем измеряют и оптимизируют адресно.

Смешанный подход часто выглядит рациональным: GDScript применяется там, где важна тесная работа со сценами и быстрые правки поведения, а C# – там, где важны контракты и долговечность систем. Это действительно рабочая стратегия, но при условии, что проект заранее решает две ключевые задачи: определяет, какие подсистемы составляют ядро игры и должны иметь стабильные контракты, и фиксирует правила взаимодействия между языками – через ограниченный набор входов и выходов (события, команды, структуры данных), а не через прямое дотягивание до внутренностей модуля. Смешанный подход требует также дисциплины в диагностике: ошибки и логи должны указывать на слой, где принято решение, а не только на слой, где оно проявилось. В командных процессах, выходящих за пределы

одиночной разработки, ценность языка определяется тем, насколько легко фиксировать правила кодовой базы как проверяемые, обеспечивать воспроизводимость сборок и зависимостей, проводить масштабные изменения без ручного аудита каждого файла и поддерживать единый стиль. C# обычно удобен в таком окружении, тогда как GDScript может быть частью пайплайна, но характер контроля будет иным: больше опоры на договорённости и сценарные проверки.

Итоговая рамка для принятия решения опирается на три оси: масштаб, срок жизни и процесс. Для небольшой игры с ограниченным числом механик и локальными изменениями GDScript часто рационален, тогда как для сети подсистем с богатой моделью состояния C# чаще снижает стоимость развития. При коротком цикле разработки и редких структурных изменениях скриптовый подход бывает эффективнее; если ожидаются годы сопровождения и серия крупных рефакторингов, выбор языка с сильной поддержкой рефакторинга становится архитектурным преимуществом. В режиме быстрой итерации без сложного пайплайна GDScript даёт минимальные накладные расходы; при построении процесса с CI, тестами и распределённой командой C# обычно лучше ложится в этот контур. Смешанный подход с GDScript для локального сценарного поведения и C# для системного слоя работает при соблюдении осознанных границ и ограниченном, понятном взаимодействии между слоями.

Существуют и неидеологические факторы, которые разумно учитывать: компетенции команды и возможности найма, инструментальная среда, требования к повторному использованию кода. Если решение неочевидно, полезно начать с вопроса «что именно должно быть стабильным»: контент и локальное поведение или структура систем и контрактов. Первый тип стабильности чаще ведёт к GDScript, второй – к C# для системного слоя.

1.5. Установка и настройка окружения

Среда разработки – часть архитектурной дисциплины. Архитектура живёт в изменениях: вы переименовываете понятия, перестраиваете зависимости, добавляете подсистемы. Если среда не воспроизводима и не даёт быстрой обратной связи, проект начинает избегать рефакторинга, копировать код вместо выделения модулей и держаться за случайные структуры, то любое изменение становится рискованным. Поэтому с самого начала нужно понимать, какие компоненты требуются для C# в Godot, где проходят границы их ответственности и как проверять, что всё собрано корректно.

Что вы устанавливаете на самом деле

Для работы на C# в Godot нужны четыре компонента, и путать их роли нельзя.

Редактор Godot со встроенной поддержкой C# отвечает за загрузку проекта, сцен и ресурсов, исполняет игровой цикл и предоставляет API к под-

системам. .NET SDK содержит компилятор, инструменты сборки и стандартные библиотеки – без него C#-код просто не во что собирать. IDE или редактор кода обеспечивает навигацию, диагностику и рефакторинг. Наконец, конфигурационные файлы проекта – .sln и .csproj – связывают всё перечисленное в единую систему. Ошибки окружения почти всегда возникают оттого, что роль одного компонента пытаются возложить на другой. Редактор движка не заменит сборочную инфраструктуру .NET. IDE не исправит неверную версию SDK. Файл .csproj – не просто список исходников, а контракт: он фиксирует, какие пакеты подключены, под какую платформу идёт сборка и какие параметры компиляции действуют. Если этот контракт не совпадает с тем, что ожидает SDK или Godot, проект перестаёт собираться, и диагностика начинает врать. Типичный сценарий: вы устанавливаете Godot, затем .NET SDK, открываете проект, а IDE настойчиво утверждает, что не видит типы движка. Причина почти наверняка в том, что Godot – сборка без C#-поддержки, либо версия .NET SDK не совпадает с той, под которую собран движок. Решение не в том, чтобы «покрутить настройки IDE», а в том, чтобы проверить версию SDK командой `dotnet --version` и сверить её с документацией Godot для вашей версии движка. Такая проверка занимает минуту и снимает половину проблем ещё до их возникновения.

Как Godot и .NET работают вместе

Godot с поддержкой C# отличается от обычной сборки наличием слоя интеграции. При запуске проекта движок загружает сцены и ресурсы, затем находит собранную .NET-сборку и подключает её. С этого момента управляемый код становится частью пользовательской логики: движок вызывает методы ваших скриптов, а скрипты – методы API движка. Зоны ответственности здесь строго разделены. Godot владеет циклом игры, объектной моделью сцены и подсистемами. .NET-рантайм исполняет управляемый код и управляет памятью управляемых объектов. Слой связывания между ними делает объекты движка видимыми в C# через управляемые обёртки. У каждого такого объекта два аспекта: нативная часть внутри движка и оболочка в C#. Их жизненные циклы не совпадают – об этом придётся помнить, когда мы доберёмся до владения объектами и утечек памяти в главе 2. Сборка C#-проекта происходит до запуска: исходный код и .csproj определяют, что компилировать, компилятор создаёт assembly, движок её загружает. Между «изменил файл» и «увидел результат» всегда стоит компиляция, и это архитектурное преимущество, а не техническая деталь. Сборка проверяет консистентность типов, наличие зависимостей и совместимость с выбранной версией SDK. Если проект не собирается, безопасные изменения невозможны – вы теряете главный механизм обратной связи. Поэтому проект должен собираться постоянно, а не раз в день. Падающая сборка означает, что типы и контракты перестали работать как система раннего предупреждения.

Согласование версий и типичные ошибки

Большинство проблем окружения упираются в одну причину: Godot, IDE и командная строка используют разные версии .NET SDK или разные параметры сборки. В результате проект ведёт себя по-разному на разных машинах – классическое «на моей машине работает». Лечится это фиксацией версии SDK в настройках проекта и единым источником истины для сборки. Если IDE показывает одни ошибки, а командная строка – другие, сначала устраните расхождение. Обсуждать архитектуру при рассинхронизированной среде бессмысленно.

Вот минимальный чек-лист, который стоит выполнять при первой настройке и после каждого обновления SDK или Godot. Откройте проект в Godot – редактор должен распознать C#-часть без дополнительных действий. Откройте решение в IDE – среда должна загрузить .sln и .csproj, увидеть зависимости и построить модель типов без фантомных ошибок. Выполните сборку из командной строки: `dotnet build` должен отработать без ошибок и выдать тот же результат, что и сборка из IDE. Запустите проект из Godot – движок должен загрузить `assembly` и выполнить ваш код без ошибок интеграции.

Если все четыре шага проходят, среда настроена корректно. Если какой-то шаг не проходит – проблема локализована, и вы знаете, в каком слое копать. Эти проверки не привязаны к конкретной операционной системе и занимают не более пары минут.

Поддержание порядка со временем

Окружение портится не при установке, а со временем: SDK обновляется, зависимости меняются, настройки IDE дрейфуют, в репозиторий попадают артефакты сборки. Порядок держится на нескольких привычках. Фиксируйте версию SDK в репозитории. Файл `global.json` на уровне решения позволяет указать конкретную версию .NET SDK, и все инструменты – IDE, командная строка, CI – будут использовать именно её. Это не устная договорённость, а машиночитаемое правило. Разделяйте исходники и артефакты сборки: вторые – производные, их не хранят как источник истины и не добавляют в систему контроля версий. Стандартный `.gitignore` для Godot-проектов на C# уже исключает папки `bin/` и `obj/`, и этого достаточно. Относитесь к диагностике компилятора как к интерфейсу системы: предупреждения и ошибки – не шум, который можно игнорировать до релиза, а канал обратной связи. Если их копить, архитектура теряет механизм самоконтроля, а вы – возможность безопасного рефакторинга. С инженерной точки зрения хорошая среда обладает тремя свойствами: наблюдаемостью (вы всегда знаете, какой инструмент выполняет сборку и какой SDK используется), повторяемостью (сборка на разных машинах даёт одинаковый результат) и изолируемостью (проблемы локализуются в одном слое, а не проявляются как хаотическое поведение). Эти свойства поддерживаются не объёмом инструкций, а мини-

мальным набором привычек: фиксировать версии, держать сборку зелёной, не смешивать артефакты и проверять изменения конфигурации так же тщательно, как изменения кода.

1.6. Первый минимальный C#-скрипт

Минимальный скрипт в Godot решает две задачи: подтвердить, что среда работает как единая система, и зафиксировать базовую структуру пользовательского кода. Прежде чем проект начнёт обрастать механиками, необходим гарантированный контур обратной связи: изменил код → собрал → запустил → увидел результат. Без него невозможно безопасно улучшать структуру – вы просто не знаете, исполнился ли ваш код вообще или движок молча проигнорировал изменения. Представьте: вы переименовали метод `TakeDamage` во всех пятнадцати местах, запускаете сцену, а персонаж молча умирает посреди боя. Это не баг игровой логики – это сборка не подхватила новый файл, а вы об этом не знали. Мы создадим намеренно крошечный пример: один класс, один метод, один наблюдаемый эффект. Он не показывает «как делать игру». Его роль – инструмент проверки, `probe`: где находится пользовательская логика, как она цепляется к объектной модели Godot и как исходный файл превращается в исполнение. Скрипт всегда живёт в контексте узла. `Node` здесь достаточно понимать как минимальную единицу, к которой движок привязывает поведение. Между движком и пользовательским кодом действует контракт. Движок работает не с файлами, а с объектами определённых типов. Он вызывает у них ограниченный набор точек входа с фиксированными именами и сигнатурами. Скрипт не существует отдельно от узла, к которому прикреплён. Граница ответственности прочерчена жёстко: движок отвечает за «когда» и «в каком контексте», разработчик – за «что означает это событие». Именно этот контракт станет опорой для разговора о жизненном цикле узлов в следующих главах.

Вот сам код:

```
using Godot;
public partial class MinimalProbe : Node
{
    public override void _Ready()
    {
        GD.Print("MinimalProbe: _Ready() called");
    }
}
```

Он намеренно пустой по смыслу и плотный по форме. Разберём структурные элементы.

`using Godot;` – мост к API движка. Без него не получится ни унаследоваться от `Node`, ни вызвать `GD.Print` без полного квалифицированного имени.

`public partial class MinimalProbe : Node` – здесь два важных слова. `: Node` фиксирует архитектурный факт: вы пишете не обычный класс, а компонент поведения, встроенный в объектную модель Godot. `partial` – часть соглашения: движок дополняет тип своей инфраструктурой, а ваш код и сгенерированная часть объединяются в один класс. Не нужно сейчас копаться в механизме – достаточно знать, что это не украшение, а условие работы.

`public override void Ready()` – точка входа. Слово `override` означает, что базовый `Node` уже объявил виртуальный метод, а вы даёте ему реализацию. Здесь закладывается ключевая модель мышления: не вы дёргаете движок, а движок дёргает вас. Основной цикл исполнения живёт в движке, ваш код встраивается в него через колбэки. Порядок и частота вызовов вам не принадлежат. Для архитектуры это означает, что вы проектируете не «процедуру, которая делает всё», а набор реакций на события платформы.

`GD.Print(...)` – датчик, не логгер. Одна строка в консоли однозначно сообщает: сборка загружена, тип найден, узел создан, движок дошёл до пользовательского кода. Если сообщения нет – проблема в сборке или привязке, а не в алгоритме.

`_Ready()` выбрана как точка входа не случайно. Она не требует внешних условий вроде ввода игрока, а её вызов хорошо интерпретируется: появилось сообщение – цепочка работает. Это также задаёт правильную мысль на будущее: инициализация должна происходить в точках, контролируемых платформой, а не где-то в порядке «само собой получилось». Теперь о том, как код становится исполнением. Цепочка выглядит так: вы меняете C#-файл в проекте (.csproj), .NET компилирует исходник в сборку, Godot загружает сборку и по метаданным типов связывает классы со скриптами, при запуске сцены движок создаёт экземпляр узла и вызывает `_Ready()`. Из этой цепочки вытекают два типовых класса проблем. «Код не исполняется» – сборка не произведена или не загружена. «Код исполняется не тот» – среда использует другой артефакт сборки, или проект открыт не в той конфигурации. Минимальный скрипт – тест именно на эту целостность, а не на логику.

Чтобы скрипт исполнился, его нужно связать с узлом в сцене. Схема концептуальная: сцена хранит узел как данные, узел ссылается на скрипт, при запуске движок создаёт экземпляр узла и связывает его с классом, затем дёргает `_Ready()`. Узел и скрипт – разные сущности. Узел принадлежит данным сцены и задаёт композицию, скрипт – управляемому коду и задаёт правила. Это разделение станет критичным, когда появятся сложные системы: вы захотите, чтобы структура мира управлялась данными, а поведение – кодом. Связь между классом и узлом держится на идентичности: файл → тип → сборка. Если файл переименовали, а в сцене осталась ссылка на старый тип – привязка молча ломается. Или файл изменили, а сборка не пересобралась – вы правите не тот артефакт. Отсюда простое правило, которое стоит держать с первого дня: имя файла, имя класса и имя скрипта в сцене либо совпадают, либо явно соотносятся. Это не эстетика. Когда в проекте полсотни узлов, навигация без явных соответствий превращается в гадание. Вывод `GD.Print` попадает в консоль редактора при запуске из редактора и в стан-

дартный вывод процесса при автономном запуске. На практике «сообщение не видно» сплошь и рядом означает «вы смотрите не в тот канал», а не «метод не вызвался». Имеет смысл сразу зафиксировать, где именно вы наблюдаете результат, чтобы не тратить часы на диагностику исправного кода. Что подтверждает корректность: в выводе появляется заданная строка, сообщение возникает один раз за запуск, изменение строки в `GD.Print` меняет вывод после пересборки. Этого достаточно. На этом этапе легко наделать архитектурных ошибок. Не надо вводить зависимости от других узлов, менять состояние сцены, использовать сложные конструкции языка или имитировать игровые механики. Минимальный скрипт – `probe`, а не заготовка будущей системы. В реальных проектах пробники имеют свойство приживаться и со временем превращаться в странные зависимости, о происхождении которых никто не помнит. Полезная привычка – держать их либо явно временными, либо оформлять так, чтобы они не торчали наружу входами и выходами, которые кто-то начнёт использовать как API.

Понимание движка

2.1. Знакомство с Nodes и Scenes

Когда вы впервые открываете Godot, то видите редактор сцен и панель с перечнем узлов. Движок предлагает добавить Node, создать Scene, сохранить её. Новичок часто пропускает этот момент и начинает перетаскивать объекты на экран, радуясь, что что-то работает. Но понимание разницы между узлом и сценой – фундамент, на котором держится вся архитектура игры. Ошибка здесь не видна сразу, но когда проект разрастается до десятков сущностей, нескольких уровней и системы сохранений, последствия кривой структуры начинают отнимать часы.

Узел – минимальная единица ответственности

Представьте конструктор. У вас есть детали: колёса, балки, окошки, оси. Каждая деталь делает что-то одно. Из колеса нельзя построить стену, а из балки не собрать крышу. В Godot узел – такая же деталь. Это минимальная исполняемая единица, из которой строится игра. Узел `Sprite2D` показывает картинку, узел `AudioStreamPlayer` воспроизводит звук, узел `Timer` отсчитывает интервалы. У каждого своя ответственность, и за неё он отвечает полностью. Главное правило, которое нужно запомнить с самого начала: хорошо спроектированный узел можно описать одним предложением. «Этот узел считает очки», «этот узел двигает камеру», «этот узел обнаруживает столкновения». Плохо спроектированный узел отвечает за всё сразу: тут и очки, и музыка, и ввод, и сохранение. Такой код превращается в кашу за несколько дней и отказывается масштабироваться.

Давайте посмотрим на примере, как выглядит хорошая и плохая архитектура. Допустим, мы делаем игрока. У него есть здоровье, он двигается и может получать урон. Новичок часто пишет единый скрипт на корневом узле:

```

public partial class Player : CharacterBody2D
{
    public int Health = 100;
    public int Speed = 400;
    public override void _PhysicsProcess(double delta)
    {
        Vector2 velocity = Velocity;
        velocity.X = Input.GetAxis("ui_left", "ui_right") * Speed;
        velocity.Y = Input.GetAxis("ui_up", "ui_down") * Speed;
        Velocity = velocity;
        MoveAndSlide();
        if (Health <= 0)
            QueueFree();
    }
    public void TakeDamage(int damage)
    {
        Health -= damage;
        GetNode<AudioStreamPlayer>("HurtSound").Play();
        GetNode<AnimationPlayer>("AnimationPlayer").Play("hurt");
    }
}

```

С виду всё работает. Игрок двигается, урон наносится, звук хлюпает. Но попробуйте теперь сделать врага, у которого тоже есть здоровье, но другая анимация и скорость. Вам придётся либо копировать код (здоровствуй, дублирование логики), либо городить наследование `Enemy : Player` (здоровствуй, враг, умеющий ходить по WASD). Оба пути ведут в тупик при росте проекта.

Правильный подход – вынести здоровье в отдельный компонент. Создадим `HealthComponent`, который отвечает только за хранение здоровья и оповещение об изменениях. Он ничего не знает ни о движении, ни об анимациях, ни о том, кто именно им владеет:

```

public partial class HealthComponent : Node
{
    [Export] public int MaxHealth { get; set; } = 100;
    public int CurrentHealth { get; private set; }
    [Signal]
    public delegate void HealthChangedEventHandler(int currentHealth);
    [Signal]
    public delegate void DiedEventHandler();
    public override void _Ready()
    {
        CurrentHealth = MaxHealth;
    }
    public void TakeDamage(int amount)
    {
        CurrentHealth = Mathf.Max(CurrentHealth - amount, 0);
        EmitSignal(SignalName.HealthChanged, CurrentHealth);
        if (CurrentHealth == 0)
            EmitSignal(SignalName.Died);
    }
}

```

Теперь этот компонент можно прикрепить к игроку, к врагу, к хрупкой вазе, которая разбивается от удара, к двери, которую нужно взорвать. Он везде работает одинаково, потому что его ответственность строго ограничена. Скрипт игрока при этом становится тоньше: он больше не хранит здоровье, а лишь реагирует на сигналы компонента. Такой подход называется композицией, и Godot заточен именно под него.

Полезно также различать две роли, которые узел может играть. Узел-адаптер связывает вашу логику с механизмами движка: физикой, рендером, интерфейсом. Узел-модуль домена хранит правила игры: инвентарь, здоровье, логику взаимодействия. Обе роли допустимы, но смешивать их в одном узле без границ нельзя. Доменная логика начинает зависеть от деталей инфраструктуры, и любое изменение перестаёт быть локальным. `HealthComponent` в примере выше – чистый домен: он не знает, кто его использует, и не обращается к API движка за пределами сигналов.

Сцена – шаблон для сборки узлов в готовый объект

Если узел – деталь конструктора, то сцена – инструкция по сборке. Сцена описывает, какие узлы входят в состав объекта и как они связаны друг с другом. Например, сцена игрока содержит корневой `CharacterBody2D`, дочерние `Sprite2D` для отрисовки, `CollisionShape2D` для физического контура и наш `HealthComponent` для здоровья. Всё это сохраняется в файл `.tscn` и может быть переиспользовано в любом месте проекта.

Критически важно понимать разницу между сценой-шаблоном и экземпляром сцены. Шаблон лежит на диске и описывает, как объект выглядит по умолчанию. Экземпляр – это живой набор узлов, созданный по шаблону в рантайме. Одна сцена врага может быть инстанцирована сорок раз на уровне, и каждый экземпляр будет независим: убили одного – остальные продолжают жить, шаблон на диске не меняется.

Эта разница особенно важна при сохранении игры. Когда вы сохраняете прогресс, вы записываете состояние экземпляров – здоровье, позицию, активные эффекты, – но никогда не перезаписываете шаблон. Если вы спавните врага кодом через `enemyScene.Instantiate()`, вы получаете независимую копию, никак не связанную с другими экземплярами и с исходным файлом сцены.

Одна из главных ошибок новичков – воспринимать сцену исключительно как уровень. Да, уровень удобно хранить сценой. Но точно такой же сценой может быть кнопка меню, иконка предмета в инвентаре, эффект взрыва, диалоговое окно или отдельный враг. Если вы научитесь видеть в сценах универсальные шаблоны, а не «контейнер для всего», вы сможете переиспользовать наработки бесконечно.

Как сцены управляют сложностью без тонны классов

Когда в игре появляются десятки типов врагов, сотни предметов и разные виды UI-панелей, возникает вопрос: как управлять этим разнообразием?

Новичок с опытом ООП хватается за наследование: `Enemy` → `ShootingEnemy` → `Boss` → `FinalBoss`. Иерархия растёт, контракты размываются, переиспользование превращается в случайные компромиссы. Godot предлагает другой путь – композицию.

Представьте, что у вас есть базовая сцена врага с узлами `Sprite2D`, `AnimationPlayer`, `HealthComponent` и `CollisionShape2D`. Дизайнер просит сделать врага, который стреляет. Вместо создания класса `ShootingEnemy` : `Enemy` вы просто добавляете в сцену ещё один узел – `ShootingBehaviour`. Теперь враг стреляет. Через час дизайнер просит убрать у врага здоровье – вы удаляете `HealthComponent`, и враг становится бессмертным. Ещё через час нужен враг, который и стреляет, и лечится сам, – вы добавляете `HealingBehaviour`. Вся вариативность достигается перетаскиванием узлов в редакторе или изменением параметров, а не переписыванием кода.

Три механизма композиции закрывают практически все сценарии. Параметризация: одна сцена с разными значениями скорости, урона и таймингов даёт несколько разных врагов без единой новой строчки кода. Композиционные различия: в одну сцену добавлен `ShootingBehaviour`, в другую – `HealingBehaviour`, состав узлов разный, но базовый код общих компонентов не меняется. Замена узла при сохранении контракта: сцена остаётся тем же объектом, но узел анимации заменён на альтернативную реализацию, и внешний код не замечает подмены. Все три способа работают на одном принципе, где вариативность выражается через данные и композицию, а не через наследование и ветвление по типам. Есть ещё одна важная штука, без которой сцены не работают как модули, – публичная поверхность. Представьте пульт от телевизора: кнопки «включить», «переключить канал», «изменить громкость». Вы пользуетесь кнопками и не лезете отвёрткой внутрь корпуса. Так же должна работать и сцена, внешний код общается со сценой через публичные методы и сигналы корневого узла, но никогда не обращается к дочерним узлам напрямую. Плохо: `enemyScene.GetNode<AnimationPlayer>("Body/AnimationPlayer").Play("die")`. Внешний код знает, что у сцены есть `Body`, внутри него `AnimationPlayer` и как он называется. Переименуйте `Body` в `Visuals` – сломается проект. Хорошо: корневой узел сцены предоставляет публичный метод `PlayDeathAnimation()`, внутри которого вызывает `GetNode<AnimationPlayer>("AnimationPlayer").Play("die")`. Внешний код знает только о методе, но не о внутренней структуре. Переименование узлов внутри сцены никому не ломает жизнь. Если запомнить эту аналогию с пультом, вы избежите десятков часов отладки разваливающихся зависимостей, а ваши сцены останутся модульными даже при росте проекта.

2.2. Дерево сцены

Новичок открывает Godot, видит вкладку `Scene` с аккуратной иерархией узлов и приходит к естественному, но опасному выводу: `Scene Tree` представ-

ляет собой просто список объектов, по которому удобно искать и перемещать элементы. Это первая и самая дорогая ошибка. Scene Tree не является визуальной диаграммой, это модель исполнения. Порядок обновления, владение объектами и распространение изменений – всё, что происходит в игре во время работы, завязано на данную структуру. Если относиться к ней как к проводнику, архитектура проекта начинает разрушаться с первого дня.

Расхождение шаблона и реальности

Начать стоит с различия, которого избегают даже многие документации. Сцена как ресурс, сохранённый в файле .tscn, служит описанием, статическим чертежом, фиксирующим набор узлов, их связи и значения свойств по умолчанию. Scene Tree, напротив, представляет собой актуальный, живой набор экземпляров этих узлов в памяти. Когда вы нажимаете Run, движок берёт сцену-шаблон и создаёт по ней конкретные объекты, но дальше эти объекты живут собственной жизнью: они могут удаляться, рядом с ними появляются новые, целые поддеревья заменяются или выключаются. Шаблон остаётся неизменным, а дерево исполнения уже выглядит иначе.

Представьте сцену Enemy.tscn со спрайтом, коллайдером и скриптом здоровья. В редакторе вы видите одно поддерево. Вы запускаете уровень, и движок создаёт пятнадцать копий этого шаблона – пятнадцать независимых веток, у каждой своё значение health, своя позиция, своя фаза поведения. Все они сосуществуют в одном Scene Tree, и каждая выступает полноценным участником исполнения. Если бы сцена и дерево были одним и тем же, пятнадцать врагов считали бы себя одним объектом.

Любое дерево имеет корень, и в Godot он играет роль не просто главного узла, а композиционного центра, с которого начинается обход всей рантайм-структуры. Он задаёт контекст существования всех дочерних элементов: определяет порядок обработки, владение и доступ к глобальным сервисам. Архитектурно корень выполняет роль клея, соединяющего подсистемы в единое целое.

Типичная ошибка заключается в превращении корня в «божественный объект», управляющий всем на свете. В него начинают сваливать логику переключения сцен, инициализацию всех менеджеров, глобальные переменные. Через месяц в Root.gd уже тысяча строк, и любое изменение ломает что-то неожиданное. Здоровый взгляд противоположен: корень служит минимальным контейнером, который лишь связывает независимые модули. Мир игры, интерфейс, система сохранений висят в дереве как отдельные ветки, а корень только предоставляет им место и общий контекст. В коде эта идея выражается через явную композицию:

```
public partial class Root : Node
{
    [Export] private World worldScene;
    [Export] private UIManager uiManager;
    public override void _Ready()
    {
```

```

        AddChild(worldScene.Instantiate());
        AddChild(uiManager);
        // Корень не знает, как работает UI или мир, – он просто соединяет.
    }
}

```

Никакой логики, только внедрение и активация. Именно такую роль исполняет корень в правильно спроектированном проекте.

Родитель и ребёнок как контекст исполнения

Связь parent-child в Scene Tree не сводится к косметической вложенности. Она определяет три критичных аспекта: контекст существования объекта, владение жизнью и смертью, а также встраивание в цикл движка, диктующее момент и порядок вызова методов. Когда вы переносите узел из одной ветки в другую, вы меняете ему среду обитания – это действие нельзя сводить к простому переключению папки.

Рассмотрим конкретный сценарий. Панель инвентаря висит внутри сцены игрока по пути Player/InventoryPanel, и её код ожидает, что родительский Player предоставит доступ к своему состоянию:

```

var player = GetParent<Player>();
player.AddItem("sword");

```

Пока панель внутри игрока, всё работает. Затем вы решаете, что инвентарь должен стать частью общего UI, и переносите узел в ветку UI/InventoryPanel. Код внезапно падает: GetParent возвращает уже не Player, а UI, который ничего не знает об AddItem. Перенос узла изменил контекст исполнения, и скрытая зависимость от структуры немедленно дала о себе знать. Отсюда следует простой вывод: если код опирается на конкретное место в дереве, архитектура уже хрупкая, а все зависимости должны быть явными и не привязанными к иерархии.

Godot намеренно ограничивает структуру исполнения деревом, а не произвольным графом. Наличие у каждого узла ровно одного родителя гарантирует однозначное владение и детерминированный обход. Движок может точно определить, в каком порядке проходить узлы за кадр, не решая сложную задачу обхода циклических связей. Такой подход обеспечивает предсказуемость, которая для движка важнее гипотетической гибкости. Реальные игровые зависимости, конечно, выходят за пределы одного дерева: врагу нужно знать о плее, плее о UI, UI о настройках. Но эти связи строятся поверх дерева через сигналы, ссылки и сервисы, не ломая базовую структуру. Дерево служит позвоночником, а все остальные связи – мышцами и сухожилиями, которые крепятся к нему, не становясь позвонками.

Ловушка порядка и антипаттерн обхода

Одна из самых коварных ошибок заключается в неявном использовании порядка детей для управления логикой. Движок обходит дерево в опреде-

лѐнной последовательности, и новичок быстро замечает: если узел А стоит в иерархии выше узла В, то методы `_Ready()` и `_Process()` у А вызываются раньше. Возникает соблазн строить на этом причинно-следственные связи.

Предположим, вы делаете систему, в которой враг должен реагировать на обновлѐнные данные игрового мира. Вы помещаете ноду `WorldUpdater` выше `Enemy` в дереве, чтобы `_Process` обновлял данные мира раньше, чем враг их прочитает. Неделю всё работает. Потом вы рефакторите сцену, объединяете врагов в контейнер `Enemies`, который оказывается выше `WorldUpdater`, и логика молча переворачивается. Враг начинает действовать на устаревших данных, а причина остаѐтся незаметной. Правило здесь однозначно: порядок, задаваемый движком, является деталью реализации, а не доменным контрактом. Если что-то должно произойти раньше чего-то, такая зависимость выражается явно через сигнал, вызов метода или общее состояние с контролируемым порядком обновления. Никогда не полагайтесь на очерѐдность узлов в дереве как на архитектурный механизм.

Теперь о том, с чего началась эта глава в сознании новичка: использование `GetNode()` для поиска любых нужных объектов. Такой подход служит самым быстрым способом сделать проект неподдерживаемым и одновременно первой мыслью, которая приходит в голову при виде иерархии.

Типичный код, который пишут в первый день:

```
var ui = GetNode<UIManager>("/root/World/UIManager");
ui.UpdateHealth(50);
```

Подобное решение ведѐт к катастрофе по нескольким причинам. Строка пути выступает магической константой: если вы переименуете узел `World` в `GameLevel`, компилятор не выдаст ошибки, игра запустится, враг перестанет обновлять интерфейс, а вы пойдѐте искать баг в боевой системе, хотя проблема кроется в жѐсткой ссылке на структуру. Кроме того, вы предполагаете, что дерево всегда будет выглядеть одинаково, однако динамическая композиция остаѐтся одной из главных возможностей Godot. Узлы создаются и удаляются, сцены инстансируются в разных местах. Сегодня `UIManager` живѐт в `/root/World/UIManager`, завтра он переезжает в отдельную ветку, чтобы переживать смену уровней, и все вызовы `GetNode` по старому пути умирают.

Самое опасное последствие кроется в закладывании фундамента под лавину. Стоит одному скрипту использовать абсолютный путь, как остальные начинают делать то же самое. Через месяц каждый уголок кодовой базы знает точную структуру сцены, и малейший рефакторинг вызывает десятки ошибок в неожиданных местах. Дерево перестаѐт быть структурой исполнения и превращается в глобальный реестр зависимостей, захардкоженный в строках.

Правильное связывание компонентов

Правильный подход заключается в инвертировании направления зависимостей. Не дочерний узел ищет родительский или соседний, а тот, кто создаѐт и владеет контекстом, связывает их явно.

Первый, самый простой и безопасный способ состоит в использовании ссылок через атрибут [Export]. Сцена-контейнер сама соединяет те части, которые должны знать друг о друге:

```
public partial class World : Node
{
    [Export] private Enemy enemy;
    [Export] private UIManager uiManager;
    public override void _Ready()
    {
        enemy.HealthChanged += uiManager.UpdateHealth;
    }
}
```

Модуль World выступает посредником. Он знает о существовании врага и интерфейса, что входит в его зону ответственности как контейнера, но не диктует, как именно враг устроен внутри. Враг лишь вызывает событие HealthChanged, а UI подписан на него. Если завтра UI переедет в другую ветку, изменится только строка в [Export] у World, а компилятор проверит, что тип по-прежнему совместим. Никаких путей, никаких магических строк. Второй способ заключается в использовании сигналов напрямую между узлами без посредника, когда это уместно. Кнопка в UI может испускать сигнал Pressed, а игровой мир подписываться на него, но подписку всё равно должен настроить тот, кто владеет контекстом, – родитель или специальный менеджер, а не сама кнопка через обход дерева. Третий, более продвинутый механизм представляет собой внедрение зависимости через метод инициализации, когда узел получает ссылку на нужный объект при создании:

```
public partial class Enemy : Node
{
    private UIManager uiManager;
    public void Initialize(UIManager ui)
    {
        uiManager = ui;
        HealthChanged += uiManager.UpdateHealth;
    }
}
```

Такой подход делает врага полностью независимым от структуры дерева, позволяя инстансировать его где угодно – зависимость придёт извне. Выбор конкретного способа зависит от масштаба, но все они объединены одним принципом: дерево служит для владения и порядка исполнения, а связи между модулями устанавливаются через явные контракты, а не через навигацию по иерархии.

Динамическая композиция и структурное мышление

Сила Scene Tree заключается в возможности динамически добавлять и удалять целые поддеревья. Это позволяет загружать уровень, показывать экран паузы или диалог как отдельную сцену, которая инстанцируется и вставляется в нужное место. Однако динамика становится управляемой только

тогда, когда поддереву представляет собой настоящий модуль с чёткими границами.

Представьте включение экрана паузы по нажатии Escape. Здоровый подход выглядит так:

```
PauseScreen pauseScreen = pauseScreenScene.Instantiate();
AddChild(pauseScreen);
pauseScreen.QueueFree();
```

Поддереву `PauseScreen` внутри может содержать десятки кнопок и обработчиков, но снаружи оно выступает единым объектом. Никто за пределами поддерева не обращается к внутренним кнопкам через `GetNode`. Если завтра вы перепроектируете интерфейс паузы, внешний код не пострадает. Граница модуля проходит по его корневому узлу с публичными методами и сигналами.

Полезный тест на модульность формулируется вопросом: можете ли вы взять это поддерево и вставить в совершенно другую сцену, не меняя ни кода поддерева, ни кода принимающей сцены? Отрицательный ответ означает, что модуль тянет за собой скрытые зависимости от конкретного окружения в дереве. Строить иерархию следует не по принципу «как красивее» или «как получилось», а начиная с трёх вопросов. Где проходит граница модуля и что он выставляет наружу? Какие зависимости ему действительно нужны и можно ли предоставить их через явные ссылки или сигналы, минуя положение в дереве? Какие узлы выступают деталями реализации, которые разрешено свободно менять, не нарушая работу остального проекта? Ответы формируют архитектуру, в которой `Scene Tree` остаётся прозрачной и предсказуемой структурой исполнения, а не запутанным клубком скрытых связей. Применяйте эти принципы с первого дня, даже если проект кажется крошечным. Когда он вырастет до десятков сцен и сотен узлов, вы скажете себе спасибо, а переучиваться потом окажется многократно больше.

2.3. Жизненный цикл Node

2.3.1. Контракт на исполнение: фазы вместо хаоса

Жизненный цикл узла в `Godot` – не просто список методов, которые «надо переопределить». Это договор между движком и вашим кодом. Движок обещает вызвать определённые методы в строго заданные моменты, а вы обещаете разместить в них код, соответствующий текущей фазе существования узла. Нарушение этого договора не бросается ошибкой компиляции, но по мере роста проекта превращает поведение игры в лотерею: код работает «иногда», баги зависят от порядка загрузки сцен, а зависимости возникают «сами собой» в самый неподходящий момент.

Первое, что нужно осознать: узел проходит два принципиально разных состояния. Когда вы пишете `var node = new MyNode()`, узел существует в памяти, но он мёртв для движка. Он не включён в дерево сцены, у него нет родителя, он не получает вызовов `_Process` или `_Ready`. Это просто объект, как в обычном C#-приложении. Всё меняется в момент входа в Scene Tree – когда вы вызываете `AddChild(node)` или движок инстанцирует сцену. Только тогда узел становится живым участником исполнения, получает окружение и подчиняется правилам обхода.

Ошибка, которую совершают почти все новички, – пытаться настроить узел в конструкторе. В C# конструктор выглядит как естественное место для инициализации, но здесь это ловушка. Давайте посмотрим на типичный провальный код:

```
public partial class Enemy : Node
{
    private HealthBar healthBar;
    public Enemy()
    {
        // Пытаемся найти HealthBar в сцене.
        healthBar = GetNode<HealthBar>("../UI/HealthBar");
    }
}
```

В конструкторе узел ещё не в дереве, метод `GetNode` не работает, `healthBar` останется `null`, и всё, что полагается на эту ссылку, сломается. При этом компилятор молчит – ведь синтаксически код корректен. Заказчик бага приходит в рантайме, и не всегда сразу. Правильный путь – разделять создание объекта и его инициализацию. Конструктор (instantiation) только для установки внутренних полей, не зависящих от окружения: начальные значения переменных, флаги, создание простых подобъектов. Вся работа, требующая знания о структуре сцены, доступе к родителю или детям, должна быть отложена до момента, когда движок сам скажет: «Теперь всё готово». Эти моменты и есть фазы жизненного цикла.

Фаз можно выделить пять, но запоминать их как мантру не стоит, важнее понимать, какая работа в какую фазу ложится.

1. **Создание (Instantiation).** Объект существует в памяти, но не в дереве. Нет родителя, нет контекста сцены. Здесь можно задать значения по умолчанию, которые не требуют внешних ссылок.
2. **Вход в дерево (Entering the tree).** Вызов `AddChild()` или инстанцирование сцены добавили узел в Scene Tree. Появился родитель, узел стал частью иерархии. В этот момент можно предположить, что родитель уже существует, но нет гарантии, что все дочерние узлы уже на месте и готовы к работе.
3. **Состояние готовности (Ready state).** Движок завершил построение композиции узла и его детей. Это та самая «точка готовности», когда можно безопасно искать зависимости, обращаться к дочерним узлам и финализировать настройку.

4. **Регулярное обновление (Processing/Update).** Узел живёт в игровом цикле. Каждый кадр вызывается `_Process(delta)`, и здесь выполняется повторяющаяся логика – движение, проверки, обновление состояния. Но не инициализация.
5. **Выход из дерева (Exiting the tree).** Узел удалён из дерева (например, через `RemoveChild` или `QueueFree`). Он перестаёт получать `_Process` и `_Ready`. Ссылки на контекст больше не действительны.

Именно эта последовательность позволяет отделить «сборку» системы от «работы» и избежать реактивного ада.

2.3.2. Ключевые *callback*'и и их роли

Godot предоставляет четыре основных метода, соответствующих этим фазам: `_EnterTree`, `_Ready`, `_Process` и `_ExitTree`. Их не нужно заучивать как заклинания, полезнее понимать, какая архитектурная задача решается в каждом. `_EnterTree()` вызывается сразу после того, как узел попал в дерево. На этом этапе вы уже можете обращаться к родителю (`GetParent()`), но ещё не факт, что все дочерние узлы этого родителя уже на месте. Типичная работа для `_EnterTree`, регистрация узла в какой-либо подсистеме, которая следит за структурой дерева. Например, враг может сообщить менеджеру врагов, что он появился:

```
public override void _EnterTree()
{
    // EnemyManager – узел в той же сцене, но мы не уверены,
    // что он уже готов, поэтому просто запоминаем факт входа.
    GD.Print("Enemy entered the tree");
}
```

Не стоит здесь искать соседей или детей, которых ещё может не быть.

`_Ready()` – страховочный рубеж. Движок обещает: когда он вызывает `_Ready` у конкретного узла, все его дети (и дети детей) уже прошли свои `_EnterTree` и, скорее всего, свои `_Ready`. Это гарантирует, что внутренняя структура сцены полностью построена. Именно здесь можно безопасно получить ссылки на дочерние узлы, связать компоненты и выполнить финальную настройку. Классический пример – получить ссылку на UI-элемент, который находится внутри того же префаба:

```
public override void _Ready()
{
    healthBar = GetNode<HealthBar>("HealthBar");
    healthBar.MaxValue = maxHealth;
}
```

Важная дисциплина: не стоит пихать в `_Ready` игровую логику вроде «начать движение» или «спавнить врагов». Потому что `_Ready` у разных узлов может вызываться в разном порядке, и нельзя рассчитывать, что остальная

сцена уже полностью готова к взаимодействию. Инициализация закончена – да, но общая координация между разными подсистемами ещё не наступила. Движок гарантирует готовность внутренней композиции, но не глобальную синхронизацию всей игры.

`_Process(double delta)` – сердце кадрового цикла. Сюда попадает всё, что должно происходить каждый кадр: передвижение, анимация, проверка столкновений. Правило: к моменту первого вызова `_Process` узел уже должен быть полностью инициализирован. Если внутри `_Process` вы проверяете, не `null` ли ссылка, которую должны были получить в `_Ready`, – значит, архитектура дала трещину. Исправлять её нужно не добавлением `if`, а переносом получения ссылки в нужную фазу.

Код из жизни новичка, который мы часто видим:

```
public override void _Process(double delta)
{
    if (healthBar == null)
        healthBar = GetNode<HealthBar>("HealthBar");
}
```

Тут не оптимизация, здесь симптом того, что инициализация поехала в фазу обновления. Последствия: в первом кадре здоровье не обновится, а если узел-владелец `HealthBar` ещё не готов, будет исключение (или молчаливый пропуск). `_ExitTree()` – точка выхода. Здесь нужно отменить всё, что было сделано при входе: отписаться от сигналов, убрать узел из списков активных объектов, очистить внешние ссылки. Если враг регистрировался в менеджере, удаление из дерева должно эту регистрацию снимать:

```
public override void _ExitTree()
{
    EnemyManager.Unregister(this);
}
```

Пропуск очистки приводит к «висящим» ссылкам: другой узел продолжает обращаться к удалённому объекту, возникает ошибка доступа к `disposed-узлу`, и найти источник проблемы становится всё сложнее.

2.3.3. Типичные ошибки и как они убивают архитектуру

Все проблемы с жизненным циклом растут из одной и той же ошибки: код пытается выполнять работу не в своей фазе. Разберём три самых частых сценария, которые превращают проект в тыкву.

Первое: инициализация в `_Process`. Выше мы уже видели пример с `if (healthBar == null)`. Разработчик не был уверен, что в `_Ready` всё будет доступно, и, вместо того чтобы разобраться с порядком загрузки, добавил «защитный» поиск в каждом кадре. Такой код закапывает проблему, не решая её. Если узел в какой-то момент перестанет находиться в ожидаемом месте, ошибка

станет плавающей. Кроме того, поиск по дереву каждый кадр – это лишняя нагрузка, которая на крупных проектах складывается в просадки FPS. Лучшее решение – гарантировать, что зависимость установлена в `_Ready`, а если она не может быть установлена (например, опциональный компонент), использовать явный флаг и однократную проверку, но не полагаться на `_Process`.

Второе: логика в `_Ready` как «первый кадр». Новичок часто думает, что раз `_Ready` вызывается после загрузки, можно начинать двигать объекты, инициировать бой или открывать двери. Но `_Ready` не синхронизирован глобально. Представьте: в сцене есть `GameManager` и `Player`. `Player._Ready` может выполняться раньше `GameManager._Ready`, и если игрок попытается обратиться к менеджеру, чтобы запросить текущий уровень, он получит `null` или неинициализированное состояние. Правильная архитектура – не разбрасывать запуск логики по разным `_Ready`, а ввести точку входа для начала игры (например, метод `StartGame` в `GameManager`), который будет вызван после того, как все узлы отчитаются о готовности, или через сигнал `Ready` от всей сцены. В Godot есть метод `_Ready`, но нет глобального «все готовы»; вам нужно создавать его самим через композицию.

Третье: отсутствие очистки при выходе. Узел может быть удалён из дерева по множеству причин: переключение сцены, телепортация, выгрузка неиспользуемых областей. Если он подписан на сигнал другого объекта, а при выходе не отписался, то при следующем вызове сигнала произойдёт попытка обратиться к уничтоженному объекту. Движок выбросит ошибку `ObjectDisposedException` или аналогичную, и стек вызовов будет вести к чужому коду. Часто новички закрывают ошибку проверкой `IsValid(obj)`, не понимая, что это лечит симптом, а не причину – забытую отписку. Правило: каждая подписка в `_EnterTree` или `_Ready` должна иметь парную отписку в `_ExitTree`. Это симметрия, которая держит систему в чистоте.

Отдельная беда – повторный вход в дерево. Godot позволяет вынуть узел через `RemoveChild` и позже снова добавить. Если вы не сбрасываете накопленные в `_Ready` ссылки или не очищаете регистрации, второе добавление приведёт либо к задвоению, либо к использованию «мёртвых» ссылок. Например:

```
public override void _Ready()
{
    enemyManager?.Register(this); // При первом входе - ок.
}
public override void _ExitTree()
{
    // Забыли отменить регистрацию.
}
```

При повторном добавлении в дерево узел снова вызовет `_Ready` и снова зарегистрируется. Менеджер врагов получит две записи, и один и тот же враг будет обрабатываться дважды. Поэтому `_ExitTree` обязан отменять всё, что сделали `_EnterTree` и `_Ready`, – это называется идемпотентностью жизненного цикла.

Всё это подводит к главной инженерной привычке: мыслить фазами, а не реактивными заплатками. Когда вы пишете код для узла, сразу задавайте

вопрос: «В какой фазе я сейчас и что мне гарантировано?» Если для работы требуется родитель – это `_EnterTree` или позже. Если нужны дети – точно `_Ready`. Если действие должно происходить каждый кадр – `_Process`. Если нужно убрать за собой – `_ExitTree`. Эта дисциплина, введённая с первого дня, убережёт от бессонных ночей, когда в проекте уже сотня сцен. И да: не пытайтесь обмануть систему, помещая всю логику в `_Ready` и вызывая `SetProcess(false)`, – это работает только в тривиальных случаях, а потом порождает неконтролируемые зависимости от порядка инициализации.

Мы уже видели, как выглядит правильный минимальный пример, в разделе 1.6. Здесь можно его расширить, чтобы продемонстрировать все четыре callback'а в действии:

```
using Godot;
public partial class LifecycleProbe : Node
{
    private Node context;
    public override void _EnterTree()
    {
        GD.Print("Probe enters tree. Parent: " + (GetParent()?.Name ?? "none"));
    }
    public override void _Ready()
    {
        context = GetParent(); // Теперь можно безопасно сохранить родителя.
        GD.Print("Probe ready. Context set.");
    }
    public override void _Process(double delta)
    {
        // Только регулярная логика. Никаких проверок на null.
        // Здесь она пустая, потому что это probe.
    }
    public override void _ExitTree()
    {
        GD.Print("Probe exits tree. Clearing context.");
        context = null; // Разрываем связь.
    }
}
```

Каждый метод выполняет строго ту работу, для которой предназначен. И это не надуманное чистоплюйство – это фундамент, на котором держится предсказуемость системы.

2.4. Сигналы и поток событий

2.4.1. От прямых вызовов к модели «факт – реакция»

Когда проект состоит из пары сцен и десятка узлов, проще всего общаться напрямую: получить ссылку на соседний узел и дёрнуть его метод. Такой код пишется мгновенно и выглядит очевидным. Но по мере роста игры прямые вызовы превращают дерево сцен в плотную сеть жёстких связей. Один узел

знает внутреннее устройство другого, изменение структуры ломает логику в неожиданных местах, а добавление новой реакции требует правки исходного модуля. Сигналы в Godot – это механизм, позволяющий перевернуть направление зависимости: вместо «я говорю тебе, что делать» узел сообщает «со мной что-то произошло», а все заинтересованные реагируют сами.

Сигнал – именованное уведомление, которое один объект может испустить (emit), а другие объекты – слушать, предварительно подписавшись. Отправитель не знает, кто именно его слушает, и не управляет последствиями; он лишь фиксирует факт в своей зоне ответственности. Архитектурно такая модель даёт три преимущества: во-первых, разрывается прямая ссылка, и отправитель перестаёт зависеть от API получателя; во-вторых, один и тот же факт могут независимо обрабатывать несколько подсистем; в-третьих, добавление новой реакции не требует изменений в источнике события.

Разница с прямым вызовом проявляется не в синтаксисе, а в направлении контроля. Когда UI-панель вызывает `player.TakeDamage(10)`, интерфейс командует игровой логикой. Когда же `player` сам испускает сигнал `HealthChanged` с новым значением, а UI-панель на него подписана, владельцем решения остаётся игрок, а интерфейс лишь отображает последствия. Первый подход – командный, второй – событийный. Команды хороши внутри одного модуля, где родитель управляет детьми, события – на границах подсистем, где важна независимость.

Классическая модель издатель–подписчик в Godot выглядит так: узел-издатель (publisher) владеет состоянием или процессом и публикует событие, когда это состояние меняется; узлы-подписчики (subscribers) самостоятельно решают, что с этим фактом делать. Издатель не должен знать о существовании подписчиков, а подписчики не должны лезть внутрь издателя за дополнительными данными. Пример из реальной практики: узел `HealthComponent` отвечает за здоровье персонажа. Когда здоровье меняется, он испускает сигнал `HealthChanged(int newHealth)`. На этот сигнал могут быть подписаны: `HealthBar` (обновить полосу), `AudioPlayer` (проиграть звук ранения), `AchievementManager` (проверить условие «выжить с низким здоровьем»). Сам `HealthComponent` не знает ни об одном из них – он просто сообщает факт.

Связь сигналов с жизненным циклом узла критична. Подписка не должна быть вечной. Если узел-подписчик покидает дерево (например, UI-панель скрывается), он обязан отписаться, иначе при следующем изменении здоровья возникнет ошибка: движок попытается вызвать метод на уже неактивном узле. Поэтому подписки обычно создаются в `_EnterTree` или `_Ready`, а разрываются в `_ExitTree`. Это не просто ритуал – это гарантия, что поток событий соответствует реальному составу сцены.

2.4.2. Как проектировать сигнал: контракт, имя, данные и локальность

Сигнал – это контракт, не просто «удобный способ дёрнуть метод». Если относиться к нему как к чему-то временному, проект быстро зарастает сотнями

невнятных уведомлений, и распутать их становится невозможно. Контракт состоит из трёх частей: имени, смысла и передаваемых данных.

Имя должно отражать факт, который произошёл, а не действие, которое нужно совершить. Хорошие имена – `HealthChanged`, `DoorOpened`, `InventoryUpdated`. Плохие – `UpdateUI`, `DoStuff`, `Notify`. Когда имя звучит как команда, подписчики начинают воспринимать сигнал как приказ, и событийная модель скатывается в скрытое управление. Однажды вы увидите в коде сигнал `RefreshEverything`, который эмитится из пяти разных мест по разным поводам – и каждый раз с непредсказуемыми последствиями. Это красный флаг. Смысл сигнала должен быть узким. `SomethingHappened` с произвольным словарём данных – не контракт, а дыра в архитектуре. Подписчик, получивший такое, вынужден угадывать, что именно случилось, и залезать внутрь издателя за деталями. Вместо этого сигнал должен чётко отвечать на вопрос «какой факт зафиксирован?». Например, сигнал `DoorOpened` – факт, `DoorLocked` – другой факт. Не нужно объединять их в `DoorStateChanged` с параметром, который подписчики будут интерпретировать. Данные в сигнале должны быть минимальны и достаточны. Если сигнал сообщает об изменении здоровья, передайте новое значение (и, возможно, старое, если оно нужно для анимации). Не передавайте весь объект `Player`, чтобы подписчик мог дёрнуть `player.GetWeapon()`. Как только вы начинаете тащить через сигнал ссылки на крупные объекты, подписчик получает доступ к внутренностям издателя – и вы теряете всё, ради чего затевали событийную модель. Правило большого пальца: сигнал несёт данные о самом факте, а не об объекте, который этот факт породил.

Отдельно стоит разделять события-состояния и события-команды. События-состояния (`HealthChanged`, `ScoreUpdated`) описывают свершившийся факт. Они безопасны: подписчики реагируют как хотят, не влияя на логику издателя. События-команды (`RequestDoorOpen`) – это завуалированный вызов метода. Они могут быть полезны, когда нужно оповестить систему о намерении, но требуют осторожности: если на `RequestDoorOpen` подписаны три разных модуля, и каждый пытается открыть дверь по-своему, результат будет хаотичным. Для команд часто честнее использовать прямой вызов конкретного владельца логики – например, через интерфейс `IOpenable`.

Локальность сигнала – ещё один критерий здоровой архитектуры. Сигналы работают лучше всего, когда область их действия ограничена модулем или взаимодействием нескольких соседних модулей. Если сигнал начинает кочевать через всё дерево сцены, он превращается в глобальную шину, а отладка – в поиск всех подписчиков по проекту. В небольшом прототипе это терпимо, в большой игре – смертельно. Держите сигналы близко к их источнику: здоровье персонажа – сигнал внутри его поддерева, событие открытия двери – внутри сцены комнаты. Если реакции нужны глобально (например, достижения), лучше ввести выделенный слой – `EventBus` или сервис достижений, – но это тема для отдельной главы. Порядок обработки сигналов – скрытая цена десвязывания. Когда у сигнала несколько подписчиков, порядок их вызова не гарантирован. Если ваша логика требует, чтобы сначала обновился UI, а потом сработал звук, сигнал – плохой инструмент. Зависимость от по-

рядка – признак того, что вы зашили в обработчики неявную последовательность. Либо перепроектируйте так, чтобы порядок был неважен, либо введите явный этап обработки: например, все реакции на `HealthChanged` могут только читать новое состояние, но не менять его, а любые изменения инициируются отдельно.

2.4.3. Типичные ошибки и дисциплина подписок

Самая распространённая ошибка – использовать сигналы как замену архитектуре. Вместо того чтобы продумать, какие модули есть и как они общаются, разработчик начинает «просто кидать сигналы». Результат: десятки неявных связей, причину бага невозможно отследить, потому что событие проходит через пять узлов-посредников. Сигнал не устраняет необходимость в модульности, он лишь меняет форму связи. Прежде чем вводить сигнал, спросите себя: какие две подсистемы я связываю? Какая из них владеет фактом? Ответы дадут границу, а сигнал ляжет ровно на эту границу.

Другая типичная ловушка – подписка с поиском по дереву. Новичок хочет, чтобы враг при смерти оповещал UI. Он пишет в скрипте врага:

```
var ui = GetNode<UIManager>("/root/World/UI");
ui.EmitSignal("EnemyDied");
```

Этот код содержит две ошибки. Во-первых, враг теперь знает, где в дереве находится UI, и любое перемещение интерфейса сломает врага. Во-вторых, враг сам решает, кому и как сообщать о своей смерти, – он стал командиром. Правильно инвертировать направление: враг объявляет сигнал `Died`, а UI сам подписывается на него через родительский узел, который знает обоих:

```
// Enemy.cs .
[Signal] public delegate void DiedEventHandler();
public void Die()
{
    EmitSignal(SignalName.Died);
    QueueFree();
}
// World.cs – родитель, связывающий врагов и UI.
public override void _Ready()
{
    foreach (var enemy in GetNode("Enemies").GetChildren())
    {
        enemy.Connect(Enemy.SignalName.Died, Callable.From(OnEnemyDied));
    }
}
private void OnEnemyDied()
{
    uiManager.UpdateKillCount();
}
```

Сигнал остался внутри модуля, зависимость от структуры исчезла.

Следующая по популярности ошибка – забытые подписки. Узел подписался на сигнал в `_Ready`, но не отписался в `_ExitTree`. Сцена перезагружается, узел уничтожается, а где-то в памяти остаётся висячая ссылка. При следующем вызове сигнала движок выдаёт ошибку вроде «Attempted to call a method on a previously freed instance». Новички часто лечат это проверкой `IsValid`, не осознавая, что борются с симптомом, а не с причиной – забытой отпиской. Дисциплина проста: каждая подписка, созданная в `_EnterTree` или `_Ready`, должна иметь парную отписку в `_ExitTree`. Для этого удобно использовать лямбды с сохранением ссылки или метод `Connect` с флагами, но проще всего – ручное управление:

```
public override void _EnterTree()
{
    healthComponent.Connect(HealthComponent.SignalName.HealthChanged, Callable.
From<int>(OnHealthChanged));
}
public override void _ExitTree()
{
    healthComponent.Disconnect(HealthComponent.SignalName.HealthChanged, Callable.
From<int>(OnHealthChanged));
}
```

Если узел может многократно входить и выходить из дерева, такая симметрия гарантирует, что подписки не задвоятся и не останутся висеть.

Цепочки событий – ещё один источник хаоса. Один сигнал вызывает реакцию, которая эмитит следующий сигнал, тот – третий, и через пять шагов понять, с чего всё началось, уже невозможно. Такие цепочки возникают, когда сигнал используют как неявный способ оркестрации. Например, враг умер → сигнал `EnemyDied` → achievement manager проверяет условие и эмитит `AchievementUnlocked` → UI анимации показывает попап → по окончании анимации эмитится `AnimationFinished` → и так далее. Часть этой цепочки можно заменить прямым вызовом там, где это внутреннее дело модуля, а сигналы оставить только для действительно межмодульных уведомлений. Правило: сигнал должен сообщать о факте, а не запускать сценарий. Сценарии должны быть собраны в одном месте, а не размазаны по цепочке подписчиков.

Напоследок – о разнице между Godot signals и C# events. В Godot сигналы встроены в движок: они видны в редакторе, могут сохраняться в сценах, поддерживают группы и автоматическую связь с узлами. Это нативный инструмент для коммуникации внутри мира движка. C# события (event Action) – чисто языковая конструкция, удобная для доменной логики, которая не привязана к узлам. Если вы пишете библиотеку для расчёта повреждений, не зависящую от Godot, используйте event. Если связываете узел здоровья с узлом интерфейса – берите сигналы Godot. Смешивать оба подхода без чёткого правила опасно: вы получите два разных механизма уведомлений с разными жизненными циклами, и отладка усложнится. Договоритесь в команде, где какой инструмент применяется: всё, что внутри сцены и связано с узлами, – сигналы; всё, что в чистых C#-классах, – события.

2.5. Память и владение объектами

Понимание сигналов и жизненного цикла не спасёт проект, если вы не знаете, кто отвечает за существование объектов и когда они перестают существовать. В обычном C#-приложении всё просто: объект жив, пока на него есть ссылки, а потом сборщик мусора тихо убирает мусор. В Godot с C# эта модель ломается, потому что вы имеете дело с двумя параллельными реальностями. Первая – мир .NET с управляемой памятью и сборщиком мусора. Вторая – нативный мир движка на C++, где объекты живут по собственным правилам. Ваш код на C# работает с мостом между этими мирами, и если архитектура не выравнивает два жизненных контура, начинают происходить странные вещи: обработчики сигналов срабатывают на уже удалённых узлах, интерфейс продолжает обновляться после выгрузки уровня, а ошибки пахнут мистикой – «работает через раз».

2.5.1. Два времени жизни, которые нужно подружить

Каждый узел, с которым вы работаете из C#, – это не просто объект. Это управляемая обёртка над нативным экземпляром, которым распоряжается движок. Вы держите в руках C#-ссылку и наивно полагаете, что она гарантирует существование узла. Но гарантирует она только существование обёртки в управляемой куче. Сам узел как участник дерева сцен может быть уже удалён. И наоборот: если вы потеряли C#-ссылку и сборщик мусора подобрал обёртку, это не значит, что нативный узел исчез – движок может продолжать его обслуживать, если тот всё ещё в дереве. Типичная история новичка: персонаж умер, вы удалили его узел методом `QueueFree()`, но где-то в UI осталась ссылка на `healthBar`, которая была привязана к персонажу. Через секунду UI пытается обновить полосу здоровья, вызывает `healthBar.Update()` и получает исключение – объект уже не существует в мире движка. C#-ссылка при этом может быть не `null`, потому что обёртка ещё не собрана сборщиком мусора. Вы проверяете `if (healthBar != null)`, проверка проходит, и всё равно происходит падение. Это и есть рассинхронизация двух времён жизни. Решение – явно определить, кто владеет объектом и когда заканчивается его логическая жизнь. Если персонаж умер и его узел удалён, все ссылки на него должны быть обнулены немедленно, а не ждать сборщика мусора. Этим должны заниматься те, кто на него ссылался, в фазе `_ExitTree`.

2.5.2. Дерево сцен как основа владения

В Godot владение узлами выражается через дерево сцен. Родитель владеет детьми: пока узел находится в поддереве, его время жизни определяется временем жизни этого поддерева. Когда вы удаляете родителя из дерева, все его потомки тоже перестают быть частью исполнения. Именно это даёт возможность управлять целыми модулями одним движением: убрали корень под-

дерева – вся сцена уровня выгружается. Но здесь кроется ловушка: удаление из дерева и уничтожение объекта – два разных события. Метод `RemoveChild()` просто вынимает узел из иерархии, но объект остаётся в памяти, и если вы сохранили на него ссылку, можете снова куда-то добавить. Метод `QueueFree()` и `Free()` добавляют к удалению из дерева ещё и освобождение нативного объекта. Однако управляемая обёртка может висеть в памяти до следующего пробега сборщика мусора. Поэтому после вызова `QueueFree()` хорошей практикой будет сразу обнулить свою ссылку, чтобы не использовать её случайно.

Рассмотрим простой пример управления временем жизни временного эффекта. Допустим, у нас есть звук выстрела, который должен проиграться и исчезнуть:

```
public partial class Weapon : Node
{
    private AudioStreamPlayer shotSound;
    public override void _Ready()
    {
        shotSound = GetNode<AudioStreamPlayer>("ShotSound");
    }
    public void Shoot()
    {
        shotSound.Play();
        // После окончания звука удаляем узел из дерева.
        shotSound.Finished += () => shotSound.QueueFree();
    }
}
```

Здесь временный узел `ShotSound` существует в рамках поддеревя оружия. Когда звук закончился, он сам себя удаляет. Никакие внешние системы не держат на него ссылок, и проблемы висячих ссылок не возникает. Но если бы UI где-то сохранил ссылку на этот звуковой узел и пытался с ним работать после удаления, проблем было бы не избежать.

2.5.3. Висячие ссылки, забытые подписки и утечки – как это лечить

Ссылка становится висячей не тогда, когда она `null`, а когда объект, на который она указывает, уже не должен участвовать в системе. Например, вы сохранили в переменной узел врага, враг умер, но ссылка осталась и продолжает использоваться в расчётах. В .NET это не ошибка – память не повреждена, – но логика игры сломана. Такие ошибки трудно отлаживать, потому что они проявляются далеко от причины. Основной источник висячих ссылок – забытые подписки на сигналы. Представьте: враг при входе в дерево подписался на сигнал `PlayerMoved` от игрока, но при выходе из дерева не отписался. Через минуту игрок перемещается, движок пытается вызвать метод на уже отключённом узле врага – ошибка. Новичок в панике добавляет проверку `if`

(IsValid(this)) внутри обработчика, но это не решение, правильно – управлять подписками симметрично.

Вот как выглядит здоровая дисциплина подписок внутри одного модуля:

```
public partial class Enemy : Node
{
    [Export] private Player player;
    public override void _EnterTree()
    {
        player.Moved += OnPlayerMoved;
    }
    public override void _ExitTree()
    {
        player.Moved -= OnPlayerMoved;
    }
    private void OnPlayerMoved(Vector2 newPosition)
    {
        // Реакция на движение игрока.
    }
}
```

Если враг выходит из дерева, подписка снимается, и никаких «призрачных» вызовов не происходит. Тот же принцип касается любых ссылок на внешние сервисы или глобальные объекты: в `_ExitTree` нужно явно обнулить всё, что больше не должно использоваться. Другая распространённая ошибка – использование статических полей или глобальных коллекций для хранения ссылок на узлы сцены. Скажем, вы сделали статический список `Enemy.AllEnemies`, чтобы быстро находить врагов. При загрузке нового уровня старый список не очистился, и в нём остались мёртвые узлы с предыдущей сцены. Игра пытается обновить их, возникают ошибки. Лечится это либо отказом от статики в пользу локальных реестров внутри сцены (которые очищаются вместе со сценой), либо явной регистрацией и deregистацией в `_EnterTree/_ExitTree`. Никакой «магии самоочистения» в .NET нет.

Утечки памяти в таком проекте редко бывают классическими «забыли вызвать free». Обычно это «забыли отписаться» или «забыли обнулить ссылку». Сборщик мусора не может освободить объект, пока на него кто-то ссылается. Если у вас есть подписка на долгоживущий сигнал, весь граф подписчика висит в памяти до конца игры. Например, вы подписали метод UI-окна на сигнал `GameManager.GameStateChanged`, но при закрытии окна не отписались. Окно закрыто, но его объект продолжает висеть и даже реагировать на события – вы просто этого не видите. Рано или поздно это приводит к пропадам производительности и лавине неожиданных эффектов.

2.5.4. Стратегия чистого владения: кто создаёт, тот и уничтожает

Чтобы не тонуть в проверках валидности, с самого начала заведите правило: у каждого значимого объекта или подсистемы должен быть единственный

владелец, который отвечает за его появление и исчезновение. Для узлов это, как правило, родительское поддерево. Для чистых C#-объектов (не унаследованных от Node) – тот модуль, который их создал. Не пытайтесь растягивать время жизни узлов дольше, чем живёт их сцена. Если вам нужно сохранить состояние между сценами (здоровье игрока, инвентарь, настройки), выносите эти данные в отдельные объекты, не привязанные к дереву, например в классы, помеченные как [GlobalClass] и хранящиеся в ресурсах или синглтонах. Узел же должен быть лёгким и умирать вместе со своей сценой. Это называется разделением на «эпизодические узлы» и «долгоживущие данные». Пример:

```
// Данные живут отдельно, могут пережить сцену.
public class PlayerData
{
    public int Health;
    public Inventory Inventory;
}
// Узел живёт только в сцене, ссылается на данные.
public partial class Player : Node
{
    private PlayerData data;
    public void Init(PlayerData data)
    {
        this.data = data;
    }
}
```

При загрузке новой сцены вы просто передаёте тот же PlayerData новому экземпляру Player. Никаких попыток «сохранить узел игрока и воскресить его» – это путь к хаосу. Наконец, полезно помнить простую мантру отладки: если у вас есть баг, связанный с «внезапными» ошибками доступа, первым делом проверьте не содержимое метода, а жизненный цикл. Где создаётся объект? Кто его удаляет? Не осталось ли подписок? Не удерживается ли ссылка в статике? В восьми случаях из десяти проблема решается восстановлением симметрии входа и выхода, а не расстановкой дополнительных if.

Сводя всё вместе: управление памятью в Godot с C# – не про ручное выделение и освобождение, а про строгую дисциплину владения и согласование двух параллельных жизненных контуров. Если вы явно определяете, кто кем владеет, когда начинается и заканчивается жизнь объекта, и следите за симметрией подписок, то ни висячих ссылок, ни утечек, ни «магических» багов у вас не будет.

2.6. Основы отладки

Отладка в инженерном смысле – не хаотичное тыканье в код в надежде, что ошибка исчезнет, а систематическое расследование: наблюдение → гипотеза → проверка → исправление причины. В Godot с его деревьями сцен, жиз-

ненными циклами и сигналами соблазн «быстро поправить и посмотреть, заработало ли» особенно велик, но именно он превращает проект в непредсказуемый набор исключений. Хорошая новость: как только вы понимаете модель исполнения движка, отладка перестаёт быть магией и становится почти механическим применением нескольких принципов. Цель этого раздела – показать, что «умение отлаживать» в Godot на 90 % состоит из архитектурной грамотности, а не из знания горячих клавиш. Мы пройдем от цикла гипотез до конкретных инструментов и научимся видеть за каждым симптомом нарушение контракта между движком и кодом.

2.6.1. Цикл гипотез: от наблюдения к первопричине

Любая отладка начинается с чёткого наблюдения. Не «что-то сломалось», а «на пятом уровне враг не получает урон, когда игрок нажимает атаку». Уже на этом этапе важно отделить симптом от контекста. Симптом – это факт: урон не начисляется. Контекст – это ответы на вопросы: в какой сцене это происходит, на каком узле, в какой фазе игры, при каких условиях. Без контекста вы начинаете гадать по коду и чаще всего правите не ту строку.

Следующий шаг – сформулировать гипотезу о причине. Хорошая гипотеза звучит как структурное утверждение: «Обработчик сигнала атаки не вызывается, потому что подписка создаётся в `_Ready()` врага, но игрок появляется позже, и сигнал испускается до того, как враг успел подписаться». Плохая гипотеза – «наверное, что-то с условиями». Разница в том, что хорошую можно проверить одним точным действием, а плохая заставляет менять код наугад.

Проверка гипотезы должна быть максимально дешёвой и однозначной. Вместо того чтобы переписывать всю систему подписок, вы добавляете пару `GD.Print` прямо в месте предполагаемой подписки и в обработчике, запускаете сцену и смотрите, в каком порядке идут сообщения. Если в консоли нет строки «Враг подписался на сигнал» до момента атаки – гипотеза подтверждена. Если есть, но обработчик всё равно не вызывается – ищите дальше. Такой подход снижает неопределённость за минуты, а не за часы.

Ключевое правило: проверка должна отличать одну гипотезу от другой. Если ваша проверка не исключает альтернативное объяснение, вы остаётесь в том же болоте предположений. Например, вы подозреваете, что урон не начисляется из-за того, что враг уже удалён из дерева к моменту обработки. Добавьте в обработчик строчку `GD.Print($"Враг в дереве: {IsInsideTree()}")`. Если при срабатывании вы видите `false`, причина именно в этом. Если `true` – проблема в чём-то другом, и вы не будете зря копать в жизненном цикле.

2.6.2. Инструменты наблюдаемости: как заставить систему заговорить

Самый доступный инструмент в Godot – `GD.Print`. Но его ценность резко возрастает, если логировать не «что-то случилось», а контекстные факты:

имя узла, фазу, факт подписки или отписки, состояние дерева. Представьте, что вы пытаетесь понять, почему UI перестаёт обновляться после перезагрузки уровня. Один грамотно расставленный GD.Print в `_ExitTree()` интерфейса с сообщением «UI выходит из дерева» и второй в `_Ready()` с «UI готов, подписываюсь на здоровье игрока» сразу покажут, пересоздаётся ли интерфейс, когда нужно, и не остаётся ли висеть старая подписка. Точки останова (breakpoints) хороши для локального анализа: вы останавливаете выполнение в конкретном методе, смотрите значения переменных, идёте по шагам. Их слабость – они меняют временные характеристики игры и неудобны для проблем, возникающих из-за последовательности событий на протяжении нескольких кадров. Поэтому комбинируйте: логи дают общую картину потока событий, а точки останова – «микроскоп» в подозрительном месте.

Отдельно стоит упомянуть **Remote Scene Tree**, доступный в редакторе во время работы игры. Это малоизвестная, но мощная возможность: вы видите реальное дерево узлов, которые существуют в данный момент, их свойства и сигналы. Если объект, который по логике должен быть, отсутствует в дереве – вы мгновенно узнаете об этом, не вставляя ни одного `GetNode`. Если он есть, но расположен не там, где ожидалось, – вы видите точный путь. Эта утилита незаменима при диагностике проблем с динамической композицией сцен, когда узлы создаются и удаляются на лету. Используйте её вместе с логами: логи расскажут, когда и что произошло, а Remote-просмотр – в каком состоянии дерево оказалось. Стек-трейсы исключений читайте как карту маршрута. Если вы видите в стеке вызов `Enemy.OnPlayerMoved`, затем `Player.EmitSignal`, а потом снова какой-то обработчик, это прямой намёк на цепочку событий, возможно, замкнутую. Не ограничивайтесь последней строчкой – проследите путь, и часто обнаружите, что ошибка возникла из-за того, что какой-то узел уже вышел из дерева, но остался в цепочке подписчиков.

2.6.3. Карта ошибок: связываем симптомы с архитектурными нарушениями

Большинство багов в Godot-проектах – это не ошибки в алгоритмах, а нарушение контрактов, которые мы разбирали в предыдущих разделах. Зная устройство Scene Tree, жизненный цикл, сигналы и владение, вы можете почти сразу локализовать проблему.

Проблемы с порядком инициализации проявляются как null-ссылки или «метод вызван, но объект ещё не настроен». Классика: в `_Ready()` врага вы пытаетесь обратиться к игроку через `GetNode(«/root/World/Player»)`, но сцена мира ещё не готова или игрок добавлен позже. Симптом: при первом запуске всё ломается, при повторном – иногда работает, потому что порядок загрузки кешируется. Решение: не искать глобальные узлы по абсолютному пути, а использовать внедрение зависимостей либо через `[Export]`, либо через сигналы, а инициализацию, требующую внешнего контекста, откладывать

до момента, когда этот контекст гарантированно появится (например, через сигнал Ready от родительской сцены или явный вызов из менеджера сцены).

Ошибки с сигналами чаще всего сводятся к забытым подпискам или подпискам не в той фазе. Симптом: обработчик срабатывает дважды, либо срабатывает на уже удалённом объекте, либо не срабатывает вовсе. Как искать: добавьте GD.Print при каждой подписке (в _EnterTree или _Ready) и при отписке (в _ExitTree). Запустите игру, дождитесь бага и посмотрите в консоль. Если на момент события нет сообщения о подписке – она не была создана (возможно, узел ещё не в дереве). Если сообщение о подписке есть, а отписки нет, но объект уже удалён – утечка. Если подписка создаётся дважды (два сообщения подряд без промежуточной отписки) – обработчик будет вызван дважды, и это может приводить к задвоению эффектов.

Проблемы владения и времени жизни выглядят как внезапные «Object-DisposedException» при обращении к узлу или как работа логики после того, как сцена должна была выгрузиться. Причина всегда одна: кто-то держит ссылку на объект, который уже не должен участвовать в системе, и продолжает его использовать. Лучший способ найти такого «держателя» – посмотреть, где сохраняются ссылки на проблемный узел. Часто это статические списки, кеш в синглтонах или подписки в глобальных менеджерах. Исправляется явной регистрацией/дерегистрацией: если объект при создании добавляет себя в глобальный реестр, то при выходе из дерева он обязан себя удалить.

Отдельно отметим **ошибки порядка в дереве**, когда код предполагает, что узел A обрабатывается раньше узла B, потому что он выше в иерархии. Симптом: поведение меняется при перестановке узлов в редакторе. Диагностика: проверьте, не используете ли вы GetParent() или абсолютные пути для связи подсистем, полагаясь на конкретное место в дереве. Дерево должно определять владение и исполнение, а не причинно-следственные цепочки.

2.6.4. Ловушки мышления и профилактическая архитектура

Даже зная все инструменты, легко попасть в ловушки, которые затягивают отладку на часы. Первая ловушка – **лечение симптома**. Вы видите null и добавляете if (node != null). Ошибка исчезает, но настоящая причина – неправильный жизненный цикл – остаётся и породит ещё пять багов в будущем. Всегда спрашивайте: «Почему здесь оказался null? Должен ли этот объект существовать в данной фазе?» Если ответ «нет», ищите, почему он существует или почему код выполняется в этой фазе.

Вторая ловушка – **подтверждающее смещение**. Вы предполагаете, что проблема в методе TakeDamage, и начинаете менять только его, игнорируя то, что ошибка может быть выше по потоку – в подписке на сигнал, в порядке загрузки сцен. Расширяйте обзор: проверьте, кто вызывает проблемный код и в каком контексте. Remote Scene Tree и логи на границах модулей здесь бесценны.

Третья ловушка – **слишком широкий поиск**. Новичок ставит `GD.Print` в каждый метод, получает лавину сообщений и теряется. Вместо этого задайте конкретный вопрос: «На каком именно шаге состояние становится неверным?» – и поставьте логи только в двух-трёх точках, отсекающих возможные развилки. Например, чтобы понять, почему не работает анимация смерти, проверьте: вызывается ли вообще метод `Die()`, устанавливается ли флаг `isDead`, входит ли `AnimationPlayer` в состояние `death`. Трёх `Print` часто достаточно.

Самая эффективная стратегия – **профилактическая отладка**, то есть проектирование кода так, чтобы ошибки было легко заметить. Когда вы соблюдаете симметрию подписок (подписались в `_EnterTree` – отписались в `_ExitTree`), когда не храните ссылки на узлы дольше их жизни, когда используете сигналы вместо прямого обхода дерева – вы сужаете пространство возможных ошибок. В такой системе баг почти всегда локализован: он сидит в одном модуле, а не каскадом расходится по всему проекту. И тогда отладка превращается в быстрое сопоставление симптома с одним из известных контрактов, а не в бесконечное гадание.

Рассмотрим практический пример, демонстрирующий эти принципы. Допустим, у нас есть враг, который должен при смерти отправлять сигнал `Died`, а UI – обновлять счётчик. Мы реализовали это так:

```
public partial class Enemy : Node
{
    [Signal] public delegate void DiedEventHandler();
    public void Die()
    {
        EmitSignal(SignalName.Died);
        QueueFree();
    }
}

public partial class KillCounter : Node
{
    private int kills = 0;
    public override void _Ready()
    {
        // Подписываемся на всех врагов, которые уже есть в сцене.
        var enemies = GetTree().GetNodesInGroup("enemies");
        foreach (var enemy in enemies)
            enemy.Connect(Enemy.SignalName.Died, Callable.From(OnEnemyDied));
    }
    private void OnEnemyDied()
    {
        kills++;
        GD.Print($"Убито: {kills}");
    }
}
```

Первая проблема: подписка происходит в `_Ready()` только на тех врагов, которые уже есть. Враг, добавленный позже, не будет учтён. Вторая: при уда-

лении врага подписка не снимается – если вы зачем-то оставите узел в памяти, но вне дерева, возникнет утечка. Мы можем отладить так: добавим в `Enemy.Die()` строчку `GD.Print($"Враг {Name} умирает. В дереве: {IsInsideTree()}")`, а в `KillCounter._Ready()` – `GD.Print($"Подписываемся на {enemies.Count} врагов")`. Запустив сцену, мы сразу увидим, сколько врагов поймано и что при смерти нового врага сообщение появляется, но счётчик не растёт – значит, подписки нет. Исправляем: переносим подписку в `_EnterTree()` каждого врага, чтобы он сам регистрировался в группе, либо используем сигнал `SceneTree.NodeAdded` для динамического отслеживания. Но ключевой шаг отладки – не гадание, а добавление двух `Print`, которые показали рассинхрон.

Закончим важным наблюдением: архитектурные принципы, описанные в предыдущих разделах, не просто «правильные» – они напрямую сокращают время отладки. Когда вы мыслите в терминах дерева, фаз, сигналов и владения, каждый баг превращается не в катастрофу, а в задачу с известным алгоритмом решения. И это, пожалуй, главный инженерный навык в разработке на Godot.

3

С# как инструмент системного дизайна

3.1. Классы и ответственность

Когда вы только начинаете писать на С# для Godot, класс воспринимается как синтаксическая конструкция: место, где лежат переменные и методы. Это правда, но лишь техническая. С точки зрения архитектуры класс – это в первую очередь **граница ответственности**. Он фиксирует, какая часть игры за что отвечает, какие данные себе забирает и какие правила защищает. Если границы не проведены или проведены как попало, код довольно быстро превращается в кашу, где изменение механики ломает интерфейс, а добавление нового врага требует переписывания половины проекта. Цель этого раздела – показать, как с помощью обычных классов навести порядок, не усложняя жизнь раньше времени.

3.1.1. Класс как архитектурный рубеж

Представьте, что вы собираете конструктор. У вас есть детали: кубики, шестерёнки, колёса. Если свалить всё в одну коробку без надписей, собрать даже простую машинку будет мучительно: вы тратите время на поиск, путаете похожие элементы, а при любой доработке приходится перерывать всё заново. Класс – это такая коробка с этикеткой. «Здоровье», «Инвентарь», «Ввод персонажа» – каждая коробка содержит только то, что относится к её названию, и отвечает за сохранность своего содержимого.

В коде ответственность класса выражается в трёх вещах. Во-первых, он определяет **роль**: например, Health хранит количество здоровья, PlayerInput

превращает нажатия клавиш в намерения, ScoreCounter считает очки. Во-вторых, класс задаёт **контракт**: снаружи можно вызвать метод ApplyDamage, и объект гарантирует, что здоровье не уйдёт в минус, а при смерти отправит сигнал. В-третьих, класс защищает **инварианты** – правила, которые не должны нарушаться ни при каких условиях. Например, количество патронов в обойме никогда не превышает максимальную вместимость, а здоровье не становится отрицательным. Если ответственность не определена, правила игры распределяются по всем углам проекта. В одном скрипте здоровье уменьшается, в другом проверяется на ноль, в третьем отрисовывается полоска, и все они напрямую меняют одну публичную переменную `int health`. Пока игра состоит из одного персонажа и одной кнопки, это работает. Но как только появляется второй тип врагов с бронёй, модификаторы урона или способность регенерировать здоровье вне боя, вы начинаете раз за разом обходить все места, где `health` меняется, дописывая условия. Это и есть главный признак отсутствия границ: изменение правил требует правок по всему проекту, а не в одном месте. Одна из самых частых ошибок новичков – объединять в одном классе всё, что связано с персонажем, потому что «это всё про игрока». Получается Player-монстр, который знает и про анимации, и про ввод, и про инвентарь, и про сохранение, и про UI. Такой класс меняется по десяти разным причинам одновременно, и каждая правка рискует сломать остальные девять функций. В инженерии это называют «единственной причиной изменения». Хороший класс меняется только тогда, когда меняются требования к его конкретной роли. Если вы решили иначе рисовать полоску здоровья – меняется только класс HealthBar, но не сам Health. Если добавили геймпад – меняется InputHandler, а правила начисления урона остаются нетронутыми.

Другая крайность – делать классы просто мешками для данных. Например, объявить `public int Health` и везде, где надо, писать `player.Health -= damage`. Такой подход лишает вас возможности контролировать правила: никто не гарантирует, что здоровье не уйдёт в минус, что при нуле сработает смерть, что интерфейс узнает об изменении. В Godot это особенно опасно, потому что ссылки на объекты разбросаны по сценам и сигналам, и отловить, кто именно сейчас изменил переменную, почти невозможно. Поэтому ответственность всегда должна быть упакована: данные и методы, которые с ними работают, живут в одном классе, а внешний мир только подаёт запросы. Вернёмся к примеру: вместо `player.Health -= damage` правильно вызывать `player.TakeDamage(damage)`, а уже внутри TakeDamage проверять броню, ограничивать значение, вызывать событие HealthChanged при необходимости.

3.1.2. Связность и цена размытых границ

Качество архитектуры часто описывают двумя понятиями: **cohesion** (сцепление внутри класса) и **coupling** (связность между классами). Высокий cohesion означает, что все методы и поля класса работают на одну задачу. Низкий coupling – что класс мало знает о других и не зависит от их внутренностей.

Идеальная, но недостижимая цель: классы-«коробки» с чёткой надписью, соединённые тонкими проводами контрактов, а не кашей из перекрёстных ссылок. В играх на Godot coupling часто разрастается незаметно. Типичный сценарий: вы хотите, чтобы при получении урона враг мигал красным. Для этого в скрипте врага вы находите узел Sprite2D через GetNode, вызываете у него метод `Modulate = Colors.Red`. Позже вы решаете заменить спрайт на анимированный или вынести эффект в отдельный узел `ShakeEffect` – и код врага ломается, потому что он слишком много знал о внутреннем устройстве собственного представления. Правильно было бы, чтобы враг просто вызывал у себя метод `PlayDamageEffect()`, а тот уже внутри разбирался с деталями. Ещё одна форма coupling – зависимость от порядка вызовов. Когда класс А меняет здоровье, а класс В ждёт, что А сделает это строго до того, как В начнёт свою проверку, и всё держится на том, что А висит в дереве выше В, – это скрытая связность. Она работает до первого рефакторинга сцены, а потом приводит к «мистическим» багам, которые мы обсуждали в разделе 2.2.

Отсутствие явных границ порождает и так называемые «God-классы» – узлы или модули, которые знают всё обо всём. У новичков это часто корневая сцена уровня или сам `Player`. Пример такого класса, который мы уже видели и который стоит разобрать ещё раз:

```
using Godot;
public partial class PlayerManager : CharacterBody2D
{
    public int Health = 100;
    public int Score = 0;
    public Label HealthLabel;
    public override void _Process(double delta)
    {
        if (Input.IsActionJustPressed("attack")) Score += 10;
        if (Input.IsActionJustPressed("heal")) Health += 5;
        Velocity = new Vector2(Input.GetAxis("left", "right") * 200, Velocity.Y);
        MoveAndSlide();
        HealthLabel.Text = $"HP: {Health}";
        if (Health <= 0) GetTree().ChangeSceneToFile("res://ui/game_over.tscn");
        FileAccess.Open("user://save.txt", FileAccess.ModeFlags.Write)
            .StoreLine($"{Health}:{Score}");
    }
}
```

Код компактный, но катастрофичный. Один класс отвечает за движение, боевую систему, UI, сохранение и переключение сцен. Если вы захотите изменить кнопку атаки, добавить анимацию смерти, переработать формат сохранения или вывести интерфейс в отдельную сцену – править придётся именно этот файл. Причём каждая правка рискует задеть несвязанные функции: меняете сохранение – ломается движение, потому что случайно переопределили `Velocity`. Обратите внимание, что проблема не только в размере класса, а в том, что **доменные правила** (как начисляются очки, что такое смерть) приклеены к **интеграции с движком** (ввод, физика) и **пред-**

ставлению (UILabel). Вы не можете протестировать расчёт очков отдельно от нажатия кнопок, не можете переиспользовать здоровье для врага без копирования кода, не можете заменить UI без того, чтобы не трогать физику.

3.1.3. Проектируем классы: от хаоса к ролям

Как этого избежать? Нужно с самого начала раскладывать функциональность по ролям, даже если кажется, что проект маленький и «можно всё в одну кучу». Роли выделяются по принципу «что меняется вместе и по какой причине». В большинстве игр просматриваются следующие стабильные группы.

- **Доменное состояние и правила:** здоровье, инвентарь, опыт, таланты. Эти классы не знают ни про движок, ни про UI – только данные и проверки.
- **Ввод и управление:** преобразование сырых событий клавиатуры или геймпада в намерения («прыгнуть», «выстрелить»). Они зависят от Go-dot, но не от правил игры.
- **Представление и эффекты:** UI-элементы, визуальные эффекты, звуки. Они читают состояние из доменных объектов и реагируют на изменения, но никогда не меняют правила.
- **Инфраструктура:** сохранение и загрузка, смена сцен, глобальные сервисы. Они связывают всё вместе, но не должны диктовать логику.

Давайте перепишем тот же PlayerManager, разделив ответственность. Это не окончательная архитектура, а первый шаг к порядку, который можно сделать в любом прототипе:

```
// Доменный объект: только данные и правила.
public sealed class Health
{
    public int Value { get; private set; } = 100;
    public int MaxValue { get; }
    public Health(int max) { MaxValue = max; Value = max; }
    public void ApplyDamage(int amount)
    {
        Value = Math.Max(0, Value - amount);
    }
}

// Интеграция с движком: персонаж как физическое тело, управляемое вводом.
public partial class PlayerController : CharacterBody2D
{
    public Health Health { get; } = new Health(100);
    public override void _PhysicsProcess(double delta)
    {
        float direction = Input.GetAxis("left", "right");
        Velocity = new Vector2(direction * 200, Velocity.Y);
        MoveAndSlide();
    }
}
```

```

    }
    public void TakeHit(int damage)
    {
        Health.ApplyDamage(damage);
        if (Health.Value == 0)
            GD.Print("Player died");
    }
}
// Представление: отдельный UI-узел, который читает состояние.
public partial class HealthBar : Control
{
    [Export] private PlayerController player;
    [Export] private Label label;
    public override void _Process(double delta)
    {
        if (player != null)
            label.Text = $"HP: {player.Health.Value}/{player.Health.MaxValue}";
    }
}

```

Теперь изменения изолированы. Если вы добавите броню или регенерацию, вы полезете только в класс `Health`. Если замените клавиатуру на геймпад или сделаете управление для мобильных устройств, изменится только `PlayerController` (или отдельный `InputHandler`). Если решите перерисовать полосу здоровья, все правки локализуются в `HealthBar`. Самое важное – правила здоровья больше не зависят от того, как персонаж двигается и как выглядит интерфейс. Вы можете использовать тот же самый класс `Health` для врагов, бочек или компаньонов, просто создав новый экземпляр. Начинать так с самого начала проще, чем кажется. Достаточно перед написанием класса спросить себя: «Какую одну вещь этот класс делает и по какой причине он может измениться?» Если причин больше двух – делите. Если класс называется `Manager` или `Utils` – задумайтесь, не свалка ли это разнородных функций. Имена вроде `Health`, `EnemySpawner`, `SaveSystem` сами подсказывают границы и удерживают от соблазна запихнуть туда лишнее.

Конечно, абсолютно независимых классов не бывает: контроллеру нужно знать о `Health`, UI нужно читать из `PlayerController`. Но эта зависимость направленная и явная. `HealthBar` знает, что существует `PlayerController` (через `[Export]`), но не наоборот. `PlayerController` знает о `Health`, но не о том, что его отображают. Направление зависимостей всегда должно идти от более конкретных и нестабильных вещей (UI, конкретный ввод) к более стабильным и абстрактным (доменные правила). Тогда при замене UI стабильное ядро игры остаётся нетронутым.

Уже на уровне этих простых правил архитектура перестаёт быть «чем-то сложным для больших проектов» и становится повседневной привычкой, экономящей часы отладки. В следующих разделах мы углубимся в то, как эти классы связывать через сигналы и сервисы, не разрушая достигнутой независимости.

3.2. Инкапсуляция

Разделение на классы даёт проекту границы, но сами по себе они ничего не гарантируют. Если любая строчка кода может в любой момент изменить здоровье персонажа, количество патронов или счёт, границы становятся фикцией. Инкапсуляция в C# – это механизм, который делает границу реальной: вы определяете, что видно снаружи, а что – внутренняя кухня класса и, главное, кто и как может менять его состояние. Без неё правила игры рассеиваются по всем скриптам, и через месяц разработки вы тратите часы, выясняя, почему у врага вдруг стало отрицательное здоровье, а полоска интерфейса показывает 150 %.

3.2.1. Инкапсуляция как архитектурная защита и язык дизайна

Представьте банковский счёт. Никто не может просто так изменить ваш баланс, придя в банк и сказав «поменяйте эту цифру на миллион». Есть чёткие протоколы: внести наличные, перевести средства, снять деньги. Каждая операция проверяет условия (достаточно ли денег, не превышен ли лимит) и только после этого меняет баланс. Если бы баланс был просто публичной переменной, которую любой кассир мог бы поправить ручкой в тетради, банковская система рухнула бы за один день. Инкапсуляция для класса – это такой же банковский протокол: вы даёте доступ к данным только через проверенные методы, гарантирующие, что объект всегда остаётся в осмысленном состоянии. В C# у вас есть три главных инструмента, чтобы реализовать это: модификаторы доступа, свойства и методы. Модификаторы (`private`, `public`, `protected`, `internal`) сразу задают, кто видит деталь. Свойства позволяют контролировать чтение и запись. А методы – это те самые протоколы, через которые внешний мир может попросить объект что-то сделать. В правильно спроектированном классе все поля, хранящие состояние, помечены как `private`. Снаружи доступ разрешён только к тому, что составляет контракт: например, можно узнать текущее здоровье (`public int Health { get; private set; }`), но нельзя просто так присвоить ему значение. Изменить здоровье можно только через вызов `TakeDamage()` или `Heal()`, внутри которых и спрятаны все проверки.

Почему нельзя просто сделать все поля публичными? Потому что тогда каждое присваивание становится частью негласного контракта, разбросанного по всему проекту. Сегодня вы в скрипте врага пишете `player.Health -= damage`, завтра в скрипте ловушки – `player.Health = 0`, послезавтра в UI-кнопке – `player.Health += 50`. Если потом понадобится добавить броню, или ограничить максимальное здоровье, или отправлять сигнал при смерти, вам придётся найти и переписать каждое такое присваивание во всех файлах. А если вы что-то пропустите, в игре появятся неуловимые баги: здоровье

скакнуло выше максимума, смерть не наступила, звук ранения не проигрался. Искать причину придётся долго, потому что точек изменения много и все они выглядят невинно.

Пример плохой инкапсуляции из реальной практики: класс `PlayerStats`, который поначалу кажется удобным.

```
public class PlayerStats
{
    public int Health;
    public int MaxHealth;
    public int Mana;
    public int Score;
}
```

Любой узел в сцене может сделать `GetNode<Player>("/root/Game/Player").Health += 10`. Это работает, но при первом же рефакторинге всё ломается. Допустим, через неделю вы решили, что здоровье не может превышать `MaxHealth`. Вы добавляете в одном месте проверку, но никто не гарантирует, что в десяти других местах кто-то не присвоил `Health` напрямую. Хуже того: вы можете забыть, где именно происходит изменение. Инкапсуляция решает эту проблему радикально: она делает так, что изменить состояние можно только одним способом – через специально предназначенный для этого метод. Тогда любое нарушение инварианта означает ошибку внутри этого метода, а не поиск иголки в стоге сена.

3.2.2. Как построить надёжный контракт: свойства, методы и инварианты

Инвариант – это условие, которое всегда должно быть истинным. «Здоровье не может быть отрицательным», «количество патронов не превышает размер обоймы», «очки опыта не убывают». Инкапсуляция позволяет взять эти правила и запереть их внутри конкретного класса так, чтобы снаружи их невозможно было нарушить. Достигается это за счёт комбинации приватных полей и публичных методов-операций, каждая из которых сама защищает инвариант.

Посмотрите на улучшенную версию управления здоровьем:

```
public class Health
{
    public int Value { get; private set; }
    public int Max { get; }
    public Health(int max, int initial)
    {
        Max = Math.Max(1, max);
        Value = Math.Clamp(initial, 0, Max);
    }
    public void Heal(int amount)
```



```

{
    if (amount <= 0) return;
    Value = Math.Min(Max, Value + amount);
}
public void TakeDamage(int damage)
{
    if (damage <= 0) return;
    Value = Math.Max(0, Value - damage);
}
}

```

Здесь Value имеет private set – никто снаружи не может написать health. Value = 999. Сеттер скрыт, а публичные методы Heal и TakeDamage выполняют всю работу, гарантируя, что здоровье останется в интервале [0, Max]. Что бы ни случилось в игре – зашли ли вы в зону с ядом, получили ли хилку от союзника, загрузили ли сохранение, – вы прогоняете изменение через эти два метода, и инвариант всегда соблюдается. Если в каком-то сценарии здоровье становится неверным, вы точно знаете, где искать ошибку: в Heal, TakeDamage или конструкторе, но не в десятках скриптов по всему проекту. Отдельного внимания заслуживает проблема «утекающих» ссылок на изменяемые объекты, которую часто упускают даже опытные разработчики. Представьте класс инвентаря, который хранит список предметов:

```

public class Inventory
{
    private List<Item> items = new List<Item>();
    public List<Item> Items => items; // Катастрофа!
}

```

Снаружи кто угодно может написать inventory.Items.Clear() или добавить предмет в обход всяких правил (вес, размер, уникальность). Хотя поле items приватное, ссылка на сам список утекла наружу. Правильное решение – либо возвращать неизменяемое представление (например, IReadOnlyList<Item>), либо, ещё лучше, вообще не давать доступ к внутренней коллекции, а предоставить методы AddItem(Item item), RemoveItem(Item item), HasItem(string id). Тогда все проверки (не превышен ли лимит, можно ли носить этот предмет) останутся внутри класса, и никто их не обойдёт. То же касается любых других изменяемых объектов. Если метод возвращает наружу ссылку на массив, список, словарь или даже на другой объект с публичными сеттерами – вы теряете контроль над состоянием. Инкапсуляция требует, чтобы всё, что может быть изменено, оставалось под полным контролем владельца.

3.2.3. Инкапсуляция на практике: отладка, масштабирование и работа в команде

Как только вы начинаете последовательно прятать состояние за методами, отладка мутирует из кошмара в рутину. Допустим, у вас есть баг: после за-

грузки сохранения у персонажа оказывается 200 % здоровья. Если здоровье – публичное поле, вы должны проверить все места, где оно меняется: скрипт загрузки, скрипт лечения, скрипт баффов, может быть, какой-то триггер на уровне. В большом проекте таких мест может быть десятки, и вы потратите часы. Если же здоровье меняется только через `Heal` и `TakeDamage`, вы ставите точку останова в этих двух методах, через минуту видите, что загрузка дергает `Heal` с неверным аргументом, и исправляете ошибку в одном месте. Инкапсуляция работает как фильтр: она заставляет все изменения идти по узкому каналу, где вы можете их отследить, логировать и проверять. Ещё один довод в пользу инкапсуляции – масштабирование. Сегодня у вас простая модель урона: `Health -= damage`. Завтра вы вводите систему брони, временные щиты, типы урона. Если до этого вы использовали публичное поле, вам придётся переписать каждую строчку кода, где это поле менялось. Вы вносите правки в двадцать файлов, рискуя что-то сломать. Если же был метод `TakeDamage(int damage)`, вы меняете только его внутренности: добавляете учёт брони, звонок к щитам, вызов события `ShieldAbsorbed`. Ни один из двадцати скриптов, вызывающих `TakeDamage`, не меняется. Они продолжают работать, как раньше, а сложность растёт внутри изолированного блока.

В командной разработке инкапсуляция незаменима как дисциплина. Представьте, что над игрой работают три человека. Один отвечает за интерфейс, другой – за врагов, третий – за спецэффекты. Если здоровье – публичное поле, UI-разработчик может нечаянно обнулить его при инициализации, вражеский скрипт – установить в отрицательное значение, а эффект – поднять выше максимума. Каждый думает, что использует значение правильно, но суммарно инварианты рушатся. Инкапсуляция даёт жёсткое правило: вы не имеете права менять состояние чужого объекта, кроме как через его публичные методы. Это снижает количество конфликтов и ошибок интеграции, потому что контракт класса становится частью инженерного договора внутри команды. Вместо того чтобы гадать, можно ли трогать поле, разработчик смотрит на доступные методы и видит: «Ага, я могу только применить урон или вылечить. Значит, чтобы сбросить здоровье, нужно использовать `TakeDamage` с большим числом». Крайней формой инкапсуляции является неизменяемость (*immutability*). Если объект после создания вообще не может быть изменён, все проблемы с контролем состояния исчезают. В играх это полезно для конфигурационных данных: параметры оружия, константы баланса, описания предметов. Такой объект можно спокойно передавать куда угодно, не боясь, что кто-то его испортит. Достигается это через автосвойства только для чтения (`public int Damage { get; }`) и установку всех значений исключительно в конструкторе. В остальных случаях, когда состояние должно меняться, инкапсуляция даёт вам ровно столько контроля, сколько нужно, чтобы система оставалась предсказуемой и легко менялась. В конечном счёте инкапсуляция – не столько технический приём, сколько привычка мыслить в терминах защиты ответственности. Вы задаёте себе вопросы: «Кто имеет право менять это значение?», «Какие условия должны быть соблюдены при

изменении?», «Как гарантировать, что никто не обойдёт эти условия?» – и ответы на них оформляете кодом. В маленьком проекте это сэкономит вас от глупых багов, а в большом станет фундаментом, на котором держится вся архитектура.

3.3. События и делегаты

Вы наверняка уже сталкивались с такой ситуацией: персонаж получает урон, и тут же должны произойти сразу несколько вещей – обновиться полоска здоровья в интерфейсе, проиграться звук боли, возможно, сработать эффект красной вспышки на экране, а если здоровье упало до нуля, запуститься сцена смерти. Можно, конечно, прописать все эти действия прямо в методе `TakeDamage`:

```
public void TakeDamage(int damage)
{
    health -= damage;
    ui.UpdateHealthBar(health);
    audio.PlayHurtSound();
    effects.FlashRed();
    if (health <= 0) Die();
}
```

Пока игра состоит из двух комнат и одного персонажа, это работает. Но когда проект разрастётся, такой подход превратится в кошмар. Классу, отвечающему за здоровье, придётся знать обо всех уголках игры: о каждом элементе интерфейса, о каждом звуке, о каждой системе достижений. Добавление новой реакции, например экранной тряски или срабатывания ловушки, потребует правки этого класса. А если вы захотите использовать ту же логику здоровья у врагов или разрушаемых объектов, вам придётся либо копировать код, либо тащить за собой весь этот ворох зависимостей. Именно для решения таких проблем в C# существуют **делегаты** и **события**. Они позволяют объекту оповестить всех заинтересованных о том, что с ним что-то произошло, не зная при этом, кто именно слушает и что они будут делать. Объект просто сообщает факт: «Моё здоровье изменилось». А все остальные части системы сами решают, как на это реагировать.

3.3.1. Делегаты: ссылки на методы как значения

Прежде чем говорить о событиях, нужно понять, что такое делегат. Проще всего представить делегат как переменную, которая вместо числа или строки хранит ссылку на метод. Вы можете вызвать эту переменную так же, как и обычный метод. Главное, чтобы сигнатура (входные параметры и возвращаемый тип) совпадала.

Объявляется делегат при помощи ключевого слова `delegate`:

```
// Делегат, который принимает два целых числа и возвращает целое.
public delegate int ScoreRule(int basePoints, int combo);
```

Теперь `ScoreRule` – это новый тип данных. Переменная такого типа может указывать на любой метод с такой же сигнатурой. Например:

```
// Метод, который просто суммирует очки и комбо.
int DefaultScore(int basePoints, int combo) => basePoints + combo;
// Присваиваем делегату ссылку на этот метод.
ScoreRule rule = DefaultScore;
// Вызываем делегат как обычный метод.
int total = rule(100, 3); // result = 103.
```

Но самое интересное – вы можете передавать делегаты как параметры в другие методы или хранить их как поля класса. Это позволяет менять поведение на лету, не прибегая к громоздким цепочкам `if-else`. Представьте, что у вас есть класс, отвечающий за подсчёт очков, и в зависимости от режима игры (обычный, хардкорный, турбо) очки должны считаться по-разному. Вместо флага и кучи условий вы просто передаёте нужную функцию-правило при создании счётчика:

```
public class ScoreCounter
{
    private readonly ScoreRule _rule;

    public ScoreCounter(ScoreRule rule) => _rule = rule;

    public int Add(int basePoints, int combo) => _rule(basePoints, combo);
}
// Где-то в коде переключения режимов.
ScoreCounter counter;
if (currentMode == Mode.Turbo)
    counter = new ScoreCounter((base, combo) => base * combo * 2);
else
    counter = new ScoreCounter((base, combo) => base + combo);
```

Делегаты в C# являются **мультикастовыми** (multicast). Это значит, что одна переменная-делегат может хранить список из нескольких методов, и при вызове они выполнятся по очереди. Именно на этой возможности строятся события: одно событие может иметь множество подписчиков, и все они будут оповещены.

3.3.2. События: контролируемая рассылка уведомлений

Если делегат – это просто ссылка на метод, то **событие** (event) – это специальная обёртка вокруг делегата, которая накладывает важное ограничение. Сна-

ружи класса-владельца к событию можно применять только две операции: подписаться (+=) и отписаться (-=). Вызывать событие или перезаписывать его список подписчиков имеет право только сам класс-владелец. Это и есть та самая инкапсуляция, о которой мы говорили в предыдущем разделе: объект сообщает о факте, но никто снаружи не может сфальсифицировать этот факт или помешать рассылке.

Давайте превратим наш класс здоровья в источник события. Вместо того чтобы дёргать UI и звуки напрямую, мы объявим событие Changed, которое будет уведомлять всех о том, что значение здоровья изменилось:

```
public class Health
{
    public event Action<int> Changed; // Событие с передачей нового значения.

    public int Value { get; private set; } = 100;

    public void TakeDamage(int amount)
    {
        if (amount <= 0) return;
        Value = Math.Max(0, Value - amount);
        Changed?.Invoke(Value); // Вызываем событие, только если есть подписчики.
    }
}
```

Теперь любой внешний объект, которому важно знать об изменении здоровья, может подписаться на это событие:

```
public class HealthBar
{
    private Health _health;

    public void Bind(Health health)
    {
        _health = health;
        _health.Changed += OnHealthChanged; // Подписываемся.
    }

    private void OnHealthChanged(int newValue)
    {
        // Тут обновляется полоска здоровья в UI.
        GD.Print($"Health: {newValue}");
    }

    public void Unbind()
    {
        if (_health != null)
            _health.Changed -= OnHealthChanged; // Обязательно отписываемся!
    }
}
```

Обратите внимание: класс Health понятия не имеет о существовании HealthBar. Он лишь сообщает, что значение изменилось. HealthBar, в свою

очередь, просто получает уведомление и обновляет себя, не трогая внутренности Health. Это и есть слабая связанность (low coupling), которую мы стремимся получить.

3.3.3. Жизненный цикл подписки и утечки памяти

Самая коварная ошибка новичков при работе с событиями – забытая отписка. Когда вы подписываетесь на событие, издатель сохраняет ссылку на подписчика. Если издатель живёт дольше, чем подписчик должен существовать, возникает утечка: подписчик не может быть удалён сборщиком мусора, потому что на него всё ещё ссылаются. Более того, если событие вдруг срабатывает, оно попытается обратиться к уже «мёртвому» объекту, что приведёт к трудноуловимым багам.

Особенно остро эта проблема стоит в игровых движках, где узлы и сцены постоянно создаются и уничтожаются. Предположим, у вас есть временный UI-элемент (всплывающее окно), который подписался на здоровье игрока. Если окно закрывается (удаляется из дерева), а отписка не происходит, оно останется висеть в памяти до тех пор, пока жив игрок. И при каждом изменении здоровья будет безуспешно пытаться обновить несуществующий интерфейс.

К счастью, решение простое и стандартное: **парность подписки и отписки**. Где подписались, там и отписываемся. В мире Godot, где всё завязано на жизненный цикл узлов, удобно привязывать подписки к `_EnterTree/_Ready` и `_ExitTree`. Если ваш скрипт является наследником `Node`, вы можете сделать так:

```
public partial class HealthBarNode : Control
{
    [Export] private Health _health;

    public override void _Ready()
    {
        if (_health != null)
            _health.Changed += OnHealthChanged;
    }

    public override void _ExitTree()
    {
        if (_health != null)
            _health.Changed -= OnHealthChanged; // Снимаем подписку при удалении узла.
    }

    private void OnHealthChanged(int newValue) => GetNode<Label>("Label").Text = newValue.ToString();
}
```

Но что, если подписчик – не узел, а обычный C#-объект? Тогда управление жизненным циклом ложится на создателя этого объекта, и вам нужно

предусмотреть метод типа `Dispose()` или иной способ корректной очистки. В любом случае принцип неизменен: подписка – это ресурс, который нужно освобождать. Также очень опасно подписываться на **статические события**. Статическое событие живёт всё время работы программы, и любой подписчик, однажды подписавшись, будет удерживаться вечно, если явно не отписаться. Это прямой путь к утечкам памяти и «ожившим мертвецам». Используйте статические события только с чётким пониманием, кто и когда отписывается.

3.3.4. События, прямые вызовы и сигналы Godot: когда что использовать

После знакомства с событиями может возникнуть соблазн перевести всё общение между классами на эту рельсу. Это не всегда оправдано. Важно различать **факты** (то, что случилось) и **команды** (то, что нужно сделать).

События идеальны для реакции на факты. «Здоровье изменилось», «предмет подобран», «уровень загрузился». У такого факта может быть много независимых наблюдателей, а инициатору они безразличны. Если же вам нужно, чтобы один объект прямо сказал другому «Прыгни!» или «Сохрани игру», честнее будет использовать прямой вызов метода. Такой код проще читать и отлаживать, потому что поток управления остаётся на виду, а не прячется за подписками. В контексте Godot у вас есть ещё и встроенный механизм **сигналов** (signals). Сигналы очень похожи на события: один узел испускает сигнал, другие подписываются. Но сигналы глубже интегрированы в движок: их можно настраивать визуально в редакторе, они привязаны к дереву сцены и так далее. Возникает вопрос: что же использовать?

Хорошее правило: **сигналы – для общения между узлами в сцене**, где важна визуальная компоновка и связь с жизненным циклом Godot. **События C# – для чистой доменной логики**, которая не зависит от движка и которую вы, возможно, захотите тестировать отдельно или переиспользовать вне Godot. Например, класс `Health` из примеров выше ничего не знает о Godot – это обычный C# класс с событием. Сигнал здесь был бы избыточен и привязал бы логику к API движка. А вот узел, который отображает здоровье и является наследником `Control`, как раз может через сигнал или через подписку на C#-событие обновлять текст. Смешивать оба подхода в одном модуле без крайней нужды не стоит – это запутывает поток данных. Выберите что-то одно для конкретной подсистемы и придерживайтесь этого.

Несколько советов

- **Давайте событиям осмысленные имена**, отражающие факт: `HealthChanged`, `CoinsCollected`. Имя `UpdateUI` для события – плохой тон, оно диктует подписчику, что делать, а не сообщает, что случилось.
- **Передавайте минимально необходимые данные**. Не стоит прикреплять к событию ссылку на весь объект-источник, если достаточно одно-

го числа. Это снижает связность и не позволяет подписчикам влезать в чужие внутренности.

- **Остерегайтесь цепочек событий:** когда обработчик события А испускает событие В, которое вызывает событие С, логика становится невероятно запутанной. Как правило, если вам нужна строгая последовательность действий, её лучше реализовать прямыми вызовами в рамках одного управляющего метода.
- **Проверяйте событие на null перед вызовом** (или используйте ?.Invoke). Отсутствие подписчиков – нормальная ситуация, это не ошибка.

3.4. Интерфейсы: как договариваться о возможностях, не привязываясь к деталям

Представьте, что вы проектируете систему взаимодействия в игре. Игрок нажимает кнопку «использовать», и мир должен отреагировать. Сегодня это дверь: нажали – дверь открылась. Завтра вы добавляете сундук с добычей, послезавтра – рычаг, включающий мост. Хочется, чтобы игрок просто «применял действие» к тому, на что он смотрит, и не думал, дверь перед ним или терминал. Но если писать код в лоб, получится что-то вроде:

```
if (target is Door door)
    door.Open();
else if (target is Chest chest)
    chest.Open();
else if (target is Lever lever)
    lever.Toggle();
```

Пока объектов три – это терпимо. Когда их станет тридцать, а логика взаимодействия размажется по всем игровым скриптам, поддерживать такое станет невозможно. Каждый новый тип объекта заставит вас перелопачивать все места, где он может быть использован. Более того, скрипт игрока окажется завязан на конкретные классы Door, Chest, Lever. Изменится внутренность сундука – и, возможно, придётся править игрока. Архитектурно это тупик. Ровно для таких ситуаций в C# придуманы **интерфейсы** (interface). Они позволяют сказать: «Мне всё равно, что это за объект, главное – чтобы он умел делать X». Интерфейс – это как универсальный пульт с несколькими кнопками. Вы можете направить его на телевизор, кондиционер или музыкальный центр, и кнопка «включить» сработает, даже если внутри они устроены совершенно по-разному. В программировании такой пульт – это **контракт**, который описывает, какие методы должен реализовать класс, но не говорит, как именно. Сам интерфейс не содержит кода – только названия методов и их сигнатуры. А класс, который соглашается выполнять этот контракт, обязуется наполнить методы реальной логикой.

Зачем это нужно и как спасает архитектуру

Главная проблема, которую решают интерфейсы, – **жёсткая связанность** (coupling). Когда один класс знает о другом слишком много, любое изменение в деталях вызывает цепную реакцию. Интерфейс разрывает эту связь, оставляя только знание о роли. Потребителю (например, системе урона) не нужно знать, что перед ним: игрок, враг или бочка с динамитом. Ему достаточно знать, что этот объект **умеет получать урон**. Мы описываем эту способность интерфейсом `IDamageable`:

```
public interface IDamageable
{
    void TakeDamage(int amount);
}
```

Теперь класс врага, класс игрока и даже класс бочки могут реализовать этот интерфейс, каждый по-своему. А система урона будет работать с ними одинаково:

```
public class DamageSystem
{
    public void ApplyDamage(IDamageable target, int damage)
    {
        // Система не знает, кто перед ней: враг, игрок или бочка.
        // Она просто вызывает оговорённый метод.
        target.TakeDamage(damage);
    }
}
```

Ключевое преимущество – расширяемость. Если через месяц вы решите добавить в игру хрупкие сталактиты, падающие на голову, вам не придётся лезть в код `DamageSystem`. Достаточно создать новый класс сталактита и реализовать в нём `IDamageable`. Система подхватит его автоматически. Это и есть та самая «свобода реализации», о которой говорят опытные разработчики: вы меняете внутренности или добавляете новые сущности, а внешний мир этого даже не замечает.

Интерфейс как роль: «can-do» против «is-a»

Посмотрите на иерархию наследования. Обычно мы мыслим категориями «является» (is-a): враг – это персонаж, персонаж – это игровой объект. Такая цепочка полезна, когда у объектов действительно общая природа и общая реализация. Но как только появляются перекрёстные возможности, дерево начинает трещать по швам. Враг и бочка оба могут получать урон, но они не являются родственниками. Сундук и дверь оба открываются, но один – контейнер, другая – проход. Делать для них общего предка `OpenableObject` бессмысленно, потому что у них нет общей реализации, и мы лишь создадим искусственную связь. Интерфейсы предлагают другую модель мышления – «умеет» (can-do). Мы не спрашиваем, кем является объект, мы спрашиваем,

какую роль он может исполнить. Один и тот же объект может совмещать несколько ролей. Например, враг может быть и `IDamageable`, и `IMovable`, и `IInteractable` (если с ним можно поговорить). Стул может быть `IInteractable` (чтобы сесть) и `IDamageable` (чтобы сломать). Интерфейсы позволяют «приклеить» к объекту ровно те ярлыки, которые нужны, не выстраивая громоздких бабашен наследования. Именно это называют ролевым дизайном, и для игр он подходит идеально.

Как интерфейсы помогают отделить правила от движка

Представьте, что вы делаете игру, которая должна работать и на ПК с клавиатурой, и на телефоне с тачскрином, а в будущем, возможно, и по сети. Правила игры (например, «персонаж поворачивается в сторону нажатия») не должны зависеть от того, откуда пришли данные – от кнопки или от пальца. Здесь снова спасает интерфейс. Мы вводим контракт `IInputSource`:

```
public interface IInputSource
{
    Vector2 GetMovementDirection();
    bool IsActionPressed(string action);
}
```

Класс игрока будет зависеть от этого интерфейса, а не от конкретного `Input`. В результате мы можем легко подменить реализацию: для ПК – чтение клавиш, для телефона – джойстик на экране, для записи повтора – проигрывание сохранённых команд. Доменному коду (правилам движения) всё равно; он продолжает работать, потому что контракт соблюлён. Такая «инверсия зависимостей» (когда высокоуровневая логика зависит от абстракции, а детали – от неё же) краеугольный камень поддерживаемой архитектуры.

Тестирование: интерфейсы как дверь в изолированную проверку логики

Одно из самых приятных следствий использования интерфейсов – возможность тестировать игровые правила без запуска движка. Если ваш `DamageSystem` зависит от `IDamageable`, вы в тесте можете создать простой объект-заглушку:

```
public class TestTarget : IDamageable
{
    public int LastDamageTaken { get; private set; }
    public void TakeDamage(int amount)
    {
        LastDamageTaken = amount;
    }
}
```

Теперь вы проверяете логику чисто: создали `DamageSystem`, передали `TestTarget`, вызвали `ApplyDamage` и убедились, что `LastDamageTaken` равен ожидаемо-

му числу. Никаких загрузок сцен, ожиданий кадра и танцев с бубном. Такой подход окупается стократно, когда в игре появляются сложные механики с модификаторами урона или правилами, зависящими от типа цели.

Осторожно: взрыв интерфейсов и жирные контракты

Как любой мощный инструмент, интерфейсы можно использовать во вред. Самая частая ошибка новичков – начать вводить интерфейсы на каждый чих. Появляются `IHealth`, `IPlayer`, `IEnemy`, которые попросту дублируют классы и не добавляют никакой гибкости. Запомните: **интерфейс оправдан только тогда, когда он фиксирует роль, у которой может быть несколько независимых реализаций**. Если реализация всегда одна и вы не планируете её подменять или расширять, интерфейс не нужен – достаточно конкретного класса. Другая крайность – перегруженные контракты. Если ваш `IEntity` содержит методы `TakeDamage`, `Heal`, `Move`, `Jump`, `Save`, `Load`, то это уже не роль, а божественный объект. Потребитель такого интерфейса будет вынужден знать всё обо всём. Правило простое: **интерфейс должен быть узким и осмысленным**. Обычно он содержит от одного до трёх методов, которые относятся строго к одной задаче. Лучше иметь несколько маленьких интерфейсов (`IDamageable`, `IMovable`, `ISavable`), чем один гигантский. Это называется «принципом разделения интерфейсов», и он прямым образом влияет на масштабируемость: изменения в системе движения не затронут систему урона.

Детали языка: как это выглядит в коде

В C# интерфейс объявляется с ключевым словом `interface`. По соглашению его имя начинается с большой буквы `I` (`IDamageable`, `IInteractable`). Внутри интерфейса могут быть методы, свойства, события и индексаторы, но не поля. Реализация интерфейса в классе прописывается через двоеточие, так же как наследование, только здесь можно перечислить несколько интерфейсов через запятую. Например, класс может быть объявлен как `public class Enemy : Node, IDamageable, IInteractable`.

Переменная типа интерфейса может хранить ссылку на любой объект, который этот интерфейс реализует. Вы не можете создать экземпляр интерфейса через `new`, ведь в нём нет реализации. Но вы можете передавать его как параметр, возвращать из методов, создавать списки `List<IDamageable>`. Именно так строятся гибкие системы: метод принимает интерфейс, а вы передаёте ему то, что нужно в данный момент. Компилятор проследит, чтобы у передаваемого объекта точно были все нужные методы. В более старых версиях языка интерфейсы не могли содержать реализации, но начиная с C# 8.0 появились `default interface methods` – методы по умолчанию, которые могут иметь тело. Впрочем, в контексте архитектуры Godot это редко используется, и для простоты можно считать интерфейс чистым списком требований.

Пример из жизни: универсальный инвентарь

Допустим, вы хотите реализовать инвентарь, способный хранить любые предметы. У вас есть зелья, оружие, ключи, ресурсы. Все они должны уметь что-то делать при использовании, но каждое по-своему. Интерфейс `IUsable` решает эту задачу:

```
public interface IUsable
{
    void Use(Character user);
}
```

Теперь класс зелья реализует этот интерфейс и в методе `Use` восстанавливает здоровье владельцу. Класс оружия даёт сигнал атаки. Класс ключа открывает соответствующую дверь. А код инвентаря остаётся предельно общим: при двойном клике по предмету он просто вызывает `item.Use(player)`. Если через месяц появится свиток телепортации, вы добавите ещё один класс, реализующий `IUsable`, и больше ни строчки кода не измените.

3.5. Асинхронность: как не заморозить игру и не потерять контроль

Игровой движок Godot каждый кадр обновляет сотни объектов, считает физику, принимает ввод и отрисовывает графику. Весь этот процесс умещается в 16 миллисекунд, если вы целитесь в 60 FPS. Но что будет, если посреди кадра вы попытаетесь прочитать большой файл сохранения, отправить запрос на сервер или сгенерировать уровень? Главный поток просто встанет. Игра замрёт. Пользователь увидит подвисший экран, а операционная система может даже предложить закрыть приложение. Именно для таких ситуаций в C# существует асинхронность – не для того, чтобы «ускорить вычисления», а чтобы дать игре возможность продолжать дышать, пока выполняются долгие операции. Но асинхронность – это не просто кнопка «сделать фоном». Она добавляет в архитектуру новое измерение – время. Код, который раньше выполнялся строка за строкой, теперь разрывается паузами, и после каждой паузы мир может быть уже другим. Если не держать это в голове, баги становятся случайными, а отладка – кошмаром. Понять, как управлять этой силой, – наша задача.

3.5.1. *Task и async/await: как работают обещания и паузы*

В центре асинхронной модели C# лежит класс `Task`. Представьте себе талончик из гардероба. Вы сдаёте пальто, получаете номерок – и можете идти пить кофе,

не стоя у стойки. Когда пальто понадобится, вы по номерку его заберёте, возможно, уже через полчаса. Task – это такой номерок. Он говорит: операция запущена, но её результат пока не готов. Вы можете передать этот номерок кому-то ещё, проверить, готов ли он, или просто подождать его с помощью ключевого слова `await`. Само `await` – это не магическая пауза, которая замораживает весь мир. На самом деле компилятор превращает метод после `await` в «продолжение» (continuation): код до `await` выполняется сразу и возвращает управление движку, чтобы тот мог нарисовать следующий кадр. А когда задача завершится, продолжение будет вызвано снова, и выполнение метода возобновится. Весь фокус в том, что между этими точками игра жила своей жизнью: кадры сменялись, объекты двигались, игрок нажимал кнопки. Это и есть **неблокирующее ожидание**. Очень важно не путать асинхронность и многопоточность. `async/await` по умолчанию не создаёт новых потоков. Если вы просто напишете `await Task.Delay(1000)`, ваш код не будет выполняться в фоне – он просто освободит поток на секунду, а потом вернётся в него же. Многопоточность (запуск `Task.Run`) используется, когда нужно вынести тяжёлые вычисления из основного потока, но для большинства игровых сценариев (загрузка файла, ожидание ответа сервера) достаточно именно неблокирующих операций. Ваш метод как бы «разрезан» на две части: до ожидания и после. Первая часть выполняется здесь и сейчас, вторая – когда-нибудь позже.

Рассмотрим простой пример: способность с кулдауном. Без `async` вам пришлось бы завести таймер, вручную отсчитывать кадры и обновлять флаг готовности. С `async` это выглядит предельно линейно:

```
public class Ability
{
    public bool IsReady { get; private set; } = true;
    public async Task ActivateAsync()
    {
        IsReady = false;
        await Task.Delay(2000);
        IsReady = true;
    }
}
```

Вызываем этот метод где-то в обработчике нажатия кнопки, и в течение двух секунд способность неактивна. Код читается так же, как синхронный: «сначала выключаем готовность, затем ждём две секунды, потом включаем». Но между «выключаем» и «включаем» проходит два реальных кадра – и это ключевой момент.

3.5.2. Асинхронность в мире Godot: где она уместна, а где опасна

Godot построен вокруг синхронного кадрового цикла. Методы `_Process`, `_PhysicsProcess`, обработчики сигналов – всё это вызывается в строго опреде-

лённые моменты на основном потоке. Здесь же работает физика, анимации и большая часть API. Асинхронные операции ломают этот ритм, потому что их продолжения могут выполняться в любой момент, а не только в начале кадра. Возникает соблазн разбросать `async/await` по всему проекту, но это почти всегда приводит к проблемам. Представьте: персонаж начинает способность, которая внутри себя грузит асинхронно файл конфигурации. Через секунду, пока файл ещё не загрузился, персонаж умирает, и узел удаляется из дерева. Но асинхронная задача продолжает жить. Когда файл наконец загрузится, код попытается применить результат к уже уничтоженному узлу – и вы получите либо ошибку доступа, либо «призрачный» эффект. Поэтому правило номер один: **владелец операции должен управлять её временем жизни**. Если узел инициирует асинхронную работу, он обязан отменить её, когда покидает дерево.

В Godot для этого есть удобный механизм – `CancellationToken`. Когда узел выходит из дерева, мы подаём сигнал отмены, и задача завершается без результата. Вот как это выглядит на примере загрузки профиля игрока, которую нужно прервать, если игрок вышел в главное меню:

```
public partial class ProfileLoader : Node
{
    private CancellationTokenSource _cts;
    // Этот метод вызывается, когда сцена загружена.
    public async void LoadProfile(string userId)
    {
        _cts = new CancellationTokenSource();
        try
        {
            // Асинхронно получаем данные (в реальном проекте – запрос к серверу).
            string profileData = await FetchProfileAsync(userId, _cts.Token);

            // Проверяем, не успели ли мы выйти из дерева, пока ждали.
            if (IsInsideTree())
                ApplyProfileData(profileData);
        }
        catch (OperationCanceledException)
        {
            GD.Print("Загрузка профиля отменена");
        }
    }

    private async Task<string> FetchProfileAsync(string userId, CancellationToken ct)
    {
        await Task.Delay(1500, ct); // Эмуляция сетевой задержки.
        return $"Profile-{userId}";
    }

    public override void _ExitTree()
    {
        // Когда узел покидает дерево, отменяем все его активные операции.
        _cts?.Cancel();
    }
}
```



```

        _cts?.Dispose();
    }
}

```

Обратите внимание: метод `LoadProfile` объявлен как `async void`. В обычных приложениях это считается дурным тоном, потому что `async void` не позволяет вызывающему коду дождаться завершения или поймать исключение. Но в Godot это приемлемо для обработчиков, которые срабатывают по сигналу или нажатию кнопки – мы не можем изменить их сигнатуру на `Task`. Исключения внутри `async void` приводят к краху приложения, поэтому обязательно используйте `try-catch` для всех таких методов. В остальных случаях всегда возвращайте `Task`, чтобы потребитель мог контролировать выполнение.

3.5.3. Состояние между мирами: главная ловушка *await*

Когда вы пишете `await`, то передаёте управление обратно движку. После возврата из `await` весь контекст, который был до паузы, может оказаться недействительным. Классический пример:

```

public async void OnButtonPressed()
{
    var enemy = GetNode<Enemy>("Enemy");
    await Task.Delay(1000);
    enemy.TakeDamage(50);
}

```

За эту секунду враг мог быть удалён из сцены другим игроком, событием или даже выйти из дерева. Если вы не проверите `IsValid(enemy)`, код упадёт с исключением. Ссылки на узлы, захваченные до `await`, – это битые стёкла: вы видите объект, но не знаете, жив ли он на самом деле. Поэтому хорошей практикой является делать «снимок» минимально необходимых данных до `await` и заново проверять существование объекта после. Гораздо безопаснее перенести через `await` не ссылку на узел, а простые типы (ID, строки, числа). Например, запомнить `enemy.Name` или `enemy.GetInstanceId()` и после паузы попытаться найти узел заново. Если он не найден – операция потеряла смысл.

Ещё более глубокая проблема – гонки. Два разных асинхронных продолжения могут попытаться изменить одно и то же поле в непредсказуемом порядке. Это не гонка потоков в классическом смысле, а гонка во времени: оба продолжения выполняются последовательно (на одном потоке), но вы не знаете, какое первым. Чтобы избежать таких сюрпризов, применяйте простой принцип: **результаты асинхронной операции должны применяться через явную точку синхронизации**, например флагом в `_Process`, а не напрямую по факту завершения. Так вы сохраняете контроль над тем, в какой фазе кадра происходят изменения.

3.5.4. Где асинхронность действительно нужна, а где лучше без неё

Асинхронность как мощный спортивный автомобиль: на трассе он великолепен, но в пробке бесполезен. В игровом коде есть всего несколько сценариев, где `async/await` действительно упрощает жизнь:

- **Загрузка и выгрузка контента:** подгрузка уровней, ресурсов, текстур, чтобы избежать фризов при открытии дверей или перемещении по миру.
- **Сетевые запросы:** обращение к серверу за списком матчей, сохранение прогресса в облаке, получение уведомлений. Здесь неблокирующее ожидание критично.
- **Фоновые вычисления:** генерация карты, обсчёт AI-поведения, не влияющего на текущий кадр. Для таких задач часто используют `Task.Run`, чтобы вынести работу на пул потоков, но результат возвращают в кадр через синхронные точки.
- **Таймеры и кулдауны:** компактный способ выразить задержку, как в примере со способностью.

Во всех остальных случаях, особенно в высокочастотных методах вроде `_Process`, `async/await` не нужен и вреден. Не пытайтесь сделать асинхронным движение персонажа или обновление анимации. Кадровый цикл уже даёт идеальную синхронизацию времени. Смешивание двух моделей (кадровой и событийной) приводит к несогласованности, которую очень трудно отлаживать.

Хорошая архитектура прячет асинхронность за стабильными интерфейсами. Доменная логика (правила игры) не должна знать, откуда взялись данные: из файла, из сети или из памяти. Она получает готовый результат. Асинхронным должен быть только код инфраструктуры, который добывает эти данные и затем передаёт их в домен через контракты. Это позволит вам сегодня загружать конфигурацию из локального JSON, а завтра – с удалённого сервера, не трогая основные механики.

3.6. Как не превратить проект в тыкву: типичные архитектурные ошибки и их профилактика

Архитектурные ошибки никогда не выглядят как одна «неправильная строчка». Это всегда цепочка компромиссов, каждый из которых на момент написания казался самым быстрым и естественным решением. Вы добавляете публичное поле, потому что «потом сделаем красиво». Вы дёргаете `GetNode` из

скрипта врага, чтобы обновить интерфейс. Вы подписываетесь на событие и забываете отписаться. По отдельности эти грехи безобидны, но вместе они размывают границы ответственности, и через пару месяцев проект превращается в хрупкий картонный домик: любое изменение радиуса в пять строк лавиной обрушивает поведение в неожиданных углах. Цель этого раздела – научиться видеть не сами ошибки, а ту **механику**, которая к ним приводит, и закрепить привычку думать структурно ещё до того, как код написан.

Давайте проследим историю одного вымышленного проекта – экшн-платформера «Месть Самурая». Это будет живая иллюстрация того, как отсутствие дисциплины порождает хаос и как те самые принципы из предыдущих разделов возвращают системе устойчивость.

Шаг 1: «божественный» PlayerController и публичные поля как API

Игра начинается с прототипа: есть сцена World, в ней игрок, пара врагов и полоска здоровья в UI. Наш разработчик Артём создаёт класс PlayerController и рассуждает так: «Зачем плодить кучу классов, если всё нужное умещается в одном?» Он пишет:

```
public partial class PlayerController : CharacterBody2D
{
    [Export] public int Health = 100;
    [Export] public int Score = 0;
    [Export] private Label healthLabel;
    [Export] private Label scoreLabel;
    public override void _Process(double delta)
    {
        // Движение.
        Vector2 velocity = Velocity;
        velocity.X = Input.GetAxis("left", "right") * 300;
        Velocity = velocity;
        MoveAndSlide();
        // UI-обновления.
        healthLabel.Text = $"HP: {Health}";
        scoreLabel.Text = $"Score: {Score}";
        // Смерть.
        if (Health <= 0)
        {
            GD.Print("Player died");
            GetTree().ReloadCurrentScene();
        }
    }
    public void TakeDamage(int amount)
    {
        Health -= amount;
        if (Health <= 0)
            File.WriteAllText("save.txt", $"{Score}");
    }
}
```

Пока игра микроскопическая, всё работает. Артём счастлив. Но вскоре приходит дизайнер и просит:

- добавить систему брони, которая поглощает часть урона;
- сделать вспышку экрана при получении урона;
- сохранять не только очки, но и максимальное здоровье;
- чтобы враги тоже умирали и начисляли очки.

Артём понимает, что трогать `TakeDamage` страшно: метод уже отвечает и за звук, и за UI, и за сохранение. Добавление брони потребует править то, что никак не связано с защитой. Врагам нужна такая же логика смерти, но копировать всего игрока с движением и интерфейсом бессмысленно. А самое неприятное – изменение радиуса одной правки (скажем, формата сохранения) теперь требует перекомпиляции всего, что тянет `PlayerController`.

Корень проблемы – **отсутствие границ ответственности**. Класс собрал в себе движение, здоровье, очки, отображение и файловый ввод-вывод. Любая из этих сущностей может измениться по своим причинам, но все они заперты в одном файле. В терминах предыдущего раздела это «God class» – божественный объект, который знает всё и командует всем. Он удобен на старте, но убивает расширяемость.

Как Артёму следовало поступить? Первым делом – разделить роли. Здоровье – отдельная ответственность. Очки – отдельная. Отображение здоровья – отдельная. Вот как выглядит первый рефакторинг:

```
// Доменный объект, не зависящий от Godot.
public class Health
{
    public int Value { get; private set; }
    public int Max { get; }
    public Health(int max, int initial)
    {
        Max = max;
        Value = Math.Clamp(initial, 0, Max);
    }
    public void TakeDamage(int amount)
    {
        if (amount <= 0) return;
        Value = Math.Max(0, Value - amount);
    }
}

// Контроллер игрока отвечает только за движение и владеет здоровьем.
public partial class PlayerController : CharacterBody2D
{
    public Health Health { get; } = new Health(100, 100);
    public override void _PhysicsProcess(double delta)
    {
        Vector2 velocity = Velocity;
        velocity.X = Input.GetAxis("left", "right") * 300;
        Velocity = velocity;
        MoveAndSlide();
    }
}
```

```

    }
}
// UI-компонент, который следит за здоровьем.
public partial class HealthBar : Label
{
    [Export] private PlayerController player;
    public override void _Process(double delta)
    {
        if (player != null)
            Text = $"HP: {player.Health.Value}/{player.Health.Max}";
    }
}
}

```

Теперь здоровье изолировано: добавить броню или регенерацию можно внутри класса `Health`, не касаясь движения или UI. Очки можно вынести в отдельный `ScoreCounter`. Врагам достаточно создать свой экземпляр `Health` и так же принимать урон. Внешний код не может случайно испортить значение, потому что `Value` имеет `private set`, а изменения проходят через осмысленные методы.

Но это только половина беды. Второй архитектурный грех оригинала – **публичные поля как API**. Артём выставил `Health` и `Score` публичными, разрешив любому скрипту менять их напрямую. В этом хаосе невозможно отследить, кто и когда изменил состояние. Инкапсуляция решает эту проблему: состояние скрыто, а мутация возможна только через контролируемые точки (`TakeDamage`), где можно добавить проверки, логирование или вызовы событий. Это та самая дисциплина, которая на масштабе экономит часы отладки.

Шаг 2: прямые зависимости и скрытое знание о структуре сцены

Игра развивается. Появляется экран окончания уровня с кнопкой «Выйти в меню». Артём в скрипте кнопки пишет:

```
GetNode<GameManager>("/root/World/GameManager").EndGame();
```

Сработало. Затем добавляется сундук, который должен открываться при нажатии, и он тоже ищет `GameManager` по абсолютному пути. Позже дизайнер решает переместить `GameManager` в другое место иерархии, и всё ломается – причём ошибка всплывает не на этапе компиляции, а в рантайме. Что хуже, сундук теперь зависит от точного пути в дереве сцен, а значит, его нельзя переиспользовать в другом уровне без танцев с бубном.

Это классическая ошибка **скрытых зависимостей**. Класс не заявляет о своих потребностях явно (через параметры конструктора или `[Export]`), а добывает их сам через глобальный поиск. Симптомы: код падает в новом контексте, появляются «магические» требования к именам узлов, и переиспользование сцены превращается в расследование.

Правильный подход – **делать зависимости явными**. Если кнопке нужен доступ к GameManager, пусть она получит его через экспортируемое поле:

```
public partial class EndGameButton : Button
{
    [Export] private GameManager manager;
    public override void _Pressed()
    {
        manager.EndGame();
    }
}
```

Теперь связь видна в редакторе, проверяется при сборке, и кнопка не делает предположений об остальной структуре уровня. Если GameManager переедет, достаточно перетащить ссылку в инспекторе. Это не просто удобство, это гарантия, что компонент честно сообщает о своих зависимостях и может быть использован в любом контексте.

Шаг 3: события-команды и забытые подписки

Дальше Артём хочет, чтобы при получении урона игрок мигал красным, а враги издавали звук. Он вспоминает: «Ага, надо использовать события!» И добавляет в Health событие:

```
public event Action<int> Damaged;
```

В скрипте игрока он подписывается и меняет цвет спрайта. Во врагах тоже подписывается. Всё работает. Но потом Артём замечает странности: при перезагрузке уровня враги, которые уже были убиты, продолжают «крякать» при атаках игрока. Оказывается, при удалении врага из дерева подписка не снимается, и объект-призрак продолжает реагировать на события здоровья, которого у него нет. Это **утечка подписок**, классическая ловушка асинхронности и событийной модели.

Лекарство простое, мы его уже разбирали: подписка – это ресурс с жизненным циклом. Всякий раз, когда вы говорите +=, вы должны гарантировать парный -=. В Godot это удобно делать в паре _EnterTree/_ExitTree, а для обычных классов – через явный метод отписки, вызываемый владельцем. В нашем случае враг мог бы управлять подпиской так:

```
public partial class Enemy : Node, IDamageable
{
    private Health _health = new Health(50, 50);
    public override void _EnterTree()
    {
        _health.Changed += OnHealthChanged;
    }
    public override void _ExitTree()
    {
        _health.Changed -= OnHealthChanged;
    }
}
```

```

private void OnHealthChanged(int newValue)
{
    if (newValue <= 0)
        QueueFree();
}
}

```

Теперь при удалении врага подписка гарантированно рвётся, и никаких «призраков» не остаётся.

Другая крайность, в которую чуть не впал Артём, – использование событий как командного центра. Он хотел сделать глобальное событие `GameOver`, на которое подписывались бы UI, звук, сохранение, аналитика. Но тогда изменение порядка подписчиков или добавление нового могло бы сломать логику завершения. Вместо этого лучше оставить `GameOver` фактом, а конкретные действия (например, сохранение перед показом экрана) инициировать явным вызовом из одного места. События – для оповещения о свершившемся, а не для скрытой оркестрации. Это сохраняет поток управления прозрачным.

Шаг 4: асинхронность, которая живёт своей жизнью

Артём решает, что загрузка уровней должна быть плавной, и добавляет в кнопку «Начать игру» такой код:

```

public async void OnStartPressed()
{
    await Task.Delay(500);
    GetTree().ChangeSceneToFile("res://levels/level1.tscn");
}

```

Работает. Потом он добавляет загрузку с сервера ежедневных наград. Всё асинхронно. Но вдруг начинают появляться ошибки вида «Attempted to call a method on a freed instance». Оказывается, игрок нажимает кнопку, запускается пятисекундная асинхронная операция, а за это время он может передумать и выйти в главное меню. Сцена выгружается, узел кнопки уничтожается, но `Task` продолжает выполняться. Когда он завершается, код пытается вызвать `GetTree()` уже на несуществующем объекте. Катастрофа.

Артём усвоил урок и теперь всегда привязывает асинхронные операции к жизненному циклу узла-инициатора. Он вводит `CancellationTokenSource`, который отменяется в `_ExitTree`. Переписанный код загрузки уровня выглядит так:

```

public partial class LevelLoader : Node
{
    private CancellationTokenSource _cts;
    public async void LoadLevel(string path)
    {
        _cts = new CancellationTokenSource();
        try
        {

```



```

        // Здесь может быть реальный await загрузки ресурса или запроса.
        await Task.Delay(500, _cts.Token);
        if (!IsInsideTree()) return; // Дополнительная страховка.
        GetTree().ChangeSceneToFile(path);
    }
    catch (OperationCanceledException)
    {
        GD.Print("Загрузка уровня отменена");
    }
    finally
    {
        {
            _cts?.Dispose();
            _cts = null;
        }
    }
    public override void _ExitTree()
    {
        {
            _cts?.Cancel();
        }
    }
}

```

Правило простое: **асинхронная операция должна иметь хозяина**. Если хозяин умирает, операция отменяется или её результат игнорируется. Вдобавок Артём понял, что нельзя захватывать ссылки на узлы через `await` без проверки их актуальности. Лучше захватить `GetInstanceId()` и перепроверить объект после возобновления.

Шаг 5: глубокие иерархии и преждевременные абстракции

В какой-то момент Артём увлёкся наследованием. Он создал класс `Character`, от него унаследовал `Player` и `Enemy`, а от `Enemy` – `FlyingEnemy` и `GroundEnemy`. Всё шло неплохо, пока не потребовалось сделать врага, который одновременно летает и стреляет. Наследование зашло в тупик: от кого наследоваться? Артём начал дублировать код, нарушив стройность иерархии. А когда он попытался вынести общую логику урона в базовый `Character`, наследники сломались, потому что ожидали разный порядок вызовов. Ошибка в том, что **наследование описывает отношение «is-a», а для «can-do» лучше подходят интерфейсы и композиция**. Вместо того чтобы строить хрупкую башню классов, Артём рассыпал поведение на роли: `IHealth`, `IMovement`, `IAttack`. Каждый враг стал собираться из компонентов, как из кубиков. Это позволило переиспользовать логику и избежать конфликтов.

Другая крайность – **преждевременная абстракция**. Напуганный ошибками, Артём чуть не начал оборачивать каждую мелочь в интерфейс и фабрику. Но вовремя остановился. Абстракция нужна тогда, когда у вас есть минимум две реальные реализации, которые действительно должны скрываться за общим контрактом. Если пока есть только один класс, дополнительные слои лишь усложнят чтение кода и затормозят разработку. Простое правило: не обобщайте на будущее – обобщайте тогда, когда боль от дублирования начинает перевешивать боль от абстракции.

Симптомы дрейфа и как вовремя остановиться

Вернёмся к нашему герою. В один прекрасный день Артём заметил, что добавление простого параметра в оружие требует править пять разных мест. Он остановился и вспомнил, что мы называли **радиусом изменений**: если правка одной фичи растекается по проекту, значит, границы модулей проведены неверно. Другие тревожные звоночки: появляются «обязательные ритуалы»: «прежде чем вызвать этот метод, не забудь проинициализировать вот то», множатся проверки `if (obj != null)` без явного понимания, почему объект может отсутствовать, классы получают названия `Helper`, `Manager`, `Utils`, под которыми скрывается всё что угодно.

Артём провёл ревизию и задал себе по каждому подозрительному модулю четыре вопроса, ставших для него теперь привычными:

1. Какую одну чёткую роль выполняет этот класс? (Если не можешь ответить одним предложением – дели.)
2. Зависит ли он от деталей реализации других классов или от стабильного контракта?
3. Кто владеет временем жизни его объектов и подписок?
4. Что сломается, если я перенесу этот модуль в другую сцену или проект? (Если слишком многое – значит, зависимости скрыты.)

Эти вопросы помогают удерживать систему в здоровом состоянии без перегрузки формализмом. Не бойтесь переписывать неудачные куски, пока они малы, потому что цена переделки растёт экспоненциально с размером кода. Идеальной архитектуры не существует, но есть архитектура, которая не мешает вам жить, а помогает. Вот к ней и стоит стремиться.

4

Структурирование игровой логики

4.1. Разделение ответственности (Separation of Concerns)

На старте любого проекта главное – чтобы всё работало. Вы набрасываете механики, проверяете гипотезы и собираете петлю геймплея. Такая свобода оправдана и даже необходима, потому что на этапе прототипа важна скорость проверки идей, а не архитектурная чистота. Но как только проект перерастает стадию эксперимента, риск смещается: теперь судьбу разработки определяет не столько работоспособность отдельных функций, сколько способ организации кода. Если логика размазана по проекту без ясных границ, любое изменение вызывает цепную реакцию, а отладка превращается в расследование. На этом рубеже появляется дисциплина, которую в инженерии называют **разделением ответственности** (Separation of Concerns).

4.1.1. *Concern как единица архитектурной устойчивости*

Слово «concern» часто переводят как «задача», «аспект» или «ответственность», но ни один перевод не передаёт всей сути. В архитектурном смысле concern – это не фича и не класс, а **кластер ответственности, объединённый общей причиной для изменений**. Представьте, что вы проектируете

систему здоровья персонажа. Правила расчёта здоровья (максимум, восстановление, неуязвимость) образуют один concern, потому что они будут меняться вместе, если вы решите переработать механику урона. Отображение полоски здоровья в интерфейсе – другой concern: он изменится, когда вы переделаете HUD, даже если правила останутся прежними. Сохранение состояния здоровья между сессиями – третий: он зависит от формата файлов и стратегии записи на диск.

Если вы поместите все три аспекта в один класс, они окажутся склеены. Тогда изменение интерфейса заставит вас трогать класс, отвечающий за сохранения, а изменение правил урона может неожиданно сломать UI. И наоборот, если вы искусственно разделите то, что всегда меняется синхронно, например разнесёте расчёт урона и проверку на смерть по разным модулям без веской причины, вы получите лишнюю фрагментацию. Суть разделения ответственности в том, чтобы найти естественные границы, внутри которых изменения локализованы, а между которыми – лишь узкие, контролируемые контракты.

Как понять, что граница проведена верно? Задайте себе два вопроса. Первый: «Если мне придётся переписать эту часть системы, сколько других мест я буду вынужден затронуть?» Если число ненулевое, но предсказуемо мало – граница работает. Второй вопрос: «Могу ли я объяснить, за что отвечает этот модуль, в одном предложении, не используя союзы “и”, “а также” и “или”?» Если нет, модуль, скорее всего, объединяет несколько разных concerns.

4.1.2. Три слоя архитектуры: представление, правила, инфраструктура

В игровых проектах ответственность естественным образом группируется в три крупных слоя, которые полезно выделять даже в небольшой кодовой базе. Они не обязаны лежать в разных папках или сборках, но должны существовать как архитектурные роли, которые вы осознаёте при проектировании.

Presentation logic – логика представления. Это всё, что отвечает за отображение состояния игры: HUD, меню, всплывающие подсказки, экраны загрузки, визуальные эффекты интерфейса. Задача слоя представления – перевести состояние системы в форму, понятную игроку. Он не должен принимать решения, влияющие на правила игры, и не должен знать, откуда берутся данные. Его дело – показать, а не вычислять. Когда вы меняете шрифт, переставляете кнопки или добавляете адаптивную вёрстку под разные экраны, меняется только этот слой. Если же правка UI тянет за собой изменения в коде правил, значит, граница была нарушена.

Gameplay logic – доменная логика, или правила игры. Это ядро, в котором живут инварианты: здоровье не может быть отрицательным, инвентарь не вмещает предметов сверх лимита, очки начисляются только при выполнении условий. Слой правил ничего не знает о шрифтах, кнопках и анимациях. Он не знает, в каком формате и куда сохраняются данные. Он оперирует по-

нениями предметной области: «здоровье», «урон», «предмет», «статус». Эта независимость позволяет переиспользовать одни и те же правила в разных контекстах – для одиночной и сетевой игры, для разных платформ, для автоматических тестов.

Infrastructure logic – логика обеспечения. Сюда попадают операции, которые связывают игру с внешним миром: сохранение и загрузка, сетевое взаимодействие, работа с файловой системой, аналитика, конфигурация, логирование. Инфраструктура всегда изменчива: сегодня вы сохраняете прогресс в JSON на диске, завтра – в облачную базу данных; сегодня игра работает офлайн, завтра появляется мультиплеер. Если слой правил напрямую зависит от инфраструктуры, любая такая замена вызывает каскад правок в самом ядре. Поэтому инфраструктура обслуживает домен, а не определяет его: она получает запрос «сохрани этот снимок» или «дай настройку сложности» и возвращает результат, скрывая детали реализации.

Взаимодействие между слоями подчиняется строгому протоколу, который полезно держать в голове как чек-лист. Слой представления формирует **намерение** (intent): игрок нажал кнопку «использовать предмет», UI вызывает соответствующий метод в слое правил, не проверяя допустимость и не меняя состояние самостоятельно. Слой правил обрабатывает намерение, проверяет условия, изменяет состояние и при необходимости публикует **факт** (event) – «здоровье изменилось», «предмет использован». Слой представления, подписанный на эти факты, обновляет отображение. Если требуется сохранить игру, слой правил не обращается к файловой системе напрямую; он публикует факт «состояние изменилось», а инфраструктурный обработчик реагирует на него и запускает сохранение. Такой поток данных однонаправлен и предсказуем.

4.1.3. Цена смешения: как невинные *shortcuts* превращаются в системный долг

Легко понять теорию, но на практике границы размываются незаметно. Вы пишете прототип, и всё кажется естественным: кнопка инвентаря напрямую лечит игрока, удаляет предмет и обновляет HUD. Код короткий и работает. Проблема не в том, что код «неправильный», а в том, что он связывает три разных concerns в одном месте, и каждое последующее требование усиливает этот клубок.

Представьте, что через месяц вы добавляете горячую клавишу для использования предмета. Поскольку логика лечения зашита в UI-кнопке, вам приходится либо дублировать её в обработчике клавиши, либо хитро эмулировать нажатие кнопки из кода. Дублирование немедленно создаёт риск расхождения: условия, проверки, побочные эффекты копируются с ошибками. Через некоторое время появляются баги в духе «лечение с горячей клавиши не учитывает бафф, который учитывает кнопка». Ещё через месяц вы решаете, что использовать предмет можно только вне боя. Теперь нужно найти

и поправить все места, где иницируется лечение, – и не забыть, что одно из них прячется глубоко в сценарии туториала, где проходят автоматические действия.

Теперь добавим инфраструктуру. Дизайнер просит, чтобы при использовании редкого предмета игра автоматически сохранялась, предотвращая потерю прогресса. Вы добавляете вызов сохранения прямо в метод лечения. Казалось бы, всё логично. Но через неделю выясняется, что сохранение при лечении замедляет отклик на слабых устройствах, и тимлид просит вынести сохранение в фоновый поток или делать его не чаще раза в минуту. Поскольку вызов зашит в доменной логике, оптимизация инфраструктуры требует переписывания правил. Игра начинает сохраняться не после каждого лечения, а по таймеру, но в коде по-прежнему остаётся старый прямой вызов, который теперь конфликтует с новой системой.

Такие цепочки – классический пример того, что инженеры называют ripple effect: изменение одной детали распространяется кругами по всей системе, затрагивая модули, которые по смыслу не должны были пострадать. Разделение ответственности не устраняет необходимость взаимодействия между частями игры, но заставляет проводить его через узкие, стабильные контракты. Благодаря этому радиус изменений остаётся под контролем.

4.1.4. От хаоса к границам: рефакторинг на примере инвентаря

Рассмотрим конкретный код, который на первый взгляд кажется нормальным, но содержит все признаки смешения слоёв:

```
using Godot;
public partial class InventoryPanel : Control
{
    [Export] private NodePath _playerPath;
    private Player _player;
    public override void _Ready()
        => _player = GetNode<Player>(_playerPath);
    private void OnUsePotionPressed()
    {
        _player.Health += 25;
        _player.Inventory.Remove("Potion");
        _player.SaveNow();
        GetNode<Label>("%HealthLabel").Text = _player.Health.ToString();
    }
}
```

Здесь UI-панель напрямую меняет здоровье персонажа (правила), удаляет предмет из инвентаря (правила), запускает сохранение (инфраструктура) и обновляет текстовую метку (представление). Одна кнопка выполняет работу всех трёх слоёв. На этапе прототипа это допустимо, но как только проект начинает расти, такой код становится источником постоянных проблем.

Первым шагом к порядку будет отделение намерения от исполнения. Кнопка должна не управлять правилами, а лишь сообщать о желании игрока. Мы вводим в слой правил метод `UseItem`, который принадлежит инвентарю или персонажу:

```
// В классе Player или Inventory.
public bool UseItem(string itemId)
{
    if (!Inventory.Contains(itemId))
        return false;
    var item = Inventory.Get(itemId);
    if (!item.IsUsable)
        return false;
    // Применяем эффект предмета – это знает только сам предмет.
    item.Apply(this);
    Inventory.Remove(itemId);
    return true;
}
```

Теперь панель инвентаря вызывает только `_player.UseItem(«Potion»)` и ничего не знает о том, как именно предмет влияет на здоровье и сколько здоровья добавится. Более того, метод `UseItem` возвращает булево значение, позволяя UI показать ошибку, если предмет использовать нельзя, – и это решение остаётся в UI, не проникая в домен.

Следующий шаг – убрать прямое сохранение из доменной логики. Вместо вызова `SaveNow` внутри `UseItem` мы делаем так, чтобы класс здоровья или инвентаря при изменении своего состояния публиковал событие. Инфраструктурный сервис сохранения подписывается на эти события и сам решает, когда и как выполнять запись:

```
public class GameSaveService
{
    public GameSaveService(Player player)
    {
        player.Health.Changed += (_, _) => Save(player);
        player.Inventory.Changed += (_, _) => Save(player);
    }
    private void Save(Player player)
    {
        // Сериализация и запись в файл – забота инфраструктуры.
    }
}
```

Теперь, если вы захотите изменить стратегию сохранения (например, сохранять не чаще раза в секунду или отправлять данные на сервер), весь код изменений локализуется внутри `GameSaveService`. Класс `Player` об этом не знает.

Наконец, UI перестаёт вручную обновлять текстовые метки. Вместо этого `HealthBar` как отдельный компонент подписывается на событие `Changed` у здоровья и сам обновляет своё отображение. При изменении модели дан-

ных представление реактивно подтягивается, и кнопка лечения полностью освобождается от ответственности за внешний вид.

Такой рефакторинг не требует сложных фреймворков. Это просто последовательное применение принципа: каждый кусок кода должен отвечать на вопрос «к какому слою относится эта операция?» и не выполнять работу чужого слоя.

4.1.5. Модульное мышление и отладка в разделённой системе

Когда слои разделены, отладка перестаёт быть гаданием. Вы видите, что здоровье персонажа не обновляется после использования предмета, и вместо поиска по всему проекту проверяете три точки: не заблокирован ли вызов `UseItem` в UI, корректно ли отработал метод предмета и подписан ли `HealthBar` на событие изменения. Причина почти всегда локализуется в одной из этих границ. Без разделения вы бы искали баг, который мог прятаться в кнопке, в таймере, в анимации или в формате сохранения, потому что все эти модули могли напрямую менять здоровье. Кроме того, разделение ответственности снижает порог входа для новых участников. Разработчику, которому поручили переработать интерфейс, не нужно разбираться в механике урона и сетевых протоколах. Он ограничен слоем представления, а контракты с другими слоями служат ему документацией. Аналогично программист, оптимизирующий сохранение, не обязан изучать, как работают кнопки инвентаря. Важно не впадать в крайность и не плодить слои ради самих слоёв. Если вы пишете крошечную игру-кликер, которая никогда не будет развиваться, разделение на три слоя может быть избыточным. Но как только в проекте появляется хотя бы два источника ввода (кнопка и клавиша), два потребителя состояния (HUD и звук) или два формата сохранения – выгода от проведённых границ многократно окупает усилия по их созданию. Начинайте с малого: выносите в отдельные классы здоровье и инвентарь, используйте события для обновления UI и никогда не вызывайте `File.WriteAllText` из метода `TakeDamage`. Остальное приложится по мере роста.

4.2. Композиция поведения: от монолитных сущностей к набору ролей

Начинающие разработчики почти всегда описывают игровые объекты через наследование. Есть базовый класс `Enemy`, от него наследуется `FlyingEnemy`, а от того – `ShootingFlyingEnemy`. Пока комбинаций мало, иерархия выглядит стройной и понятной. Но игры имеют свойство разрастаться, и вскоре вы обнаруживаете, что вам нужен враг, который одновременно летает, стреля-

ет очередями, но не имеет брони. Затем – летающий враг без стрельбы, но с регенерацией. Потом – наземный враг со щитом и стрельбой. Каждая новая комбинация требует нового класса, а изменение базового поведения ломает половину наследников. Архитектура превращается в запутанное дерево, где правки даются всё тяжелее, а дублирование кода растёт экспоненциально. Компонентный подход предлагает радикально иную точку зрения: игровая сущность – не звено в иерархии наследования, а контейнер, собираемый из независимых модулей поведения. Враг перестаёт быть `FlyingShootingEnemy`; вместо этого он состоит из компонента движения, реализующего полёт, компонента атаки, отвечающего за стрельбу, и компонента здоровья, управляющего получением урона. Каждый модуль – самостоятельная единица с чёткой ответственностью, собственными инвариантами и контрактом. Модули ничего не знают друг о друге, а их взаимодействие настраивается явно, через события или через координатор-контейнер. Такой подход снижает связность, делает переиспользование кода естественным, а расширение системы – безопасным.

4.2.1. Роль как архитектурная единица: контракт, состояние и границы

Центральное понятие компонентного мышления – роль. Роль описывает не конкретный объект, а тип поведения, который может быть присущ разным объектам. «Получение урона» – это роль. «Перемещение в пространстве» – роль. «Способность атаковать» – роль. Каждая роль оформляется в виде компонента – класса, который инкапсулирует связанные данные и операции, защищает инварианты и предоставляет внешнему миру узкий, стабильный контракт.

Рассмотрим компонент здоровья. Это не просто число, а полноценный модуль, отвечающий за границы допустимых значений, реакцию на лечение и урон, оповещение об изменениях. В правильно спроектированном компоненте все поля приватны, а мутация состояния возможна только через публичные методы, гарантирующие целостность. Внешний код не может напрямую написать `health.Current = 999` – он обязан пройти через `TakeDamage` или `Heal`, внутри которых и сосредоточены все проверки. Это та же инкапсуляция, которую мы обсуждали в разделе 3.2, но применённая к модулю поведения, а не к целой сущности.

Код компонента здоровья в минимальном, но достаточном для практического использования виде:

```
public class HealthComponent
{
    private int _current;
    private readonly int _max;
    public int Current => _current;
    public int Max => _max;
```

```

public bool IsDead => _current <= 0;
public event Action<int, int> Changed;
public HealthComponent(int max, int initial)
{
    _max = Math.Max(1, max);
    _current = Math.Clamp(initial, 0, _max);
}
public void TakeDamage(int amount)
{
    if (amount <= 0) return;
    _current = Math.Max(0, _current - amount);
    Changed?.Invoke(_current, _max);
}
public void Heal(int amount)
{
    if (amount <= 0) return;
    _current = Math.Min(_max, _current + amount);
    Changed?.Invoke(_current, _max);
}
}

```

Обратите внимание: компонент не знает, к какой сущности он прикреплен, не зависит от Godot API и не обращается к файловой системе или сети. Он владеет только своим состоянием и правилами его изменения. Событие `Changed` позволяет внешним наблюдателям – UI, звуковым эффектам, системе сохранения – реагировать на факт изменения, не зная вызвавшей его причины. Это и есть граница ответственности в действии: компонент делает свою работу и сообщает о результате, но не управляет последствиями.

Компонент движения демонстрирует другой тип ответственности – интеграционный. Он находится на стыке доменной логики и движка, преобразуя абстрактное намерение в конкретные вызовы физического API:

```

public class MovementComponent
{
    private readonly CharacterBody2D _body;
    private Vector2 _velocity;
    public Vector2 Velocity
    {
        get => _velocity;
        set => _velocity = value;
    }
    public MovementComponent(CharacterBody2D body)
    {
        _body = body;
    }
    public void ApplyMovement(float delta)
    {
        _body.Velocity = _velocity;
        _body.MoveAndSlide();
    }
}

```

Здесь уже есть зависимость от `CharacterBody2D`, и это нормально. Компонент не претендует на роль чисто доменного модуля – его задача в том, чтобы инкапсулировать детали взаимодействия с физикой движка и предоставить остальной системе простой интерфейс: «установи скорость – примени движение». Если в будущем вы перейдёте на другую физическую модель или на кастомное движение через `CharacterBody3D`, изменения локализуются внутри этого компонента.

Важно понимать разницу между компонентом и обычным полем класса. Если вы просто поместите `int health` внутрь `Player` и будете менять его отовсюду, вы не получите компонентной архитектуры – вы получите агрегацию данных, не защищённую инвариантами. Компонент обязан владеть своей ответственностью полностью: он сам следит за корректностью состояния, сам публикует события и сам предоставляет контракт. Внешний мир взаимодействует с ним только через публичные методы, не имея доступа к внутренним деталям.

4.2.2. Контейнер сущности: сборка, оркестрация и связывание

Когда компоненты готовы, встаёт вопрос: кто владеет их экземплярами и координирует работу? Эту роль выполняет контейнер – класс, представляющий игровую сущность. В Godot контейнером обычно выступает узел сцены: `Player`, `Enemy`, `DestructibleCrate`. Но сам контейнер больше не содержит игровой логики. Он лишь создаёт компоненты, передаёт им необходимые зависимости и подписывается на их события, связывая роли в единое целое.

Рассмотрим сборку игрока:

```
public partial class Player : CharacterBody2D
{
    public HealthComponent Health { get; private set; }
    public MovementComponent Movement { get; private set; }
    private AttackComponent _attack;
    public override void _Ready()
    {
        Health = new HealthComponent(max: 100, initial: 100);
        Movement = new MovementComponent(this);
        _attack = new AttackComponent();
        // Связываем компоненты: смерть отключает движение и атаку.
        Health.Changed += (current, max) =>
        {
            if (current <= 0)
            {
                Movement.Velocity = Vector2.Zero;
                _attack.Enabled = false;
            }
        };
    }
}
```

```

    public override void _PhysicsProcess(double delta)
    {
        Movement.ApplyMovement((float)delta);
    }
    public void Move(Vector2 direction, float speed)
    {
        Movement.Velocity = direction * speed;
    }
    public void TakeHit(int damage)
    {
        Health.TakeDamage(damage);
    }
}

```

Класс Player стал тонким и предсказуемым. Он не вычисляет урон, не управляет анимациями смерти и не обновляет UI. Он лишь предоставляет внешнему коду (например, скрипту ввода или системе столкновений) осмысленные методы – Move, TakeHit – и делегирует их соответствующим компонентам. Если потребуется изменить поведение при смерти, мы модифицируем лямбду в _Ready или переносим эту логику в отдельный компонент DeathHandler. Если нужно добавить рывок, мы расширяем MovementComponent, не трогая Player.

Создание врага столь же лаконично. Мы не наследуемся от Enemy, а собираем новую сущность из тех же компонентов, но с другими параметрами:

```

public partial class Enemy : CharacterBody2D
{
    public HealthComponent Health { get; private set; }
    private MovementComponent _movement;
    public override void _Ready()
    {
        Health = new HealthComponent(max: 50, initial: 50);
        _movement = new MovementComponent(this);
        Health.Changed += (current, max) =>
        {
            if (current <= 0) QueueFree();
        };
    }
    public override void _PhysicsProcess(double delta)
    {
        _movement.ApplyMovement((float)delta);
    }
}

```

Разница между игроком и врагом свелась к разной конфигурации компонентов и разной реакции на событие смерти. Логика здоровья и движения не продублирована – она переиспользована. Если понадобится создать разрушаемый ящик, мы просто добавим HealthComponent на соответствующий узел. Никакого копирования кода, никаких искусственных базовых классов.

Критически важно, чтобы компоненты не знали друг о друге и не дергали методы соседей напрямую. Если AttackComponent внутри своей ло-

гики начнёт искать `HealthComponent` у цели и вызывать `TakeDamage`, возникнет жёсткая связь, и атаку нельзя будет использовать в контексте, где у цели другая модель здоровья. Правильный подход – оповещать координатор (контейнер или специализированную шину событий) о факте атаки, а координатор уже решает, как применить урон к целевым объектам. Например, `AttackComponent` при выполнении атаки генерирует событие `TargetsHit(IEnumerable<Node> targets)`, а контейнер слушает его и для каждой цели вызывает метод `TakeDamage`, если у цели есть `HealthComponent`. Так атака остаётся независимой от реализации здоровья и может взаимодействовать с любым объектом.

4.2.3. Гранулярность компонентов и предотвращение архитектурного шума

У компонентного подхода есть своя ловушка: увлечение дроблением. Разработчик, вдохновлённый идеей чистых ролей, начинает создавать отдельный компонент на каждую микроскопическую деталь. Максимальное здоровье выделяется в `MaxHealthComponent`, текущая скорость – в `VelocityComponent`, флаг оглушения – в `StunComponent`. Внешне система остаётся компонентной, но фактически распадается на множество мелких классов, которые вынуждены постоянно координироваться, потому что реальная ответственность не разделена, а раздроблена.

Критерий гранулярности – **инвариант**. Если два значения всегда должны быть согласованы, например текущее и максимальное здоровье, они принадлежат одному компоненту, иначе вы будете вынуждены синхронизировать их вручную и рискуете получить расхождение. Другой критерий – **причина изменений**. Если два аспекта поведения всегда меняются вместе (добавили тип урона – добавили и сопротивление к нему), их искусственное разделение лишь увеличит количество мест, которые нужно править. Компонент должен быть настолько большим, чтобы защищать свой инвариант, и настолько маленьким, чтобы не смешивать независимые причины изменений.

Хорошая проверка: можете ли вы объяснить ответственность компонента одним предложением? «Управляет текущим здоровьем, обеспечивая его нахождение в допустимом диапазоне и оповещая об изменениях» – это одна ответственность. «Управляет здоровьем и воспроизводит звуки получения урона» – уже две, и их стоит разделить, потому что звук может измениться при смене аудиосистемы, а здоровье – при ребалансе.

4.2.4. Компонентное мышление в контексте Godot: две плоскости композиции

Godot изначально построен на идее композиции: сцены состоят из узлов, каждый узел выполняет узкую функцию – `Sprite2D` отображает текстуру, `Col`

lisionShape2D задаёт форму, AudioStreamPlayer проигрывает звук. Это низкоуровневая, визуальная композиция. Компонентное мышление, которое мы обсуждаем, работает на уровень выше – на уровне доменных правил. Оно не заменяет систему узлов, а дополняет её.

Вы можете реализовать компонент двумя способами. Первый – как обычный C#-класс, не унаследованный от Node, который получает необходимые движковые зависимости через конструктор или метод инициализации. Такой подход максимально отделяет правила от инфраструктуры, упрощает тестирование и позволяет переиспользовать компонент вне Godot. Второй способ – как узел-наследник, если компонент тесно интегрирован с деревом сцены и должен участвовать в его жизненном цикле. Например, компонент, управляющий визуальными эффектами, может быть Node со своими дочерними спрайтами и анимациями.

Часто разумно комбинировать подходы: доменные компоненты (HealthComponent, InventoryComponent) оставлять чистыми C#-классами, а интеграционные (VisualEffectsComponent, InputComponent) делать узлами. Граница между ними должна быть явной: доменный компонент публикует событие, а интеграционный узел подписывается на него и преобразует в визуальные или звуковые реакции. Тогда смена визуального стиля игры не затронет правила, а ребаланс урона не сломает анимации. Когда вы мыслите компонентами, вы начинаете видеть игру не как набор объектов (Player, Enemy, Boss), а как таблицу ролей и их комбинаций. Такой взгляд особенно полезен на этапе прототипирования, когда дизайнеры активно экспериментируют со свойствами персонажей. Вместо того чтобы плодить классы под каждую итерацию, вы меняете конфигурацию сборки – и получаете новый вариант поведения за минуты, а не за часы. Эта гибкость – прямое следствие архитектурного решения, принятого на старте.

4.3. Наследование и композиция: два способа выразить вариативность

В предыдущем разделе мы утвердились в мысли, что игровая сущность – это набор компонентов поведения. Но как только вы начинаете собирать объекты из частей, возникает архитектурный вопрос: каким механизмом выражать различия между этими объектами? В C# для этого есть два фундаментальных инструмента – наследование и композиция. Их часто противопоставляют, называя наследование устаревшим и хрупким, а композицию – современной и правильной. Однако в инженерной практике оба механизма решают разные задачи, и слепое следование моде лишь порождает новые проблемы. Ценность представляет не «правильный выбор», а понимание, в какой момент какой инструмент укрепляет архитектуру, а в какой – размывает её.

4.3.1. Два отношения: «является» и «содержит»

Наследование выражает отношение **is-a** – «является». Класс `FlyingEnemy` является разновидностью `Enemy`. Это отношение хорошо моделирует ситуации, где существует устойчивая общая природа объектов. Если все враги в игре действительно разделяют общие инварианты, например у всех есть здоровье, все обязаны реагировать на смерть по единому протоколу и все должны регистрироваться в системе целей, разумно вынести этот общий код в базовый класс. Тогда наследники получают его бесплатно, а вызывающий код работает с ними полиморфно, не различая конкретные подтипы. Это классика объектно-ориентированного дизайна, и в правильном контексте она прекрасно работает. Композиция, напротив, опирается на отношение **has-a** – «содержит». Объект не «является» чем-то, а «обладает» определёнными возможностями. Враг не «является летающим врагом», он «обладает компонентом полёта». Его поведение складывается из поведения частей, а не определяется предком в иерархии. Это принципиально иной способ организации зависимостей: изменчивость переносится из структуры типов в структуру сборки.

Чтобы почувствовать разницу, представьте, что вы моделируете дверь. Через наследование вы могли бы создать класс `Door`, а от него унаследовать `LockedDoor`, `TrappedDoor`, `LockedTrappedDoor`. Пока свойств мало, это терпимо. Но если позже понадобится дверь с таймером, дверь, открывающаяся только ночью, и дверь-портал, иерархия раздуется, а любое изменение базового поведения (например, добавление анимации открытия) потребует проверки всех наследников. В композиционной модели дверь – это контейнер, в который можно поместить компонент `Lock`, компонент `Trap`, компонент `Timer`. Комбинаторика собирается из кубиков, и добавление нового свойства не затрагивает существующие. С практической точки зрения выбор между «is-a» и «has-a» сводится к вопросу: **что будет изменяться чаще – общая природа объектов или их индивидуальные возможности?** Если природа стабильна (все враги всегда умирают одинаково, и это не поменяется через год), наследование даёт простую и понятную структуру. Если же возможности наращиваются, комбинируются и переключаются динамически – композиция избавляет от каскадных правок.

4.3.2. Ловушки наследования: неявные контракты и комбинаторный рост

Проблемы начинаются, когда наследование применяют не для отражения устойчивой природы, а для добавления новых «фишек». Предположим, у нас есть враг. Потом появляется стреляющий враг – мы наследуем `ShootingEnemy` от `Enemy`. Потом летающий – `FlyingShootingEnemy` от `ShootingEnemy`. Потом бронированный – `ArmoredFlyingShootingEnemy` от `FlyingShootingEnemy`. Через месяц в игре нужен летающий враг, который не стреляет, но накладывает

проклятие. Мы не можем унаследоваться от `FlyingShootingEnemy`, потому что он стреляет, а проклятие – другой тип атаки. Приходится создавать `FlyingCursingEnemy` и дублировать код полёта. Если нужно ещё и отключить полёт при ранении, начинают плодиться флаги в базовом классе, и логика расползается. Этот комбинаторный взрыв – прямое следствие того, что иерархия классов пытается отразить не родовидовые отношения, а набор независимых ролей. Летание и стрельба – не подтипы врага, а ортогональные поведенческие оси. Когда они скрещиваются в дереве наследования, каждый новый вариант требует нового класса. А поскольку базовый класс вынужден предоставлять всё более общую функциональность, он превращается в хрупкую точку, где собраны политики и исключения, нужные разным наследникам.

Неявные контракты – другая сторона той же медали. В иерархии порядок вызовов методов часто превращается в устную договорённость, которую невозможно проверить компилятором. Вам нужно помнить, что сначала вызывается `base.Update`, потом ваша логика, и ни в коем случае не наоборот. Если в базовый класс добавляют новый виртуальный метод, все наследники должны его переопределить в нужном порядке – а если не переопределят, получат поведение по умолчанию, которое может быть некорректным. Чем глубже дерево, тем больше таких неформальных обязанностей и тем выше цена ошибки.

Рассмотрим конкретный код, демонстрирующий эту проблему:

```
public abstract class Enemy
{
    public virtual void Update(float dt) { }
    public virtual void TakeDamage(int amount) { }
}
public class RangedEnemy : Enemy
{
    public override void Update(float dt) { /* aim + shoot */ }
}
public class FlyingRangedEnemy : RangedEnemy
{
    public override void Update(float dt)
    {
        // Сначала летим, потом вызываем базовую стрельбу
        base.Update(dt);
    }
}
public class BossFlyingRangedEnemy : FlyingRangedEnemy
{
    public override void TakeDamage(int amount)
    {
        // Фазовая логика босса, которая должна выполняться после базовой?
        base.TakeDamage(amount);
    }
}
```

Внешне всё аккуратно, но неявных соглашений уже несколько. В каком порядке вызывать `base.Update`? Что делает базовый `TakeDamage` и не ломает ли босса вызов его до проверки фаз? Если завтра в `Enemy` добавят виртуальный `OnSpawn`, все наследники обязаны его поддерживать, иначе новая логика инициализации просто проигнорируется. Такая структура крайне чувствительна к изменениям в верхних уровнях иерархии.

4.3.3. Композиция как инструмент вариативности и динамической замены

Композиция решает описанные проблемы, заменяя древовидную таксономию плоской сборкой ролей. Вернёмся к нашему врагу и выразим его через компоненты:

```
public interface IMoveBehavior { void Tick(float dt); }
public interface IAttackBehavior { void Tick(float dt); }
public sealed class Enemy
{
    private IMoveBehavior _move;
    private IAttackBehavior _attack;
    public Enemy(IMoveBehavior move, IAttackBehavior attack)
    {
        _move = move;
        _attack = attack;
    }
    public void Update(float dt)
    {
        _move.Tick(dt);
        _attack.Tick(dt);
    }
    public void SetAttack(IAttackBehavior newAttack)
    {
        _attack = newAttack;
    }
}
```

Класс `Enemy` больше не знает, как именно он движется или атакует. Эти знания инкапсулированы в отдельных объектах, реализующих интерфейсы `IMoveBehavior` и `IAttackBehavior`. Если завтра понадобится враг, который летает, но не стреляет, – мы передадим `FlyMove` и `NullAttack`. Если нужен стреляющий и ходящий – `WalkMove` и `RangedAttack`. Комбинации создаются без новых классов. Более того, композиция естественно поддерживает динамическое изменение поведения. Метод `SetAttack` позволяет поменять атаку во время игры: подобрал новое оружие, получил бафф, перешёл в режим ярости. В иерархической модели для этого пришлось бы вводить флаги и условные ветки внутри класса, размывая инварианты. Здесь же стратегия поведения – это просто заменяемый объект. Изоляция изменений тоже становится гораз-

до более локальной. Если мы хотим изменить алгоритм полёта, мы правим класс `FlyMove`. Враги, использующие его, не требуют перекомпиляции и не нуждаются в переписывании. Если мы хотим добавить новый тип атаки, мы создаём новую реализацию `IAttackBehavior` и внедряем её в контейнер. Вариативность наращивается горизонтально, а не через углубление иерархии.

Разумеется, композиция не бесплатна. Появляется больше объектов, и системе нужно собирать явно: где-то в коде вы должны создать экземпляры компонентов и передать их врагу. Это требует дисциплины, но затраты окупаются на стадии поддержки. Кроме того, важно не перестараться с дроблением: если вы создадите отдельный интерфейс для каждого микроскопического параметра, сборка станет громоздкой. Ориентиром служит инвариант: роли, которые всегда меняются вместе, лучше держать в одном компоненте.

4.3.4. Реалистичный гибрид и практические ориентиры

В реальном проекте почти никогда не бывает «только наследования» или «только композиции». Здоровая архитектура использует наследование там, где оно действительно отражает устойчивую природу вещей, а композицию – там, где поведение вариативно и должно комбинироваться. Например, все узлы в `Godot` наследуются от `Node` – это осмысленная иерархия, потому что каждый узел действительно «является» элементом дерева сцен с общим жизненным циклом. Но доменная логика игры почти всегда богаче, и моделировать её исключительно через наследование – значит покупать на иллюзию простоты.

Как понять, что вы зашли не туда? Есть несколько верных признаков. Если новый класс создаётся только ради добавления одного флага или режима – вы используете наследование как костыль для ветвления, и композиция здесь уместнее. Если вы ловите себя на мысли «сначала вызвать базовый метод, потом мой», а через месяц забываете, в каком порядке, – это симптом неявного контракта. Если изменение в базовом классе заставляет вас править десяток наследников – радиус изменений неприемлемо велик. Все эти сигналы указывают на то, что пора выносить вариативное поведение в отдельные компоненты.

Рефакторинг от иерархии к композиции не обязан быть катастрофическим. Выберите одну ось изменчивости, которая порождает больше всего проблем (обычно это атака или движение), и вынесите её в интерфейс с реализациями. Затем удалите соответствующие виртуальные методы из базового класса и передавайте реализации через конструктор или сеттер. Повторите для следующей оси. Постепенно базовый класс станет тонким, а большинство расширений перекоچуют в компоненты. Такой пошаговый переход сохраняет проект рабочим и не требует одномоментного переписывания всего. На практике это означает, что наследование остаётся полезным для инфраструктурных слоёв: общие контейнеры, адаптеры движка, базовые

типы узлов. Композиция же обслуживает доменный слой, где требования меняются от билда к билду, а разнообразие сущностей растёт с каждым новым дизайнерским решением. Удерживая эти инструменты в их естественных границах, вы получаете архитектуру, которая не сопротивляется изменениям, а охотно их принимает.

4.4. Организация больших проектов: от стихийного кода к управляемой структуре

На ранних этапах разработки, когда весь проект умещается в дюжине файлов, а в команде один или два человека, организация кода кажется делом десятым. Гораздо важнее быстро проверить механику, настроить управление и увидеть, как игра откликается на действия игрока. В этот период любой порядок – или его отсутствие – работает, потому что цена навигации нулевая: вы помните, где что лежит, и держите всю архитектуру в голове. Но игры растут неравномерно. Появляется больше механик, затем интерфейс, потом сохранения, аналитика, сетевой режим. В какой-то момент файлов становится не десятки, а сотни, и проект из знакомого района превращается в мегаполис без указателей. Организация кода перестаёт быть вопросом вкуса и превращается в инженерную дисциплину, от которой напрямую зависят скорость разработки и стоимость ошибок.

4.4.1. От прототипа к сотням файлов: три эффекта разрушения порядка

Типичный проект-прототип в Godot устроен незамысловато: в корне проекта лежат папки Scenes, Scripts, Assets. Внутри Scripts файлы появляются по мере необходимости: сначала Player.cs, потом Enemy.cs, следом HealthBar.cs и SaveManager.cs. Пока игра мала, такая группировка по типу файла не вызывает трудностей. Вы знаете, что за здоровье отвечает Player, а за запись прогресса – SaveManager, и этого достаточно.

Первый тревожный звонок – **деградация навигации**. Вы точно помните, что логика начисления очков где-то есть, но уже не можете с ходу сказать, где именно: она расплзлась между Player, GameManager и HUD. Поиск превращается в ползание по папкам и использование глобального поиска по всем файлам. Разработка замедляется, хотя количество кода ещё невелико.

Второй эффект – **конфликтующие изменения**. Если разные люди, или даже один человек в разное время, постоянно правят одни и те же файлы, что-то идёт не так. Класс Player начинает отвечать и за движение, и за ин-

вентарь, и за отображение здоровья, и за сохранение состояния – потому что «всё равно он центральный». Любая новая фича требует трогать этот файл, и в нём сталкиваются правки, не связанные по смыслу. Архитектурно это означает, что файл перестал быть модулем – он стал общей тетрадью.

Третий, самый разрушительный эффект – **смещение слоёв как норма**. Рядом лежат UI-скрипты, доменные правила и инфраструктура; они ссылаются друг на друга без ограничений. В результате границы между «что является истиной игры» и «как это выглядит» растворяются. Поменять интерфейс становится страшно, потому что можно сломать правила. Перенести логику сохранения в фоновый поток – невозможно, потому что вызов `File.WriteAllText` зашит прямо в методе `TakeDamage`. Код не столько работает, сколько балансирует между случайными зависимостями.

В этот момент и приходит понимание: структура нужна не для красоты, а для выживания. Она должна сообщать, где лежит ответственность, какие зависимости разрешены и куда смотреть, когда что-то сломалось.

4.4.2. Слои, фичи и вертикальные срезы: поиск осей организации

Когда мы говорим «организовать проект», на самом деле мы решаем задачу группировки кода по какому-то принципу. Самых распространённых принципа два, и каждый имеет свои сильные стороны.

Группировка по слоям (layered organisation) подчиняется архитектурным ролям. Вы создаёте отдельные неймспейсы или папки для доменной логики, интеграции с движком, представления и инфраструктуры. Например, `Game.Domain` содержит классы `Health`, `Inventory`, `ScoreCounter` – они не знают о `Godot` и UI. `Game.Presentation` живёт отдельно и только читает состояние из домена, не меняя его. `Game.Infrastructure` скрывает детали сериализации и сетевых запросов. Такая структура предельно ясно выражает границы ответственности. Глядя на неё, вы можете мгновенно сказать: «Вот истина, вот способ её показать, вот способ сохранить». Нарушение границы видно невооружённым глазом – если внутри `Game.Domain` появляется `using Godot`, это красный флаг. Однако у этого подхода есть недостаток: фича, например «система квестов», окажется размазанной по всем слоям. Чтобы добавить новое свойство квеста, вам придётся пройти по домену, UI и, возможно, инфраструктуре, что увеличивает радиус изменений.

Группировка по фичам (feature-based) действует наоборот: всё, что относится к «инвентарю», лежит в одном месте. Папка `Inventory` содержит и доменный класс `Inventory`, и его визуальное представление `InventoryPanel`, и адаптер для сохранения `InventorySaveAdapter`. Когда вы работаете над инвентарём, весь код перед глазами. Локальность изменений максимальна – правка интерфейса или правил не требует прыжков по проекту. Но цена – дублирование инфраструктурных решений. Каждая фича может начать изобретать свой способ сохранения или свой способ обращения к движку. Без

дисциплины фичи превращаются в изолированные островки, которые ничего не знают друг о друге, но при этом бессистемно тянут общие зависимости.

На практике устойчивее всего работает **гибрид**, который часто называют вертикальными срезами. Вертикальный срез – это фича, организованная так, чтобы внутри неё самой соблюдались архитектурные слои. Например, срез Inventory содержит собственное доменное ядро (правила и состояние инвентаря), собственные адаптеры к движку (представление предметов в UI через Godot-ноды) и собственные точки интеграции с инфраструктурой (контракт InventorySave, реализованный где-то в общем слое сохранений). Но при этом доменное ядро инвентаря ничего не знает о нодах, а его публичные методы не тащат за собой зависимости от Sprite2D или Label. Срез замкнут по смыслу, но открыт по контрактам. Такой подход даёт главное: вы получаете локальность изменений, не разрушая архитектурные границы. Когда дизайнер просит переработать интерфейс инвентаря, вы работаете внутри среза и не трогаете общие инфраструктурные сервисы. Когда нужно изменить правила инвентаря, правки локализованы в доменном ядре. А общие модули (сохранения, аналитика, конфигурация) остаются стабильными, потому что срезы зависят от них через интерфейсы.

Как выбрать между этими подходами? Ориентируйтесь на два триггера. Если вы замечаете, что добавление новой фичи требует создавать файлы во всех папках слоёв по очереди – вероятно, вам нужен вертикальный срез. Если же вы видите, что внутри разных фич начинают возникать одинаковые утилиты и адаптеры – пора выделять общий слой. Проект не обязан следовать одному принципу догматически; он может начать с грубого слоистого разделения, а по мере роста вычленять фичи в собственные пространства.

4.4.3. Границы модулей, неймспейсы и конвенции: удержание порядка в долгой перспективе

Структура папок – лишь внешнее проявление. Настоящая организация живёт в **неймспейсах** и в **дисциплине зависимостей**. Если вы просто раскидали файлы по красивым папкам, но внутри каждый класс объявлен public и свободно расхватывается соседями, порядок рухнет через месяц. Неймспейсы в C# – это и навигационный словарь, и сигнал компилятору. Хорошая конвенция: корневой неймспейс проекта, например MyGame, и далее осмысленные ветви – MyGame.Domain.Inventory, MyGame.Presentation.HUD, MyGame.Infrastructure.Save. Глядя на using-директивы файла, вы сразу видите, какие области он затрагивает. Если доменный класс тянет MyGame.Presentation, это грубое нарушение, заметное и на ревью, и при автодополнении в IDE.

Стоит с самого начала договориться о простых, но строгих правилах. Домен не ссылается на Godot-типы и UI-сборки. Инфраструктура не зависит от домена конкретной игры, если только не реализует его контракты. Презентация может читать домен и вызывать его публичные методы, но никогда

не менять состояние напрямую, минуя контракт. Эти правила не требуют сложного инструментария; достаточно держать их в голове и следить за импортами. Постепенно они войдут в привычку, и нарушение будет восприниматься как ошибка.

Огромную роль играет **именование**. Имена папок и неймспейсов должны отражать домен, а не технические ярлыки. `Combat`, `Progression`, `Dialogue` сразу сообщают, о чём идёт речь. А вот `Managers`, `Helpers`, `Common` – это признак того, что вы не смогли сформулировать ответственность. Папка с названием `Utils` особенно опасна: она становится свалкой, куда попадает всё, что «не подошло» к остальным. Если вы видите, что `Utils` растёт быстрее других модулей, остановитесь и перераспределите лежащие там классы по осмысленным неймспейсам. В 99 % случаев внутри обнаруживаются куски доменной логики, маленькие сервисы и адаптеры, которым просто не нашли дом. Ещё одна практика – **контроль публичной поверхности**. Не каждый класс, который вы написали, должен быть `public`. Если тип используется только внутри своего модуля, делайте его `internal`. Это снижает количество точек, за которые могут зацепиться другие части системы, и упрощает рефакторинг. В больших `Godot`-проектах это особенно важно, потому что движок часто поощряет делать скрипты публичными для доступа из редактора. Различайте: `public` для движка (чтобы узел был виден в сценах) и `public` для кода. Иногда стоит разделить класс на два: тонкий публичный узел-фасад и внутренний модуль с логикой, которая не торчит наружу. Это дополнительная работа, но она окупается, когда вы захотите переработать внутренности, не ломая сцены и префабы.

В растущей команде конвенции становятся единственным способом сохранить связность. Договоритесь, где лежат доменные модели, как именуются сервисы, как оформляются адаптеры к движку. Не стремитесь к идеальной схеме с первого дня – начните с малого и уточняйте конвенции по мере появления проблем. Главное, чтобы правила были явными и соблюдались. Когда новый разработчик открывает проект и видит, что классы в `Domain` нигде не импортируют `Godot`, он с большой вероятностью продолжит эту традицию. Если же он находит в `Domain` вызовы `GetNode`, то с чистой совестью добавит ещё один, потому что «здесь так принято». Структура либо воспитывает, либо развращает.

4.4.4. Когда и как перестраивать структуру: триггеры роста и рефакторинг без катастрофы

Одна из самых вредных идей – «мы сделаем идеальную структуру сразу, чтобы потом не переделывать». Это невозможно, потому что вы не знаете, какой станет игра через полгода. Преждевременная детализация приводит к пустым папкам и абстрактным слоям, в которых никто не ориентируется. Вместо этого стоит ориентироваться на **триггеры роста** – симптомы, при которых текущая организация начинает мешать.

Первый триггер: вы всё чаще ищите файлы глобальным поиском, а не переходя по папкам. Это значит, что ментальная карта проекта больше не совпадает с его файловой структурой. Второй триггер: один и тот же файл правится по несвязанным причинам в разных задачах. Например, `Game-Manager.cs` одновременно меняется и при добавлении новой способности, и при правке экрана загрузки. Это сигнал, что класс отвечает за несколько независимых областей. Третий триггер: появляются папки-свалки (`Common`, `Utils`), которые растут быстрее доменных модулей. Четвёртый: добавление простой фичи требует раскидать код по пяти разным папкам, и вы постоянно забываете какую-нибудь из них.

Когда таких сигналов становится критически много, пора проводить рефакторинг структуры. Важно понимать, что рефакторинг структуры – это не эстетическое переименование папок, а **перенос ответственности**. Вы не просто перекладываете файлы; вы фиксируете архитектурные границы, которые до этого были размыты. Начинайте с самого больного места. Если вы заметили, что весь проект завязан на класс-монстр `Player`, не пытайтесь сразу разнести его по слоям. Выделите одну ось, например здоровье, и вынесите её в отдельный доменный класс в `Game.Domain`. Затем перенесите UI-отображение здоровья в `Game.Presentation.HUD`. Убедитесь, что связи стали односторонними. Только после этого беритесь за следующую ответственность.

Рефакторинг структуры отличается от обычного рефакторинга тем, что он меняет «адрес» ответственности. Поэтому важно сохранять историю изменений – используйте систему контроля версий, делайте атомарные коммиты, чтобы можно было откатиться. Если вы пользуетесь IDE вроде `Rider` или `Visual Studio`, перемещение файлов с автоматическим обновлением неймспейсов облегчает задачу. Но главное – после рефакторинга задайте себе вопрос: стало ли легче объяснить новому человеку, где что лежит? Если нет – вы поменяли структуру, но не архитектуру. Перестройка структуры может затрагивать и **сборку**. Если вы выделили доменный слой, который не должен зависеть от движка, имеет смысл поместить его в отдельную сборку (`C# Class Library`). Это даст жёсткую гарантию: домен не сможет использовать `Godot` даже случайно, потому что у него нет ссылки на `GodotSharp`. Это более продвинутая техника, но в больших проектах она окупается. Даже без отдельных сборок, просто разнеся код по проектам внутри решения, вы улучшите время компиляции и повысите дисциплину. Наконец, организация проекта – это живой организм. То, что работало вчера, может не работать завтра. Регулярно, раз в несколько месяцев, проводите «архитектурную ревизию» – просматривайте структуру на предмет разросшихся свалок, циклических зависимостей и нарушений конвенций. Делайте это в одиночку или вместе с командой. Такая профилактика дешевле, чем разгребание хаоса, в который проект скатится через год без присмотра.

Структура, построенная на этих принципах, не гарантирует отсутствия ошибок. Но она гарантирует, что ошибки будут локальными, а изменения –

предсказуемыми. А это и есть главная цель архитектурной организации большого проекта.

4.5. Направление зависимостей: как защитить правила игры от деталей реализации

В предыдущих разделах мы разложили по полочкам компоненты, отделили домен от инфраструктуры и обсудили, как организовать код. Теперь самое время соединить эти навыки в единую дисциплину, которая удерживает архитектуру от развала по мере роста проекта. Речь пойдёт о **направлении зависимостей** – о том, кто в вашей системе имеет право знать о ком и почему ответ на этот вопрос решает, будет ли игра расширяемой или превратится в хрупкий картонный домик. Зависимость в архитектурном смысле – это не просто строчка `using SomeNamespace`. Это любое знание, которое делает один модуль чувствительным к изменениям в другом. Если вы поменяли формат сохранений и вам пришлось править код начисления очков, у вас есть зависимость между правилами и инфраструктурой. Если вы переименовали кнопку в UI и сломалась логика открывания дверей – зависимость между представлением и доменом. Зависимости бывают **типовыми** (код не скомпилируется без чужого класса), **поведенческими** (код рассчитывает на определённый порядок вызовов), **временными** (код предполагает, что другая операция завершится раньше) и **конфигурационными** (код работает только при наличии глобального объекта или настройки). Все эти формы связей имеют свойство накапливаться, и в какой-то момент проект перестаёт быть гибким. Единственное, что спасает от этого, – сознательно выбранное и поддерживаемое направление зависимостей.

Представьте себе электрический щиток в доме. В центре – входной рубильник и предохранители, от которых питаются все комнаты. Если проводку проложили грамотно, то перестановка розетки в гостиной не затронет освещение в спальне, а замена лампочки не вырубит холодильник. Но если бы лампочка сама управляла рубильником, любая мелочь могла бы обесточить весь дом. В программировании так же: есть стабильное ядро (правила игры и доменные инварианты) и изменчивая периферия (UI, файлы, сеть, API движка). Направление зависимостей должно идти от периферии к ядру, а не наоборот. Ядро предоставляет контракты, а периферия их реализует и подключается на границах. Когда это правило нарушается, начинаются знакомые многим новичкам симптомы: невозможно перерисовать интерфейс, не боясь поломать геймплей; любое изменение формата сохранения тянет за собой переписывание боевой системы; тестирование предполагает

запуск всей игры, потому что класс здоровья внутри себя рисует полосу и пишет в базу данных. Такой код быстро становится беззащитным перед ростом требований.

4.5.1. Почему ядро не должно знать о периферии: пример с растущей энтропией

Рассмотрим типичного нарушителя – класс `PlayerHealth`, который отвечает за здоровье игрока, но при этом «заодно» обновляет UI и сохраняет прогресс:

```
public sealed class PlayerHealth
{
    private readonly Hud _hud;
    private readonly SaveService _save;
    public int Value { get; private set; } = 100;
    public PlayerHealth(Hud hud, SaveService save)
    {
        _hud = hud;
        _save = save;
    }
    public void TakeDamage(int amount)
    {
        Value -= amount;
        _hud.SetHealth(Value); // Прямое обновление интерфейса.
        _save.SaveNow();       // Прямой вызов сохранения.
    }
}
```

Пока игра состоит из двух экранов и одного разработчика, этот код работает прекрасно. Но через месяц приходит дизайнер и просит: «Пусть после получения урона экран немного краснеет, а сохранение происходит только раз в пять секунд, чтобы не лагало». В текущей архитектуре разработчику придётся лезть внутрь `PlayerHealth` и либо добавлять флаги, либо править логику сохранения. Доменный класс, который должен отвечать лишь за границы значений здоровья, начинает заниматься таймерами, цветами и форматами файлов. Через полгода в нём поселится и тряска камеры, и сетевые запросы, и блокировка интерфейса во время анимации. Класс превратится в свалку, которую страшно трогать.

Корень проблемы – направление зависимостей. `PlayerHealth` знает о конкретных классах `Hud` и `SaveService`. Это низкоуровневые модули (механизмы отображения и хранения), а `PlayerHealth` – высокоуровневый модуль (правило). Зависимости текут наружу: от домена к периферии. Теперь любое изменение периферии рискует сломать домен, и наоборот. Чтобы исправить ситуацию, нужно развернуть зависимости: домен должен определять, что ему нужно от внешнего мира, через абстрактные контракты. А конкретные реализации (UI, сохранение) должны эти контракты исполнять, находясь на периферии.

Эта же логика касается и Godot API. Если доменный класс напрямую работает с нодами, вызывает `GetNode` или использует `Input`, он становится завязан на конкретный движок и конкретную структуру сцены. Замена клавиатурного ввода на геймпад, или перенос UI в другую ветку дерева, или запуск логики на сервере без графики – всё это либо невозможно, либо требует мучительного переписывания. Здоровый подход – изолировать такие детали за интерфейсами и подключать их через адаптеры.

4.5.2. Инверсия зависимостей через контракты и адаптеры

Перепишем `PlayerHealth` так, чтобы он зависел только от контрактов – интерфейсов, описывающих **потребности** домена, а не возможности конкретных сервисов:

```
public interface IHealthView
{
    void Show(int currentHealth);
}
public interface ISaveCheckpoint
{
    void RequestSave();
}
public sealed class PlayerHealth
{
    private readonly IHealthView _view;
    private readonly ISaveCheckpoint _save;
    public int Value { get; private set; } = 100;
    public PlayerHealth(IHealthView view, ISaveCheckpoint save)
    {
        _view = view;
        _save = save;
    }
    public void TakeDamage(int amount)
    {
        Value -= amount;
        _view.Show(Value);           // Сообщили представлению о факте.
        _save.RequestSave();         // Попросили инфраструктуру сохранить.
    }
}
```

Теперь `PlayerHealth` не знает ничего о Godot, о файлах или о кнопках. Он общается с внешним миром через два узких интерфейса, которые сформулированы на языке домена: «показать здоровье» и «запросить сохранение». Как именно это будет сделано – решать периферии. Обратите внимание: интерфейсы принадлежат доменному слою, они вытекают из потребностей ядра, а не из устройства периферии. Это важнейший момент: если бы контракт диктовала конкретная реализация (например, метод `Show` принимал бы

ссылку на `Godot.Label`), направление зависимостей снова было бы нарушено.

Теперь соберём реальные реализации этих интерфейсов в `Godot`. Они будут адаптерами, живущими на границе системы:

```
// Адаптер, связывающий домен с конкретным UI-элементом движка.
public class HealthBarAdapter : IHealthView
{
    private readonly Label _label;
    public HealthBarAdapter(Label label)
    {
        _label = label;
    }
    public void Show(int currentHealth)
    {
        _label.Text = $"HP: {currentHealth}";
    }
}

// Адаптер, отвечающий за сохранение.
public class FileSaveAdapter : ISaveCheckpoint
{
    private readonly string _path;
    public FileSaveAdapter(string path)
    {
        _path = path;
    }
    public void RequestSave()
    {
        // Реальная логика записи в файл или облако.
        File.WriteAllText(_path, "saved");
    }
}

В сцене Godot всё это связывается воедино:
public partial class GameRoot : Node
{
    public override void _Ready()
    {
        // Создаём конкретные реализации.
        IHealthView healthView = new HealthBarAdapter(GetNode<Label>("HealthLabel"));
        ISaveCheckpoint save = new FileSaveAdapter("user://save.dat");
        // Внедряем зависимости в доменный объект.
        var playerHealth = new PlayerHealth(healthView, save);
        // Теперь можно использовать playerHealth, не боясь за его внутренности.
        playerHealth.TakeDamage(10);
    }
}
```

Посмотрите, как изменился граф зависимостей. Раньше `PlayerHealth` тащил `Hud` и `SaveService` напрямую. Теперь он зависит от абстракций, а конкретные реализации подставляются на границе – в `_Ready` корневой сцены. Это и есть инверсия зависимостей без мудрёных терминов: домен перестал быть рабом деталей, а стал заказчиком, которому периферия предоставляет услуги.

Стоит уточнить разницу между `compile-time` и `runtime`-зависимостями, потому что новички часто путают эти понятия. Когда `PlayerHealth` использует интерфейс `IHealthView`, на этапе компиляции он не зависит от `Godot.Label` или от файла `save.dat`. Это разрывает жёсткую связность и позволяет поменять реализацию в тестах или на другой платформе без перекомпиляции домена. Однако в рантайме зависимость остаётся: класс `HealthBarAdapter` всё ещё существует и должен быть передан в `PlayerHealth`. Важно, чтобы эта передача происходила в одной, хорошо известной точке сборки – обычно это `_Ready` корневого узла или специализированный загрузчик сцены. Если же вы разбросаете создание адаптеров по разным углам программы, `runtime`-зависимости снова станут неуправляемыми.

4.5.3. Профилактика циклов и сохранение дисциплины в долгосрочной перспективе

Когда зависимости текут в правильном направлении, циклы практически невозможны. Цикл – это ситуация, когда модуль `A` зависит от `B`, а `B` – от `A`. В архитектуре это равносильно короткому замыканию: любое изменение в одном модуле заставляет пересобрать и перепроверять второй, а причинно-следственные связи теряются. Циклы часто возникают из-за неявных ссылок: например, доменный объект испускает событие, на которое подписан `UI`, а `UI` в ответ дёргает метод того же доменного объекта. Формально стрелок зависимостей нет, но поведенческий цикл налицо. Лечится это выносом оркестрации: контейнер-сборщик слушает оба события и принимает решение о вызовах, разрывая прямую связь.

Практических эвристик для удержания направления достаточно трёх, но они покрывают почти все кейсы. Первая – **проверка импортов**. Откройте любой файл из папки `Domain` и посмотрите на `using`. Если там есть `Godot`, `UI`, `File`, `HttpClient` – скорее всего, ядро залезло в периферию. Единичные исключения возможны, но массовое присутствие таких импортов – красный флаг. Вторая эвристика – **контроль за точкой сборки**. Всё, что связывает домен с реальным миром, должно происходить в одном месте: в начале игры, при старте новой сцены, в корневом загрузчике. Если адаптеры создаются «внутри» доменных классов или через глобальный поиск (`GetNode`), вы теряете надзор над зависимостями. Третья – **тест радиуса изменений**. Задайте себе вопрос: «Если я полностью перепишу `UI`, сколько строк в папке `Domain` мне придётся исправить?» Правильный ответ: ноль. Домен должен переживать смену интерфейса без единой правки. Если это не так – у вас утечка зависимостей.

Эффект от соблюдения этих правил особенно заметен на отладке и тестировании. Представьте, что вы нашли баг: после пятого удара по врагу звук смерти проигрывается дважды. Без архитектуры вы будете проверять всё подряд: коллизии, таймеры, анимации, код аптечки. В системе с контролируемыми зависимостями маршрут поиска очевиден: ошибка либо в домен-

ном правиле подсчёта ударов, либо в адаптере звука, который неправильно управляет подписками. Вы изолируете доменный класс и прогоняете его на цепочке вызовов, подставив заглушки. Баг локализуется за минуты, а фикс не ломает другие части игры.

В долгоживущих проектах именно направление зависимостей отделяет код, способный к эволюции, от кода, который проще переписать с нуля. Игры живут годами, обрастают дополнениями, портируются на новые платформы, получают онлайн-режимы. В каждом из этих переходов меняются периферийные механизмы, но ядро – правила, баланс, доменные инварианты – остаётся. Если вы с самого начала приучите себя к тому, что правила ничего не знают о мире за пределами своего контракта, вы заложите фундамент, который не потребует сноса через пару лет. А это, пожалуй, лучший подарок, который разработчик может сделать будущему себе.

Игрок как система

5.1. Абстракция ввода: учим персонажа слушаться не кнопок, а намерений

Представьте себе водителя за рулём автомобиля. Он не думает: «Сейчас я нажму правую педаль на 3 сантиметра, а руль поверну на 15 градусов». У него в голове совсем другие категории: «Я хочу плавно ускориться и перестроиться в левый ряд». Автомобиль, в свою очередь, устроен так, чтобы преобразовывать эти высокоуровневые намерения в низкоуровневые команды для двигателя, коробки передач и колёс. Педали и руль – это всего лишь интерфейс, переводчик между желанием человека и поведением машины. В мире кода всё точно так же. Игрок жмёт на пробел или наклоняет стик – это физический сигнал. Но персонаж игры не должен знать, какой именно палец и с какой силой надавил на пластик. Персонажу важно лишь одно: «Чего ты от меня хочешь, человек? Двигаться? Прыгнуть? Атаковать?» Прямая зависимость геймплея от конкретных клавиш и джойстиков так же абсурдна, как если бы водителю для ускорения приходилось напрямую дёргать тросик дроссельной заслонки. Это хрупко, неудобно и делает любые изменения чрезвычайно болезненными.

Первый шаг к устойчивой, масштабируемой игровой архитектуре – построить этот самый слой-переводчик. На инженерном языке он называется абстракцией ввода. Его задача – раз и навсегда отделить то, **как** приходит сигнал, от того, **что** он означает для вашей игры.

5.1.1. Главная ошибка новичка: рождение «божественного метода»

Давайте посмотрим, как обычно рождается монстр. Вы начинаете новый проект, полный энтузиазма. Создаёте сцену игрока, вешаете скрипт и в `_PhysicsProcess` быстро набрасываете заветные строчки:

```
using Godot;
public partial class Player : CharacterBody2D
{
    [Export] public float Speed = 300.0f;
    [Export] public float JumpForce = 400.0f;
    public override void _PhysicsProcess(double delta)
    {
        // 1. Движение.
        Vector2 velocity = Velocity;
        velocity.X = Input.GetAxis("move_left", "move_right") * Speed;

        // 2. Прыжок.
        if (Input.IsActionJustPressed("jump") && IsOnFloor())
            velocity.Y = -JumpForce;

        // 3. Взаимодействие.
        if (Input.IsActionJustPressed("interact"))
            TryInteract();
        Velocity = velocity;
        MoveAndSlide();
    }
    private void TryInteract() { /* ... */ }
}
```

Этот код работает. Персонаж бежит, прыгает, всё прекрасно. Но это ловушка. В этом крошечном фрагменте уже заложено архитектурное землетрясение, которое произойдёт через несколько месяцев разработки.

Почему? Потому что ваш `Player` стал не просто игровым персонажем. Теперь представьте, какие новые требования прилетят от геймдизайнера через месяц. Всё это – следствие того, что высокоуровневая игровая логика оказалась намертво прикручена к низкоуровневому API движка. На языке архитектуры это называется **нарушением направления зависимостей**. Ваши правила игры (персонаж может прыгать только с земли) стали рабами детали реализации (клавиша `jump`). Хорошая архитектура требует обратного.

5.1.2. Проектирование намерения: контракт, понятный и кнопке, и ИИ

Решение элегантно и при этом не требует никаких сложных фреймворков. Мы просто создадим «переводчик с языка кнопок на язык намерений». На-

мерение, или `intent`, – это простая структура данных, которая отвечает на один вопрос: «Что игрок хочет сделать прямо сейчас, если забыть о том, чем именно он играет?»

Для нашего платформера намерение может выглядеть так:

```
/// <summary>
/// Структура, описывающая сырое намерение игрока в этом кадре.
/// Он не знает о кнопках, клавиатурах и джойстиках.
/// </summary>
public readonly struct PlayerIntent
{
    /// <summary>
    /// Желаемое направление движения: от -1.0f (влево) до 1.0f (вправо).
    /// 0.0f – стоим на месте.
    /// </summary>
    public float MoveDirection { get; init; }

    /// <summary>
    /// Истина в том кадре, когда игрок начинает давить на кнопку прыжка.
    /// </summary>
    public bool JumpRequested { get; init; }

    /// <summary>
    /// Удерживается ли кнопка бега (модификатор действия).
    /// </summary>
    public bool RunHeld { get; init; }

    /// <summary>
    /// Запрос на взаимодействие (открыть дверь, взять предмет).
    /// </summary>
    public bool InteractRequested { get; init; }
}
```

Посмотрите на эту структуру. `MoveDirection` – это уже готовое, нормализованное значение, а не «сырая ось с мёртвой зоной и непонятным ID устройства». `JumpRequested` – не «нажата ли физическая клавиша Space», а булево значение, выражающее желание. Это язык домена нашей игры. На нём с персонажем может разговаривать кто угодно, если он освоит этот язык.

Следующий шаг – формализовать «переводчик». Его роль будет выполнять интерфейс `IInputProvider`, единственная задача которого – вернуть актуальный `PlayerIntent` за этот кадр.

```
/// <summary>
/// Интерфейс источника намерений. Отделяет игровую логику от способа ввода.
/// </summary>
public interface IInputProvider
{
    /// <summary>
    /// Получить намерение игрока на текущий кадр.
    /// </summary>
    PlayerIntent Read();
}
```

С этого момента всё, что находится «выше» этого интерфейса (ваш будущий контроллер игрока, AI, система реплеев), будет зависеть только от него. Всё, что «ниже», – конкретные реализации для Godot или другого движка – останутся на периферии. Мы развернули стрелку зависимости в правильную сторону.

Теперь напомним одну из таких «низкоуровневых» реализаций для Godot. Она будет знать об Input, об Action Map и о том, как из этого получить наш PlayerIntent.

```
using Godot;
public class GodotInputProvider : IInputProvider
{
    // Константы, специфичные для устройства, живут только здесь.
    private const float GamepadDeadzone = 0.2f;
    private const float MoveSmoothing = 5.0f;
    public PlayerIntent Read()
    {
        // 1. Получаем сырую ось движения от любого устройства, привязанного к `move`
        // actions.
        float rawAxis = Input.GetAxis("move_left", "move_right");

        // 2. Нормализуем её, фиксируя мёртвую зону – это проблема геймпада,
        // а не персонажа.
        float moveDirection = Mathf.Abs(rawAxis) < GamepadDeadzone
            ? 0.0f
            : Mathf.Clamp(rawAxis, -1.0f, 1.0f);

        // 3. Собираем намерение, переводя action names в поля структуры.
        return new PlayerIntent
        {
            MoveDirection = moveDirection,
            JumpRequested = Input.IsActionJustPressed("jump"),
            RunHeld = Input.IsActionPressed("run"),
            InteractRequested = Input.IsActionJustPressed("interact")
        };
    }
}
```

Вся грязь, связанная с мёртвыми зонами, названиями кнопок и особенностями Input.* API, герметично заперта в этом единственном классе. Если завтра вы решите переделать управление, добавить геймпад с другой мёртвой зоной или даже перенести игру на другую платформу, где Input работает иначе, вы измените только GodotInputProvider. Ни одна строчка в коде игрового персонажа не будет тронута.

5.1.3. Контроллер как потребитель намерений: логика без привязки к железу

Время переписать наш класс Player. Он превратится в чистого и ясного потребителя намерений.

```

public partial class PlayerController : CharacterBody2D
{
    [Export] private float speed = 300f;
    [Export] private float jumpForce = 400f;

    private IInputProvider _inputProvider;
    // ...
    public override void _Ready()
    {
        // В реальном проекте провайдер будет передан извне (хост сцены, DI),
        // но для первого шага создадим его прямо здесь.
        _inputProvider = new GodotInputProvider();
    }
    public override void _PhysicsProcess(double delta)
    {
        PlayerIntent intent = _inputProvider.Read();
        ProcessMovement(intent, (float)delta);
        ProcessJump(intent);
        ProcessInteraction(intent);
    }
    private void ProcessMovement(PlayerIntent intent, float delta)
    {
        float desiredSpeed = intent.MoveDirection;
        if (intent.RunHeld) desiredSpeed *= 1.5f;

        Vector2 velocity = Velocity;
        velocity.X = desiredSpeed * speed;
        Velocity = velocity;
        MoveAndSlide();
    }
    private void ProcessJump(PlayerIntent intent)
    {
        if (intent.JumpRequested && IsOnFloor())
        {
            Vector2 velocity = Velocity;
            velocity.Y = -jumpForce;
            Velocity = velocity;
        }
    }
    private void ProcessInteraction(PlayerIntent intent)
    {
        if (intent.InteractRequested)
        {
            // Логика взаимодействия.
        }
    }
}

```

Код стал самодокументированным и понятным. Читая метод `ProcessMovement`, мы видим: «Если намерение говорит двигаться вправо, мы двигаемся вправо. Если ещё и бежим – быстрее». Всё. Никаких `Input.GetAxis`, никаких

магических строк. Класс `PlayerController` больше не зависит от `Godot.Input`. Он зависит от нашего собственного контракта `IInputProvider` и структуры `PlayerIntent`.

Теперь мы можем сделать то, что раньше казалось фантастикой, – **протестировать управление персонажем без запуска Godot**. Мы просто создадим тестового провайдера, который будет «врать» персонажу, подсовывая заранее заготовленные намерения.

```
public class TestInputProvider : IInputProvider
{
    private readonly Queue<PlayerIntent> _actions;
    public TestInputProvider(Queue<PlayerIntent> actions) => _actions = actions;
    public PlayerIntent Read() => _actions.Count > 0 ? _actions.Dequeue() : default;
}
// Где-то в тестовом окружении.
public void TestJump()
{
    var actions = new Queue<PlayerIntent>();
    // Симулируем: стоим на месте, нажимаем прыжок.
    actions.Enqueue(new PlayerIntent { MoveDirection = 0, JumpRequested = true });

    var testInput = new TestInputProvider(actions);
    // Создаём игрока, внедряем testInput, вызываем _PhysicsProcess.
    // Проверяем, что Velocity.Y стала отрицательной.
    Assert.IsTrue(player.Velocity.Y < 0);
}
```

Мы только что превратили проверку игрового поведения из ручного прогона сцены в автоматизированный, на 100 % воспроизводимый тест. Это меняет всё. Баг в прыжке? Мы пишем тест, который воспроизводит последовательность намерений, и видим, где ломается логика.

5.1.4. Магия буферизации: как простить игроку ошибку на 100 миллисекунд

Одно из самых ценных архитектурных решений, которое открывает слой намерений, – это буферизация ввода. Представьте, что игрок подбегает к краю платформы и нажимает пробел для прыжка в самый последний момент. Из-за скорости обработки кадров `_PhysicsProcess` может быть вызван в тот краткий миг, когда персонаж уже на миллиметр оторвался от земли. Нажатие клавиши прыжка было, но `IsOnFloor()` возвращает `false`, и драгоценный прыжок «съеден». Игрок зол, он же нажал вовремя! Это лечится элегантным буфером на несколько кадров. Идея проста: после того как флаг `JumpRequested` получен, мы не проверяем его сиюминутно, а запоминаем его на небольшой промежуток времени (например, 100 миллисекунд). Если в течение этого времени ноги персонажа снова коснутся земли, заветный прыжок будет выполнен автоматически.


```

public partial class PlayerController : CharacterBody2D
{
    private float _jumpBufferTimer;
    private const float JumpBufferTime = 0.1f; // 100 миллисекунд.
    private void ProcessJump(PlayerIntent intent, float delta)
    {
        // 1. Если намерение прыгнуть пришло, "заряжаем" буфер.
        if (intent.JumpRequested)
            _jumpBufferTimer = JumpBufferTime;
        // 2. Уменьшаем таймер буфера с каждым кадром.
        _jumpBufferTimer -= delta;
        // 3. Прыжок произойдёт, когда буфер ещё активен, а персонаж на полу.
        if (_jumpBufferTimer > 0f && IsOnFloor())
        {
            Vector2 velocity = Velocity;
            velocity.Y = -jumpForce;
            Velocity = velocity;
            _jumpBufferTimer = 0f;
        }
    }
}

```

Посмотрите на красоту архитектуры. Мы улучшили **ощущение от игры** (game feel), внося изменения исключительно в доменную логику контроллера. Мы не трогали ни `IInputProvider`, ни его реализации. Мы просто по-другому обработали то же самое намерение. Это и есть обещанная гибкость и масштабируемость. Буферизация – не единственный фокус, который можно повернуть: точно так же можно добавить «койот-время» (возможность прыгнуть чуть-чуть после того, как персонаж сорвался с края) или любые другие техники, делающие игру приятнее на ощупь.

5.1.5. Реплеи, AI и мультиплеер: один интерфейс, чтобы править всеми

Когда в вашем проекте появляется такой `IInputProvider`, архитектурные чудеса только начинаются. Вы получаете возможность подключать к персонажу любой источник намерений, не меняя его код. Запись и воспроизведение действий (replays) становится тривиальной задачей. Вместо того чтобы писать сложную систему записи нажатий на клавиши, вы просто на каждом кадре записываете структуру `PlayerIntent` в файл. Чтобы воспроизвести заезд, пишется ещё одна реализация `IInputProvider` – `ReplayInputProvider`, которая читает этот файл кадр за кадром и возвращает записанный `intent`.

```

public class ReplayInputProvider : IInputProvider
{
    private readonly Queue<PlayerIntent> _recordedIntents;
    public ReplayInputProvider(Queue<PlayerIntent> recording) => _recordedIntents =
recording;
}

```

```

public PlayerIntent Read() =>
    _recordedIntents != null && _recordedIntents.Count > 0
    ? _recordedIntents.Dequeue()
    : default;
}

```

AI работает точно по такому же принципу. Вы пишете `AIInputProvider`, который анализирует мир (где враг, где стена, каков тайминг) и, используя любой алгоритм, генерирует `PlayerIntent`. Он может говорить персонажу: «В этом кадре я хочу двигаться вправо и прыгнуть». Вся та же физика, проверка `IsOnFloor()` и правила игры применяются и к AI, и к игроку абсолютно идентично. Это невероятно упрощает балансировку и делает поведение ботов естественным, они играют по тем же правилам, что и человек. Мультиплеер здесь тоже встаёт на своё место. Сетевые пакеты от клиента к серверу будут пересылать не координаты и не нажатия кнопок, а последовательности `PlayerIntent`. Серверный «призрак» игрока будет управляться тем же самым `PlayerController`, только ввод для него будет поступать не с `GodotInputProvider`, а с `NetworkInputProvider`. И вы можете быть уверены, что физика просчитается одинаково, потому что логика одна.

5.1.6. Подводные камни: о чём нельзя забывать

Даже такая стройная модель, как `Input Abstraction`, может быть испорчена ошибками реализации. Самая частая из них – это **чрезмерная абстракция**. Новичок, вдохновлённый идеей, может начать делать интерфейс `IInputProvider` «пухлым», как булгаковский Коровьев, засунув туда два десятка методов на все случаи жизни. Или, наоборот, сделать `PlayerIntent` настолько общим, что он превратится в бесформенный мешок параметров. Помните: этот слой – переводчик, а не экскаватор. Он должен быть как можно тоньше и проще. Вторая ошибка – **разбросанные чтения**. Когда в проекте уже есть аккуратный `IInputProvider`, а где-то в углу в скрипте UI вдруг снова вызывается `Input.IsActionJustPressed("ui_accept")`, «потому что было лень». Система начинает получать ввод из двух независимых источников, и предсказуемость рушится. Железное правило: **код, который управляет игровым персонажем или объектом, никогда не должен обращаться к `Godot.Input` напрямую**. Единственным источником истины для игровых действий должен быть ваш `IInputProvider`.

5.2. Архитектура движения: конвейер от намерения к позиции

Теперь, когда персонаж больше не зависит от конкретных клавиш, а управляется абстрактным намерением, пора навести порядок в том, как он это

намерение выполняет. Движение – это вторая половина головоломки. Самое время спроектировать систему, которая будет брать желание игрока, просчитывать его последствия по законам физики и воплощать результат в игровом мире. Прямо сейчас мы заложим фундамент, который не даст вашему коду превратиться в лапшу из условий и побочных эффектов.

5.2.1. Конвейер движения: три шага, которые нельзя смешивать

В основе любого хорошо организованного движения лежит идея, простая, как конвейер на фабрике. Представьте, что персонаж – это сложный механизм, который перерабатывает сырьё в продукт. Сырьём является намерение, а продуктом – новое положение в мире. Чтобы процесс был надёжным и предсказуемым, мы разобьём его на три стадии, каждая из которых отвечает за свою задачу и не вмешивается в дела соседей.

Первая стадия – **получение снимка намерения**. В начале каждого кадра наш контроллер обращается к источнику ввода, будь то клавиатура, AI или сетевой пакет, и получает структуру `PlayerIntent`. Это его единственная связь с внешним миром. Дальше намерение фиксируется на весь кадр и не меняется. Это критически важно: если скорость или состояние персонажа начнут влиять на то, как читается ввод, или если разные модули будут читать ввод в разное время, предсказуемость улетучится.

Вторая стадия – **обновление внутренней модели движения**. У нас есть специальный класс, назовём его `MovementModel`, который владеет всей физической правдой о персонаже: его скоростью, ускорением, текущим состоянием (на земле, в воздухе). Этот класс берёт фиксированное намерение, текущую скорость и, используя чёткие правила (трение, гравитацию, ограничения), вычисляет новую скорость. Важнейший момент: на этом этапе персонаж никуда не двигается. Мы только считаем математику. И эта математика абсолютно чиста – она не зависит ни от движка Godot, ни от ввода, ни от анимаций.

Третья стадия – **применение к физическому телу**. Только теперь, когда новая скорость вычислена, мы отдаём её движку. В случае Godot это произойдёт в `_PhysicsProcess`, где мы присвоим посчитанное значение полю `Velocity` и вызовом `MoveAndSlide`. Движок, в свою очередь, просчитает столкновения, подвинет персонажа и вернёт нам скорректированные данные, например новое состояние «на земле / в воздухе».

Эта трёхфазная структура – не усложнение, а спасательный круг. Она гарантирует, что изменение физических констант не заставит вас переписывать код ввода, а замена клавиатуры на геймпад не сломает физику. Каждая часть конвейера меняется по своим причинам и в своей песочнице. Чтобы увидеть, как эта схема оживает в коде, давайте напомним `MovementModel` – сердце нашей системы. Этот класс является чистым C#-кодом, который можно протестировать хоть в консольном приложении.

```

public class MovementModel
{
    // Конфигурация, которая может быть загружена из ресурсов/конфигов.
    private readonly float _walkAcceleration;
    private readonly float _maxWalkSpeed;
    private readonly float _groundFriction;
    private readonly float _airControlMultiplier;
    private readonly float _gravity;
    private readonly float _jumpImpulse;
    // Состояние движения.
    public Vector2 Velocity { get; private set; }
    public bool IsGrounded { get; private set; }
    // Входные данные для шага симуляции.
    public void Step(PlayerIntent intent, float delta, bool isGrounded)
    {
        IsGrounded = isGrounded;
        ProcessHorizontalMovement(intent, delta);
        ProcessGravity(delta);
    }
    public void ApplyJump()
    {
        if (IsGrounded)
        {
            Velocity = new Vector2(Velocity.X, -_jumpImpulse);
            IsGrounded = false;
        }
    }
    private void ProcessHorizontalMovement(PlayerIntent intent, float delta)
    {
        float targetSpeed = intent.MoveDirection * _maxWalkSpeed;
        float acceleration = IsGrounded ? _walkAcceleration : _walkAcceleration * _
airControlMultiplier;

        // Вычисляем разницу между желаемой и текущей скоростями.
        float speedDifference = targetSpeed - Velocity.X;
        // Применяем ускорение в нужном направлении, но не больше, чем надо.
        float accelerationAmount = Mathf.Clamp(acceleration * delta *
            Mathf.Sign(speedDifference),
            0, Mathf.Abs(speedDifference));

        Velocity = new Vector2(Velocity.X + accelerationAmount, Velocity.Y);
    }
    private void ProcessGravity(float delta)
    {
        if (!IsGrounded)
            Velocity = new Vector2(Velocity.X, Velocity.Y + _gravity * delta);
    }
}

```

Обратите внимание на кристальную ясность этого кода. У нас есть константы, определяющие «характер» персонажа: как быстро он разгоняется, как сильно на него действует гравитация. Эти константы можно вынести

в отдельный файл конфигурации и настраивать без единой правки логики. Сам же метод `Step` делает ровно то, что мы задумали: обрабатывает горизонтальное движение и гравитацию, используя физически корректные формулы с учётом ускорения и дельты времени. Такой подход позволяет добавлять новые механики, не разрушая старые. Захотели сделать платформы-батуты? Добавляете публичный метод `ApplyExternalImpulse(Vector2 impulse)`, который просто прибавляет переданное значение к скорости. Захотели зелье скорости? Добавляете временный модификатор к `_maxWalkSpeed`. Логика всего движения остаётся нетронутой, потому что она спроектирована как система, а не как набор разрозненных `if`.

5.2.2. Владение скоростью: почему так важно, чтобы у каждой величины был хозяин

В проектах новичков скорость часто становится «бесхозной вещью». Один скрипт меняет `Velocity.X`, чтобы подвинуть персонажа в кат-сцене. Другой – чтобы отбросить его взрывом. Третий – при входе в зону замедления. Вроде бы всё работает, но когда персонаж начинает двигаться рывками или не реагировать на управление, найти причину практически невозможно. Скорость превращается в общежитие, где каждый меняет всё под себя и никто не отвечает за порядок. Мы решаем эту проблему, объявляя `MovementModel` единственным владельцем скорости. `Velocity` здесь – приватное поле с публичным геттером. Изменить её извне можно только через осмысленные, специально созданные методы, такие как `ApplyJump()` или условный `ApplyKnockback()`. Это даёт нам гарантию, которую невозможно получить с публичным полем: скорость не будет изменена случайно, и в любой момент времени мы можем отследить, какая операция к ней привела. Эта гарантия распространяется и на другие системы. Анимация хочет знать, быстро ли бежит персонаж? Пожалуйста, вот тебе свойство `Velocity`. Звук хочет проиграть шаги, когда персонаж касается земли? Движение выставит флаг `IsGrounded`. Но ни анимация, ни звук, ни интерфейс никогда не получают права писать в `Velocity`. Они – потребители, а не производители. Эта односторонняя связь гигантски упрощает отладку и делает поведение персонажа предсказуемым.

5.2.3. Детерминизм и отладка: превращаем ощущения в науку

«Персонаж какой-то ватный», «прыжок неотзывчивый», «скользкий пол» – львиная доля багов в управлении не связана с ошибками в коде. Они связаны с непредсказуемостью этого кода. Где-то вы применили гравитацию до того, как учли ввод игрока, и персонаж не успел разогнаться. Где-то наоборот, сначала обработали нажатие, а потом уже силу тяжести, и персонаж взлетел выше, чем задумано. Архитектурная дисциплина детерминизма – лучшее

лекарство от таких болезней. Мы договариваемся, что наш конвейер всегда работает в одном и том же порядке. Внутри нашего `MovementModel.Step()` порядок жёстко задан: сначала обрабатываем горизонтальное движение, потом – гравитацию. Эффект от этого потрясающий: если вы подадите на вход один и тот же `PlayerIntent`, одно и то же начальное `Velocity` и одну и ту же `delta`, вы гарантированно получите одну и ту же скорость на выходе. Это превращает отладку движения из гадания на кофейной гуще в рутинную инженерную задачу. Вы записываете в лог: «В кадре 1000 персонаж был на земле, имел скорость (100, 0), получил намерение `MoveRight=1.0`, после `Step`'а его скорость стала (120, 0)». Если она стала другой, вы точно знаете, что ошибка внутри `MovementModel`, а не в таинственных взаимодействиях с UI или анимациями. Вы можете написать десяток юнит-тестов, которые подадут на вход модели разные намерения и проверяют, что скорость меняется корректно, не запуская игру ни разу. Согласитесь, это совсем другой уровень контроля над качеством.

5.2.4. Жёсткая граница: почему анимация не имеет права двигать персонажа

Это один из самых соблазнительных и опасных архитектурных хаков. Представьте: вы делаете анимацию атаки мечом. Чтобы удар чувствовался мощным, персонаж должен сделать небольшой шаг вперёд. Самое простое – внутри анимации, в определённом кадре, дернуть метод вроде `player.Position += new Vector2(10, 0)`. Или, что ещё хуже, записать что-то в `Velocity`. Это кажется гениальным: и красиво, и логично, анимация же знает, когда происходит шаг. Но это архитектурная мина замедленного действия. Как только анимация начинает управлять позицией или скоростью, у персонажа появляются два хозяина: `MovementModel` и анимационный клип. И они друг о друге не знают. В момент рывка анимация может конфликтовать с физикой: стена не даёт персонажу двигаться, а анимация толкает его вперёд, вызывая прохождение сквозь текстуры. В сетевой игре на сервере этого визуального рывка вообще не будет, и клиент и сервер рассинхронизируются. А если вы решите заменить спрайтовую анимацию на кости, то обнаружите, что вся логика рывка размазана по клипам, и её нужно переносить вручную. Здоровое решение – сделать так, чтобы анимация не управляла движением, а только реагировала на его состояние. Персонаж сделал рывок? Эффект должен рождаться в доменной логике, например через вызов `movementModel.ApplyExternalImpulse(new Vector2(rushDistance, 0))`. Этот вызов может сделать контроллер игрока, активируя способность. А анимация, увидев внезапно изменившуюся скорость или факт начала способности, уже сама решит, какой клип проиграть. Зависимость строго односторонняя: движение → анимация, и никогда в обратную сторону. Это делает вашу физическую модель главной и неприкасаемой, а визуальную часть – легко заменяемой декорацией.

5.2.5. Собираем всё воедино: как это выглядит в Godot-сцене

Пришло время примерить нашу замечательную архитектуру на реальный Godot-узел. Это последний шаг, где мы соединяем конвейер намерений из раздела 5.1 с конвейером движения из 5.2. Класс `Player` станет тем самым тонким контроллером, который только связывает части, не внося своей логики.

```
using Godot;
public partial class Player : CharacterBody2D
{
    // Здесь мы внедряем наши абстракции.
    private IInputProvider _input;
    private MovementModel _movement;
    public override void _Ready()
    {
        // 1. Создаём источник ввода. В реальном проекте он будет передан как зависимость.
        _input = new GodotInputProvider();
        // 2. Загружаем конфигурацию движения.
        var config = LoadMovementConfig();
        // 3. Создаём модель движения.
        _movement = new MovementModel(config);
    }
    public override void _PhysicsProcess(double delta)
    {
        // Этап 1: получаем снимок намерения.
        PlayerIntent intent = _input.Read();
        // Этап 2: обновляем нашу доменную модель движения.
        _movement.Step(intent, (float)delta, IsOnFloor());
        // Дополнительно обрабатываем запрос на прыжок, который тоже влияет на модель.
        if (intent.JumpRequested)
            _movement.ApplyJump();
        // Этап 3: применяем итоговое состояние к телу движка.
        Velocity = _movement.Velocity;
        MoveAndSlide();
        // Этап 4 (необязательный, но полезный): обновляем флаги в модели
        // на основе того, что движок сказал нам после столкновений.
        _movement.IsGrounded = IsOnFloor();
    }
    private MovementConfig LoadMovementConfig()
    {
        // В реальном проекте это может загружаться из Godot.Resource или JSON-файла.
        return new MovementConfig
        {
            WalkAcceleration = 1200f,
            MaxWalkSpeed = 200f,
            GroundFriction = 800f,
            AirControlMultiplier = 0.6f,
        };
    }
}
```

```

        Gravity = 900f,
        JumpImpulse = 400f
    };
}
}

```

Посмотрите, как мало кода в нашем главном классе. Вся сложность аккуратно упакована в `MovementModel` и в реализации `InputProvider`. Сам же `Player` превратился в дирижёра, который взмахивает палочкой, вызывая по очереди нужные методы. Его легко читать, легко тестировать и, что самое приятное, легко изменять. Замена физической модели с `CharacterBody2D` на `RigidBody2D` или добавление новых механик вроде подката либо двойного прыжка станут локальными изменениями в соответствующих модулях. Наш персонаж больше не развалится в руках от первой же правки геймдизайнера. Он готов к будущему.

5.3. `PhysicsBody` и `CharacterBody`: выбор физической основы для движения

Разрабатывая движение персонажа, мы уже построили чистый доменный конвейер: он принимает намерение, вычисляет скорость и применяет её к телу. Но в Godot на последнем шаге нас ждёт фундаментальная развилка. Движок предлагает два принципиально разных способа взаимодействия с физическим миром, и от того, какой из них мы выберем, зависит не просто пара строчек кода, а весь характер управления, предсказуемость поведения и даже архитектура будущей машины состояний. Не будет преувеличением сказать, что выбор между симуляцией физики и контроллером – это один из тех моментов, который формирует «характер» всей последующей разработки. Тема эта не умозрительная: переделка игрока с одной модели на другую в середине проекта обычно стоит как полноценный рефакторинг. Поэтому давайте раз и навсегда разложим по полочкам, чем отличаются эти подходы, и научимся выбирать между ними осознанно.

5.3.1. Две модели движения: симуляция физики и контроллер

Godot 4 предоставляет два основных типа тел для объектов, которые двигаются и сталкиваются. За классом `RigidBody2D` (и его трёхмерным собратом `RigidBody3D`) стоит целая физическая модель: движок симулирует движение объекта как в настоящем физическом мире, используя массу, инерцию, силы и импульсы. Второй тип, `CharacterBody2D` (и соответствующий `CharacterBody3D`), работает совершенно иначе – он предоставляет удобные методы

для перемещения, при которых вы сами решаете, куда и с какой скоростью двигаться, а движок лишь помогает корректно обработать столкновения.

За этими двумя классами стоят две противоположные философии, определяющие, кто главный. Попробуем понять их суть без привязки к названиям конкретных нод.

В **симуляционной модели** (реализуемой RigidBody) авторитет над движением принадлежит физическому миру. Вы не можете приказать объекту: «переместись на два метра вправо». Вместо этого вы говорите: «к этой точке пространства приложена сила такой-то величины». А дальше движок, подобно огромному калькулятору, просчитывает ускорение, учитывает инерцию, проверяет контакты с другими телами, сталкивает объекты, разрешает коллизии и в результате вычисляет итоговую траекторию. Такой подход невероятно хорош для создания правдоподобного хаоса: ящики, бочки, обломки – всё то, что должно выглядеть как часть живого физического мира. Но он же становится источником архитектурной боли, когда вы пытаетесь применить его к главному герою, от которого требуется молниеносная реакция на нажатия и строго определённая высота прыжка.

Контроллерная модель (реализуемая CharacterBody), напротив, оставляет бразды правления в ваших руках. Авторитет над движением полностью принадлежит вашему коду. Вы вычисляете желаемую скорость или вектор смещения, руководствуясь игровыми правилами (ускорением, гравитацией, ограничениями), а затем вызываете метод, например MoveAndSlide(). Движок в этой модели играет роль ассистента: он берёт ваш вектор, аккуратно двигает персонажа и, если на пути встречается стена, надёжно его останавливает, не позволяя пройти сквозь текстуру. Именно вы, а не симуляция, решаете, затухнет ли скорость после отпускания клавиши, можно ли развернуться в воздухе и как высоко должен прыгнуть персонаж.

Ключевая разница здесь – в понятии **владельца скорости**. В симуляции скорость – это результат взаимодействия множества сил, она живёт где-то в недрах движка. В контроллере вы храните её самостоятельно (в нашей модели MovementModel), и она является вашим доменным состоянием. Это означает, что внешний наблюдатель или система анимаций не могут «подкрутить» её незаметно, в обход вашего конвейера. Контроль над состоянием остаётся у вас.

5.3.2. Архитектурные последствия: управляемость, отладка и предсказуемость

Выбор между этими двумя моделями пронизывает всю архитектуру игрока. Начнём с самого чувствительного для геймплея аспекта – управляемости.

Представьте, что вы делаете платформер с очень точными, выверенными прыжками. Вы хотите, чтобы персонаж всегда прыгал на 200 пикселей, независимо от того, стоял ли он на краю платформы или в её центре. В контроллерной модели это требование реализуется одной строкой: при нажатии

кнопки вы присваиваете вертикальной скорости значение `-jumpForce`. Инвариант соблюден железно. В симуляционной модели вы прикладываете импульс. Но если персонаж в момент прыжка начинает сползать по склону или в него врежется другой объект, результирующая сила решателя может частично погасить ваш импульс, и высота прыжка станет плавающей. Чтобы добиться такой же стабильности, вам придётся писать костыли: на время прыжка отключать часть внешних воздействий, принудительно править позицию и гасить нежелательные скорости. Вы начнёте бороться с инструментом, вместо того чтобы решать игровую задачу.

Отсюда вытекает вопрос предсказуемости и воспроизводимости. Одна из главных ценностей архитектуры, которую мы закладываем уже несколько разделов, – детерминизм. При одинаковом намерении, одинаковом начальном состоянии и одинаковом контексте мы должны получать одинаковый результат. С контроллерной моделью это достижимо: наш конвейер `Movement-Model` работает изолированно, чётко пересчитывая `Velocity`. В симуляции же между вашим намерением и итоговой позицией находится «чёрный ящик» общего физического мира, который добавляет в уравнение множество переменных: микроскопические неровности коллизий, переменное количество итераций решателя, контакты с другими телами. Результат может меняться от кадра к кадру, и это делает отладку крайне нетривиальной. Отладка в симуляции заставляет вас мыслить иначе: «почему этот импульс не привёл к ожидаемому ускорению? почему персонаж застревает в этом углу? почему трение ведёт себя по-разному на визуально идентичных поверхностях?» Ответы на эти вопросы часто лежат не в вашем коде, а в устройстве физического окружения или даже в настройках движка. В контроллерной модели вы всегда идёте по своему коду: вы видите `Velocity`, которое выставили до `MoveAndSlide`, и видите, что изменилось после. Граница между вашей логикой и помощью движка кристально ясна.

Это различие напрямую влияет на то, как вы будете строить систему **состояний** персонажа (бег, прыжок, оглушение). В контроллерной модели каждое состояние может являться отдельным методом или даже классом, который просто по-своему конфигурирует параметры (ускорение, трение) или накладывает ограничения на итоговую скорость. В симуляционной модели состояниям пришлось бы выражаться в терминах сил, максимальных скоростей и жёстких ограничений для решателя, что значительно запутывает код и смешивает чисто игровые правила с физическими механизмами.

5.3.3. Практические сценарии и критерии выбора

В долгоживущем проекте эти две модели редко конкурируют за одну и ту же роль. Они просто занимают свои, предназначенные им места. Объекты, для которых естественна **симуляционная модель**: всё, что должно «жить физикой». Ящики и бочки, которые игрок двигает и разбивает. Обломки после взрыва. Кубики-льдинки, скользящие по поверхности. Любой объект, для которого важнее взаимодействие с другими телами и реакция на столкновения,

чем строго контролируемые правила поведения. Там, где правила просты («пусть летит и отскакивает»), симуляция будет самым быстрым и правдоподобным решением. Объекты, для которых естественна **контроллерная модель**: сам игрок, его союзники и противники – все, от кого требуется чуткость и предсказуемость управления. Также сюда относятся любые сущности, чьё движение определяется дизайном, а не физической случайностью. Если вам нужно, чтобы враг летал по строго определённой синусоиде, не сбиваясь на коллизиях, – это контроллер.

Инженерный критерий выбора прост: задайте себе три вопроса.

- Должен ли я иметь полный контроль над скоростью и её производными или я готов довериться движку? Если да – CharacterBody.
- Является ли реакция на столкновения с другими объектами частью моих правил игры или взаимодействие должно происходить само собой? Если вы готовы описывать каждое взаимодействие явно (толчок, передача импульса) – берём CharacterBody. Если вам нужно, чтобы тела сами сталкивались и разлетались, – RigidBody.
- Насколько критична воспроизводимость для отладки, сетевой игры и системы состояний? Чем она критичнее, тем больше доводов в пользу контроллерной модели.

Очень часто в проектах пытаются сделать гибрид: CharacterBody для управляемости, но с возможностью получать физические толчки и влиять на другие тела. Это абсолютно нормальная практика, если подходить к ней дисциплинированно и оформлять взаимодействие как явный контракт. Нельзя просто взять и подправить Velocity чужому CharacterBody из скрипта взрыва, это разрушит его инварианты. Вместо этого вы вводите метод, например ApplyExternalImpulse(Vector2 impulse), который делает это легально, и владелец движения (модель) сам решает, как применить этот импульс с учётом текущих ограничений. С другой стороны, если вы используете RigidBody, но хотите сделать управление более «персонажным», вы строите контроллер поверх симуляции, что будет гасить нежелательные ускорения, фиксировать вращение и принудительно ограничивать скорость. Это тоже возможно, но такая модель требует отдельного слоя-адаптера, иначе логика ограничений размажется по всему коду.

Чтобы закрепить теорию, давайте посмотрим на два минимальных примера, иллюстрирующих разницу в коде.

```
// Симуляционная модель: персонаж как RigidBody2D.
public partial class PhysicsDrivenMover : RigidBody2D
{
    [Export] private float _moveForce = 1200f;
    public override void _PhysicsProcess(double delta)
    {
        var intentX = Input.GetAxis("move_left", "move_right");
        // Наше намерение превращается в физическую силу.
        ApplyCentralForce(new Vector2(intentX * _moveForce, 0f));
    }
}
```

Даже в этом крошечном фрагменте видно архитектурное следствие: убрав на секунду вызов `Input.GetAxis`, мы видим, что намерение напрямую отдаётся на откуп симуляции. Захотим мы заменить ввод – придётся разрывать эту связку, вынося намерение в отдельный слой.

А теперь посмотрим на контроллерный подход:

```
// Контроллерная модель: персонаж как CharacterBody2D.
public partial class CharacterDrivenMover : CharacterBody2D
{
    [Export] private float _speed = 200f;
    [Export] private float _gravity = 900f;
    public override void _PhysicsProcess(double delta)
    {
        var intentX = Input.GetAxis("move_left", "move_right");
        // Мы сами вычислили скорость на основе намерения и физических констант.
        Velocity = new Vector2(intentX * _speed, Velocity.Y + _gravity * (float)delta);
        // И попросили движок нас подвигать, корректно обрабатывая столкновения.
        MoveAndSlide();
    }
}
```

Здесь `Velocity` – это наша собственная переменная (свойство унаследованного класса `CharacterBody`, которое мы полностью контролируем). Вызов `MoveAndSlide` выполняет «грязную работу» по остановке у стен, но не вмешивается в логику изменения скорости. Это и есть та самая граница ответственности: наша модель движения устанавливает скорость, а работа движка на этом шаге – чистая механика перемещения с учётом коллизий.

Выбор между этими двумя подходами – не вопрос моды или абстрактных предпочтений. Это решение о том, в какой части вашего кода будет жить истина о движении. Истина в симуляции означает, что ваша задача – настроить физическую среду так, чтобы она порождала нужный результат. Истина в контроллере означает, что вы сами определяете этот результат, а среда лишь предоставляет необходимые для расчёта факты. Для подавляющего большинства игровых персонажей, особенно в проектах, которые планируют расти и усложняться, контроллерная модель на базе `CharacterBody` является надёжным выбором, который сбережёт вам нервы и время в долгосрочной перспективе. Следующую главу о машине состояний мы уже будем писать, твёрдо стоя на этом фундаменте.

5.4. Реализация машины состояний для движения персонажа

До этого момента мы старательно собирали наш конвейер движения из независимых частей. Ввод превращался в намерение, намерение обрабатывалось моделью движения, а модель применялась к выбранному физическому телу.

Всё работает предсказуемо и линейно ровно до тех пор, пока поведение персонажа остаётся простым: бежим вперёд, прыгаем, падаем. Но игры устроены иначе. Уже в прототипе появляются рывки, подкаты, оглушения, режимы скольжения по льду или плавания под водой. И тогда наш аккуратный код начинает покрываться наростом из флагов и условных операторов, превращая чтение и отладку в кошмар. Проблема не в том, что флагов много, а в том, что они начинают жить собственной жизнью, порождая неявные комбинации. Вы добавляете булево поле `isDashing`, и теперь в методе движения нужно помнить, что во время рывка гравитация действует слабее, а управление не работает. Следом идёт `isStunned`, который требует полностью игнорировать ввод и, конечно же, отменять рывок. Потом выясняется, что оглушение может наступить и в воздухе, а в воздухе прыжок уже недоступен, но персонаж должен падать с особой скоростью. И вот вы сидите перед десятком `if`, вложенных друг в друга, и боитесь дышать, чтобы не сломать хрупкий баланс взаимных исключений. Это называется «взрыв булевых флагов», и для его лечения архитектура располагает мощным, но в то же время простым инструментом – **машиной состояний**.

5.4.1. От неявных флагов к явным состояниям

Машина состояний базируется на идее, что в каждый момент времени персонаж находится ровно в одном из конечного числа режимов поведения. Этот режим определяет, как обрабатывается намерение игрока, какие ограничения накладываются на скорость, какие переходы возможны и что происходит при входе в режим и выходе из него. Вместо того чтобы угадывать поведение по прихотливой комбинации флагов, вы смотрите на текущее состояние и точно знаете, по каким законам живёт персонаж в этом кадре.

Чтобы прочувствовать разницу, давайте представим фрагмент контроллера, который пытается обслужить три режима: обычное движение по земле, прыжок и оглушение. Типичный «домашинный» код выглядит примерно так:

```
public override void _PhysicsProcess(double delta)
{
    var intent = _input.Read();
    if (!isStunned)
    {
        if (_isDashing)
        {
            // Логика рывка: скорость фиксирована, таймер тикает.
            _dashTimer -= delta;
            if (_dashTimer <= 0)
            {
                _isDashing = false;
                _velocity.X *= 0.5f; // Гасим инерцию после рывка.
            }
        }
    }
    else
```

```

{
    // Обычная обработка движения и прыжка.
    if (IsOnFloor())
    {
        if (intent.JumpRequested)
        {
            _velocity.Y = -_jumpForce;
            _isGrounded = false;
        }
        else
        {
            _velocity.X = intent.MoveDirection * _speed;
        }
    }
    else
    {
        _velocity.X = intent.MoveDirection * _airControl;
        _velocity.Y += _gravity * delta;
    }
}
}
else
{
    // В оглушении: ввод игнорируется, ждём таймер.
    _stunTimer -= delta;
    if (_stunTimer <= 0)
        _isStunned = false;
}
Velocity = _velocity;
MoveAndSlide();
// Не забываем обновить флаг _isGrounded после MoveAndSlide.
_isGrounded = IsOnFloor();
}

```

Этот код не является ужасным для пятиминутного прототипа, но архитектурно он уже обречён. Правила рывка, прыжка и оглушения размазаны по одному методу. Чтобы добавить ещё одно состояние, например скольжение, вам придётся вклинить в эту паутину условий и, скорее всего, случайно сломать одно из уже существующих. Приоритеты состояний неочевидны: может ли оглушение прервать рывок? Из кода видно, что да, но только потому, что проверка `_isStunned` стоит первой. Это скрытое знание, которое живёт в порядке строк, а не в контракте. Машина состояний решает эту проблему радикально. Мы выделяем каждое состояние в отдельный класс, который сам отвечает за свою логику и знает, из каких соседних состояний в него можно прийти и в какие он может выйти. Контроллер же превращается в дирижёра, который просто вызывает у текущего состояния метод `Tick` и, если состояние возвращает запрос на переход, производит смену. Правила игры становятся видимыми и изолированными.

5.4.2. Минимальный каркас: состояния, переходы и фазы жизни

Чтобы эта идея не осталась абстрактной, напомним минимальный, но достаточный интерфейс для состояния персонажа. Это не фреймворк, а скорее аккуратная заготовка, которую вы будете наполнять жизнью под нужды своей игры.

```
public interface IPlayerState
{
    /// <summary>Вызывается при входе в состояние.</summary>
    void Enter();
    /// <summary>Владелец состояния. Возвращает запрос на переход или null.</summary>
    IPlayerState Update(PlayerContext context);
    /// <summary>Вызывается при выходе из состояния.</summary>
    void Exit();
}
```

Обратите внимание на три метода, соответствующих трём фазам жизни состояния. Enter иницирует всё, что должно произойти однократно в момент начала режима: сбросить таймер, зафиксировать направление рывка, применить начальный импульс. Update выполняется каждый кадр и получает на вход контекст – объект, который содержит всё необходимое для принятия решений: текущее намерение, факты контакта с миром (на земле ли мы, есть ли столкновение), ссылку на модель движения для изменения скорости и, возможно, сервис времени. В ответ Update возвращает либо null, сигнализируя «я ещё не закончило», либо другое состояние, в которое пора перейти. Наконец, Exit подчищает за режимом: снимает временные ограничения, вызывает необходимые завершающие события.

Сам контроллер состояний, который живёт внутри нашего игрока, выглядит обманчиво просто:

```
public class PlayerStateMachine
{
    private IPlayerState _current;
    public PlayerStateMachine(IPlayerState initial, PlayerContext context)
    {
        _current = initial;
        _current.Enter();
    }
    public void Update(PlayerContext context)
    {
        var next = _current.Update(context);
        if (next != null)
        {
            _current.Exit();
            _current = next;
            _current.Enter();
        }
    }
}
```

Эта простота и есть архитектурная мощь. Она принудительно делает явными две вещи. Во-первых, точка смены состояния теперь строго централизована: вы не можете случайно выйти из рывка в середине кадра, изменение произойдёт только после того, как текущий Update отработает до конца. Во-вторых, гарантируется парность вызовов Enter и Exit, а значит, никаких «забытых» таймеров или висящих ограничений. Вы точно знаете, что при переходе из Dashing в Grounded у первого вызовется Exit, очистив свои флаги, а у второго – Enter, инициализировав затухание скорости.

Контекст PlayerContext – это единый пакет, который вы передаёте всем состояниям, чтобы они не зависели от конкретного класса игрока и могли быть протестированы изолированно.

```
public class PlayerContext
{
    public PlayerIntent Intent;
    public bool Grounded;
    public MovementModel Movement;
    public float Delta;
    // Сюда можно добавить сервис времени, события внешнего мира.
}
```

Теперь давайте посмотрим, как может выглядеть конкретное состояние GroundedState. Оно будет отвечать за обычное движение по земле и инициировать прыжок.

```
public class GroundedState : IPlayerState
{
    public void Enter() { /* Ничего особенного, можно сбросить таймер пыли */ }

    public IPlayerState Update(PlayerContext ctx)
    {
        var mov = ctx.Movement;
        // Обрабатываем намерение движения
        mov.SetHorizontalIntent(ctx.Intent.MoveDirection, ctx.Intent.RunHeld);
        // Применяем гравитацию (нулевая, т. к. стоим на земле, но для устойчивости)
        mov.ApplyGravity(0f);
        // Переход: запрошен прыжок.
        if (ctx.Intent.JumpRequested)
            return new AirborneState(initialJumpVertical: -mov.JumpForce);
        // Переход: потеряна опора (упали с платформы).
        if (!ctx.Grounded)
            return new AirborneState(initialJumpVertical: null);
        return null; // Остаёмся в этом состоянии.
    }

    public void Exit() { /* Здесь можно установить время последнего контакта. */ }
}
```

А теперь состояние AirborneState, которое в нашем примере обрабатывает и начальный прыжок, и свободное падение.


```

public class AirborneState : IPlayerState
{
    private readonly float? _initialJumpVertical;

    public AirborneState(float? initialJumpVertical) => _initialJumpVertical =
initialJumpVertical;

    public void Enter()
    {
        // Если это был прыжок, применяем импульс.
        if (_initialJumpVertical.HasValue)
            // Предполагается метод в MovementModel для прямого управления вертикальной
            скоростью.
            // ctx.Movement.SetVerticalVelocity(_initialJumpVertical.Value); .
            ;
    }

    public IPlayerState Update(PlayerContext ctx)
    {
        var mov = ctx.Movement;
        mov.SetHorizontalIntent(ctx.Intent.MoveDirection, false, airControlMod: 0.6f);
        mov.ApplyGravity(mov.Gravity);
        // Переход: коснулись земли.
        if (ctx.Grounded)
            return new GroundedState();
        return null;
    }

    public void Exit() { /* Сбросили флаг "в воздухе". */ }
}

```

Сразу видно, как легко читается логика. Открыв класс `GroundedState`, вы сразу понимаете, что он делает на земле и при каких условиях передаёт управление воздушному режиму. Никаких сюрпризов из соседних флагов, потому что их просто нет. Если завтра дизайнер попросит, чтобы во время дождя персонаж на земле скользил, а в воздухе – нет, вы измените только `GroundedState`, и это никак не заденет `AirborneState`. Радиус изменений локализован, как мы и договаривались в предыдущих главах.

5.4.3. Интеграция с моделью движения и обработка прерываний

Осталось разобраться, как эта элегантная конструкция стыкуется с нашей тщательно выстроенной моделью движения. Принцип прост: состояния не должны выполнять физическое перемещение или лезть в детали Godot. Их задача – **настраивать** модель движения так, чтобы та, следуя своим внутренним правилам, дала нужный результат. Состояние – это политик, а модель – исполнитель. В методе `Update` каждого состояния мы видим вызовы вроде `ctx.Movement.SetHorizontalIntent(...)` или `ctx.Movement.ApplyGravity(...)`. Это

и есть тот самый язык настройки. Состояние говорит: «Сейчас я хочу, чтобы горизонтальное движение управлялось намерением с таким-то ускорением и максимальной скоростью», и модель применяет эти настройки к своему внутреннему состоянию Velocity. Само же применение к физическому телу (присваивание Velocity и вызов MoveAndSlide) происходит, как и раньше, в _PhysicsProcess нашего тонкого контроллера Player, который следует за вызовом _stateMachine.Update(context). Это сохраняет наш драгоценный детерминизм: сначала мы коллективно решаем, какой должна быть скорость, а потом разовым актом воплощаем это решение в мире. Этот же подход идеально ложится на обработку **прерываний** – событий высокого приоритета вроде получения урона с нокдауном или начала кат-сцены. Представьте, что в персонажа попал файербол, и независимо от того, бежал он, прыгал или махал мечом, он должен перейти в состояние Stunned. Если мы оставим этот переход на откуп самим состояниям, каждое из них должно будет знать об оглушении и проверять соответствующий флаг. Это вернёт нас к связанности.

Вместо этого мы вводим простую процедуру в контроллере игрока. До вызова _stateMachine.Update, когда все внешние события уже собраны, мы проверяем, не хотим ли мы принудительно сменить состояние, минуя обычную логику переходов.

```
public partial class Player : CharacterBody2D
{
    private PlayerStateMachine _stateMachine;
    private PlayerContext _ctx;
    public override void _PhysicsProcess(double delta)
    {
        // 1. Собираем контекст.
        _ctx.Intent = _input.Read();
        _ctx.Grounded = IsOnFloor();
        _ctx.Delta = delta;
        // 2. Обрабатываем прерывания (высокий приоритет).
        if (_stunRequested)
        {
            _stunRequested = false;
            _stateMachine.ForceChange(new StunnedState(2.0f), _ctx);
        }
        // 3. Обновляем текущее состояние (и, возможно, переходим).
        _stateMachine.Update(_ctx);
        // 4. Забираем итоговую скорость у модели и применяем.
        Velocity = _ctx.Movement.Velocity;
        MoveAndSlide();
    }
    // Метод, вызываемый из системы урона.
    public void ApplyStun()
    {
        _stunRequested = true;
    }
}
```

Теперь логика прерывания централизована. Состояния остаются в счастливом неведении о том, что их могут принудительно завершить. Ответственность за приоритеты лежит на контроллере. Это вносит ясность и убивает последнюю причину для невнятных проверок вроде `if (isStunned)` внутри каждого состояния.

5.4.4. Проектирование переходов и предотвращение хаоса

Машина состояний решила проблему флагов, но тут же подбрасывает новую – неаккуратно спроектированные переходы могут создать не менее тугой узел. Когда каждое состояние знает, в какое другое состояние оно может перейти, и напрямую создаёт его экземпляр, между ними возникает связанность. В небольшом наборе из трёх-четырёх состояний это совершенно нормально и не требует усложнений. Однако если количество режимов переваливает за десяток, появляется смысл ввести посредника – **фабрику** или **словарь переходов**, чтобы сами состояния не зависели от конкретных типов друг друга.

Но гораздо важнее – дисциплина самих переходов. Переход не должен быть хаотичным действием, которое разрывает логику состояния пополам. Отличное правило: `Update` состояния должен лишь вычислить, какой будет следующая фаза, и вернуть её из метода. Никаких других побочных эффектов вроде неожиданного обнуления скорости до вызова `Exit`. Если переход требует сложных вычислений, они выполняются внутри `Update`, а любое воздействие на мир (например, генерация события «персонаж приземлился») должно происходить либо в `Exit` старого состояния, либо в `Enter` нового.

Придерживаясь этого сценария `Compute` → `Return` → `Commit`, вы защищаетесь от «растянутых переходов», когда после возврата нового состояния из `Update` вдруг выясняется, что старое состояние ещё пытается что-то сделать в том же кадре. Контроллер состояний выполняет `Exit` строго после того, как `Update` отработал, и строго перед `Enter` нового. Это и есть та самая транзакционность, которая делает поведение машины предсказуемым.

Ещё один враг порядка – анимации. В следующей главе мы будем говорить об этом подробно, но уже сейчас важно запомнить твёрдое правило: состояние не должно говорить анимации, что делать. Вместо этого оно публикует объективный факт: «я – состояние `Dashing`», или «скорость `X` равна 10». А уже визуальный слой, отделённый глухой стеной зависимости от движения, решает, какой анимационный клип проиграть. Смешение этих двух слоёв приводит к циклическим зависимостям, когда анимация меняет скорость, а скорость меняет анимацию, и всё это происходит в рамках одного кадра. Такую систему невозможно отлаживать.

В итоге машина состояний, построенная на этих принципах, не усложняет, а укрощает сложность. Она заменяет молчаливую и запутанную систему джентльменских соглашений между флагами на чёткий, видимый и расширяемый контракт. Каждый новый режим – это чистый лист в виде класса, где

вы описываете его законы. А контроллер остаётся тем же дирижёром, которому не нужно знать партитуру каждого инструмента, чтобы оркестр звучал слаженно. Это именно то, ради чего мы и затевали весь этот архитектурный разговор.

5.5. Разделение поведения и анимации: односторонний поток данных

Мы подошли к тому рубежу, где игра начинает обретать не только логику и управление, но и визуальную душу. Персонаж больше не абстрактная точка с вектором скорости – он должен бежать, подпрыгивать, выполнять рывок, и всё это должно выглядеть естественно и сочно. Однако именно на стыке поведения и анимации кроется одна из самых коварных архитектурных ловушек. Стоит один раз разрешить анимационному клипу напрямую управлять состоянием персонажа, и вы рискуете превратить код в непредсказуемый клубок зависимостей, который ломается при каждой смене визуального стиля.

В этом разделе мы зафиксируем строгую, но при этом удивительно освобождающую дисциплину: анимация не командует, она слушается. Она является потребителем фактов о мире, но ни при каких условиях не диктует, персонажу бежать или стоять, разрешён ли сейчас прыжок и когда закончится рывок. Такой подход не усложняет, а упрощает систему, делая её расширяемой и устойчивой к будущим изменениям.

5.5.1. Ловушка обратной связи: когда анимация начинает управлять поведением

Представьте, что вы реализуете рывок. Вы делаете красивый анимационный клип, в котором персонаж на полсекунды превращается в размытую линию. Самое простое, что приходит в голову, – поставить в конце клипа вызов метода `OnDashFinished`, который сбросит флаг `_isDashing` обратно в `false`. Это работает из коробки, требует пары строк и кажется очень логичным: ведь рывок и правда должен заканчиваться вместе с анимацией.

```
// Опасный путь: анимация становится источником истины.
public partial class PlayerAnimDriver : Node
{
    [Export] private AnimationPlayer _anim;
    private bool _isDashing;
    public void StartDash()
    {
        _isDashing = true;
```

```

    _anim.Play("dash");
}
// Вызывается по окончании клипа через AnimationPlayer.
private void OnDashFinished()
{
    _isDashing = false;
}
}

```

Проблема этого подхода не в синтаксисе, а в фундаментальном сдвиге ответственности. Истина о том, находится ли персонаж в состоянии рывка, теперь хранится не в машине состояний и не в модели движения, а в тайминге визуального клипа. Механика игры попадает в зависимость от ресурса, который может быть случайно изменён художником, ускорен для динамичности или вовсе удалён при редизайне. Если другой персонаж будет использовать другой клип с другой скоростью, механика неожиданно изменится. Если клип прервётся (например, из-за смешивания с анимацией получения урона), рывок может вообще никогда не завершиться – персонаж застынет в этом состоянии навсегда.

Ещё страшнее ситуация, когда в анимацию добавляют так называемые «события анимации» (animation events) и через них управляют критическими игровыми моментами: «в этом кадре активировать хитбокс», «на этом движении разрешить поворот», «на этом кадре оттолкнуть персонажа вперёд». В небольшом прототипе это выглядит удобно: всё видно на таймлайне. Но по мере роста игры выясняется, что правила урона и движения оказались распределены между кодом и анимационными ресурсами, а отладка превращается в гадание «почему удар прошёл?» – «потому что в клипе enemy_attack_v3_2 хитбокс активируется на 0.12 секунды». Механика становится хрупкой, её невозможно ни нормально протестировать, ни перенести на другой тип персонажа.

Архитектурно грамотный подход требует обратного: **поведенческий слой должен быть единственным источником истины**, а анимация – лишь её визуальным отражением. Это означает, что не клип говорит, когда закончился рывок, а машина состояний, опираясь на таймер и собственные правила, сама решает, что рывок завершён. Анимация же просто реагирует на это событие, проигрывая соответствующий переход.

5.5.2. Контракт визуального представления: факты вместо команд

Как же тогда анимация узнаёт, что ей делать? Через явный, односторонний контракт. Поведенческий слой (контроллер, машина состояний, модель движения) формирует небольшой пакет данных – **снимок визуального состояния** (visual state snapshot). Этот пакет содержит не команды «проиграй то-то», а объективные факты о том, что происходит с персонажем прямо сейчас.

Такой снимок может выглядеть как простая структура:

```
/// <summary>Факты о персонаже, предназначенные только для чтения визуальным слоем.</summary>
public struct PlayerVisualState
{
    public PlayerMode Mode;
    public Vector2 Velocity;      // Текущая скорость (можно только модуль и направление).
    public bool IsGrounded;
    public bool HasMovementIntent; // Есть ли осознанное намерение двигаться.
}
```

Анимационный контроллер, существующий отдельно от логики игрока, в каждом кадре получает этот снимок и на его основе принимает все решения о том, какой клип выбрать, как смешать анимации и какие визуальные эффекты активировать. Ключевое слово здесь – «на основе». Он не имеет права менять поля в `PlayerVisualState` или пытаться через них повлиять на поведение.

В коде Godot это часто реализуется через отдельный узел `PlayerAnimationController`, который в своём `_Process` читает состояние родительского игрока:

```
public partial class PlayerAnimationController : Node
{
    [Export] private AnimationPlayer _animator;
    [Export] private PlayerController _player; // Ссылка на того, кто владеет поведением.
    public override void _Process(double delta)
    {
        // 1. Получаем актуальный снимок поведения.
        PlayerVisualState state = _player.GetVisualState();
        // 2. Переводим факты в параметры анимационного графа.
        _animator.Set("parameters/Mode", (int)state.Mode);
        _animator.Set("parameters/Speed", state.Velocity.Length());
        _animator.Set("parameters/Grounded", state.IsGrounded);
        _animator.Set("parameters/MovementIntent", state.HasMovementIntent);
    }
}
```

Граница становится непроницаемой в правильную сторону: `PlayerController` вообще не знает о существовании `PlayerAnimationController`, а значит, не может случайно начать управлять анимацией. В то же время `PlayerAnimationController` полностью зависит от контракта `GetVisualState()` и не лезет во внутренности движения. Если дизайнер решит полностью сменить визуальный стиль и набор клипов, изменения будут локализованы исключительно в этом одном узле. Правила игры останутся нетронутыми.

Это разделение даёт ещё один мощный инструмент – возможность **заменять визуальный слой** без переписывания логики. Представьте, что ваш персонаж должен выглядеть по-разному в зависимости от экипировки или вы хотите использовать одну и ту же логику для игрока и для ботов, но с разными анимациями. Вы просто создаёте другую реализацию анимационного

контроллера, который читает те же самые факты, и подключаете его. Поведение остаётся единым. В мире, где анимация командует, вам пришлось бы копировать и модифицировать поведенческие скрипты, порождая сотни новых багов.

5.5.3. Синхронизация, тайминги и проблема *root motion*

Сразу возникает практический вопрос: как быть с таймингами? Ведь удар мечом должен нанести урон не абстрактно, а именно в тот визуальный момент, когда лезвие соприкасается с врагом. Окончание рывка должно выглядеть плавно, без визуального разрыва между тем, когда персонаж перестал двигаться рывком, и тем, когда закончился клип. Ответ кроется в том, чтобы **владельцем времени оставалось поведение**, а анимация лишь синхронизировалась с ним.

Рассмотрим состояние атаки. В коде оно может быть реализовано не как один непрерывный блок, а как три последовательные фазы: подготовка (Windup), активная фаза (Active), восстановление (Recovery). Каждая фаза – это просто интервал времени, заданный таймерами внутри состояния.

```
public class AttackState : IPlayerState
{
    private float _timer;
    private AttackPhase _phase = AttackPhase.Windup;
    public IPlayerState Update(PlayerContext ctx)
    {
        _timer += ctx.Delta;
        switch (_phase)
        {
            case AttackPhase.Windup:
                if (_timer >= WindupDuration) { _phase = AttackPhase.Active;
ActivateHitbox(); }
                break;
            case AttackPhase.Active:
                if (_timer >= WindupDuration + ActiveDuration) { _phase = AttackPhase.
Recovery; DeactivateHitbox(); }
                break;
            case AttackPhase.Recovery:
                if (_timer >= TotalDuration) return new GroundedState();
                break;
        }
        return null;
    }
    // Публичные данные для визуального слоя.
    public AttackPhase CurrentPhase => _phase;
    public float NormalizedTime => _timer / TotalDuration;
}
```

Анимационный контроллер, получив доступ к `CurrentPhase` и нормализованному времени из снимка визуального состояния, может идеально синхронизировать клип: для `Windup` проигрывается замах, для `Active` – удар и так далее. Если клип анимации немного короче или длиннее механического окна, визуальный слой может растянуть или сжать его через параметр скорости проигрывания, либо смешать с другими клипами. Главное, что истина о том, когда удар попадает по врагу, остаётся в коде, в фазе `Active`, и не зависит от художественных решений аниматора.

Точно так же решается проблема с **root motion** – техникой, когда анимация содержит перемещение корневого объекта и персонаж визуально движется вместе с клипом. Сама по себе идея неплоха, но её реализация часто скатывается к тому, что анимация напрямую двигает персонажа, отбирая авторство у модели движения. В устойчивой архитектуре анимация не должна перемещать персонажа самовольно. Вместо этого она через специальный контракт может лишь **предоставить расчётное смещение** за кадр как предложение, своего рода ещё один источник намерения для движения. Модель движения получает это смещение, совмещает его с текущей скоростью, проверяет коллизии (чтобы персонаж не протолкнуло в стену из-за анимации) и только потом применяет итоговое перемещение.

```
public class MovementModel
{
    // ... поля ...
    // Обычный вход с намерением игрока.
    public void ApplyIntent(PlayerIntent intent, float delta) { /* ... */ }
    // Вход с root motion смещением от анимации.
    public Vector2 CalculateMovement(Vector2 rootMotionDelta, float delta)
    {
        // Добавляем смещение к текущей скорости, ограничиваем, проверяем.
        Velocity += rootMotionDelta / delta; // Грубое приближение.
        // ... дальнейшая обработка ...
        return Velocity * delta; // Возвращаем итоговое перемещение для MoveAndSlide.
    }
}
```

Таким образом, авторитет над физикой сохраняется за движением. Анимация подсказывает, но не командует. И если в будущем вы решите заменить обычный `AnimationPlayer` на процедурную анимацию, всё, что потребуется, – изменить способ вычисления `rootMotionDelta` в визуальном слое. Правила движения не изменятся.

5.5.4. Влияние на командную работу, отладку и мультиплеер

Возможно, сейчас, работая над проектом в одиночку, вам кажется, что строгое разделение – это лишняя бюрократия. Однако представьте, что через месяц к проекту присоединится аниматор, который будет править клипы

и их тайминги, а вы – заниматься балансом и новыми способностями. Если анимация управляет поведением, вы обречены бесконечно отлавливать мигрирующие баги, возникшие после того, как кто-то случайно сдвинул ключевой кадр. С чёткой границей, выраженной через `PlayerVisualState`, ваши зоны ответственности чётко разделены. Аниматор может менять визуал как угодно, пока машина состояний и контракт фактов остаются нетронутыми. Это в десятки раз ускоряет итоговую полировку игры.

Отладка также разделяется на два независимых слоя. Если персонаж не двигается, но анимация бега проигрывается – проблема в движении, и вы идёте в `MovementModel`. Если персонаж бежит, но визуально стоит – проблема в анимационном контроллере или его связке с клипами, и вы лезете только туда. Без этой границы вы получаете единую кашу, где логика и визуал переплетены настолько, что любое отклонение может быть вызвано чем угодно.

Наконец, для тех, кто смотрит в сторону сетевой игры, разделение становится критически важным. Анимация недетерминирована по своей природе: на разных клиентах кадры могут пропускаться по-разному, скорость проигрывания может немного отличаться. Если от неё зависят игровые правила, сетевая синхронизация превращается в кошмар. Когда же реплицируются только факты поведения (Режим=`Airborne`, Скорость= $(10, -5)$), а каждый клиент локально визуализирует их как может, сетевая модель становится на порядок проще и устойчивее. Это не просто «фича для мультиплеера», а общий принцип: чем чище разделены намерение и его визуальное воплощение, тем легче вашей архитектуре масштабироваться в любом направлении.

В этом и заключается главный урок раздела: анимация – это потрясающий инструмент для создания атмосферы и отзывчивости игры, но она ни в коем случае не должна становиться фундаментом для игровой логики. Построив односторонний мост из фактов поведения в визуальный слой, вы не только обезопасите свой код от каскадных поломок, но и получите свободу творить, экспериментировать и менять визуальный облик игры без страха сломать то, что работает. А это, пожалуй, одна из главных целей хорошей архитектуры.

6

Мир и взаимодействие

6.1. Коллизионные слои и маски: проектирование политики взаимодействия

Когда вы только начинаете собирать игровой мир в Godot, взаимодействие объектов кажется делом нехитрым. Персонаж должен упираться в стены, враги – получать урон от пуль, а монетки – подбираться при касании. Вы бросаете на сцену `CharacterBody2D`, вешаете `CollisionShape2D`, и всё работает. Но чем больше сущностей появляется в игре, тем чаще вы начинаете наткаться на странные, почти мистические проблемы.

Почему подобранная аптечка вдруг заблокировала пролетающий мимо файербол? Отчего враг споткнулся о невидимый триггер, предназначенный только для игрока? И почему при добавлении нового типа снаряда пришлось перепроверять чуть ли не каждый узел на сцене? В такие моменты и приходит понимание, что коллизии – это не просто «чтобы не проходить сквозь текстуры», а полноценная архитектурная система, требующая осмысленной, декларативной политики. И фундаментом этой политики являются **коллизионные слои и маски**.

6.1.1. Декларация ролей и политики обнаружения

В основе механики столкновений Godot лежит простая, но очень мощная идея: каждый физический объект одновременно отвечает на два вопроса. Первый: «Кто я такой?» Ответ записывается в **коллизионный слой** (`collision layer`). Это ваша декларация о принадлежности к одной или нескольким группам. Второй: «Кого я должен замечать?» Ответ на него хранится в **кол-**

лизионной маске (collision mask). Маска определяет, объекты из каких слоёв будут для нас «видимы» и вызовут реакцию – будь то жёсткое столкновение или сигнал от триггера.

Представьте себе театральное представление. Слой – это бейдж на груди у каждого участника: «Актёр», «Зритель», «Осветитель», «Реквизит». Этот бейдж не говорит, что человек делает; он лишь обозначает его роль в системе. Маска же – это инструкция для каждого: «Актёр взаимодействует со сценой и другими актёрами», «Осветитель видит только пульт и софиты», «Зритель не должен выходить на сцену и сталкиваться с реквизитом». Если выдать осветителю инструкцию «взаимодействовать со всеми», он начнёт мешать актёрам, а зрители начнут застревать в декорациях. Когда вы мыслите слоями и масками, вы перестаёте думать о коллизиях как о попарном «А столкнётся с В» и начинаете проектировать граф допустимых взаимодействий на уровне самого мира. Первый шаг к созданию такой политики – инвентаризация всех сущностей в вашей игре и выделение устойчивых ролей. Для типичного платформера этот список может выглядеть так:

- **Мир (Environment)**: стены, платформы, пол, статические препятствия.
- **Игрок (Player)**: управляемый персонаж.
- **Враги (Enemy)**: все противники, с которыми игрок сталкивается.
- **Снаряды игрока (PlayerProjectile)**: всё, что выпускает игрок во врагов.
- **Предметы (Pickup)**: монетки, аптечки, ресурсы, которые подбираются при касании.
- **Зоны обнаружения (Trigger)**: невидимые области для активации скриптов, открывания дверей и ловушек.

Обратите внимание: на этом этапе мы не думаем, кто с кем сталкивается. Мы лишь присваиваем логические роли, которые имеют разные причины для изменения. Например, «Снаряды игрока» и «Снаряды врагов» часто должны вести себя по-разному, поэтому это разные роли, даже если визуально они похожи. А вот добавить отдельную роль для «летающих врагов» не стоит, если они сталкиваются с окружением по тем же правилам, что и наземные. Различие в их движении – это вопрос поведения, а не физического взаимодействия.

После того как роли определены, мы формулируем **политику масок** – правила, определяющие, какой слой кого видит. Эта политика фиксирует, какие взаимодействия в принципе возможны в нашем мире. Вот как могла бы выглядеть такая политика для нашего примера:

Игрок видит **Мир** (чтобы упираться в стены), **Врагов** (чтобы не проходить сквозь них и получать урон при касании) и **Предметы** (чтобы подбирать их). Он **не видит** вражеские снаряды и зоны обнаружения, потому что столкновение со снарядом обрабатывает сам снаряд, а триггеры – это отдельный канал взаимодействия.

Враг видит **Мир** и **Игрока**, но, возможно, не видит **Предметы** (они ему не нужны) и другие **Снаряды врагов** (чтобы они не сталкивались друг с другом, если это не требуется по дизайну).

Снаряд игрока видит **Мир** и **Врагов**: его задача – врезаться в препятствие или в цель, но не в игрока или другие снаряды.

Ключевая мысль, которую стоит здесь усвоить: маска не обязана быть симметричной. То, что игрок видит врага, не означает, что враг обязан видеть игрока через свою маску. В нашем примере за столкновение с врагом может отвечать сам игрок (если он атакует телом) либо снаряд. Мы можем настроить мир так, чтобы враги просто были «осязаемыми» для игрока, но сами не инициировали контакт. Это мощный инструмент для уменьшения связанности: мы сокращаем количество каналов, по которым системы должны знать друг о друге.

6.1.2. Разделение каналов: блокировка и обнаружение

Одна из главных причин, по которой коллизионный хаос поглощает проект, – смешивание двух принципиально разных задач под одной вывеской «столкновения». Первая задача – **блокирующая геометрия**, физическое препятствие, которое не даёт объектам пройти друг сквозь друга. Стены, платформы и персонажи обычно блокируют друг друга. Вторая задача – **обнаружение**, то есть получение сигнала о том, что два объекта пересеклись, без какого-либо физического сопротивления. Это зоны, активирующие скрипты, предметы, которые нужно подобрать, или датчики радиуса действия.

Если эти две задачи решаются одними и теми же слоями и масками, предсказуемость мира рушится. Вы начинаете сталкиваться с «невидимыми стенами», когда триггер, размещённый на входе в локацию, вдруг оказывается непроходимым для персонажа, потому что вы забыли убрать у него маску блокировки. Или наоборот – враги начинают проваливаться сквозь пол, потому что вы, настраивая им обнаружение игрока, случайно поменяли им маску, убрав из неё слой окружения.

Архитектурно зрелое решение состоит в том, чтобы явно разделить эти два канала. В Godot для этого есть два основных типа узлов: `CharacterBody2D`/`RigidBody2D` для блокирующих объектов и `Area2D` для зон обнаружения. Важно не только использовать правильный тип узла, но и осмысленно настроить им слои и маски.

Блокирующие объекты должны иметь маску, которая включает только те слои, с которыми они действительно должны сталкиваться физически. Как правило, это слой самого окружения и, возможно, других персонажей. Система движения персонажа, которую мы построили в предыдущих главах, будет опираться именно на эти факты: вызов `MoveAndSlide` обработает столкновение и сообщит нам, что мы упёрлись в стену или стоим на полу.

Зоны обнаружения (`Area2D`), напротив, должны быть настроены на те слои, которые они «слушают». Триггер у входа в логово босса будет иметь маску, в которую входит только слой Игрок. Подбираемый предмет аналогично будет ждать контакта только с игроком. Благодаря этому вы можете быть абсолютно уверены, что обработчик сигнала `body_entered` этого предмета никогда

не сработает на врага или на снаряд. Ваш код обработки подбора становится чище: он получает только те события, которые допущены политикой мира, и может сразу переходить к доменной логике – добавлению предмета в инвентарь и воспроизведению звука.

6.1.3. Проектирование политики через константы и владение

Политика слоёв и масок – это глобальный контракт вашего проекта. Нельзя, чтобы он был размазан по сотне сцен и префабов и держался на честном слове. Как только вы определились с логическими ролями, их нужно материализовать в коде в виде единственного источника истины. Самый простой и действенный способ – создать статический класс с константами.

```
using Godot;
public static class CollisionPolicy
{
    // Объявляем логические роли как именованные константы.
    // Значение – это степень двойки, сдвинутая на номер бита.
    public const uint World = 1u << 0;    // 0001, бит 1.
    public const uint Player = 1u << 1;    // 0010, бит 2.
    public const uint Enemy = 1u << 2;    // 0100, бит 3.
    public const uint PlayerProjectile = 1u << 3; // 1000, бит 4.
    public const uint Pickup = 1u << 4;    // 10000, бит 5.
    public const uint Trigger = 1u << 5;   // 100000, бит 6.
    // Политики масок для разных ролей.
    public const uint PlayerMask = World | Enemy | Pickup; // Игрок видит мир, врагов
и предметы.
    public const uint EnemyMask = World | Player;           // Враг видит мир и игрока.
    public const uint PlayerProjectileMask = World | Enemy; // Снаряд игрока видит мир
и врагов.
    public const uint PickupMask = Player;                  // Предмет видит только игрока.
    public const uint TriggerMask = Player;                 // Триггер видит только игрока.
}
```

Теперь, когда вы создаёте новый объект, его принадлежность к слою и его политика видимости устанавливаются в одном-единственном месте, со ссылкой на понятные именованные константы. Сравните два подхода. Первый – хаотичный, где в каждом скрипте написано магическое число, которое ещё нужно мысленно перевести в биты: `CollisionLayer = 4`; `CollisionMask = 5`;. Через неделю вы уже не вспомните, что такое 5. Второй – архитектурный, где в `_Ready` вашего снаряда написано:

```
public override void _Ready()
{
    CollisionLayer = CollisionPolicy.PlayerProjectile;
    CollisionMask = CollisionPolicy.PlayerProjectileMask;
}
```

Этот подход решает сразу несколько проблем. Во-первых, политика становится видимой и читаемой. Вы смотрите на константу `PlayerProjectileMask` и видите, что она состоит из `World | Enemy`. Вам не нужно гадать. Во-вторых, появляется единая точка для изменений. Если геймдизайнер решит, что предметы должны подбираться не при касании, а по кнопке, и коллизия для пикапов больше не нужна, вы просто уберёте `Pickup` из `PlayerMask`. Ни одной правки в сценах или скриптах предметов не потребуется. В-третьих, отладка становится структурной: вы всегда можете поставить точку останова на строке `CollisionMask = ...` и увидеть, что видит объект, или заглянуть в статический класс и мгновенно считать текущую политику мира.

6.1.4. Архитектурная дисциплина: избавляемся от хаоса в коде

Кульминацией этого подхода является то, как он очищает ваш код обработки столкновений. Без внятной политики, обрабатывая сигнал `body_entered` у какого-нибудь `Area2D`, вы почти всегда пишете цепочку проверок: `if (body is Player player), else if (body is Enemy enemy), else if (body is Boss boss)`. Этот код – симптом того, что ваша коллизионная маска слишком широка, а роли не формализованы. Вы используете C# для фильтрации того мусора, который сами же и разрешили на уровне движка.

С чёткой политикой, описанной в `CollisionPolicy`, ситуация меняется кардинально. Если ваш триггер должен реагировать только на игрока, вы настраиваете его маску как `CollisionPolicy.TriggerMask`. Всё. Теперь сигнал `body_entered` на этом триггере **никогда** не сработает на врага или на файербол. Ваш обработчик перестаёт быть универсальным фильтром и превращается в сфокусированный метод, который точно знает, что зашёл тот, кому дозволено.

```
public partial class CheckpointTrigger : Area2D
{
    public override void _Ready()
    {
        // Политика настроена на уровне узла: только игрок вызывает сигнал.
        CollisionLayer = 0; // Триггеру не нужен свой слой, если он невидим для других.
        CollisionMask = CollisionPolicy.PlayerMask;
        BodyEntered += OnBodyEntered;
    }
    private void OnBodyEntered(Node2D body)
    {
        // Мы со 100%-ной уверенностью можем привести тип.
        var player = (PlayerController)body;
        player.RespawnPoint = GlobalPosition;
    }
}
```

Здесь исчезает целый класс багов, связанных с неверным приведением типов или неожиданными контактами. Ваш код становится строже, а его на-

мерения – прозрачнее. Когда новый разработчик откроет скрипт чекпоинта, он сразу увидит в `_Ready` строчку `CollisionMask = CollisionPolicy.PlayerMask` и поймёт: этот объект спроектирован так, чтобы видеть только игрока. И если он захочет, чтобы на чекпоинт можно было телепортироваться, проезжая на боевой колеснице, он будет точно знать, где именно нужно внести изменения – в классе `CollisionPolicy`, не обшаривая весь проект в поисках разбросанных битовых масок.

Такой декларативный подход к проектированию взаимодействий является одним из краеугольных камней масштабируемой архитектуры. Он переводит вопрос коллизий из разряда «магических настроек» в инженерное русло. Потратив немного времени на составление матрицы слоёв и её оформление в коде в начале разработки, вы сэкономите десятки часов в будущем, которые иначе ушли бы на отлов истеричных багов в духе «почему мой снаряд заблокировался о собственный UI». Игра станет более стабильной, предсказуемой и, в конечном счёте, более приятной как для вас, так и для ваших игроков.

6.2. Системы взаимодействия: контрактная модель вместо прямых вызовов

Нажатие кнопки «Е» перед дверью – самое первое, что хочется запрограммировать, оживляя мир игры. В прототипе это делается за минуту: повесить на игрока проверку `Input.IsActionJustPressed`, найти ближайший объект класса `Door` и вызвать у него метод `Open()`. Шаг простой и логичный, но в нём скрыта мина замедленного действия. Код, открывающий дверь, вырос в классе `Player`, и теперь `Player` знает о существовании дверей. Добавляем сундук – `Player` узнаёт о сундуках, терминал – узнаёт о терминалах, NPC – узнаёт о диалогах. Через месяц активной разработки ваш центральный скрипт превращается в энциклопедию всех интерактивных механик мира, и любое расширение требует правок в этом центре.

Архитектурная цель системы взаимодействий – разорвать эту цепочку знаний. Игрок должен оставаться системой, отвечающей за движение, состояния и намерения, но ничего не знать о том, как устроена конкретная дверь. Он должен лишь сообщать миру о своём желании совершить действие, а мир – самостоятельно решать, что это действие означает и допустимо ли оно сейчас.

6.2.1. Протокол взаимодействия: инициатор, цель и контекст

В хорошо спроектированной системе взаимодействий каждое нажатие на кнопку проходит не как вызов конкретного метода, а как соблюдение фор-

мального контракта. Участников этого контракта всегда четыре: инициатор, выражающий намерение; целевой объект, с которым взаимодействуют; контекст, описывающий условия «здесь и сейчас»; и результат, объясняющий, что именно произошло. Игрок-инициатор не говорит «я открываю дверь». Вместо этого он передаёт по цепочке запрос: «я хочу взаимодействовать с объектом, который сейчас передо мной». А дверь, получив этот запрос вместе с контекстом, уже сама решает, открываться ей сейчас или нет.

Прямой вызов `door.Open()` разрушает эту ответственность. Он закрепляет знание о внутреннем устройстве мира внутри класса игрока. Сегодня это дверь, завтра – сундук с проверкой ключа, послезавтра – терминал, которому нужно питание. И вот класс `Player` уже содержит цепочки `if-else` или `switch`, которые проверяют тип цели, чтобы дёрнуть нужный метод. Добавление нового типа взаимодействия требует переписывания игрока, а значит, мир перестаёт быть расширяемым локально.

```
// Прямой вызов, который порождает архитектурную хрупкость.
using Godot;
public partial class Player : Node
{
    [Export] private NodePath _targetPath;
    public override void _Process(double delta)
    {
        if (!Input.IsActionJustPressed("interact"))
            return;
        var door = GetNodeOrNull<Door>(_targetPath);
        if (door != null)
        {
            door.Open();
            return;
        }
        var chest = GetNodeOrNull<Chest>(_targetPath);
        if (chest != null)
        {
            chest.Open();
            return;
        }
    }
}
```

Ключевой момент: проблема не в синтаксисе, а в том, что игрок принимает решения за мир. Правила доступа к сундуку, звук открывания, проверка на наличие ключа, анимация – всё это начинает расползаться из классов двери и сундука обратно в `Player`, потому что именно там живёт логика вызова. В итоге система теряет предсказуемость: изменение правил одного объекта может потребовать правок в плеере, а правка плеера ради нового NPC может сломать сундуки.

Чтобы выправить ситуацию, мы вводим интерфейс – роль, которую может принять любой объект, готовый к взаимодействию. Для инициатора существует только эта роль, а не конкретные классы. Вместо `door.Open()` мы вызыва-

ем у цели метод `Interact()`, предварительно проверив методом `CanInteract()`, разрешено ли действие в текущем контексте.

```
public interface IInteractable
{
    bool CanInteract(InteractContext context);
    void Interact(InteractContext context);
}
```

Контекст здесь – отдельный объект, который система взаимодействия собирает на стороне инициатора и передаёт цели. Разработчик двери или сундука точно знает, какую информацию он получит для принятия решения: ссылку на инвентарь, положение инициатора, возможно, активные квесты. Этот контекст не должен быть ссылкой на весь класс игрока, потому что тогда цель опять начнёт зависеть от его внутреннего устройства. Вместо этого контекст – небольшой пакет данных, который можно собрать в тестовом окружении и проверить логику двери без запуска всей игры.

```
public class InteractContext
{
    public Vector2 InitiatorPosition;
    public IInventoryService Inventory; // Доступ к инвентарю через контракт,
                                     // а не конкретный класс.
    // ... любые другие данные мира, не привязанные к Player.
}
```

Теперь посмотрим, как выглядит дверь, реализующая такой контракт. Она инкапсулирует все свои правила, не протекая в плеер.

```
public class KeyDoor : Node, IInteractable
{
    [Export] private string _requiredKeyId = "red_key";
    public bool CanInteract(InteractContext context)
    {
        return context.Inventory.HasItem(_requiredKeyId);
    }
    public void Interact(InteractContext context)
    {
        context.Inventory.RemoveItem(_requiredKeyId);
        QueueFree(); // Дверь исчезает.
    }
}
```

Плеер же целиком освобождается от знания о том, кто перед ним. Он просто запрашивает взаимодействие, и контракт отрабатывает сам.

6.2.2. Разделение обнаружения и исполнения

Остаётся вторая половина проблемы – как плеер узнаёт, с какой именно дверью или сундуком он хочет взаимодействовать. Эта задача распадается

на две независимые ответственности: обнаружение цели и исполнение контракта. Их смешивание – вторая по частоте архитектурная ошибка, когда код обнаружения (проверка радиуса, направление взгляда, приоритет) оказывается в том же методе плеера, что и вызов `Open()`.

Обнаружение цели – чисто инфраструктурная задача, которая опирается на нашу коллизионную политику из раздела 6.1. Мы создаём специальный слой `Interaction` и помечаем им все интерактивные объекты. Затем на игрока вешается `Area2D`-детектор, настроенный на этот слой. Его единственная работа – в любой момент предоставить отсортированный список кандидатов, готовых к взаимодействию.

```
using Godot;
using System.Linq;
public partial class InteractionDetector : Area2D
{
    public override void _Ready()
    {
        CollisionLayer = 0;
        // Видим только слой Interaction, определённый в CollisionPolicy.
        CollisionMask = CollisionPolicy.Interaction;
    }
    public IInteractable GetBestTarget(InteractContext context)
    {
        var candidates = GetOverlappingAreas()
            .Concat<GodotObject>(GetOverlappingBodies())
            .OfType<IInteractable>()
            .Where(i => i.CanInteract(context));
        // Простая политика приоритета: ближайший объект.
        return candidates.OrderBy(c => c.Position.DistanceTo(GlobalPosition)).
            FirstOrDefault();
    }
}
```

Исполнение же вынесено в отдельный сервис, который связывает детектор с миром.

```
public class InteractionService
{
    private readonly InteractionDetector _detector;
    public InteractionService(InteractionDetector detector) => _detector = detector;
    public bool TryInteract(InteractContext context)
    {
        var target = _detector.GetBestTarget(context);
        if (target == null)
            return false;
        target.Interact(context);
        return true;
    }
}
```

Теперь `PlayerController`, о котором мы так заботились в предыдущих главах, остаётся девственно чистым. Его `_PhysicsProcess` не содержит ни одной

проверки типа цели. Получив намерение взаимодействия через `PlayerIntent`, он просто вызывает сервис.

```
public partial class PlayerController : CharacterBody2D
{
    private InteractionService _interaction;
    // ...
    public override void _Ready()
    {
        var detector = GetNode<InteractionDetector>("%Detector");
        _interaction = new InteractionService(detector);
    }
    public override void _PhysicsProcess(double delta)
    {
        // ... движение, обработка других намерений ...
        if (intent.InteractionRequested)
        {
            var context = new InteractContext
            {
                InitiatorPosition = GlobalPosition,
                Inventory = _inventoryService
            };
            _interaction.TryInteract(context);
        }
    }
}
```

6.2.3. Отказы как часть контракта и расширяемость системы

Молчаливое «ничего не произошло» – худший результат взаимодействия. Игрок нажимает кнопку перед закрытой дверью и теряется в догадках: дверь сломана? Кнопка не работает? Может, надо было прочитать свиток? В контрактной модели отказ становится полноценным результатом, который несёт смысл и позволяет UI отреагировать.

Доработаем наш интерфейс `IInteractable`, чтобы метод `CanInteract` возвращал не просто `bool`, а объект результата, объясняющий причину отказа.

```
public struct InteractionResult
{
    public bool IsSuccess { get; init; }
    public string FailureMessage { get; init; }
    public static InteractionResult Success() => new() { IsSuccess = true };
    public static InteractionResult Fail(string reason) => new() { IsSuccess = false,
FailureMessage = reason };
}
public interface IInteractable
{
    InteractionResult CanInteract(InteractContext context);
    void Interact(InteractContext context);
}
```

Теперь дверь в методе `CanInteract` может вернуть `Fail` (“Нужен красный ключ”), а терминал – `Fail` (“Необходимо питание реактора”). Сервис взаимодействия получает этот отказ и передаёт его слою представления через событие. UI, прослушав событие, показывает локализованную подсказку. Звуковой движок воспроизводит глухой стук. И всё это происходит без вмешательства в код плеера или самой двери.

Собранная вместе, такая система взаимодействий демонстрирует главные свойства хорошей архитектуры. Она сопротивляется лавинообразному росту сложности, потому что каждый новый интерактивный объект – это просто новая реализация `Interactable`. Игрок или AI-бот, обученный взаимодействовать с дверью, сундуком или кнопкой, делает это единообразно через `InteractionService`. Модульные тесты на логику двери пишутся подменой контекста и проверкой, что при отсутствии ключа возвращается `Fail`, а при наличии – вызов `Interact` удаляет дверь. Архитектура становится не сводом догм, а ежедневным инструментом, который позволяет не бояться расширять мир и делать его интерактивным.

6.3. Архитектура камеры: наблюдатель, а не контроллер

Достаточно бросить на сцену узел `Camera2D`, привязать его к персонажу и добавить плавное следование, чтобы игра сразу ожила. Картинка движется за героем, мир обретает перспективу, и кажется, что на этом вопрос камеры закрыт. Но первое же серьёзное расширение вскрывает обратное. Дизайнер просит, чтобы при быстром беге кадр чуть отъезжал, показывая больше пространства впереди. Сценарист хочет в начале уровня показать панораму цели, а затем плавно вернуть камеру игроку. Звукоинженер добавляет тряску при взрывах. И вот код, который мирно жил в `_Process` персонажа, превращается в месиво из условий, таймеров и конфликтующих смещений. Архитектурная проблема здесь не в сложности эффектов, а в том, что камера с самого начала была оформлена как придаток игрока, а не как самостоятельная система. Она разделяет с персонажем его жизненный цикл, читает его внутренние переменные и, что самое опасное, иногда в ответ вмешивается в его поведение. Этот раздел посвящён тому, чтобы раз и навсегда навести порядок в этой связке. Мы спроектируем камеру как отдельный модуль-наблюдатель, который потребляет факты о мире, но никогда не командует им.

6.3.1. Разделение обязанностей игрока и наблюдателя

Код, в котором персонаж сам двигает камеру, выглядит до обидного логично: ведь камера следует за игроком. Прямо в методе `_PhysicsProcess` или `_Process` появляется строка `_camera.GlobalPosition = GlobalPosition`. Она не вызывает

подозрений, пока вы не захотите добавить в следование интеллект. Как только в уравнение входят упреждение по скорости, зум при рывке или сценарный фокус на другом объекте, вы начинаете обкладывать эту строку условиями. Условия множатся, переплетаются с логикой движения и состояний, и в какой-то момент изменение радиуса обзора начинает требовать правки класса Player.

Правильная архитектурная граница прямо противоположна. Игрок – это источник фактов, а камера – их потребитель. Игрок не должен знать о существовании камеры, её положении или режиме работы. Его дело – публиковать своё состояние: позицию, вектор скорости, текущую фазу (бег, прыжок, рывок). Камера же, будучи отдельным узлом или контроллером, подписывается на эти факты и сама решает, как их интерпретировать. Такая односторонняя зависимость означает, что персонажа можно запустить вообще без камеры, и он будет полностью функционален. А камеру можно заменить с платформенного следования на изометрическое, не тронув ни строчки геймплейного кода.

Давайте сразу закрепим это в коде. Вместо прямой манипуляции камерой из плеера мы вводим класс `CameraController2D`, который сам находит цель и сам вычисляет свою позицию.

```
using Godot;
public partial class CameraController : Node
{
    [Export] private NodePath _targetPath;
    [Export] private float _followSpeed = 8f;
    private Node2D _target;
    public override void _Ready()
    {
        _target = GetNode<Node2D>(_targetPath);
    }
    public override void _Process(double delta)
    {
        if (_target == null) return;
        Vector2 desiredPosition = _target.GlobalPosition;
        float t = 1f - Mathf.Exp(-_followSpeed * (float)delta);
        GlobalPosition = GlobalPosition.Lerp(desiredPosition, t);
    }
}
```

Здесь уже виден ключевой принцип: контроллер читает позицию цели, но цель о нём не знает. Параметр `_followSpeed` управляет отзывчивостью, и его настройка безопасна – она не затронет физику персонажа. Однако это лишь базовая заготовка, которая слепо держит цель в центре экрана. Настоящая кинематографичность начинается там, где камера начинает предугадывать действия игрока.

6.3.2. Слойный пайплайн: от цели до финального кадра

Поведение хорошей игровой камеры редко сводится к одной формуле. Обычно это целый конвейер, в котором желаемая точка кадра проходит через не-

сколько стадий обработки. Разработчик, который мыслит камеру как монолитный скрипт, обречён перемешивать эти стадии в одной куче, получая непредсказуемые побочные эффекты. Гораздо продуктивнее представить камеру как пайплайн из четырёх слоёв.

Первый слой – **цель**. Это не обязательно сам игрок. Целью может быть точка между игроком и его companion'ом, объект интереса во время диалога или даже заранее заданная позиция в катсцене. Камера не привязана к конкретному классу, она привязана к контракту – интерфейсу, предоставляющему позицию и скорость.

Второй слой – **контроллер**. Он берёт сырую позицию цели и применяет к ней политики композиции. Самая важная из них – look-ahead, или упреждение. Идея в том, чтобы сместить кадр в направлении движения игрока, открывая ему обзор на то, что впереди. Мы берём горизонтальную скорость из состояния персонажа и вычисляем точку упреждения.

```
private Vector2 CalculateDesiredPosition()
{
    Vector2 basePosition = _target.GlobalPosition;
    float lookAhead = _target.Velocity.X * _lookAheadFactor;
    lookAhead = Mathf.Clamp(lookAhead, -_maxLookAhead, _maxLookAhead);
    return basePosition + new Vector2(lookAhead, 0f);
}
```

Третий слой – **ограничения**. Даже самая умная камера не должна показывать края мира или уходить за пределы арены. Этот слой получает желаемую позицию от контроллера и применяет к ней границы. В Godot для этого удобно использовать встроенный Camera2D с его свойствами LimitLeft, LimitRight, LimitTop, LimitBottom, которые можно выставлять при входе в новую зону. Но для более сложной логики, например ограничения по заранее заданной кривой или прямоугольнику, пишется собственный компонент. Важно, что он ничего не знает о цели или скорости – только о позиции и пределах.

Четвёртый, последний слой – **применение**. Он берёт итоговую целевую позицию, прошедшую через контроллер и ограничения, и назначает её реальному узлу Camera2D через сглаживание. Сглаживание здесь выступает не как «магия от тряски», а как настраиваемая политика отзывчивости. В быстрых играх, например в bullet hell, камера должна быть почти мгновенной. В медитативных платформах допустима плавная инерция.

Итоговый _Process нашего контроллера собирает весь пайплайн воедино.

```
public override void _Process(double delta)
{
    if (_target == null) return;
    Vector2 desiredPosition = CalculateDesiredPosition();
    Vector2 constrainedPosition = ApplyBounds(desiredPosition);
    GlobalPosition = SmoothTo(constrainedPosition, delta);
}
```

Такой слойный подход делает систему удивительно расширяемой. Чтобы добавить тряску при выстреле, не нужно лезть в логику следования. Вы прос-

то пишете компонент-модификатор, который добавляет случайное смещение к `GlobalPosition` уже после всех расчётов. Чтобы реализовать `brief`-фокус на статуе, достаточно временно подменить источник цели, и вся остальная машина – упреждение, ограничения, сглаживание – продолжит работать корректно.

6.3.3. Управление режимами и модификаторами

Пожалуй, самая опасная ловушка в архитектуре камеры – это борьба за авторитет. Когда скрипт катсцены говорит «покажи статую», а в этот же кадр персонаж пытается следовать за игроком, побеждает тот, кто последним вызвал `GlobalPosition`. Это приводит к визуальным рывкам и трудноуловимым багам.

Решение – ввести централизованный арбитр, который решает, какой режим активен сейчас. Сделать это можно через простой конечный автомат внутри `CameraController`. У каждого режима есть метод `Update`, возвращающий желаемую позицию. Контроллер содержит словарь режимов и ссылку на текущий. Когда сценарию нужно перехватить управление, он не двигает камеру сам, а вызывает метод `SetMode`(«cinematic»). Контроллер переключает активный режим, и на следующем кадре позиция вычисляется уже по правилам катсцены. Как только событие завершено, вызов `SetMode`(«follow») возвращает управление обычному следованию.

Модификаторы вроде тряски или динамического зума работают поверх режима, а не вместо него. Компонент `ShakeHandler`, получив сигнал об ударе, загружает в контроллер затухающую амплитуду. Итоговое смещение от тряски вычисляется в самом конце пайплайна и добавляется к позиции. Когда амплитуда падает до нуля – тряска бесшумно отключается.

```
public partial class ShakeHandler : Node
{
    private float _trauma;
    private readonly FastNoiseLite _noise = new();
    public void AddTrauma(float amount) => _trauma = Mathf.Clamp(_trauma + amount, 0f,
1f);
    public override void _Process(double delta)
    {
        if (_trauma <= 0f) return;
        _trauma = Mathf.Max(0f, _trauma - 0.8f * (float)delta);
    }
    public Vector2 GetShakeOffset()
    {
        float intensity = _trauma * _trauma * 10f;
        return new Vector2(
            _noise.GetNoise(Time.GetTicksMsec() * 0.05f, 0f) * intensity,
            _noise.GetNoise(0f, Time.GetTicksMsec() * 0.05f) * intensity
        );
    }
}
```

Пайплайн камеры от этого не меняется. Он получает смещение от `Shake-Handler` и применяет его самой последней операцией. Это и есть архитектурная дисциплина: у каждого эффекта – свой чёткий слот в конвейере.

Спроектированная таким образом камера перестаёт быть источником страха при доработках. Она превращается в понятную, настраиваемую систему, где каждый слой делает свою работу. Игрок остаётся просто источником фактов и не знает о камере ничего. Катсцены, тряска и визуальные эффекты подключаются как независимые модули. А вы получаете возможность итеративно настраивать ощущение от игры, не боясь сломать её логику.

6.4. Событийно-ориентированный геймплей: факты вместо опроса

Когда игровой мир состоит всего из пары объектов, кажется естественным проверять их состояние в каждом кадре. Мы пишем в `_Process` условие «если здоровье упало до нуля – запустить смерть», а в другом месте – «если все враги мертвы – открыть дверь». Такой подход, называемый опросом, работает идеально ровно до тех пор, пока количество механик не переваливает через критическую отметку. Как только в сцене появляются десятки интерактивных объектов, а их поведение начинает зависеть от действий игрока, погоды, времени суток и фазы луны, код, построенный на бесконечных проверках, превращается в кашу. Логика размазывается по множеству `Update`-циклов, а причинно-следственные связи становятся неясными.

Событийно-ориентированный подход предлагает развернуть эту модель на сто восемьдесят градусов. Вместо того чтобы спрашивать мир «не случилось ли чего?» каждый кадр, мы учим мир сообщать нам, когда случается что-то важное. Враг не ждет, пока кто-то проверит его здоровье; он сам объявляет: «Я умер». Дверь не опрашивает всех врагов на уровне; она просто ждет сигнала «Врагов больше нет». Такой сдвиг в мышлении кажется незначительным, но на деле он фундаментально меняет архитектуру: правила перестают зависеть от порядка проверок, а модули мира – от внутреннего устройства друг друга.

6.4.1. От постоянного опроса к подписке на изменения

Корень проблемы опроса кроется в его природе. Проверая условие в `_Process`, мы смешиваем две разные вещи: сам факт (здоровье кончилось) и момент, когда нам удобно о нём узнать (очередной кадр). Пока персонаж один, эта разница незаметна. Но представьте сцену с тремя разными системами, которые должны отреагировать на смерть врага: дверь, открывающая проход;

интерфейс, обновляющий счётчик; и квест, засчитывающий убийство. При опросе каждая из этих систем должна сама догадаться, что враг умер. Они будут постоянно проверять сцену, искать врагов, пересчитывать их – и делать это каждый кадр, просто чтобы заметить изменение, которое происходит раз в несколько минут. Это не только расточительно, но и опасно: одна система может проверить врагов до того, как другая удалит их из дерева сцены, и тогда дверь не откроется.

Событийная модель разрубает этот узел, отделяя момент свершения факта от реакции на него. Враг, умирая, публикует событие `Died`. Все, кому это важно, заранее подписались на это событие и в нужный момент получают уведомление. Врагу при этом совершенно не нужно знать, кто именно его слушает. Он просто сообщает миру о своём состоянии и уходит со сцены.

Чтобы почувствовать разницу на практике, давайте сравним два способа реализовать открытие двери после зачистки комнаты. Сначала – классический опрос в лоб, который пишут в прототипах.

```
using Godot;
public partial class DoorOpener : Node
{
    public override void _Process(double delta)
    {
        // Каждый кадр ищем всех врагов в группе и проверяем их количество.
        var enemies = GetTree().GetNodesInGroup("enemies");
        if (enemies.Count == 0)
        {
            GD.Print("Все враги мертвы – дверь открывается");
            // Открываем дверь...
        }
    }
}
```

Проблема не в производительности, а в связанности. Метод `_Process` теперь знает, что враги где-то есть, и они помечены группой `enemies`. Если разработчик забудет убрать врага из группы, дверь не откроется. Если враг умрет и мгновенно удалится из дерева, а дверь проверит счётчик до его удаления – дверь не откроется. И главное – логика открытия теперь завязана на конкретный механизм поиска узлов в сцене.

Теперь перепишем это же поведение через доменное событие. Для начала создадим легковесный диспетчер, который будет жить в контексте уровня и управлять всеми значимыми фактами.

```
using System;
public class LevelEvents
{
    // Событие "Враг умер" – чисто доменный факт.
    public event Action? EnemyDied;
    // Метод для публикации события извне.
    public void RaiseEnemyDied() => EnemyDied?.Invoke();
}
```

Каждый враг получает ссылку на этот `LevelEvents` при создании и в момент смерти вызывает `RaiseEnemyDied()`. Система двери, в свою очередь, подписывается на событие и ведёт внутренний счётчик.

```
public partial class DoorSystem : Node
{
    [Export] private int _enemiesRequired = 5;
    private int _enemiesRemaining;
    public void Initialize(LevelEvents levelEvents)
    {
        _enemiesRemaining = _enemiesRequired;
        levelEvents.EnemyDied += OnEnemyDied;
    }
    private void OnEnemyDied()
    {
        _enemiesRemaining--;
        if (_enemiesRemaining <= 0)
        {
            GD.Print("Все враги мертвы - дверь открывается");
            // Вызываем метод открытия двери.
        }
    }
}
```

Посмотрите, как изменилась структура. Враг ничего не знает о двери. Дверь знает только о факте смерти, но не о том, сколько врагов было изначально и где они находятся. Вся логика зачистки комнаты инкапсулирована в одном месте – в классе `DoorSystem`. Если дизайнер решит, что теперь врагов должно быть семь, он поменяет `_enemiesRequired` в инспекторе. Если захочет, чтобы дверь открывалась не сразу, а с задержкой – добавит таймер внутрь `OnEnemyDied`. Ни одной правки в коде врага или уровне не потребуется.

6.4.2. Область видимости событий и предотвращение хаоса

Когда в игре появляются десятки событий, возникает соблазн завести один глобальный диспетчер на весь проект и публиковать туда всё подряд. Это ловушка. Глобальная шина событий кажется удобной: подписывайся откуда хочешь. Но очень быстро выясняется, что понять, кто и почему реагирует на событие, становится невозможно. При добавлении новой сцены она неожиданно реагирует на события, оставшиеся от предыдущей. А подписки, забытые при закрытии UI, продолжают обрабатывать данные давно удалённых персонажей.

Вместо глобального монолита мы используем диспетчеры как локальные «радиостанции», привязанные к конкретной области ответственности. `LevelEvents` существует, пока жив уровень, и автоматически умирает вместе с ним, гарантируя, что все подписки будут потеряны без утечек памяти. Точно так же `PlayerEvents` может жить внутри узла игрока и рассылать факты о его со-

стоянии: `HealthChanged`, `Jumped`, `Landed`, `Died`. Система выбора диспетчера проста: если событие имеет смысл только внутри одного уровня, оно принадлежит `LevelEvents`. Если оно касается исключительно игрока – `PlayerEvents`. И только если факт действительно глобален, например завершение игры или смена профиля, для него можно завести отдельный `SessionEvents`, который будет создаваться и уничтожаться явно.

Это подводит нас к важной технической детали – полезной нагрузке события. Представьте, что событие `EnemyDied` передаёт подписчикам ссылку на узел умершего врага. Квест, получив эту ссылку, тянется к врагу, читает его координаты, тип и выпавший лут. Через секунду враг удаляется из сцены, но подписчик всё ещё хранит на него ссылку. `Managed`-обёртка может оставаться живой, но движковый объект уже уничтожен. Попытка обратиться к нему вызовет ошибку. Правильный подход – передавать в событии не ссылку на изменчивый объект, а **снимок значимых фактов**: структуру с идентификатором, позицией смерти и, скажем, типом врага.

```
public readonly struct EnemyDeathInfo
{
    public string EnemyType { get; init; }
    public Vector2 DeathPosition { get; init; }
}
// В LevelEvents:
public event Action<EnemyDeathInfo>? EnemyDied;
public void RaiseEnemyDied(EnemyDeathInfo info) => EnemyDied?.Invoke(info);
```

Такой снимок – это не просто «передача данных», а часть контракта. Он говорит всем слушателям: «Вот что произошло, и вы можете быть уверены, что эти данные неизменны». Система достижений сможет посчитать смерть `EnemyType.Orc` в статистике, а звуковой движок – проиграть звук в точке `DeathPosition`. И ни одна из них не будет зависеть от жизненного цикла узла врага.

6.4.3. Синхронный вызов против отложенной обработки

Остаётся последний штрих, без которого событийная система может преподнести сюрпризы, – порядок обработки. Когда враг вызывает `RaiseEnemyDied`, событие по умолчанию обрабатывается синхронно: все подписчики прямо сейчас, в том же стеке вызовов, запускают свою логику. Это может привести к неожиданным цепочкам: обработчик смерти меняет состояние мира, которое прямо в этом кадре ещё не готово к изменениям; другая система читает это состояние и ломается; или, того хуже, один обработчик инициирует новое событие, и цепочка становится неуправляемой.

В большинстве игровых сценариев безопаснее сначала дождаться завершения всех обновлений движения и физики, а затем, в специально отведённой фазе кадра, обработать накопившиеся события. Для этого мы вводим простую очередь в диспетчере.

```

public class QueuedLevelEvents
{
    private readonly Queue<Action> _deferredActions = new();
    public void RaiseEnemyDied(EnemyDeathInfo info)
    {
        // Захват info в лямбде безопасен, т. к. это структура.
        _deferredActions.Enqueue(() => EnemyDied?.Invoke(info));
    }
    public void ProcessDeferred()
    {
        while (_deferredActions.TryDequeue(out var action))
            action();
    }
}

```

Теперь, когда враг сообщает о своей смерти, событие не срабатывает немедленно. Оно помещается в очередь и ждёт. В конце `_Process` или `_PhysicsProcess` нужного узла мы вызываем `ProcessDeferred()`. Все реакции выполняются одна за другой, в предсказуемом порядке, отдельно от основного игрового цикла. Это делает поведение детерминированным и позволяет отлавливать ошибки, просто глядя на содержимое очереди в отладчике.

Событийная модель не отменяет необходимости в опросе целиком – физические движения и интерполяция камеры по-прежнему используют `_Process`. Но для всех дискретных фактов мира она становится фундаментом, который позволяет модулям общаться, не зная друг о друге. Архитектура перестаёт быть сетью прямых вызовов и превращается в систему с явным, читаемым потоком причин и следствий. И именно это в конечном счёте делает игру не просто работающей, но и удобной для поддержки и расширения в долгосрочной перспективе.

6.5. Базовая структура AI: конвейер восприятия, решения и действия

Первое, что приходит в голову при создании врага, – открыть его скрипт и прямо в `_Process` прописать всю логику. Проверить расстояние до игрока, если мало – атаковать, иначе – двигаться. Пять строчек, и враг оживает. Этот подход, соблазнительный своей простотой, становится фатальным, едва проект перерастает стадию прототипа. Появляются новые типы противников, каждый со своими дистанциями атаки, таймерами и особыми реакциями. Код начинает ветвиться, обрастать флагами и взаимными исключениями. Хуже того, он смешивает в одной куче три принципиально разные задачи: сбор информации о мире, принятие решения и выполнение действий. Стоит изменить форму коллайдера или способ передвижения – и весь скрипт приходится переписывать.

Устойчивая архитектура искусственного интеллекта строится на чётком разделении этих обязанностей. В основе лежит простой, как автомат с газировкой, конвейер: **восприятие (perception)** → **решение (decision)** → **действие (action)**. Враг не «видит игрока и бежит к нему», а получает факт «цель на расстоянии двадцати метров», переходит в состояние преследования и, уже находясь в нём, формирует намерение двигаться вправо. Сам он при этом не управляет ни физикой, ни анимацией – лишь отдаёт приказы тем же подсистемам, что обслуживают и игрока. Такой подход не усложняет, а упрощает жизнь, потому что каждая часть конвейера живёт по своим правилам и ничего не знает о внутреннем устройстве соседей.

Конвейер восприятия, решения и действия

Представьте себе робота-пожарного. Его сенсоры (глаза-камеры, тепловизоры) фиксируют дым и пламя – это восприятие. Бортовой компьютер, получив данные, определяет, что сейчас нужно не патрулировать, а тушить, и выбирает ближайший очаг – это решение. И лишь затем контроллеры подают питание на насосы и моторы – это действие. Если бы робот был устроен как плохой игровой AI, его программа выглядела бы как бесконечный цикл: «проверить дым – бежать – проверить пламя – лить воду». Любое изменение в конструкции насоса потребовало бы переписывания всего цикла.

В правильной архитектуре класса Enemy нет ни одного вызова MoveAndSlide или PlayAnimation. Вместо этого в нём живут три легковесных модуля, каждый со своей зоной ответственности.

Первый модуль – **восприятие (perception)**. Его задача предельно узка: перевести хаос окружающего мира в строгий набор фактов, пригодных для анализа. Он не говорит «игрок рядом», а сообщает точное расстояние до цели. Не утверждает «можно атаковать», а констатирует, что цель находится в радиусе атаки и между ними нет препятствий.

Физически восприятие часто опирается на Area2D или RayCast2D, но решение о том, как именно собирать данные, инкапсулировано внутри этого модуля. Остальной AI пользуется лишь его публичным интерфейсом, который не зависит от конкретных движковых механизмов. Так, класс EnemyPerception может внутри себя слушать события body_entered, обновлять таймер потери цели и на основе этого выставлять наружу простую структуру фактов:

```
public class EnemyPerception
{
    public Node2D Target { get; private set; }
    public float DistanceToTarget { get; private set; }
    public bool TargetInAttackRange { get; private set; }
    public void Update(Vector2 ownPosition)
    {
        // Здесь – проверка коллизий, raycast, обработка событий от Area2D,
        // таймер "потери цели" и т. д. В итоге выставляются простые поля.
    }
}
```

Важнейшая деталь: `EnemyPerception` не использует `GetNode<Player>` для поиска цели. Он получает информацию через коллизионные слои и маски, которые мы настраивали в разделе 6.1. Благодаря этому он видит игрока, но остаётся слеп к декорациям и снарядам. Это снижает количество ложных срабатываний и делает поведение стабильнее.

Когда факты собраны, в дело вступает второй модуль – **принятие решений (decision)**. Он получает на вход структуру `AIContext` и выбирает, в каком режиме сейчас должен находиться агент. Самый удобный и понятный для новичков способ реализовать этот модуль – машина состояний, похожая на ту, что мы делали для игрока в разделе 5.4.

Каждое состояние (патрулирование, преследование, атака, поиск) – это отдельный класс, реализующий интерфейс с тремя методами: `Enter`, `Update` и `Exit`. Вход в состояние сбрасывает таймеры, выход – подчищает флаги. Метод `Update` получает контекст с текущими фактами восприятия и на основе них принимает решение о переходе.

```
public interface IEnemyState
{
    void Enter(AIContext ctx);
    void Update(AIContext ctx, double delta);
    void Exit(AIContext ctx);
}
```

Представьте состояние преследования. Его логика проста и линейна: пока цель видна и находится вне радиуса атаки, оно формирует намерение двигаться к ней. Как только дистанция сокращается до ударной, состояние возвращает `new AttackState()`. Если цель пропадает из виду на достаточно долгое время, возвращается `new SearchState()`.

```
public class ChaseState : IEnemyState
{
    public void Update(AIContext ctx, double delta)
    {
        if (!ctx.Perception.TargetInSight)
        {
            ctx.TimeSinceTargetLost += delta;
            if (ctx.TimeSinceTargetLost > 3.0f)
                ctx.RequestStateChange(new SearchState());
            return;
        }
        ctx.TimeSinceTargetLost = 0f;
        Vector2 direction = (ctx.Perception.Target.GlobalPosition - ctx.Owner.
GlobalPosition).Normalized();
        ctx.MoveIntent = direction.X;
        if (ctx.Perception.TargetInAttackRange)
            ctx.RequestStateChange(new AttackState());
    }
}
```

Крайне важно, что состояние не двигает персонажа самостоятельно. Всё, что оно делает, – заполняет поле `MoveIntent` в контексте. Это и есть та самая «заявка на действие», которая будет исполнена позже, третьим модулем конвейера.

Третий модуль – **действие (action)**. Он не содержит никакой логики принятия решений и является простым исполнителем. Его задача – взять заполненный контекст (`MoveIntent`, `AttackRequested`, `InteractionTarget`) и преобразовать его в вызовы тех же самых подсистем, которые использует игрок.

Для движения это означает передать `MoveIntent` в `MovementModel`, который мы разработали в разделе 5.2. Для взаимодействия – вызвать `InteractionService` из раздела 6.2, передав ему `InteractionTarget`. Для атаки – запустить метод `RequestAttack` у боевой системы. Физика, коллизии и анимация по-прежнему обслуживаются общими модулями. AI не «двигает спрайт», а говорит: «я хочу двигаться влево со стандартной скоростью». Всё остальное делает конвейер движения, единый для игрока и врагов.

```
public class EnemyActionExecutor
{
    private readonly MovementModel _movement;
    public EnemyActionExecutor(MovementModel movement) => _movement = movement;
    public void ExecuteActions(AIContext ctx, double delta)
    {
        _movement.SetHorizontalIntent(ctx.MoveIntent);
        _movement.ApplyGravity(_movement.Gravity);
    }
}
```

Сборка всего врага в сцене теперь выглядит как сочленение трёх независимых частей:

```
public partial class Enemy : CharacterBody2D
{
    private EnemyPerception _perception;
    private StateMachine _stateMachine;
    private EnemyActionExecutor _actionExecutor;
    private MovementModel _movement;
    public override void _Ready()
    {
        _movement = new MovementModel(/* параметры */);
        _perception = new EnemyPerception(/* ... */);
        _actionExecutor = new EnemyActionExecutor(_movement);
        // Инициализация машины состояний начальным состоянием.
        _stateMachine = new StateMachine(new PatrolState());
    }
    public override void _PhysicsProcess(double delta)
    {
        // 1. Восприятие: собираем факты о мире.
        _perception.Update(GlobalPosition);
        // 2. Решение: обновляем состояние на основе фактов.
        var ctx = new AIContext { Perception = _perception, Owner = this };
    }
}
```

```

        _stateMachine.Update(ctx, delta);
        // 3. Действие: исполняем намерения.
        _actionExecutor.ExecuteActions(ctx, delta);
        // 4. Применяем движение к физическому телу.
        Velocity = _movement.Velocity;
        MoveAndSlide();
    }
}

```

Такой конвейер делает поведение предсказуемым и позволяет вносить изменения точно. Хотите, чтобы враг замечал игрока только в конусе зрения? Доработайте `EnemyPerception`. Нужно, чтобы после потери цели он бежал на её последнюю известную позицию? Модифицируйте состояние `SearchState`. Решили использовать другую модель движения? Замените `MovementModel`, и враг даже не заметит.

Интеграция с игровыми подсистемами и событийная модель

Главный архитектурный выигрыш от конвейера – AI начинает разговаривать с миром на том же языке, что и игрок. Ему не нужны особые «читерские» методы, чтобы открыть дверь или получить урон. Дверь для него – не конкретный класс `Door`, а цель, реализующая интерфейс `Interactable`. Урон он получает через тот же `HealthComponent`, что и игрок. Движением управляет через тот же `MovementModel`. Это не просто устраняет дублирование кода, а сохраняет единую систему правил. Если вы решите, что при рывке нельзя взаимодействовать с объектами, это ограничение автоматически применится и к игроку, и к врагу – просто потому, что оба проходят через один и тот же `InteractionService`.

События играют в этой картине роль клея, который связывает AI с миром без жёстких ссылок. Враг не должен каждый кадр проверять `Attribute`, не зашёл ли игрок в зону его видимости. Вместо этого на `Area2D` вешается обработчик `body_entered`, который при входе игрока дёргает метод `OnTargetEntered` у модуля восприятия. Тот, в свою очередь, выставляет внутренний флаг «цель видна». Решение о переходе в преследование принимается уже потом, в методе `Update` машины состояний, на основе этого флага.

Точно так же враг реагирует на урон. Система боя вызывает `HealthComponent.TakeDamage()`. Тот, внутри себя, кроме уменьшения здоровья публикует событие `Damaged`. Машина состояний врага, подписанная на это событие, может немедленно прервать патрулирование и перейти в состояние тревоги. При этом сам `HealthComponent` понятия не имеет ни о каком AI – он просто констатирует факт.

Отдельного упоминания заслуживает анимация. Ошибка, которую совершают почти все новички, – заставлять AI-состояния напрямую проигрывать клипы: `_animator.Play(«attack»)`. Этим вы намертво привязываете поведе-

ние к визуальному представлению. Во-первых, при смене модели врага вам придётся лезть в код поведения. Во-вторых, время анимации начинает диктовать правила игры: атака «происходит», когда заканчивается клип. Это превращает отладку в ад. Правильный подход – публикация анимационным модулем фактов о текущем режиме. Состояние `AttackState` при входе выставляет флаг `IsAttacking = true`. Анимационный контроллер врага, отделённый от логики, читает этот флаг и сам решает, какой клип проиграть. Захотели заменить спрайтовую анимацию на скелетную – поменяли только контроллер, не тронув ни строчки AI.

Масштабирование, отладка и булева ловушка

С ростом числа врагов и их типов остро встаёт вопрос масштабирования. Наивный путь – скопировать скрипт и начать править в нём условия. Через месяц у вас будут десятки похожих, но слегка отличающихся классов, и исправление общего бага превратится в кошмар. Конвейерная архитектура предлагает иную стратегию: различать не код, а конфигурацию.

Ядро врага (`Perception` → `Decision` → `Action`) остаётся единым. Разница между быстрым и медленным противниками, между снайпером и подрывником, выносится в параметры состояний или в сами состояния. `ChaseState` может принимать в конструкторе скорость преследования и безопасную дистанцию: `new ChaseState(chaseSpeed: 250f, stopDistance: 70f)`. `AttackState` может получать ссылку на реализацию интерфейса `IEnemyAttack`, который бывает ближним, дальним или магическим. Создание нового врага сводится к компоновке существующих модулей с новыми настройками, а не к написанию очередного месива из условий.

В таком модульном мире кардинально упрощается отладка. Когда враг замирает вместо преследования, вы точно знаете, куда смотреть. Проверьте `EnemyPerception` – видит ли он игрока? Если видит, проверьте `StateMachine`. `Update` – был ли совершён переход в `ChaseState`? Если переход был, проверьте `AIContext.MoveIntent` – заполнен ли он? Наконец, проверьте `EnemyActionExecutor` – передал ли он намерение в `MovementModel`? Симптом сразу указывает на сломанный модуль, а не на загадочное сочетание флагов в тысячестрочном `_Process`.

Этот же подход служит лекарством от **булевой ловушки** – главной болезни плохого AI. Начинающие разработчики обожают заводить флаги: `isAlerted`, `isReturning`, `isStunned`, `hasSeenPlayerRecently`. Поначалу они удобны, но их взаимодействие быстро становится неуправляемым. Может ли враг быть одновременно оглушён и преследовать игрока? А если он был в поиске, а потом получил урон – должен ли он перейти в преследование или остаться в поиске? Ответы на эти вопросы тонут в дебрях вложенных `if-else`.

Машина состояний решает проблему радикально: в каждый момент времени агент находится ровно в одном состоянии, которое и определяет всё его поведение. Флаг `IsStunned` не нужен, если есть состояние `StunnedState`.

Приоритеты выражаются явно, через механизм переходов. StunnedState имеет право быть единственным состоянием, которое может быть форсировано извне (при получении урона), прерывая любые другие режимы.

Напоследок стоит сказать о наблюдаемости. Поскольку весь конвейер разбит на фазы, вы можете легко добавить журналирование ключевых событий: «Враг 5 перешёл из Patrol в Chase, потому что дистанция до цели = 120». Это не требует специальных фреймворков – достаточно GD.Print в нужных местах. С таким логом вы сможете воспроизвести и понять любое решение AI, не гадая и не ставя точки останова наугад.

Спроектированный по этим принципам искусственный интеллект перестаёт быть источником постоянной головной боли. Он превращается в ясную, предсказуемую систему, которую не страшно расширять. Вы начинаете думать не «как бы запихнуть ещё одно условие в этот скрипт», а «какое новое состояние нужно добавить в машину» или «какой параметр вынести в конфигурацию». А это и есть верный признак архитектуры, которая помогает, а не мешает.

7

State Machine и проектирование поведения

7.1. Конечный автомат как фундамент игровой логики

Практически любая интерактивная система в игре – от главного героя до кнопки в меню – в каждый момент времени находится в одном из фиксированного набора состояний. Персонаж не может одновременно бежать и прыгать в атаке, дверь не может быть и открыта, и закрыта, а враг – патрулировать и сражаться. Это кажется очевидным, пока мы не начинаем писать код. В прототипе поведение естественно ложится в понятные и привычные конструкции: если нажата кнопка «вперёд» – двигаемся, если нажали «пробел» и мы на земле – прыгаем. Пять условий, два флага, и персонаж оживает.

Проблема приходит с ростом требований. Дизайнер просит добавить подкат, во время которого нельзя атаковать. Потом – состояние оглушения, которое должно прерывать вообще всё. Следом – режим плавания, в котором гравитация работает иначе, а прыжок и подкат недоступны. Каждое новое требование добавляет в код ещё один флаг вроде `isDashing`, `isStunned`, `isSwimming` и ещё десяток условий, проверяющих, не конфликтуют ли эти флаги друг с другом. Очень скоро центральный метод `Update` превращается в лапшу из вложенных `if-else`, где изменение одной механики может сломать

три другие, а отладка превращается в гадание, почему персонаж не может прыгнуть, хотя нажата правильная кнопка. Эта проблема не устраняется аккуратностью, потому что её причина – в самой структуре кода, в попытке смоделировать поведение как набор глобальных булевых переменных.

Инструмент, который решает эту проблему, называется **конечный автомат**, или Finite State Machine. Он предлагает простую, как табуретка, но невероятно мощную идею: в каждый момент времени система находится ровно в одном состоянии, которое и определяет всё её поведение. Вместо того чтобы гадать, какие флаги сейчас активны, вы смотрите на имя текущего состояния и точно знаете, по каким законам живёт персонаж в этом кадре.

Состояния и переходы как инженерный контракт

Представьте себе пульт управления стиральной машиной. Её поведение тоже можно было бы запрограммировать флагами: `isWaterFilled`, `isDrumSpinning`, `isHeaterOn`. Но этот кошмарный код никто не пишет. Вместо этого инженеры выделили конечные, всем понятные режимы: «Замачивание», «Стирка», «Полоскание», «Отжим». Каждый режим – это чёткий набор правил: какие моторы включены, какая температура воды, сколько времени это длится. Переходы между режимами тоже строго определены: нельзя перейти к отжиму, не слив воду. Вы нажимаете «Старт», и машина проживает цепочку состояний, не путаясь в показаниях датчиков.

Конечный автомат для игры работает точно так же. **Состояние** – это именнованный класс, который инкапсулирует уникальную логику поведения. `GroundedState` знает, как обрабатывать бег и инициировать прыжок. `AirborneState` знает, как применять гравитацию и ограниченное управление в воздухе. `StunnedState` знает, что нужно игнорировать любой ввод и просто ждать истечения таймера. **Переход** – это строгое, явно прописанное правило, при котором одно состояние сменяет другое. И оно всегда содержит чёткое условие: «Если находимся в `GroundedState` и нажата кнопка `Jump`, переходим в `AirborneState`». Условие может быть сложным и проверять несколько факторов, но оно всегда локализовано в одном месте.

FSM – это в первую очередь инструмент **контроля сложности через изоляцию**. Когда персонаж находится в состоянии `DashingState`, вам не нужно помнить, что в этот момент нельзя прыгать, взаимодействовать или получать урон. Сама природа архитектуры гарантирует вам это. Код подката просто не содержит проверок на прыжок, а код прыжка никогда не выполняется одновременно с кодом подката. Это не оптимизация, а смена парадигмы мышления. Вы перестаёте думать о персонаже как о едином огромном классе со списком того, что он может и не может делать, и начинаете думать о нём как о системе, которая в каждый момент подчиняется одному конкретному своду правил.

Жизненный цикл состояния: Enter, Tick, Exit

Чтобы FSM был не просто «красивой диаграммой», а работал надёжно и предсказуемо, у каждого состояния должен быть явный жизненный цикл из трёх

фаз. Этот цикл и есть тот самый контракт, который делает поведение детерминированным и удобным для отладки.

Первая фаза – Enter. Она вызывается строго один раз в тот момент, когда машина переключается в это состояние. Это идеальное место для того, чтобы подготовить сцену. Если персонаж вошёл в состояние рывка, то именно здесь мы фиксируем направление рывка, применяем начальный импульс скорости и запускаем таймер длительности. Мы делаем это один раз, и ни один другой код в игре не может случайно сбросить этот таймер или изменить направление.

Вторая фаза – Tick, или Update. Этот метод вызывается каждый кадр, пока состояние активно. Он получает на вход контекст – структуру с необходимыми данными: намерение игрока, факты о контакте с землёй, дельту времени. Его задача – реализовать уникальное поведение режима. Внутри Tick состояния GroundedState мы читаем ось движения и применяем её к физическому телу. И что критически важно, здесь же мы проверяем условия выхода. Если состояние решает, что пора смениться, оно **не делает это немедленно**, а возвращает специальный объект – запрос на переход в другое состояние.

Именно эта деталь – возврат запроса вместо прямой смены – обеспечивает архитектурную целостность. Код внутри Tick не меняет глобальное состояние машины посреди своей работы, он лишь формирует намерение смениться. Фактическая смена произойдёт позже, в строго отведённой для этого точке в контроллере FSM, после того как текущий Tick полностью отработает. Это страхует нас от целого класса багов, когда одна часть логики ещё думает, что мы в воздухе, а другая уже считает, что мы на земле.

Третья, завершающая фаза – Exit. Она вызывается перед тем, как машина покинет это состояние. Её главная задача – подчистить за собой всё, что состояние настроило. Если в Enter мы запустили таймер, в Exit мы его гарантированно остановим, чтобы он не сработал в следующем состоянии. Если мы подписались на какие-то события – здесь мы отпишемся. Это не даёт появляться страшным «призрачным багам», когда персонаж уже вышел из рывка, а забытый таймер продолжает тикать и внезапно обнуляет скорость или блокирует управление.

От теории к коду: плоская машина на интерфейсах

Давайте посмотрим, как эта элегантная теория превращается в чистый, масштабируемый код на C#. В основе лежит простой интерфейс. Он отражает наш конвейер: фазу входа, рабочий цикл и фазу выхода.

```
public interface IState
{
    void Enter();
    void Tick(StateContext context);
    void Exit();
}
```

Контекст – это простой пакет данных, который мы передаём состоянию, чтобы оно могло принимать решения. В реальном проекте он будет содержать всё необходимое: текущее намерение игрока, факты окружения вроде `IsOnFloor`, возможно, ссылку на модель движения.

```
public class StateContext
{
    public PlayerIntent Intent;
    public bool IsGrounded;
    public MovementModel Movement;
    public float Delta;
}
```

Контроллер состояний – это двигатель всей системы. Его задача проста и неизменна: хранить ссылку на текущее состояние и управлять его жизненным циклом.

```
public class StateMachine
{
    private IState _currentState;
    public StateMachine(IState initialState, StateContext context)
    {
        _currentState = initialState;
        _currentState.Enter();
    }
    public void Update(StateContext context)
    {
        var nextState = _currentState.Tick(context);
        if (nextState != null)
        {
            ChangeState(nextState);
        }
    }
    private void ChangeState(IState newState)
    {
        _currentState.Exit();
        _currentState = newState;
        _currentState.Enter();
    }
}
```

Теперь сосредоточимся на том, как конкретное состояние использует этот механизм. Рассмотрим `GroundedState`. В нём нет ни одного флага, нет проверок «а не в оглушении ли я». Оно просто делает свою работу.

```
public class GroundedState : IState
{
    public void Enter() { /* Можно сбросить таймеры эффектов земли */ }
    public IState Tick(StateContext ctx)
    {
        // 1. Применяем логику движения на земле.
        ctx.Movement.SetHorizontalIntent(ctx.Intent.MoveDirection);
    }
}
```

```

ctx.Movement.ApplyGravity(0); // на земле гравитация не нужна
// 2. Проверяем условия перехода.
// Прыжок – единственная причина покинуть это состояние.
if (ctx.Intent.JumpRequested && ctx.IsGrounded)
{
    // Возвращаем новое состояние, передавая ему начальный импульс.
    return new AirborneState(ctx.Movement.JumpForce);
}
// 3. Если ничего не изменилось, остаёмся здесь.
return null;
}
public void Exit() { /* Здесь можно записать последнюю позицию */ }
}

```

Обратите внимание на элегантность и изолированность. Воздушное состояние `AirborneState` будет делать совершенно другую работу – применять гравитацию, обеспечивать ограниченный контроль в полёте и ждать касания земли. Если в него перешли через прыжок, в конструктор был передан импульс, который будет применён в фазе `Enter`. Мы не размазываем логику прыжка по всему коду – она живёт только здесь.

Именно в этой точке многие новички совершают фатальную ошибку, сводя на нет все преимущества архитектуры. Пытаясь сэкономить пару строк, они вместо создания объектных состояний заводят `enum` с режимами: `public enum PlayerMode { Grounded, Airborne, Dashing, Stunned }`. А затем, чтобы управлять этим, пишут гигантский `switch` внутри метода `Tick`. Такой подход – это не упрощение, а мина замедленного действия. Он снова стягивает всю логику, разбросанную по состояниям, в одно место – в контроллер. И вы возвращаетесь к тем же проблемам, от которых пытались уйти: чтобы добавить новое состояние, вы должны править центральный `switch` и, скорее всего, код в нескольких ветках `case`, рискуя сломать уже работающие. Объектный FSM с интерфейсами – это не усложнение, а дисциплина, которая с ростом проекта окупает себя стократно.

Когда плоской модели становится мало

Плоская машина состояний, которую мы построили, великолепна для управления одним аспектом поведения. Но игры устроены сложнее. Очень скоро вы обнаружите, что некоторые механики являются сквозными. Например, персонаж может получить урон и отправиться в нокдаун в любой момент: когда он бежит, когда прыгает и даже когда уже лежит оглушённым. Строить для этого переход из каждого состояния в `StunnedState` – значит плодить дублирующий код. Точно так же вы столкнётесь с повторением правил. И `IdleState`, и `RunState`, и `JumpState` – все они должны реагировать на получение урона. Если в каждом из них писать этот переход, вы нарушите принцип единственной ответственности. Если же вынести эту проверку куда-то «наверх», до того как `Tick` состояния будет вызван, вы получите иерархию – ситуацию, когда одно правило (получение урона) имеет приоритет над пра-

вилами, специфичными для конкретного режима. Это признак того, что системе нужна структура, где одна машина состояний может быть вложена в другую или где прерывания обрабатываются на более высоком уровне. Архитектурная задача здесь – не «написать идеальный FSM», а удержать в голове его ограничения и вовремя заметить, что простая модель перестаёт справляться. Когда вы начнёте дублировать один и тот же переход из пяти разных состояний или писать проверки «а если я в этом состоянии, то игнорируй вот это», знайте: вы стоите на пороге следующей главы, где мы будем говорить об иерархических машинах состояний.

7.2. Иерархические конечные автоматы: управление глубиной поведения

Плоская машина состояний, которую мы освоили в предыдущем разделе, прекрасно справляется с линейными сценариями. Персонаж стоит, идёт, прыгает – набор изолированных друг от друга режимов, каждый со своим чётким контрактом. Однако настоящая игра редко остаётся такой плоской. По мере добавления механик вы начинаете замечать, что разные состояния начинают требовать одинаковой логики. И `IdleState`, и `RunState` должны реагировать на получение урона. И тот, и другой должны корректно обрабатывать потерю опоры под ногами. И оба они, по сути, являются разновидностью более общего режима – «персонаж на земле».

Если продолжать решать эту проблему в рамках плоской модели, вы неизбежно встанете перед выбором: либо дублировать один и тот же код во всех наземных состояниях, либо вынести эту общую логику обратно в контроллер, превращая его в того самого «божественного объекта», от которого мы пытались уйти. Иерархический конечный автомат, или *Hierarchical State Machine* (HSM), предлагает элегантный выход из этой дилеммы, добавляя в модель поведения уровни ответственности.

Как и любой мощный инструмент, HSM не является панацеей. Введенный не к месту, он способен утяжелить и запутать даже простую логику. Но когда количество режимов и их взаимосвязей переваливает за критическую отметку, способность HSM группировать общие правила и изолировать частные становится незаменимой. Мы начнём с простой и наглядной аналогии, чтобы прочувствовать саму идею, а затем перейдём к строгим архитектурным определениям и примерам кода.

От квартиры к подъезду: модель вложенных состояний

Представьте себе многоквартирный дом. Вы не можете одновременно находиться в двух квартирах, но вы всегда находитесь внутри какой-то более крупной структуры: квартира расположена на этаже, этаж – в подъезде, подъ-

езд – в доме. Ваше местоположение можно описать как путь: Дом № 10 → Подъезд 2 → Этаж 5 → Квартира 78. Этот путь и есть ваше активное состояние в иерархии.

У каждого уровня этого пути есть свои правила и своя зона ответственности. Выходя из дома, вы обязаны покинуть квартиру, затем этаж, потом подъезд и только потом оказаться на улице. Пожарная сигнализация, сработавшая в подъезде, эвакуирует вас из квартиры, но не требует вашего непосредственного участия в выборе квартиры – это общее правило для всего подъезда. В то же время перестановка мебели в вашей гостиной – это сугубо локальное событие, которое не касается ни соседей, ни подъезда.

Иерархический конечный автомат моделирует именно такую картину мира. GroundedState – это «подъезд», общий контекст для всех режимов на земле. Он знает, что делать при потере контакта с землей (переходить в воздушный режим), и этот переход сработает одинаково, находитесь ли вы в «квартире» IdleState или в «квартире» RunState. Сами же IdleState и RunState – это частные, листовые реализации, которые знают свой уникальный набор правил: как обрабатывать намерение игрока на движение, какую скорость набирать, какую анимацию проигрывать.

Механика иерархии: родители, потомки и конвейер обработки

В основе HSM лежит простая идея, которая тем не менее требует дисциплины в реализации. Активным является не одно состояние, а путь от самого верхнего, корневого, до самого нижнего, листового. Входные сигналы – будь то нажатия кнопок, срабатывание таймеров или изменения физических фактов – проходят через этот путь строго определённым образом. Сигнал сначала предлагается самому нижнему, текущему листовому состоянию. Если оно «понимает» этот сигнал и обрабатывает его (например, RunState реагирует на отпускание кнопки движения переходом в IdleState), на этом всё заканчивается.

Но если листовое состояние не знает, что делать с этим сигналом, он, подобно пузырьку воздуха в воде, **всплывает** к своему родителю. Тот, в свою очередь, может либо обработать его, либо передать ещё выше. Этот механизм, называемый propagation, и является архитектурной сердцевиной HSM. Он позволяет вынести общую реакцию на нужный уровень. Оглушение, критические повреждения или команда «открыть главное меню» – всё это можно один раз описать на верхнем уровне иерархии, гарантируя, что оно будет обработано независимо от того, бежал ли персонаж, прыгал или стоял на месте.

Этот же принцип применяется и к смене состояний. Когда листовое состояние решает, что пора переключиться, оно не меняет всю иерархию само, а лишь возвращает запрос. Контроллер HSM, получив его, вычисляет ближайшего общего предка (Lowest Common Ancestor) для текущего и целевого состояния. Затем он аккуратно вызывает Exit на всех уровнях, от листа до этого предка, и затем Enter на всех уровнях, от предка до нового листа. Эта

строгая, почти бюрократическая процедура гарантирует, что жизненный цикл каждого уровня будет соблюден. Таймеры и подписки, созданные на уровне `Grounded`, будут гарантированно очищены при выходе из него, и это произойдет автоматически, а не потому, что разработчик не забыл вставить `if` в нужное место.

От теории к коду: проектирование иерархического поведения

Давайте закрепим эту механику на практике. В основе нашей реализации будет лежать абстрактный класс `HierarchicalState`. Его ключевая особенность – ссылка на родительское состояние и метод `_HandleInput`, который реализует нашу логику всплытия.

```
public abstract class HierarchicalState
{
    public HierarchicalState Parent { get; private set; }
    public HierarchicalState(HierarchicalState parent = null)
    {
        Parent = parent;
    }
    public virtual void Enter() { }
    public virtual void Exit() { }
    public abstract HierarchicalState Tick(StateContext context);
    /// <summary>
    /// Пытается обработать входной сигнал. Если состояние не знает, что с ним делать,
    /// передаёт сигнал своему родителю.
    /// </summary>
    public virtual bool HandleInput(string input, StateContext context)
    {
        if (Parent != null)
            return Parent.HandleInput(input, context);
        return false;
    }
}
```

Теперь спроектируем наше первое родительское состояние – `GroundedState`. Его задача – владеть общей логикой для всех наземных режимов. В нашем примере это будет реакция на прыжок и на потерю опоры под ногами.

```
public class GroundedState : HierarchicalState
{
    public GroundedState(HierarchicalState parent = null) : base(parent) { }
    public override HierarchicalState Tick(StateContext context)
    {
        // Общая логика не выполняется, если у нас есть дочернее состояние.
        // Вместо этого мы просто возвращаем себя как контекст.
        return this;
    }
    public override bool HandleInput(string input, StateContext context)
    {

```

```

switch (input)
{
    case "JumpPressed":
        // Прыжок доступен для всех наземных подрежимов.
        return new AirborneState(context.Movement.JumpForce);
    case "LostGround":
        // Потеряли опору - уходим в воздух.
        return new AirborneState(null);
    default:
        return base.HandleInput(input, context);
}
}
}

```

А вот и одно из дочерних состояний, RunState. Оно наследуется от GroundedState и отвечает только за поведение во время бега.

```

public class RunState : GroundedState
{
    public RunState() : base(parent: null) { } // В реальном коде нужно правильно
    пробрасывать родителя.
    public override HierarchicalState Tick(StateContext context)
    {
        context.Movement.SetHorizontalIntent(context.Intent.MoveDirection);
        context.Movement.ApplyGravity(0);
        // Если игрок перестал двигаться, мы возвращаем запрос на переход в Idle.
        if (Mathf.Abs(context.Intent.MoveDirection) < 0.1f)
            return new IdleState();
        return this;
    }
}

```

Обратите внимание на элегантность, которую даёт иерархия. RunState даже не задумывается о том, можно ли сейчас прыгнуть. Логика прыжка инкапсулирована на уровне GroundedState. Если во время бега игрок нажмёт кнопку прыжка, сигнал JumpPressed будет перехвачен родителем, и переход в AirborneState произойдёт корректно, автоматически подчищая за собой и RunState, и GroundedState. Это и есть та самая изоляция ответственности, которая на масштабе превращает поддержку кода из боли в удовольствие.

Когда следует остановиться: цена иерархии и её альтернативы

Взяв в руки такой мощный инструмент, как HSM, очень легко увлечься и начать строить башни из состояний там, где хватило бы и сарая. Первый и самый важный критерий введения иерархии – это **дублирование**. Если вы ловите себя на том, что копируете один и тот же блок кода (реакцию на урон, проверку контакта с землёй) в три и более состояний, значит, у вас появился общий уровень ответственности, который просится быть выделенным в родительское состояние.

Второй критерий – **плотность переходов**. Если ваша диаграмма состояний напоминает клубок спагетти, где из каждого узла можно попасть почти в любой другой, иерархия поможет распутать этот клубок, сгруппировав узлы в контейнеры с общими переходами.

Однако есть и случаи, когда HSM избыточен. Не стоит вводить иерархию, если общая логика исчерпывается одним-двумя условиями, которые проще обработать в контроллере. Не стоит строить глубокие деревья, если каждый уровень будет содержать по одному дочернему элементу – такая структура не уменьшает, а увеличивает сложность. И наконец, сигналом о переусложнении служит ситуация, когда ради соблюдения иерархии вы начинаете жертвовать ясностью кода. Хорошая архитектура не усложняет, а проясняет. Если, бросив взгляд на вашу HSM, невозможно быстро ответить, что делает персонаж в данный момент, значит, вы где-то перешли черту.

HSM – это не «серебряная пуля», а архитектурная стратегия для управления глубиной. Она не отменяет, а дополняет принципы, которые мы изучали ранее. Ортогональные аспекты поведения, такие как «вооружён/безоружен» или «под водой / на суше», по-прежнему лучше моделировать не как отдельные ветки иерархии, а как параметры контекста или компоненты-модификаторы, чтобы избежать комбинаторного взрыва классов. Контроллер по-прежнему остаётся единой точкой входа для управления потоком исполнения, гарантируя, что фазы Enter, Tick и Exit выполняются в строгом, предсказуемом порядке.

В итоге грамотно выстроенная иерархическая машина состояний делает архитектуру поведения не только более мощной, но и более приближенной к тому, как мы, люди, интуитивно мыслим игровые режимы. Она позволяет дизайнеру сказать: «Когда персонаж на земле, он может то-то и то-то», и это напрямую отразится в коде, а не растворится в десятках разбросанных по проекту файлов. И это, пожалуй, и есть высшая цель архитектуры – говорить с разработчиком и дизайнером на одном, предметном языке.

7.3. Стратегия как механизм вариативности алгоритмов

Машины состояний, которые мы разобрали в предыдущих разделах, дали нам чёткий ответ на вопрос «в каком режиме сейчас находится система». Но внутри каждого режима часто скрывается собственная вариативность, которую нельзя выразить одной лишь сменой состояний. Персонаж может атаковать мечом или луком. Враг может патрулировать по кругу или челночным бегом. Камера может следовать за игроком с упреждением или держать цель строго в центре. Все эти различия касаются не того, что делает объект, а того, как именно он это делает.

Новичок, столкнувшись с такой задачей, почти всегда идёт по пути наименьшего сопротивления. Он вводит в класс-контроллер перечисление, хра-

нящее выбранный вариант поведения, и в каждом методе, который от этого варианта зависит, пишет условный блок.

Представьте себе боевого робота, у которого по дизайну есть два типа оружия. Первое – плазменная пушка, стреляющая лучом по прямой. Второе – ракетница, выпускающая медленный самонаводящийся снаряд. В коде новичка это чаще всего выглядит как разрастающийся switch:

```
public enum WeaponType { Plasma, Rocket }
public class RobotController
{
    private WeaponType _currentWeapon;
    public void Attack()
    {
        switch (_currentWeapon)
        {
            case WeaponType.Plasma:
                SpawnProjectile("res://projectiles/plasma_beam.tscn", GlobalPosition,
GetTargetDirection());
                Audio.Play("plasma_sfx");
                break;
            case WeaponType.Rocket:
                var rocket = SpawnProjectile("res://projectiles/rocket.tscn",
GlobalPosition, Vector2.Zero);
                rocket.Target = FindNearestEnemy();
                rocket.Speed = 150f;
                Audio.Play("rocket_launch_sfx");
                break;
        }
    }
}
```

Пока оружия два, этот код работает. Но вот дизайнер просит добавить третий тип – импульсный излучатель, который бьёт не одним лучом, а конусом из трёх. Потом появляется режим берсерка, временно меняющий поведение любого оружия на более агрессивное. Затем выясняется, что боссы должны использовать слегка модифицированные версии атак. Каждое такое изменение заставляет лезть в RobotController, увеличивая switch и повышая риск сломать уже работающую логику. Этот класс превращается в «центр алгоритмической вселенной», который знает всё о каждом типе оружия и мешает разным разработчикам работать параллельно. Паттерн «Стратегия» решает эту проблему, вынося алгоритм из контекста в отдельный взаимозаменяемый объект. Контекст перестаёт знать, как именно он атакует; он лишь держит ссылку на объект-стратегию и говорит ему: «Выполни атаку, вот необходимые данные».

Отделение алгоритма от контекста и смена в рантайме

Фундамент паттерна – простой интерфейс, который описывает контракт алгоритма. Для нашего примера с оружием он может выглядеть так:

```

public interface IWeaponStrategy
{
    void Execute(AttackContext context);
}
public class AttackContext
{
    public Vector2 OriginPosition;
    public Vector2 TargetDirection;
    public Node WorldNode; // Для спавна снарядов.
}

```

Теперь каждый тип оружия становится отдельным классом, который реализует этот интерфейс. Вся уникальная логика, включая тип снаряда, его скорость, звук и особые свойства, заперта внутри этого класса.

```

public class PlasmaGun : IWeaponStrategy
{
    public void Execute(AttackContext ctx)
    {
        var beam = ResourceLoader.Load<PackedScene>("res://projectiles/plasma_beam.tscn").
Instantiate();
        beam.GlobalPosition = ctx.OriginPosition;
        beam.SetDirection(ctx.TargetDirection);
        ctx.WorldNode.AddChild(beam);
        Audio.Play("plasma_sfx");
    }
}
public class RocketLauncher : IWeaponStrategy
{
    public void Execute(AttackContext ctx)
    {
        var rocket = ResourceLoader.Load<PackedScene>("res://projectiles/rocket.tscn").
Instantiate();
        rocket.GlobalPosition = ctx.OriginPosition;
        rocket.Target = ctx.FindNearestEnemy(); // Метод расширения для контекста
        rocket.Speed = 150f;
        ctx.WorldNode.AddChild(rocket);
        Audio.Play("rocket_launch_sfx");
    }
}

```

Сам `RobotController` после этого становится удивительно простым. Он больше не содержит ни одной строчки, специфичной для конкретного оружия. Он просто получает стратегию и вызывает её метод, передавая контекст.

```

public class RobotController
{
    private IWeaponStrategy _weaponStrategy;
    public RobotController(IWeaponStrategy initialWeaponStrategy)
    {
        _weaponStrategy = initialWeaponStrategy;
    }
}

```

```

public void ChangeWeaponStrategy(IWeaponStrategy newWeaponStrategy)
{
    _weaponStrategy = newWeaponStrategy;
}
public void Attack()
{
    var context = new AttackContext
    {
        OriginPosition = GlobalPosition,
        TargetDirection = GetTargetDirection(),
        WorldNode = GetTree().Root
    };
    _weaponStrategy.Execute(context);
}
}

```

Вся вариативность атак теперь сосредоточена в классах, реализующих `IWeaponStrategy`. Добавление третьего оружия, о котором просил дизайнер, сводится к написанию нового класса `ImpulseEmitter` и регистрации его в системе подбора оружия. Код `RobotController` не меняется вообще. Более того, мы получаем возможность менять оружие прямо во время игры, просто вызвав метод `ChangeWeaponStrategy`. Эта возможность переключения алгоритмов в рантайме и есть главное архитектурное преимущество стратегии перед наследованием. Наследование заставило бы нас создавать иерархию `PlasmaRobotController`, `RocketRobotController` и пересоздавать объект при каждой смене оружия. Стратегия же делает алгоритм просто заменяемой деталью.

Стратегия и состояние: разные оси вариативности

Теперь, когда механизм понятен, важно провести чёткую границу между стратегией и состоянием. Обе эти конструкции управляют поведением, но отвечают на принципиально разные вопросы. Состояние (из конечного автомата) отвечает на вопрос «что мы делаем?» и обладает собственным жизненным циклом с переходами, фазами `Enter` и `Exit`. Стратегия отвечает на вопрос «как мы это делаем?» и обычно не имеет жизненного цикла, кроме момента выполнения.

В контексте нашего робота, «атака» – это состояние. Вход в него может включать подготовку оружия и блокировку движения. А вот то, как именно будет выполнена атака – лучом или ракетой, – это стратегия, которую состояние атаки дёргает из своего контекста. Состояние владеет временным потоком, стратегия владеет алгоритмом.

Важно не путать эти уровни. Если ваша стратегия начинает требовать фазы «подготовки», «восстановления» и сложные переходы по таймерам, значит, она переросла стратегию и должна быть повышена до состояния. И наоборот, если ваше состояние атаки превратилось в мешанину из `if`, чтобы поддерживать разные типы урона, это верный сигнал, что пора выделить стратегию.

Стратегии также могут быть не полностью стерильными. Они могут хранить внутреннее состояние, необходимое для реализации алгоритма. Напри-

мер, у плазменной пушки может быть механика «нагрева»: при каждом выстреле накапливается тепло, и при достижении лимита пушка блокируется на перезарядку. Это состояние – часть алгоритма стрельбы, а не режим робота.

```
public class PlasmaGun : IWeaponStrategy
{
    private float _heat = 0f;
    private const float MaxHeat = 100f;
    private const float CoolingRate = 25f;
    public void Execute(AttackContext ctx)
    {
        if (_heat >= MaxHeat)
            return; // Перегрев, ждём.
        FireBeam(ctx);
        _heat += 15f;
    }
    // Этот метод нужно вызывать извне, например каждый кадр.
    public void CoolDown(float delta)
    {
        _heat = Mathf.Max(0, _heat - CoolingRate * delta);
    }
}
```

Хранение такого состояния внутри стратегии абсолютно нормально до тех пор, пока оно инкапсулировано и не влияет на глобальный контекст. Главное, чтобы ответственность за это состояние несла сама стратегия, а не внешний код.

Когда стратегия избыточна: ловушка преждевременной гибкости

Любой инструмент можно использовать во вред, и паттерн «Стратегия» не исключение. Самая частая ловушка – введение стратегии там, где вариативность отсутствует или крайне маловероятна. Если вы делаете игру про рыцаря, который от начала и до конца будет махать одним и тем же мечом, оборачивание его единственной атаки в интерфейс `IMeleeStrategy` не принесёт пользы. Это добавит ненужный слой косвенности, который затруднит чтение кода, но не даст взамен ни гибкости, ни переиспользования.

Признак того, что стратегия не нужна, – это наличие ровно одной реализации интерфейса в проекте. Прежде чем создавать стратегию, спросите себя: «Появится ли у меня в обозримом будущем необходимость переключать это поведение в рантайме? Есть ли у меня сейчас хотя бы два чётко различающихся варианта?» Если ответ «нет» – не усложняйте. Прямой вызов метода внутри класса гораздо понятнее и не менее архитектурно чист, пока он не начинает ветвиться.

Ещё одна частая ошибка – «стратегический монолит», когда интерфейс стратегии пытается покрыть все возможные аспекты поведения сразу. Например, создаётся интерфейс `INemyBehavior` с методами `Move`, `Attack`, `React`–

ToDamage, SelectTarget. Такой интерфейс не является стратегией, он пытается быть целым AI-контроллером. Правильный подход – дробить стратегии по осям вариативности: отдельно стратегия движения, отдельно стратегия атаки. Это позволяет комбинировать их независимо, создавая, скажем, быстрого врага с ближней атакой и медленного врага с дальней атакой из одних и тех же компонентов.

И наконец, не стоит делать стратегии всезнающими. Если стратегия атаки для своей работы начинает искать узлы в дереве сцены через GetNode или обращаться к глобальному состоянию напрямую, она теряет свою главную ценность – изолированность. Все необходимые для работы данные должны приходить через контекст, и только через него. В идеале стратегия – это чистая функция, которая получает входные данные и вызывает побочные эффекты через переданные ей абстракции.

Паттерн «Стратегия» – это мощный рычаг для управления сложностью в играх, где вариативность механик может расти лавинообразно. Он позволяет собирать поведение из готовых блоков, заменять их на лету и тестировать независимо. Но его сила – в дисциплине. Чётко разделяя режимы (состояния) и способы их реализации (стратегии), вы создаёте архитектуру, которая легко переживёт бесконечные итерации гейм-дизайна, не превращаясь в лапшу из условий. А это, пожалуй, и есть главный инженерный вызов в разработке игр.

7.4. Связывание модулей через уведомления: Наблюдатель

Как только механика, управляемая машинами состояний и стратегиями, начинает влиять на мир, возникает инженерная потребность в коммуникации. Враг умер – нужно открыть дверь, добавить очки, проиграть звук, обновить квест. Прямолинейный подход, когда Enemy.Die() вызывает методы двери, UI и звука, за пару дней превращает проект в свалку взаимных зависимостей. Enemy знает про Door, дверь знает про Quest, квест знает про UI. Изменение одного элемента лавиной расходится по проекту. Паттерн Наблюдатель (observer) инвертирует эту связь. Источник (Enemy) не рассылает приказов. В момент смерти он просто публикует факт: «Произошла смерть». Любой другой модуль в игре, которому важен этот факт, заранее на него подписывается и реагирует самостоятельно. Издатель остаётся в полном неведении о том, кто его слушает. Такой подход превращает хаотичную сеть вызовов в стройную систему широковещательных уведомлений, где каждый модуль отвечает только за свою реакцию.

Сигналы и события: два инструмента для одной задачи

В арсенале Godot и C# есть два штатных механизма для реализации Наблюдателя: сигналы (signals) движка и события (event) языка. Новички часто

путают их или пытаются выбрать «единственно правильный», но для архитектуры важно не это, а граница между ними.

Сигналы Godot – нативный механизм узлов. Они встроены в объектную модель движка, сериализуются, видны в редакторе и глубоко привязаны к жизненному циклу сцены. Area2D отправляет сигнал BodyEntered, Button – Pressed. Их основная роль – сообщать о низкоуровневых движковых фактах: коллизиях, окончании анимации, нажатии кнопок интерфейса. Сигналы удобны для связи объектов на сцене, но их опасно тащить в доменную логику. Если квест подпишется на движковый сигнал Area2D.BodyEntered, он станет зависимым от геометрии уровня, которую вы в любой момент можете изменить.

События C# – механизм языка, живущий в управляемой памяти .NET. Они не привязаны к сцене, к узлам или к дереву. Они идеально подходят для выражения **доменных фактов**: здоровье изменилось, персонаж умер, оружие переключилось. Такой доменный код можно протестировать без запуска Godot, подписав на событие простую заглушку.

Главное правило, которое спасает архитектуру: **движковые сигналы не должны проникать в домен**. На границе сцены всегда должен стоять тонкий адаптер. Он слушает сигнал Godot и переводит его в C#-событие, понятное правилам игры.

Представьте GameplayBridge: узел Area2D, который висит у входа в логово босса. Его CollisionMask через политику слоёв (раздел 6.1) настроен только на игрока. Когда тот заходит в зону, GameplayBridge ловит родной сигнал BodyEntered и дёргает уже чистое C#-событие PlayerEnteredBossArena. Сам адаптер живёт на сцене и отвечает за свой жизненный цикл. Система квестов, подписанная на PlayerEnteredBossArena, ничего не знает о триггерах. Вы можете завтра выбросить мост и рассчитать вход через расстояние – квест не изменится.

Управление жизненным циклом подписок: симметричный вход и выход

Единожды подписавшись на событие, вы создаёте ссылку, которая будет жить до тех пор, пока жив издатель. Если подписчик – UI-панель – удаляется из дерева, а вы забыли отписаться, возникает утечка. Сборщик мусора C# не сможет убрать панель, потому что на неё через цепочку делегатов всё ещё ссылается живой объект вроде компонента здоровья. Это приводит не только к расходованию памяти, но и к «призрачным» вызовам, когда закрытое окно продолжает обновлять счётчики, вызывая ошибки.

Дисциплина здесь строга, но проста: **подписка и отписка всегда идут парой, привязанной к жизненному циклу узла**. В Godot для этого идеально подходит связка методов _EnterTree и _ExitTree. Подписались при входе в дерево – гарантированно отпишитесь при выходе из него. Никаких исключений.

```
public partial class KillCounter : Label
{
    private HealthComponent _bossHealth;
    public void Initialize(HealthComponent bossHealth)
```

```

{
    _bossHealth = bossHealth;
    _bossHealth.Died += OnBossDied;
}
private void OnBossDied()
{
    Text = "Босс повержен!";
}
public override void _ExitTree()
{
    if (_bossHealth != null)
        _bossHealth.Died -= OnBossDied;
}
}

```

Эта привычка должна стать такой же автоматической, как проверка `IsOnFloor()` перед прыжком. Если в вашем проекте царит полный порядок с отписками, вы избавляетесь от половины самых трудноуловимых багов, связанных с «неожиданным поведением» невидимых объектов.

Факты, а не команды: предотвращение скрытой оркестрации

Самый страшный грех при использовании Наблюдателя – превращение его в механизм неявных команд. Если событие называется `OpenDoor`, издатель перестаёт быть источником факта и превращается в командира. Он начинает знать, что существует дверь, которую надо открыть. Вы снова возвращаетесь к жёсткой связанности, просто канал связи стал более извилистым. Всегда формулируйте событие как свершившийся факт предметной области. `DoorOpened` – факт. `BossDefeated` – факт. `HealthChanged` – факт. Решение о том, что делать с этим фактом, полностью лежит на подписчиках. Дверь реагирует не на команду «открыться», а на факт «все враги в комнате мертвы». Это тонкое смещение ответственности полностью меняет структуру проекта, делая каждый модуль автономным. Именно здесь кроется ответ на вопрос, почему паттерн так важен в комплексе с машинами состояний и стратегиями. Машина состояний управляет переходами и публикует факты о вхождении в режим. Стратегия выполняет алгоритм. А Наблюдатель разносит новости об этих свершениях по всем заинтересованным сторонам, не позволяя ни одной из них залезть в чужую ответственность. Такой тройственный союз и есть фундамент поддерживаемой игровой архитектуры.

8

Event-Driven архитектура

8.1. Сигналы Godot и события C# – два инструмента для уведомлений

Когда мы в предыдущих разделах говорили, что «модуль публикует факт, а другие подписываются», мы намеренно не уточняли, как именно это делается. В арсенале разработчика на Godot и C# есть два самостоятельных механизма, решающих одну и ту же задачу: движковые сигналы и языковые события. Оба работают по одному принципу «издатель–подписчик», но живут в разных мирах и диктуют разную архитектуру. Понимать, какой из них когда брать в руки, – одна из главных инженерных привычек, которая сэкономит вам нервы и предотвратит превращение проекта в запутанный клубок.

Природа сигналов Godot и событий C#

Сигнал в Godot – это не просто функция обратного вызова. Это публичное обязательство узла или ресурса перед всем движком. Он объявляется прямо в теле класса с помощью атрибута [Signal], виден в редакторе и неразрывно связан с жизненным циклом своего владельца. Когда кнопка посылает Pressed, а область Area2D – BodyEntered, они используют именно сигналы. В этом их сила и их ограничение: сигнал идеально подходит для коммуникации внутри сцены, между объектами, которые живут и умирают вместе с ней.

Событие C# – это конструкция языка, не зависящая от движка. Оно живёт в любом классе, хоть унаследованном от Node, хоть совершенно самостоятельном, и предоставляет типизированный контракт через делегат. Подписчики добавляются через оператор +=, и с этого момента издатель, сам

того не зная, начинает удерживать ссылки на них. События C# незаменимы там, где логика не должна быть привязана к дереву сцены. Модель здоровья, счётчик очков, система прогресса – всё это может работать и тестироваться без единого узла, если оно общается через event.

Давайте посмотрим на эту разницу в коде. Вот типичный сценарный объект, реагирующий на игрока с помощью сигнала:

```
using Godot;
public partial class Lever : Node3D
{
    // Объявляем движковый сигнал. Он станет частью API этого узла.
    [Signal]
    public delegate void ActivatedEventHandler();
    public void Pull()
    {
        // Публикуем сигнал, когда рычаг дёрнули.
        EmitSignal(SignalName.Activated);
    }
}
```

А вот подписчик, который живёт на той же сцене и реагирует на этот сигнал:

```
public partial class Trapdoor : Node3D
{
    [Export] private Lever _connectedLever;
    public override void _Ready()
    {
        _connectedLever.Activated += OnLeverActivated;
    }
    private void OnLeverActivated()
    {
        GD.Print("Люк открывается по сигналу!");
        // Здесь можно запустить анимацию, поднять створку и т. д.
    }
    public override void _ExitTree()
    {
        // Никогда не забывайте отписываться, если подписчик умирает раньше издателя!
        _connectedLever.Activated -= OnLeverActivated;
    }
}
```

Теперь представьте, что нам нужно оповестить об изменении здоровья. Используя события C#, мы можем сделать класс `HealthComponent` полностью независимым от Godot-API:

```
public class HealthComponent
{
    private int _current;
    public event Action<int, int> HealthChanged;
    public void TakeDamage(int amount)
    {
        var previous = _current;
        _current = Mathf.Max(0, _current - amount);
    }
}
```

```

        HealthChanged?.Invoke(_current, previous);
    }
}

```

Система сохранения, интерфейс или звук – кто угодно может подписаться на `HealthChanged`, и издатель об этом никогда не узнает. Это чистая доменная логика, которую можно протестировать, просто вызвав `TakeDamage` и проверив, что событие было брошено с нужными аргументами.

Владение, утечки и жизненный цикл подписок

Ключевое архитектурное различие между сигналами и событиями – это владение временем жизни. Подписка на сигнал `Godot` осмыслена ровно до тех пор, пока оба её участника находятся в дереве сцены. Движок не позволит вам вызвать метод удалённого узла, но он и не заставит вас отписываться, если подписчик просто вышел из дерева. Это ложится на ваши плечи: забыли вызвать `--` и при следующем срабатывании сигнала получите ошибку, потому что узел уже невалиден. Подписка на событие `C#`, напротив, живёт по законам сборщика мусора `.NET`. Если ваш подписчик – обычный класс, а издатель – синглтон или долгоживущий объект, то забытая отписка гарантированно превратится в утечку памяти. Издатель будет вечно удерживать ссылку на подписчика, не давая сборщику мусора его удалить. При этом никаких видимых ошибок не будет, пока игра не начнёт тормозить и не упадёт с переполнением памяти. Именно поэтому правило «где подписался – там и отписался» одинаково важно для обоих механизмов. Для узлов идеальным моментом для подписки служит `_Ready`, а для отписки – `_ExitTree`. Для чистых `C#`-классов часто заводят парный метод `Dispose` или полагаются на то, что подписчик и издатель умирают одновременно.

Возведение моста: от движка к домену и обратно

Самый частый архитектурный паттерн в коммерческих проектах на `Godot` – это использование обоих механизмов вместе, но со строгим разделением зон ответственности. Сигналы живут на уровне сцены и связывают визуальные, физические и пользовательские события. События `C#` живут на уровне домена и переносят факты между чистыми моделями. Границу между ними охраняет тонкий адаптер – узел, который слушает движковые сигналы и переводит их на язык доменных событий, и наоборот. Представьте всё ту же зону входа в логово босса, которую мы рассматривали в разделе о Наблюдателе. Вместо того чтобы раздавать прямые ссылки на ворота и камеру, мы создаём узел-мост `BossArenaGate`, который висит на сцене как `Area2D`. Он слушает собственный сигнал `BodyEntered` и транслирует его в чистое `C#`-событие `BossArenaEntered`.

```

using Godot;
public partial class BossArenaGate : Area2D
{
    // Доменное событие, которое уже не зависит от Godot.

```

```

// На него подпишется система квестов или камера.
public event Action PlayerEnteredArena;
public override void _Ready()
{
    // Настраиваем коллизию так, чтобы видеть только игрока.
    CollisionLayer = 0;
    CollisionMask = CollisionPolicy.Player;
    BodyEntered += OnBodyEntered;
}
private void OnBodyEntered(Node2D body)
{
    if (body is PlayerController)
    {
        // Перевод движкового события в доменное.
        PlayerEnteredArena?.Invoke();
        // Отключаем триггер, чтобы не срабатывал повторно.
        SetDeferred("monitoring", false);
    }
}
public override void _ExitTree()
{
    BodyEntered -= OnBodyEntered;
}
}

```

Теперь система, отвечающая за квесты, может просто подписаться на `PlayerEnteredArena` и не знать ничего ни о коллизиях, ни о триггер-зонах. Если завтра дизайнер решит, что босс появляется не по входу в зону, а по нажатию рычага, мы просто поменяем один узел-адаптер на другой, ни на секунду не останавливая работу квестов. Это и есть та самая архитектурная устойчивость, которую даёт нам чёткое разделение миров движка и домена.

Хочется закрепить простую мысль. Сигналы и события – не конкуренты, а взаимодополняющие инструменты. Сигналы – это проводка вашей сцены, её нервная система. События – это язык вашего домена, его законы и правила. Как хороший электрик не путает силовые кабели с сигнальными, так и хороший архитектор держит сигналы в сцене, а события – в ядре. Проведя эту границу однажды, вы сэкономите недели отладки и рефакторинга в будущем.

8.2. Глобальная шина событий: централизованная коммуникация в масштабах всей игры

Локальные события, которые мы разбирали в предыдущем разделе, отлично работают, пока подписчики находятся рядом с источником. Компонент здоровья публикует факт `HealthChanged`, а соседний UI-узел или привязанный зву-

ковой плеер на него реагирует. Вся цепочка легко прослеживается, а время жизни объектов совпадает или строго контролируется через `_ExitTree`. Но по мере роста проекта появляются ситуации, которые локальными средствами решить без архитектурного ущерба уже невозможно.

Представьте себе факт «игрок погиб». Он должен одновременно: запустить анимацию смерти и проиграть траурную музыку, остановить всех врагов на уровне, сохранить финальную статистику, показать интерфейс перезапуска, отправить событие в систему аналитики и, возможно, синхронизировать состояние в мультиплеере. Пытаться реализовать всё это, протянув прямые ссылки от игрока к системам аналитики, менеджеру сцен и хранилищу сохранений означает превратить класс `Player` в абсолютного монстра, знающего обо всех подсистемах игры. Точно так же нельзя и заставить все эти системы подписываться на личное событие `Player.Died`, таская за собой зависимость от конкретного префаба игрока.

Ровно для таких сквозных, общеигровых фактов и вводится понятие **глобальной шины событий** (Global Event Bus). Шина – это независимый канал связи, который не принадлежит ни одной сцене или персонажу. Любой модуль может объявить о важном факте, опубликовав его в шину, и любой другой модуль, которому этот факт интересен, может на него подписаться. Издатель и подписчик при этом друг о друге не знают. Они знают только о контракте – типе события.

Когда локальных событий не хватает: рождение шины

В основе глобальной шины лежит простая идея, которая часто приходит разработчику интуитивно: раз есть статические методы и глобальный доступ, почему бы не сделать статический класс, где лежат все важные события? Формально это будет работать. Код ниже компилируется и даже выполняет задачу:

```
public static class GlobalEvents
{
    public static event Action PlayerDied;
    public static event Action<Vector2, float> ShakeCamera;
}
```

Вызывая `GlobalEvents.PlayerDied?.Invoke()` из любого места, мы получаем ту самую широковещательную доставку. Однако подобная реализация закладывает мину под фундамент проекта. Статическое событие живёт от запуска приложения до его закрытия. Любой подписчик, который забыл отписаться, будет висеть в памяти вечно, потому что статический корень не даёт сборщику мусора его удалить. Через десять перезагрузок уровня в памяти может скопиться десять неактивных слушателей, каждый из которых будет пытаться обновить уже несуществующий интерфейс.

Второй, не менее важный недостаток – это полное отсутствие границ. Когда любое событие доступно глобально, исчезает разница между «фактом уровня» и «фактом сессии». Обработчик, написанный для конкретной

сцены, может внезапно сработать в главном меню, если сцена выгрузилась, а подписка осталась. Проект теряет предсказуемость.

Чтобы шина стала архитектурным благом, а не ящиком Пандоры, мы отказываемся от статике в пользу **объекта с управляемым временем жизни**. Шина создаётся при старте игровой сессии (или загрузке уровня) и уничтожается вместе с ней. Вместе с её удалением гарантированно рвутся все подписки. Более того, мы типизируем каждое событие, превращая его из безымянного делегата в строгий доменный контракт – структуру, которая несёт минимум данных, но точно описывает факт.

Взгляните на пример такого события и самой шины. Это уже не просто статическое хранилище, а полноценная инфраструктурная единица.

```
// Доменный факт: структура данных, а не ссылка на Godot.Object.
public readonly record struct PlayerDiedEvent(Vector2 DeathPosition, int FinalScore);
public class SessionEventBus
{
    public event Action<PlayerDiedEvent> PlayerDied;
    public void Publish(PlayerDiedEvent eventData)
    {
        PlayerDied?.Invoke(eventData);
    }
}
```

Теперь, когда игрок погибает, мы отправляем в шину структуру `PlayerDiedEvent`, а все заинтересованные службы – будь то менеджер сохранений или система достижений – подписываются на это событие. При этом сам `SessionEventBus` не является статическим классом. Он создаётся в `Composition Root` (корне сцены) и передаётся туда, где нужен, либо через внедрение, либо через сервис-локатор верхнего уровня. Когда сессия завершается, шина просто выходит из области видимости, и сборщик мусора освобождает всех подписчиков.

Контракты и полезная нагрузка: только факты, никаких команд

Самый страшный грех при использовании шины – превратить её в инструмент для отдачи приказов. Если в вашей шине появляются события `OpenMainMenu` или `PlayBossMusic`, значит, вы начали использовать её не для уведомлений, а для скрытой оркестрации. Вы снова заставляете издателя знать, что должно произойти в мире. Архитектурно правильная шина работает строго с фактами: `PlayerDied`, `BossDefeated`, `LevelCompleted`. Решения о том, что делать с этими фактами, принимают сами подписчики в своей зоне ответственности. Менеджер сцен ловит `LevelCompleted` и решает, что пора загружать следующий уровень. Музыкальный контроллер ловит то же самое событие и запускает фанфары. Издателю (в данном случае триггеру окончания уровня) безразличны оба потребителя. Такое же дисциплинированное отношение должно быть и к данным, которые переносит событие. Никогда не переда-

вайте через шину ссылки на живые игровые объекты (узлы, префабы, целые скрипты вроде `Player`). Событие может быть обработано через секунду после публикации, когда объект уже покинет дерево сцены и станет невалидным. Передавайте исключительно снимки состояния: идентификаторы, позиции, счётчики, перечисления – всё, что является чистым значением. Для этого идеально подходят `C# record struct`, как в примере выше.

Дисциплина использования: как не превратить шину в хаос

Даже хорошо спроектированная шина станет проблемой, если не соблюдать эксплуатационную дисциплину. Худшее, что можно сделать, – это начать подписываться на шину из `_Process` или из середины `Update`, создавая и уничтожая подписки внутриигровой логики. Подписка на уровне сессии должна осуществляться на старте (`_Ready`), и там же должен быть привязан её разрыв.

Второй опасный момент – это расчёт на порядок вызова подписчиков. Шина вызывает обработчики синхронно, в порядке их подключения. Но полагаться на этот порядок для реализации геймплейной логики – фатальная ошибка. Если ваша система сохранений должна записать прогресс строго до того, как UI покажет окно результатов, вы должны реализовать это через отдельного координатора, который сам, в явном виде, дёрнет нужные методы. Шина в этой схеме может использоваться лишь для того, чтобы сообщить о завершении того или иного этапа.

Наконец, стоит избегать соблазна создать одну-единственную шину на все случаи жизни. Архитектурно гораздо устойчивее развести события по областям ответственности. Пусть `SessionEventBus` отвечает за прогресс, жизнь игрока и межсценовые переходы. Для внутрисценивых механик (волны врагов, открытие дверей) используйте локальный `LevelEventBus`, который живёт ровно столько же, сколько и сцена. Такое разделение не позволяет событиям покидать пределы своей ответственности и держит граф связей в чистоте.

Глобальная шина событий, при всей своей мощи, требует спартанской строгости. Чётко определив события как типизированные факты, привязав время жизни шины к сессии или уровню и безжалостно удаляя любые попытки протащить через неё оркестрирующие команды, вы получаете невероятно мощный и при этом прозрачный способ связи модулей. Системы, спроектированные вокруг такой шины, легко наращивать и тестировать, потому что каждый новый потребитель фактов просто добавляет свою подписку, не затрагивая издателя и не рискуя сломать работающую механику.

8.3. Техники ослабления связанности: контракты, события и границы модулей

Связанность – невидимая сила, которая определяет, насколько легко или тяжело будет меняться ваш проект. Когда два модуля сильно связаны, измене-

ние одного из них, будь то класс игрока или система сохранений, неизбежно вызывает каскад правок в другом. Вы словно пытаетесь разобрать плотно сцепленный механизм, где даже замена маленькой шестерёнки требует разбора всего устройства. Слабая связанность, напротив, стремится сделать модули независимыми. Вы можете полностью переписать интерфейс, и ни одна строчка в логике боя не сломается. Вы можете заменить способ сохранения с локального файла на облачную базу данных, и классы игрового мира об этом даже не узнают. Достижение такой гибкости не магия, а результат системного применения нескольких фундаментальных техник. В предыдущих разделах мы говорили о сигналах, событиях и шинах как о способах доставки фактов. Теперь наша задача – посмотреть на механику вокруг них: как проектировать контракты, которые не устаревают, как управлять направлением зависимостей и как собирать объекты в единую систему, не создавая при этом хаоса из ссылок.

Природа сильной связанности и её проявления

Прежде чем что-то ослаблять, нужно научиться видеть проблему в коде. Сильная связанность редко появляется как результат чьего-то злого умысла. Обычно она прорастает из самых невинных решений, принятых в спешке прототипирования. Один из самых ярких её симптомов – это когда изменение радиуса в пять строк в UI-панели внезапно ломает расчёт урона. Или когда добавление новой механики, например рывка, заставляет вас переписывать код врагов. Это происходит потому, что модули знают друг о друге слишком много лишнего.

Особенно коварна **временная связанность**. Она не видна в структуре классов, но жёстко проявляется в рантайме. Представьте, что вы написали две системы: `PlayerStats` публикует событие `StatsUpdated` после подсчёта бонусов от экипировки, а UI-панель `StatsPanel` подписывается на него в своём методе `_Ready`. Всё работает, пока вы однажды не решите оптимизировать загрузку сцены и не создадите `PlayerStats` немного позже. `StatsPanel._Ready` отработает раньше, подписка не происходит, и панель навсегда остаётся пустой. В коде нет ни ошибок, ни исключений – только пустой интерфейс. Это и есть классический пример бага, рождённого неявным порядком инициализации.

Другая частая форма – **связанность через знание реализации**. Она возникает, когда один класс напрямую использует конкретный тип другого класса, а не его контракт. Кнопка лечения, которая делает `player.Health += 10`, знает, что здоровье – это поле типа `int`. Она понятия не имеет об инвариантах: что здоровье нельзя поднять выше максимума или что на время действия эффекта яда лечение запрещено. Вся ответственность за соблюдение правил игры перекладывается на UI-кнопку. Добавление любого нового правила заставит вас лезть в код интерфейса и править его.

Наконец, существует **связанность через владение**. Это когда один модуль создаёт и уничтожает объекты, жизненный цикл которых ему не принад-

лежит. Если код спавна врагов находится в корневой сцене уровня, которая сама через минуту может быть выгружена, враги и их подписки могут остаться в памяти, порождая «призрачные» взаимодействия. Вы избавились от уровня, а его враги всё ещё пытаются атаковать игрока в следующей сцене.

Контракты и события как барьеры для изменений

Главное лекарство от этих болезней – превратить неявные знания в явные контракты. Вместо того чтобы позволить кнопке напрямую менять счётчик здоровья, мы заворачиваем операцию лечения в команду, а её состояние публикуем через событие. Это разбивает прямую зависимость на две независимые, управляемые связи. Возьмём тот же пример с лечением. В сильно связанном коде кнопка `ButtonHeal` делает всю работу сама: увеличивает счётчик, сохраняет игру, запускает звук и анимацию. Мы превратим это в архитектурно чистое взаимодействие. Для начала кнопка не должна знать о существовании игрока как класса. Она зависит от абстракции – интерфейса, который предоставляет ей ровно одну способность: запросить лечение.

```
public interface IPlayerActions
{
    event Action<int> HealthChanged;
    void RequestHeal(int amount);
}
```

Этот интерфейс и есть наш контракт. Он сформулирован в терминах намерения (`RequestHeal`) и доменного факта (`HealthChanged`). Кнопка, работающая с этим интерфейсом, становится поразительно простой и полностью теряет интерес к тому, как устроен игрок.

```
public partial class HealButton : Button
{
    private IPlayerActions _player;
    public HealButton(IPlayerActions player)
    {
        _player = player;
        Pressed += OnPressed;
    }
    private void OnPressed()
    {
        _player.RequestHeal(10);
    }
}
```

Кнопка просто объявляет о намерении. Она не лечит, не проверяет лимиты и не сохраняет игру. Решение о том, можно ли сейчас лечиться, полностью лежит на реализации `IPlayerActions`, которая находится на более высоком уровне. Реализация этого контракта, живущая в корне сцены, и будет тем самым местом, где собираются все правила. Она получает запрос, проверяет условия (не мёртв ли игрок, не наложен ли эффект яда), меняет модель здо-

ровья и публикует факт изменения. А другая часть той же реализации может подписаться на факт изменения здоровья и запустить сохранение или звук, не смешивая эти обязанности с логикой лечения. Таким образом, UI-слой (презентация) становится потребителем и источником намерений, а доменная логика – единственным владельцем правил. События здесь играют роль идеального клея, который не даёт деталям одного слоя просочиться в другой. Важно лишь помнить, что каждое событие – это свершившийся факт, а не замаскированная команда. `HealthChanged` говорит «вот новое значение», а не «обнови полосу здоровья». Последнее – уже забота подписчика.

Композиция и владение как основа слабой связанности

Слабую связанность можно заложить в архитектуру с помощью интерфейсов и событий, но поддерживать её на протяжении всего жизненного цикла проекта можно только с помощью дисциплины композиции. Композиция отвечает на вопрос: где и как все эти независимые части находят друг друга? Плохой ответ – «где угодно, через глобальный доступ или поиск по дереву сцены». Хороший ответ – «в единственной, хорошо известной точке сборки».

В типичном Godot-проекте такой точкой сборки является `GameRoot` – узел на самой верхушке сцены, который переживает все загрузки уровней или создаётся заново при старте сессии. Его задача – создать главные доменные объекты (например, экземпляр `IPlayerActions`), создать адаптеры, связывающие их с движком, и передать эти зависимости в UI-панели. Никакой другой узел на сцене не должен заниматься поиском своих зависимостей через `GetNode` или `GetTree().Root.GetChild`.

Такой подход радикально упрощает отладку и тестирование. Если `HealthButton` перестала работать, вы не ищете, какой `Player` она пытается найти и почему у неё не получается. Вы смотрите на конструктор или метод инициализации и видите, получила ли она реализацию `IPlayerActions`. Если нет, проблема в точке сборки. Если получила, проблема внутри реализации `RequestHealth`. Граница между модулями чётко очерчена.

Управление временем жизни – такой же важный элемент композиции, как и внедрение зависимостей. Каждая подписка на событие (особенно на глобальной шине) должна быть привязана к жизненному циклу подписчика. Если узел UI подписался на `HealthChanged`, он обязан отписаться в своём `_ExitTree`. Это не прихоть, а защита от одного из самых трудноуловимых классов багов. Представьте, что панель интерфейса закрылась, но её метод-обработчик всё ещё висит в списке подписчиков и продолжает дёргаться при каждом получении урона, пытаясь обновить уже несуществующую текстуру. В хорошей архитектуре эта проблема просто не возникает, потому что отписка так же обязательна, как и подписка.

```
public partial class StatsPanel : Panel
{
    private IPlayerActions _player;
    public void Initialize(IPlayerActions player)
```

```

{
    _player = player;
    _player.HealthChanged += OnHealthChanged;
}
private void OnHealthChanged(int newHealth)
{
    // Обновление UI.
}
public override void _ExitTree()
{
    if (_player != null)
    {
        _player.HealthChanged -= OnHealthChanged;
    }
}
}

```

В совокупности эти техники формируют тот фундамент, на котором строятся большие и долгоживущие проекты. Они не делают код проще на первый взгляд, ведь приходится писать больше абстракций. Но они делают проект предсказуемым. Вы всегда знаете, куда смотреть при ошибке, и, что важнее, можете смело менять одну часть системы, не парализуя работу всего остального. Именно это качество и является главной целью архитектуры слабой связанности.

8.4. Тестирование систем в изоляции как архитектурный инструмент

Тестирование часто воспринимают как отдельный, утилитарный вид деятельности: написал код, затем написал тесты, чтобы убедиться, что он работает. В архитектурном же контексте способность системы быть протестированной играет совсем другую роль. Она служит безошибочным индикатором того, насколько хорошо вы разделили ответственность, скрыли детали реализации и построили явные контракты между модулями. Если для проверки простого правила «игрок получает уровень после 100 очков» вам приходится запускать движок, загружать сцену и нажимать кнопки, проблема не в тестах. Проблема в том, что ваша доменная логика оказалась замурована в интеграционный код.

Тестируемость как симптом качественной архитектуры

Представьте, что вы проектируете автомобиль. Хороший инженер спроектирует двигатель так, чтобы его можно было протестировать на стенде, отдельно от кузова, колёс и руля. Он подаёт топливо, масло и электричество через стандартизированные разъёмы, снимает показания с датчиков и убеждается

в корректности работы каждого поршня. Плохой инженер намертво вваривает двигатель в раму и подключает всю проводку на скрутках. Проверить работу такого агрегата можно только одним способом: залить бензин, сесть за руль и поехать. В точности то же самое происходит с кодом игры. Если ваш класс, отвечающий за выдачу награды за уровень, внутри себя ищет узел интерфейса через `GetNode`, воспроизводит звук через глобальный аудиоменеджер и пишет лог в файл в определённом формате, вы «вварили» его в `Godot`. Протестировать правило «начислить 500 монет, если уровень пройден на золото» без запуска сцены невозможно. Изоляция и детерминизм – вот два столпа, на которых держится тестируемый код. Изоляция означает, что модуль получает всё необходимое через свои зависимости и не обращается за этим к глобальному окружению. Детерминизм означает, что при одних и тех же входах модуль всегда производит одни и те же выходы и побочные эффекты.

Давайте посмотрим на типичный пример сильно связанного кода, который невозможно протестировать, и на то, как архитектурный рефакторинг меняет ситуацию. Допустим, у нас есть класс, проверяющий условие победы в небольшом платформере: игрок собрал десять монет. Код написан прямо в узле.

```
using Godot;
using System.IO;
public partial class WinChecker : Node
{
    public override void _Process(double delta)
    {
        var player = GetNode<Player>("/root/World/Player");
        if (player.Coins >= 10)
        {
            File.WriteAllText("save.json", "{ \"won\": true }");
            GetTree().ChangeSceneToFile("res://ui/win_screen.tscn");
        }
    }
}
```

В этом фрагменте спрессованы сразу три разные ответственности: доменное правило, инфраструктура сохранения и управление сценой. Протестировать, что при девяти монетах ничего не происходит, а при десяти – игра завершается, можно, лишь вручную пройдя уровень. Это ненадёжно, медленно и никак не защищает логику от регрессий.

Чтобы отделить правило от его реализации, мы первым делом выделяем само условие победы в чистый класс, который ничего не знает о движке. Все свои потребности он выражает в виде интерфейсов.

```
public interface IProgressStore
{
    void MarkWon();
}
```

```

public interface ISceneNavigator
{
    void ShowWinScreen();
}
public sealed class WinRule
{
    private readonly IProgressStore _store;
    private readonly ISceneNavigator _navigator;
    public WinRule(IProgressStore store, ISceneNavigator navigator)
    {
        _store = store;
        _navigator = navigator;
    }
    public void OnCoinsChanged(int coins)
    {
        if (coins < 10)
            return;
        _store.MarkWon();
        _navigator.ShowWinScreen();
    }
}

```

Теперь WinRule – это чистая функция с предсказуемым поведением. Мы можем вручную, без всякого движка, создать два простых объекта-заглушки, которые запоминают, были ли они вызваны.

```

public class StubProgressStore : IProgressStore
{
    public bool MarkedWon { get; private set; }
    public void MarkWon() => MarkedWon = true;
}
public class StubSceneNavigator : ISceneNavigator
{
    public bool WinShown { get; private set; }
    public void ShowWinScreen() => WinShown = true;
}
// Пример ручной проверки, которая может жить прямо в консольном приложении.
var store = new StubProgressStore();
var nav = new StubSceneNavigator();
var rule = new WinRule(store, nav);
rule.OnCoinsChanged(9);
Console.WriteLine(store.MarkedWon == false); // true, ничего не произошло.
rule.OnCoinsChanged(10);
Console.WriteLine(store.MarkedWon == true && nav.WinShown == true); // true, условие
                                                                    // выполнено.

```

Этот пример показывает ключевую идею: тестирование не требует фреймворков. Оно требует архитектурных швов – мест, где модуль взаимодействует с внешним миром через заменяемые контракты. Если ваша система построена так, что эти швы можно подменить, вы можете проверять поведение локально, быстро и с абсолютной уверенностью в детерминированности.

Событийное и временное тестирование: делаем невидимое управляемым

События, как мы уже знаем из предыдущих разделов, прекрасно разрывают прямые ссылки. Но для тестирования они создают особый вызов: публикация факта – это действие без немедленного возврата значения, а реакция на факт часто распределена по нескольким подписчикам. Как протестировать, что, получив урон, персонаж правильно переходит в состояние смерти? Секрет в том, чтобы тестировать обработчик события как обычный метод, подавая ему на вход подготовленный контекст. Если реакцией на событие является вызов метода `Die()` на игроке, а тот, в свою очередь, публикует факт `PlayerDied`, вы проверяете это, подписываясь на это событие из теста. Ваш тест становится временным подписчиком, который ловит факт и проверяет, что он действительно был отправлен. То же самое касается времени и случайности. Никогда не вызывайте `Time.GetTicksMsec()` или `new Random()` напрямую внутри игровой логики. Вместо этого определите контракт `IGameTime` с методом `GetCurrentTime()` и интерфейс `IRandomService` с методом `Next()`. В боевом коде вы используете реальную реализацию, которая читает системное время. В тесте вы подсовываете заглушку, которая возвращает строго заданные значения: «сейчас 1000 мс», «сейчас 1100 мс», «следующее случайное число – 0.5». Тем самым вы получаете полную власть над этими скрытыми зависимостями и превращаете любой вероятностный процесс в чистую, воспроизводимую математику.

Практические границы тестирования и долгосрочная выгода

Очень важно в погоне за изоляцией не впасть в крайность и не пытаться протестировать абсолютно всё. Физический контакт, рендеринг, окончание анимации – это аспекты, которые по своей природе требуют интеграционного тестирования или визуальной проверки. Их бессмысленно покрывать юнит-тестами. Ваша архитектурная задача – сделать интеграционный слой как можно тоньше, а доменную логику – как можно толще. Если 90 % геймплейных правил живут в чистом C#-коде и только 10 % занимаются трансляцией фактов в движок, вы колоссально выигрываете в стабильности. В долгосрочной перспективе эта практика окупает себя не тем, что вы «находите баги», а тем, что вы перестаёте бояться рефакторинга. Имея набор изолированных тестов для логики прокачки, правил боя и инвентаря, вы можете в любой момент переписать внутренности системы сохранений или полностью заменить визуальное оформление. Вы запускаете тесты, и если они проходят, центральные механики игры гарантированно не сломаны. Это даёт ту свободу манёвра, которая отличает профессиональный долгоживущий проект от прототипа, который страшно трогать. И эта свобода, как мы увидели на протяжении всей книги, начинается не с первой строчки тестового кода, а с первого архитектурного решения, которое вы принимаете.

UI как отдельный слой

9.1. Архитектура UI как слоя представления

Интерфейс часто становится зоной быстрых побед в прототипе. Нужно, чтобы полоска здоровья обновлялась, – мы добавляем прямой вызов `healthBar.Value = player.Health` внутрь метода `TakeDamage`. Нужно, чтобы кнопка «Лечение» работала, – мы пишем в её колбэке `player.Health += 10` и заодно обновляем текст. На уровне прототипа это даёт мгновенный видимый результат, но закладывает мину замедленного действия под всю архитектуру. UI начинает владеть доменной логикой: правила лечения, лимиты, формат сохранений и даже управление сценами разносятся по десяткам кнопок и панелей. Изменение любого правила требует правок интерфейса, а любая переработка интерфейса рискует сломать геймплей.

Задача этого раздела – утвердить фундаментальный принцип: UI является слоем представления, потребителем состояния, а не его владельцем. Мы разберём, как выстроить однонаправленный поток данных, разделить состояние интерфейса и состояние игры и как превратить пользовательские нажатия в безопасные намерения, не давая кнопкам принимать решения за игровую механику.

Однонаправленный поток данных и два вида состояния

Главная архитектурная ошибка при работе с UI – позволить данным течь в обе стороны. Кнопка меняет здоровье, а полоска здоровья напрямую читает переменную из того же класса – это хаотичный граф зависимостей. В правильно спроектированной системе поток данных всегда однонаправлен и проходит через контролируемые точки. Событие происходит в мире: игрок нажал кнопку. В этот момент не меняется ни одна переменная домена. Вместо этого UI формирует **намерение** – вызывает метод `_session.RequestHeal(10)`. Доменная логика проверяет условия, корректирует состояние

модели и публикует факт: `HealthChanged(80, 100)`. Только услышав этот факт, UI-панель обновляет свою полосу. События идут от UI к домену, данные для отображения – от домена к UI, и эти потоки никогда не пересекаются. Такая дисциплина требует чёткого разделения двух видов истины, существующих в игре. **Игровое состояние** – это фундаментальная правда о мире: здоровье персонажа, количество монет, содержимое инвентаря. Оно живёт в доменной модели, защищено инвариантами и не зависит от того, открыт ли у игрока экран инвентаря. **UI-состояние** – это преходящая правда о процессе взаимодействия: какая вкладка сейчас активна, развёрнуто ли описание предмета, какие поля были заполнены в форме, но ещё не отправлены на сервер. Смешивать эти две истины – грубейшая архитектурная ошибка. Практика показывает, что когда `GameState` начинает хранить, например, индекс выбранной вкладки, доменная модель обрастает деталями представления и становится хрупкой. И наоборот, если уровень здоровья игрока хранится в компоненте `Label`, то любая другая система (звук, аналитика) вынуждена добывать эту информацию из UI, а не из модели. Решением будет передавать UI не весь доменный объект с кучей методов и прав на изменение, а тонкую **read model**: специальную структуру или интерфейс, содержащий только данные для отображения. Например, вместо того чтобы давать кнопке ссылку на весь `Player`, мы предоставляем интерфейс `IPlayerReadModel`, который содержит только `CurrentHealth` и `MaxHealth`. Это делает зависимость узкой и не позволяет UI случайно залезть во внутренности системы.

Контроллеры, намерения и обратная связь

Если UI не должен содержать логику, где же ей жить и как она связывается с интерфейсом? На помощь приходит паттерн-посредник, который в разных методологиях называют `Controller` или `Presenter`. Мы будем называть его контроллером экрана. Его задача – быть тонким интеллектуальным слоем между тупыми визуальными узлами и системой. Представьте сцену покупки зелий в магазине. Кнопка «Купить», реализованная как чистый `View`, не должна знать о цене зелья, о количестве монет у игрока или о том, что делать после покупки. Её единственная задача – отловить нажатие и дёрнуть метод `_screenController.OnBuyButtonPressed()`. Контроллер, в свою очередь, знает, что делать. Он получает вызов, обращается к системе через контракт `IShopService.RequestPurchase(itemId)`, и на этом его ответственность заканчивается. Он не знает, как именно сохранятся данные и спишутся монеты. Он лишь инициирует намерение.

Теперь посмотрим, как контроллер справляется с результатами операции. Результат не должен домывливаться на стороне UI. Нельзя, чтобы после нажатия кнопки UI сам вычитал у себя монеты, надеясь, что операция пройдёт успешно. Вместо этого мы используем одну из двух форм обратной связи. Первая форма – реакция на изменение доменного состояния. После успешной покупки модель игрока публикует факт `CoinsChanged`. Контроллер экрана, подписанный на этот факт, моментально обновляет `read model` интерфейса, и отображение монет меняется автоматически. Вторая форма – явный ответ

на действие. Если денег не хватает, сервис покупки может вернуть `PurchaseResult.Denied("Недостаточно средств")`. Контроллер получает этот результат и активирует локальный UI-эффект: выводит анимированную подсказку. Важно, что в обоих случаях источником истины остаётся система. UI лишь реагирует на свершившиеся факты.

Композиция и изоляция: как UI получает свои зависимости

Остаётся решить последний архитектурный вопрос: как контроллер экрана получает свои зависимости, такие как `IShopService`? Если мы позволим ему искать их через `GetNode` или глобальный синглтон, мы снова создадим скрытую связанность. Самый надёжный способ – внедрение зависимостей через композицию в точке сборки. Корневой узел сцены, будь то `GameRoot` или `LevelManager`, при создании экрана магазина извлекает из сервис-локатора или создаёт сам `IShopService` и передаёт его в конструктор контроллера или метод `Initialize`.

```
// Точка сборки, например, в Game.cs.
public partial class GameRoot : Node
{
    public override void _Ready()
    {
        // Создаём доменный сервис.
        IShopService shopService = new ShopService(playerInventory, priceList);
        // Находим UI-панель на сцене.
        var shopPanel = GetNode<ShopPanel>("UI/ShopPanel");
        // Передаём зависимость.
        shopPanel.Initialize(shopService);
    }
}
```

Это гарантирует, что UI-панель полностью независима от глобального контекста. Её можно протестировать, подсунув заглушку `IShopService`, и она будет работать, не зная ничего об остальной игре. Такой подход замыкает круг. Мы приходим к архитектуре, где UI реализован как набор глупых визуальных узлов, управляемых тонкими контроллерами. Контроллеры, в свою очередь, зависят от абстракций и ничего не знают о мире за их пределами. Абстракции реализованы доменными сервисами, которые оперируют правилами и публикуют факты. Это делает систему невероятно устойчивой к изменениям, позволяя вам перерисовывать интерфейс как угодно, не боясь сломать игровую механику.

9.2. Привязка данных в UI: от ручного хаоса к реактивной синхронизации

Когда интерфейс игры состоит из пары индикаторов, обновлять их напрямую кажется естественным. Персонаж получил урон – мы тут же, в том же методе

TakeDamage, находим нужный Label и пишем в его Text новое значение здоровья. Уровень поднялся – там же меняем иконку и полосу опыта. Пока таких мест два или три, код остаётся понятным. Но стоит игре обрасти десятком экранов, модальными окнами и несколькими источниками изменения одного и того же показателя, как ручное обновление превращается в кошмар. Один и тот же параметр, например количество монет, начинает меняться в пяти разных скриптах: после боя, при покупке, при продаже, при получении награды за квест, при загрузке сохранения. И в каждом из этих мест нужно не забыть обновить все связанные UI-элементы. Забыл – интерфейс начинает врать.

Привязка данных (data binding) – архитектурный ответ на этот хаос. Суть идеи проста: интерфейс не должен бегать за данными и подстраиваться под каждое изменение мира. Вместо этого он один раз «привязывается» к источнику истины и в дальнейшем получает уведомления о любых изменениях. Мир сказал «монет стало 250», а все заинтересованные панели сами обновили своё отображение. Такой подход не только устраняет дублирование, но и закрепляет единственно верное направление потока данных – от модели к представлению.

Однонаправленный поток и наблюдаемые значения

В основе любой правильной привязки лежит принцип, который мы уже не раз применяли: однонаправленный поток данных. Доменная модель или её проекция, read model, является единственным владельцем истины. Она не знает о существовании интерфейса. Вместо этого она предоставляет возможность подписаться на свои изменения. Когда в модели что-то меняется, она просто публикует факт: «Я изменился». Все подписчики, в том числе UI-панели, реагируют на этот факт и забирают из модели актуальные данные. Противоположный подход, когда UI напрямую записывает данные в модель, а модель пытается обновить UI через те же или другие каналы, создаёт двустороннюю связанность, которую чрезвычайно трудно отлаживать. В играх такая схема особенно опасна, потому что порядок обновлений между визуальным и доменным слоями может привести к циклам или к состояниям, где данные в модели уже новые, а на экране ещё старые, и наоборот. Практически однонаправленный поток реализуется через простые, написанные вручную наблюдаемые контейнеры. Вам не нужны для этого громоздкие библиотеки. Достаточно класса, который хранит значение и умеет оповещать подписчиков о его изменении. Например, можно создать универсальный контейнер Observable<T>, который внутри себя хранит значение и событие Changed.

```
public class Observable<T>
{
    private T _value;
    public T Value
    {
        get => _value;
        set
        {
```

```

        if (EqualityComparer<T>.Default.Equals(_value, value))
            return; // Ничего не изменилось – не дёргаем подписчиков.
        _value = value;
        Changed?.Invoke(_value);
    }
}
public event Action<T> Changed;
}

```

Теперь доменная модель, например экономика игры, может выставить количество монет как `Observable<int>`. Весь код, меняющий монеты, работает с `Observable<int>.Value`, не думая об интерфейсе, UI-панель один раз подписывается на `Changed` и живёт спокойно.

Синхронизация модели и интерфейса в Godot

Давайте проследим, как эта идея воплощается в реальном коде на сцене Godot. Представьте, что у нас есть доменный сервис `PlayerEconomy`, который владеет монетами игрока. Мы не хотим, чтобы он зависел от движка, поэтому он написан как чистый C#-класс и использует наш `Observable<int>`.

```

public class PlayerEconomy
{
    public Observable<int> Coins { get; } = new Observable<int>();
    public void AddCoins(int amount)
    {
        // Здесь может быть сложная логика, модификаторы, проверки.
        Coins.Value += amount;
    }
}

```

Теперь на сцене, в главном узле `GameRoot`, создаётся экземпляр `PlayerEconomy`. Здесь же, в `_Ready`, мы находим нашу UI-панель, которая показывает монеты, и связываем её с моделью. Сама панель ничего не ищет, она лишь получает ссылку на `PlayerEconomy` извне – через метод инициализации.

```

public partial class GameRoot : Node
{
    [Export] private CoinsPanel _coinsPanel;
    public override void _Ready()
    {
        var economy = new PlayerEconomy();
        _coinsPanel.Initialize(economy);
    }
}

```

Класс `CoinsPanel` теперь реализует реактивное поведение. При инициализации он подписывается на изменение монет и сразу выводит текущее значение. Когда значение меняется, он обновляет текст лейбла и, возможно, запускает анимацию.

```

using Godot;
public partial class CoinsPanel : Control
{
    [Export] private Label _amountLabel;
    private PlayerEconomy _economy;
    public void Initialize(PlayerEconomy economy)
    {
        _economy = economy;
        _economy.Coins.Changed += OnCoinsChanged;
        // Сразу показываем актуальное значение.
        UpdateLabel(_economy.Coins.Value);
    }
    private void OnCoinsChanged(int newAmount)
    {
        UpdateLabel(newAmount);
    }
    private void UpdateLabel(int amount)
    {
        _amountLabel.Text = $"Монеты: {amount}";
    }
    public override void _ExitTree()
    {
        if (_economy != null)
            _economy.Coins.Changed -= OnCoinsChanged;
    }
}

```

Обратите внимание на несколько важных деталей. Во-первых, панель подписывается на событие `Changed` и отписывается в `_ExitTree`. Это защищает от утечек памяти и гарантирует, что при удалении панели со сцены она не будет реагировать на новые изменения монет. Во-вторых, `CoinsPanel` абсолютно ничего не знает о том, как и где меняются монеты. Она реагирует только на факт изменения. Если завтра вы добавите монеты за уничтожение врага или за выполнение секретного задания, код интерфейса не изменится ни на строчку.

Гранулярность, пакетирование и борьба с «бурей обновлений»

Используя наблюдаемые значения, легко попасть в ловушку чрезмерных уведомлений. Представьте, что боевая система на каждый удар мечом вызывает десятки микрособытий: изменилось здоровье, изменился опыт, изменился счётчик комбо. Если каждый такой чих будет дёргать обновление UI, интерфейс начнёт тратить впустую ресурсы на перерисовку и форматирование строк, особенно если обновления приходят часто, но на экране визуально ничего не меняется.

Первый рубеж защиты встроен в наш `Observable<T>`. Мы не вызываем событие, если новое значение совпадает со старым. Это само по себе отсекает множество холостых обновлений. Второй рубеж – проектирование модели на правильном уровне абстракции. Не стоит выставлять наружу сы-

рые, часто меняющиеся поля. Вместо этого можно предоставить UI агрегированную `read model`, которая обновляется реже. Например, вместо отдельного `Observable<int>` для каждой валюты и ресурса можно иметь один `Observable<PlayerStatus>`, где `PlayerStatus` – структура, обновляемая целиком по завершении боя или при открытии инвентаря. Третий, более продвинутый приём – пакетирование обновлений. Если в одном кадре происходит несколько изменений, нет нужды перерисовывать UI после каждого из них. Можно накапливать флаг «требуется обновление» и применить его единожды в конце кадра или в специальном методе. Однако для большинства интерфейсов простого гибрида из `Observable<T>` и небольшого количества `read model` более чем достаточно. Главное – понимать, что цена подписки и реакции на событие должна быть осознанной. Наконец, важно различать однонаправленную и двустороннюю привязку. В играх, где UI – слой представления, двусторонняя привязка почти всегда ведёт к скрытой связанности. Она позволяет пользовательскому вводу напрямую менять доменное состояние, минуя валидацию и бизнес-правила. Если вам нужно дать игроку менять настройки, UI должен не писать напрямую в `Observable<int> Volume`, а вызывать команду `_settings.ApplyVolume(новое_значение)`. Система внутри себя провалидирует значение, применит его и, изменив `Observable<int>`, оповестит интерфейс о новом состоянии. Такой круг всегда держит поток данных под контролем, а инварианты – защищёнными.

Привязка данных, реализованная даже в такой простой форме, кардинально меняет характер разработки интерфейсов. Она устраняет класс ошибок, связанных с рассинхронизацией, и делает UI-код предсказуемым. Вы начинаете думать не «где мне обновить лейбл», а «какую модель я должен дать этому экрану, чтобы он был самодостаточным». А это, в свою очередь, ведёт к архитектуре, где интерфейс и игровая механика развиваются независимо, не рискуя сломать друг друга при каждой итерации.

9.3. Удержание UI в роли слоя представления

Когда интерфейс игры состоит из пары кнопок и одного индикатора, соблазн добавить туда немного логики практически непреодолим. Нужно, чтобы при нажатии на зелье лечения персонаж восстанавливал здоровье – почему бы не вычесть предмет из инвентаря и прибавить двадцать очков прямо в обработчике нажатия? Это быстро, код локален, и всё работает. Проблема приходит позже, когда таких «умных» кнопок становится двадцать. Правила потребления предметов оказываются размазаны по всем экранам, а изменение одного условия (например, лечение нельзя применять в бою) требует прочёсывания всего UI-кода. Интерфейс, который должен быть просто витриной состояния, становится неявным контроллером игры. Задача этого раздела – показать, как выстраивать дисциплину, при которой UI только передаёт намерения, а вся содержательная логика остаётся за его пределами.

Кнопка, меняющая состояние игры напрямую, похожа на пациента, который сам выписывает себе рецепт. Увидел, что болит горло, – взял с полки антибиотик и принял. В маленькой деревне, где врач далеко, это может сработать, но в современной медицине такой подход приведёт к катастрофе. Врач должен осмотреть пациента, поставить диагноз и только потом выписать лекарство. Ровно так же UI должен лишь сообщать о симптомах: «пользователь хочет использовать предмет». Диагноз и лечение – зона ответственности доменной логики. Она проверяет, можно ли сейчас лечиться, не заблокировано ли действие, применяет эффект и публикует новое состояние. Кнопка получает это новое состояние и просто перерисовывается.

Паттерны делегирования: намерения, команды и чтение

На практике такое разделение реализуется через простой набор архитектурных соглашений. UI никогда не меняет доменное состояние напрямую. Вместо этого он вызывает асинхронные по духу, но синхронные по реализации методы-команды. `_session.UseItem("Potion"), _shopService.RequestPurchase("Sword"), _game.RestartLevel()`. Это и есть **намерения** (intent). Намерение – это не просто факт клика, а осмысленное желание игрока, выраженное в терминах предметной области. Оно ничего не говорит о том, можно ли это сделать, и не предвосхищает результат. Клик – это механическое действие, а намерение – доменное. Именно поэтому UI должен оперировать намерениями, а не сырыми событиями ввода. Сама кнопка становится тонким адаптером между движковым событием и вызовом метода на контракте. Она переводит `Pressed` в `_controller.OnHealButtonPressed()`, и на этом её ответственность заканчивается. Контроллер, в свою очередь, знает, какому сервису что передать.

```
public partial class HealButton : Button
{
    private IGameCommands _commands;
    public void Initialize(IGameCommands commands)
    {
        _commands = commands;
        Pressed += OnPressed;
    }
    private void OnPressed()
    {
        _commands.RequestHeal(10);
    }
}
```

Код сервиса, реализующего `IGameCommands`, может выглядеть так:

```
public class GameSession : IGameCommands
{
    private readonly HealthComponent _playerHealth;
    public GameSession(HealthComponent playerHealth)
    {
```

```

        _playerHealth = playerHealth;
    }
    public void RequestHeal(int amount)
    {
        // Доменное правило: лечить можно только живого.
        if (_playerHealth.Current <= 0)
            return;
        _playerHealth.Heal(amount);
    }
}

```

Таким образом, UI вызывает команду, а домен решает, что делать. Важно, что UI не проверяет здоровье и не модифицирует его, он только инициирует намерение.

Одновременно с этим UI должен уметь читать и наблюдать состояние. Для этого используется отдельный набор контрактов `read model`, предназначенный только для отображения. Через него интерфейс получает текущее здоровье, количество монет или флаг доступности действия. Это может быть структура вроде `HudReadModel`, которая предоставляет свойства, подписываться на изменения которых можно через события.

```

public class HudReadModel
{
    public int CurrentHealth { get; private set; }
    public int MaxHealth { get; private set; }
    public event Action Changed;
    public void UpdateFrom(HealthComponent health)
    {
        CurrentHealth = health.Current;
        MaxHealth = health.Max;
        Changed?.Invoke();
    }
}

```

Панель здоровья подписывается на `Changed` и обновляет свой `Label`. Такой подход не только разделяет чтение и запись, но и дисциплинирует разработчика. Вы физически не можете изменить состояние через `HudReadModel`, потому что у него нет публичных сеттеров, кроме внутреннего метода обновления. Это и есть реализация однонаправленного потока данных, закреплённая на уровне типов.

Антипаттерны и путь рефакторинга

Проекты редко пишутся сразу с чистым разделением. Чаще всего логика прорастает в UI постепенно, через невинные, на первый взгляд, решения. Первое из них – прямые ссылки на доменные объекты. Когда кнопка получает `[Export] private Player _player`, у соблазна сделать `_player.Health += 10` практически нет шансов. Второе – использование глобального доступа

вроде `GetNode("/root/Game/Player")` или статических синглтонов. Зависимости становятся невидимыми, и код превращается в лапшу. Третье, самое коварное – дублирование правил доступности. Если кнопка блокируется проверкой `if (_player.Coins < price)`, то при добавлении скидок или временной блокировки вам придётся искать все экраны с такой проверкой и править их. К счастью, от такого кода можно уйти без тотального переписывания всего интерфейса. Рефакторинг делается пошагово, экран за экраном. Первый шаг – выделить намерения. Просмотреть все обработчики кнопок и заменить прямые манипуляции с состоянием на вызовы методов вроде `_commands.RequestUseItem(...)`. Саму реализацию команды пока можно оставить в том же классе игрока, но главное – убрать прямую запись из UI. Второй шаг – инкапсулировать проверки. Условия вроде «нельзя пить зелье в воздухе» переносятся из `if` в кнопку внутрь тела команды `RequestUseItem`. Третий шаг – перевести отображение на `read model`. Вместо того чтобы внутри команды обновлять конкретный `Label`, команда меняет модель здоровья, а та, в свою очередь, уведомляет UI. Результатом такого рефакторинга становится код, где интерфейс полностью освобождается от знаний о правилах игры. Даже обработка отказов становится делом доменной логики. Если сервис покупок не может продать предмет, он не выбрасывает исключение, а возвращает объект результата либо публикует факт `PurchaseFailed`. UI реагирует на этот факт, показывая подсказку, но не принимает решения.

Композиция, границы и эвристики для самопроверки

Последний архитектурный вопрос, который неизбежно встаёт при таком подходе: где все эти `IGameCommands` и `HudReadModel` создаются и как они попадают в кнопки? Если каждая кнопка будет самостоятельно добывать себе сервисы через поиск по дереву сцены, мы просто заменим одну форму связанности на другую. Решение – явная композиция в точке сборки. Узел, ответственный за весь экран, получает от корня сцены все необходимые зависимости и передаёт их дочерним UI-панелям.

```
public partial class GameHud : Control
{
    [Export] private HealButton _healButton;
    [Export] private HealthBar _healthBar;
    public void Initialize(IGameCommands commands, HudReadModel readModel)
    {
        _healButton.Initialize(commands);
        _healthBar.Initialize(readModel);
    }
}
```

Такая проводка может показаться излишней в небольшом проекте, но она моментально окупается при любой переделке интерфейса. Вы всегда знаете,

от чего зависит конкретная панель, и можете заменить реализацию контракта, не трогая вёрстку.

Чтобы такая архитектура стала привычкой, полезно держать в голове несколько простых критериев для самопроверки. Если вы открываете UI-скрипт и видите там поля доменных сущностей, условные операторы, проверяющие геймплейные условия, или вызовы, меняющие состояние игры, – граница нарушена. Правильный UI-скрипт либо преобразует ввод в вызов контракта, либо обновляет визуальное отображение на основе переданных ему данных. Он не содержит слов `if (player.Coins >= price)` или `player.Health += amount`.

10

Системы сохранения и загрузки

10.1. Сериализация как контракт данных

Невозможно написать систему сохранений, не ответив на фундаментальный вопрос: что именно считать прогрессом игрока и как этот прогресс выглядит за пределами запущенной игры? Сериализация, преобразование состояния в последовательность байтов или читаемый текст, кажется чисто технической задачей. В прототипе так и есть: достаточно взять объект игрока, вызвать `JsonSerializer.Serialize` и записать результат в файл. Но как только проект начинает развиваться, такой подход рассыпается, потому что путает две совершенно разные вещи – текущее состояние программы в оперативной памяти и долгосрочный контракт данных.

Любое сохранение пересекает границу времени. Вы записываете его сегодня, а прочитаете через неделю, месяц или год, когда код уже изменился. Десятки классов переименованы, поля добавлены или удалены, балансные константы подправлены. Если формат файла привязан к внутренностям текущей версии кода, каждое такое изменение становится критическим багом. Именно поэтому архитектурный подход к сериализации начинается не с выбора библиотеки, а с признания простой истины: модель хранения и модель исполнения – разные сущности, живущие по разным законам.

Почему прямой «дамп объекта» разрушает проект

Представьте игровой класс `Player`, который вы только что написали. В нём десяток полей: здоровье, монеты, ссылка на инвентарь, ссылка на текущий уровень, пара кешированных значений для ускорения расчётов, возможно,

флаг «находится ли в анимации получения урона». Вы решаете сохранять его целиком, потому что такой путь кажется самым коротким к работающей функции. Первый подводный камень – граф объектов. `Player` ссылается на `Inventory`, который содержит список `Item`, а каждый `Item` может ссылаться на своего владельца. Сериализатор, пытаясь обойти этот граф, либо упадёт с ошибкой циклической зависимости, либо сохранит всю эту паутину, включая служебные объекты движка, которым не место на диске. Даже если граф удалось сериализовать, настоящая беда приходит при загрузке. Восстановление ссылок – отдельная сложнейшая задача. Но главная архитектурная катастрофа случится позже, когда вы в очередной раз решите переименовать поле `Health` в `CurrentHealth` или поменять тип коллекции для инвентаря. Файлы сохранений, записанные старой версией, перестанут читаться. Внутренняя, изменчивая структура вашего кода стала публичным API, который нельзя ломать. Это не просто неудобство, а полная потеря контроля над эволюцией программы. Вторая проблема дампа – утечка временного и вычислимого состояния. Если в момент сохранения игрок получал урон и флаг `isStaggered` был `true`, такое значение запишется в файл. При последующей загрузке персонаж окажется оглушённым на ровном месте. То же касается кешей, промежуточных результатов физического движка, ссылок на визуальные эффекты. Всё это не является частью прогресса. Прогресс – то, что должно пережить перезапуск игры: количество монет, пройденные уровни, полученные предметы. Всё остальное следует безжалостно отбросить и пересчитать при загрузке.

Решение проблемы начинается с чёткого разделения. В коде должны существовать две параллельные структуры. Первая – ваши обычные доменные классы, работающие в рантайме. Они могут быть сложными, содержать ссылки, методы и агрегировать множество других объектов. Вторая – простые, как гвоздь, объекты передачи данных, DTO. Они не содержат поведения, не ссылаются на игровые сущности и существуют только для того, чтобы быть записанными в файл и прочитанными обратно. Именно такой DTO и становится нашим контрактом данных.

Взгляните на пример того, как поступать не следует и как та же задача решается с правильным контрактом.

```
// Неустойчивый подход: сериализация внутренностей рантайм-класса.
using System.Text.Json;
public sealed class Player
{
    public int Coins;
    public Inventory Inventory = new();
    // Здесь же десяток других полей, методов и событий.
}
public static class NaiveSave
{
    public static string SavePlayer(Player player)
        => JsonSerializer.Serialize(player); // Формат файла теперь – слепок внутренностей
Player!
}
```

Любой рефакторинг `Player` сломает все существующие сохранения. Теперь посмотрите на альтернативу – выделенную модель хранения.

```
public sealed class PlayerSaveData
{
    public int Version = 1;
    public int Coins;
    public string[] InventoryItemIds = [];
}
```

`PlayerSaveData` – осмысленный набор фактов, определяющих прогресс, нам не нужно сохранять ссылки на сами предметы, достаточно их идентификаторов. Нам не нужны ссылки на врагов, загрузчик сам создаст их при реконструкции уровня. Такой DTO становится тем самым API между версиями. Добавление нового поля, например `PlayerLevel`, – просто расширение контракта. Старые файлы, где такого поля нет, будут загружены со значением по умолчанию, что позволяет сохранять обратную совместимость без героических усилий.

Идентичность, ссылки и умолчания

Следующий слой проблем, мгновенно решаемый явным DTO, – работа со связями. В мире рантайма персонаж владеет мечом через прямую ссылку. В мире сохранений такое отношение должно быть выражено иначе. Мы не можем записать адрес памяти, поэтому записываем идентификатор: строку, число, `guid`. Персонаж содержит список идентификаторов предметов в инвентаре. Пройденный квест – идентификатор квеста и его состояние. Восстановление этих связей при загрузке – отдельная, прекрасно автоматизируемая процедура. Вы создаёте заново нужные объекты, складываете их в реестр по идентификаторам и во втором проходе восстанавливаете связи. Не менее важно умение отличать данные, которые нужно сохранить, от тех, которые нужно выбросить. Здоровье и координаты – да. Визуальное состояние UI вроде «открыта ли панель инвентаря» – нет. Оно должно быть сброшено в значение по умолчанию при загрузке, иначе вы получите интерфейс-призрак, показывающий предметы из прошлой игровой сессии. Хороший DTO не содержит таких полей, а загрузчик явно инициализирует все системы, не попавшие в «снимок». Тот же подход естественно решает проблему валидации. Файл сохранения – всегда недоверенный ввод. Его могла создать другая версия игры, он мог быть повреждён при записи, игрок мог попытаться отредактировать его вручную. Если ваша логика загрузки состоит из вызова `JsonSerializer.Deserialize<Player>` и последующей кучи `if`-ов в `_Ready`, вы обречены на бесконечный поток трудноуловимых багов.

Безопасный код загрузки действует иначе. Он читает данные в промежуточный DTO, а затем прогоняет его через функцию-нормализатор. Если здоровье оказалось отрицательным, оно превращается в ноль. Если идентификатор предмета в инвентаре ссылается на несуществующий предмет, такой предмет молча удаляется. Если уровень персонажа выходит за максималь-

ный, он ограничивается. И только после того, как все поля прошли проверку, нормализованные данные переносятся в настоящие доменные объекты. Это превращает загрузку из гадания в надёжный, воспроизводимый процесс.

Отдельно стоит вопрос консистентности. Представьте, что сохранение происходит во время торговли, игрок нажал кнопку «Купить», монеты уже списаны из модели, а предмет ещё не добавлен в инвентарь. Если в этот момент сработает автосохранение, в файл попадёт противоречивое состояние: монет нет, предмет не получен. Такая ситуация называется «грязным снимком», и бороться с ней нужно на архитектурном уровне. Все операции изменения состояния должны быть атомарными по отношению к сохранению. Точка сохранения либо предшествует всей транзакции, либо завершает её, но никогда не вклинивается в середину. Убедитесь, что ваш код вызывает запись на диск в безопасной фазе кадра, когда все системы уже закончили свою работу, и не делает этого внутри обработчиков UI.

Стратегия «полного снимка», когда вы каждый раз сохраняете всё состояние целиком в один файл, остаётся самой простой и надёжной для подавляющего большинства проектов. Она гарантирует консистентность данных и простоту восстановления. Частичные сохранения, записывающие только изменившиеся объекты, дают выигрыш в скорости на огромных открытых мирах, но и цену берут немалую: нужно гарантировать порядок загрузки разрозненных файлов, синхронизировать их и уметь правильно восстанавливать состояние, если один из них повреждён. Для новичка и его первого коммерческого проекта полный снимок – прагматичный выбор.

Сериализация в конечном счёте – не про то, как записать байты на диск. Она про то, как взять на себя обязательство перед будущим. Создавая явный, минимальный и валидируемый контракт данных, вы даёте себе свободу рефакторить, расширять и менять игру, не оглядываясь на миллионы сохранений, сделанных пользователями. Вы превращаете страшную фразу «трогать сохранения нельзя» в спокойное «сохранения – просто ещё один модуль системы». А это и есть признак здоровой архитектуры.

10.2. Версионирование сохранений: эволюция данных без потери прогресса

Как только в игре появляется возможность сохраниться, возникает долгоживущая ответственность перед игроком. Код будет меняться – механики, структуры классов, имена полей, балансные константы. Но пользователь, прошедший половину игры и отложивший её на месяц, ожидает, что после обновления его прогресс останется нетронутым. Версионирование формата сохранения превращает хаотичную эволюцию данных в управляемый процесс. С версией файла вы в любой момент знаете, с какой схемой данных работаете и как привести её к актуальному виду, не роняя ни одного сохранения.

Версия как ключ к обратной совместимости

Обратная совместимость означает, что игра, получившая обновление, обязана прочитать файл, оставшийся от предыдущей версии, и восстановить из него корректное состояние. Для этого нужны три вещи: явный номер версии внутри самого файла, чёткая стратегия миграции и разумные значения по умолчанию для новых или отсутствующих полей. Номер версии – фундамент контракта, он должен лежать на верхнем уровне вашего `SaveData`, чтобы загрузчик мог прочитать его до разбора всех деталей. Частая ошибка новичков – подставить в качестве версии номер билда приложения. Билд может меняться каждый день, а схема данных остаётся прежней. Версия схемы должна инкрементироваться только тогда, когда изменяется смысл или набор сохраняемых полей. Если вы переименовали `Coins` в `GoldCoins` или заменили массив идентификаторов на список структур с количеством – повышайте версию. Если просто поправили формулу расчёта урона – схема данных не изменилась, и версию трогать не нужно. Представьте себе паспортный стол. Вы приходите менять паспорт, и чиновник видит документ старого образца. Он не отказывает вам в обслуживании, а знает инструкцию: как перенести данные из старой формы в новую, какие поля добавить, что пересчитать. Ваш `PlayerSaveData` выполняет ту же роль. Увидев `Version = 1`, загрузчик запускает инструкцию миграции, превращающую старый набор полей в актуальный.

На практике это означает, что доменная модель, вся ваша сложная система правил и состояний, не должна знать о существовании старых форматов. Она получает уже очищенные, мигрированные, нормализованные данные. Если вы пытаетесь поддерживать в методе `LoadGame` ветвление «если поле отсутствует – значит, это версия 1.2», вы растворяете проблему совместимости по всему коду, делая его невероятно хрупким. Миграция – отдельный, изолированный этап, выполняемый до того, как данные коснутся домена.

Цепочка миграций как конвейер

Если ваша игра прожила год и получила десяток патчей, попытка описать миграцию «из любой версии сразу в текущую» одним гигантским блоком кода станет кошмаром. Вы получите простыню из исторических условий, в которой невозможно разобраться. Гораздо устойчивее выстроить цепочку небольших, однонаправленных шагов: `v1 -> v2 -> v3 -> ...`. Каждый шаг соответствует одному изменению контракта и легко проверяется. Вы добавляете новое поле в DTO, повышаете версию до следующей и пишете мигратор, который умеет превращать DTO предыдущей версии в текущую. Загрузочный конвейер тогда выглядит как последовательность стадий. Сначала файл читается с диска, и мы получаем сырой DTO. Первым делом считывается версия. Если версия неизвестна или выше текущей, загрузка останавливается с внятным сообщением – нельзя ожидать, что игра поймёт формат из будущего. Если версия устарела, DTO прогоняется через цепочку миграций, пока не достигнет актуальной. Только после этого он проходит валидацию и нормализацию, и уже очищенные данные попадают в доменную модель.

Такой подход локализует ошибки. Если файл повреждён, вы узнаете об этом на стадии чтения, если миграция дала сбой – на стадии преобразования, если значение некорректно – на стадии валидации. Доменный код, в свою очередь, всегда работает с заведомо корректным снимком.

Давайте рассмотрим реальный пример такой цепочки. Предположим, в первой версии игры мы сохраняли монеты и список предметов. Во второй версии заказчик попросил добавить самоцветы как отдельный ресурс. Мы добавляем поле Gems в DTO, повышаем версию до 2 и пишем миграцию, которая для старых файлов проставляет Gems = 0. В третьей версии мы объединяем монеты и самоцветы в общий словарь ресурсов. Снова меняется DTO, версия становится 3, и пишется миграция v2 -> v3, которая раскладывает старые поля по новым ключам.

```
public sealed class SaveData
{
    public int Version = 3;
    public Dictionary<string, int> Resources = new();
    public List<string> Items = new();
}

public static class SaveMigrations
{
    public static SaveData Migrate(SaveData data)
    {
        // Проходим по цепочке, пока не достигнем актуальной версии.
        while (data.Version < 3)
        {
            data = data.Version switch
            {
                1 => MigrateV1ToV2(data),
                2 => MigrateV2ToV3(data),
                _ => throw new InvalidOperationException($"Unsupported save version: {data.
Version}")
            };
            return data;
        }
        private static SaveData MigrateV1ToV2(SaveData old)
        {
            // Версия 1 не имела Gems. Мы просто добавляем ноль.
            old.Gems = 0;
            old.Version = 2;
            return old;
        }
        private static SaveData MigrateV2ToV3(SaveData old)
        {
            // Версия 3 переходит на словарь ресурсов. Монеты и самоцветы переезжают.
            old.Resources["coins"] = old.Coins;
            old.Resources["gems"] = old.Gems;
            old.Version = 3;
            return old;
        }
    }
}
```

Такой код читается как история изменений. Разработчик, пришедший в проект через год, открывает SaveMigrations и видит хронологию: что, когда и зачем было добавлено. Ему не нужно восстанавливать контекст по форумам или git-blame. Всё документировано прямо в логике преобразований.

Отдельно стоит сказать о поведении загрузчика при встрече с версией из будущего. Предположим, игрок открыл сохранение в бета-версии игры, где формат был уже 5-й, а затем решил вернуться на стабильную ветку, понимающую только версию 3. Загрузчик обязан честно отказаться загружать такой файл, а не пытаться «угадать» или молча обнулить прогресс. Потеря данных – худшее, что может случиться с пользовательским опытом, и архитектура должна гарантировать, что вы контролируете эту ситуацию.

В конечном счёте версионирование сохранений – не техническая деталь, а социальный контракт с игроком. Вы обещаете, что его время, потраченное на прохождение, не исчезнет из-за того, что вы переименовали пару переменных. Построив конвейер миграций с первого же дня, вы получаете свободу рефакторить доменную логику как угодно, не оглядываясь на старые файлы. Чистая архитектура здесь снова доказывает свою прагматическую ценность: она защищает не только код, но и пользователя.

10.3. Безопасность сохранений: защита данных в недоверенной среде

Когда игрок нажимает «Сохранить», он доверяет вам не просто последовательность байтов на диске. Он передаёт под вашу ответственность часы, проведённые в игре. Архитектурный долг разработчика – гарантировать, что прогресс не исчезнет из-за сбоя питания, не будет разрушен криво написанным патчем и не превратится в инструмент для разрушения мультиплеерного баланса. Задача этого раздела – разложить безопасность сохранений на два принципиально разных мира: надёжность локального файла и авторитет в сетевой игре.

Первый мир – одиночные кампании. Здесь главный враг – не хитрый взломщик, а случайность: выключившийся компьютер, дрогнувшая рука, записавшая на диск половину данных, или сам игрок, из любопытства залезший в JSON и поправивший пару значений. Второй мир – мультиплеер, где файл на компьютере пользователя изначально считается скомпрометированным. Любые данные, влияющие на честность матча, не должны жить на клиенте. Смешивать эти два мира в одной архитектуре – верный путь к путанице и дырам.

Локальные сохранения: целостность, а не паранойя

Представьте, что игрок проходит сложный босс-файт, побеждает, и игра записывает прогресс. В момент, когда поток байтов льётся на диск, в доме

гаснет свет. Операционная система успевает записать только часть файла. Когда питание возвращается и игра пытается загрузить повреждённый сейв, наивный код вызовет `Deserialize`, получит исключение и вылетит с ошибкой. Прогресс потерян.

Первая линия обороны от таких катастроф – атомарная запись. Суть проста: никогда не пишите результат напрямую в целевой файл. Сначала формируйте полный массив байтов во временном файле, и только когда запись полностью завершена, заменяйте старый файл новым. Файловые системы гарантируют, что операция переименования атомарна. В `Godot` для этого удобно использовать `FileAccess` с последующим вызовом `DirAccess.RenameAbsolute`. Если свет погаснет во время записи, вы потеряете только временный файл, а основное сохранение останется нетронутым.

Даже если запись прошла успешно, файл могли повредить позже. Ошибка диска, случайный сбой синхронизации с облаком, да и просто человеческий фактор: игрок открыл сохранение в блокноте и исправил `"Coins": 100` на `"Coins": 99999`. Такое редактирование не злой умысел, а скорее безобидное баловство, но код обязан справляться с ним без краша.

Здесь в игру вступает контрольная сумма. Это число, вычисленное на основе всех байтов сохранения по специальному алгоритму. Вы записываете его в тот же файл. При загрузке вы снова вычисляете хеш и сравниваете с сохранённым. Если они не совпадают – данные изменились. Что делать в таком случае, решать вам. Можно показать сообщение «Файл повреждён» и предложить загрузить автоматическую резервную копию, о которой мы поговорим чуть позже. Можно провалидировать все поля и загрузить игру с теми данными, которые уцелели. Главное – не пытаться исполнить повреждённый файл как рабочий, иначе вы получите каскад ошибок внутри доменной логики.

Важно понимать, что хеш без криптографической подписи не защищает от намеренной подмены. Если игрок поправил монеты в `JSON`, он может пересчитать и хеш. Но архитектурная цель хеша – не остановить решительного читера, а дать вам, разработчику, точку контроля. Вы знаете, что файл изменялся, и можете включить режим «безопасной загрузки», который нормализует все значения до допустимых границ. Увидел `Coins = 99999` при максимуме в `9999`? Откатывай до `9999` молча, без падения. Такой подход позволяет игре выживать при любых входных данных.

Резервная копия работает в паре с контрольной суммой. Когда вы собираетесь перезаписать основной файл сохранения, сначала переименуйте старый в `save_backup.dat`. После успешной записи нового файла бэкап можно удалить или оставить на один цикл. Если при следующей загрузке вы обнаружите, что основной файл повреждён, а резервный валиден, вы просто загружаете резервный. Игрок теряет максимум несколько минут прогресса, а не всё прохождение. Именно так, через простые инженерные меры, достигается та самая надёжность, которую пользователь ожидает от игры по умолчанию.

Мультиплеер: серверный авторитет как единственный источник истины

Совершенно иная картина складывается в сетевой игре. Здесь клиентское приложение выполняется на машине пользователя, а значит, находится в полностью недоверенной среде. Игрок может отключить любую проверку, подменить любой файл, модифицировать сборку. Любое значение, которое ваш сервер принимает от клиента как истину, является потенциальной дырой. Единственная устойчивая архитектура для мультиплеера – серверный авторитет. Это означает, что все данные, влияющие на честность игры, живут на сервере. Клиент может хранить у себя что угодно: настройки графики, громкость звука, кеш текстур, даже локальную копию прогресса для быстрого отображения. Но при любой операции, затрагивающей валюту, инвентарь, результаты матча, клиент не меняет состояние сам. Он посылает команду-запрос: «Игрок хочет купить зелье». Сервер принимает этот запрос, проверяет, достаточно ли у игрока золота, не заблокирована ли покупка, списывает средства, добавляет предмет в инвентарь и только после этого сообщает клиенту о новом состоянии. Клиентская валидация (например, блокировка кнопки «Купить», если монет не хватает) служит лишь для мгновенного отклика интерфейса и не имеет никакого отношения к безопасности.

Представьте себе банковское приложение. Было бы безумием, если бы баланс вашего счёта хранился только в телефоне, а сервер просто принимал от клиента новый баланс. Игра – тот же банк, только в ней хранятся не деньги, а труд и достижения игроков. Любая попытка защитить такое приложение шифрованием локального файла бессмысленна, потому что ключ для расшифровки всё равно лежит где-то в клиентском коде. Настоящая граница безопасности проходит не по файловой системе, а по сетевому проводу. Всё, что оказывается на стороне клиента, следует считать публичным и потенциально скомпрометированным.

Поэтому архитектура сетевой игры чётко разделяет данные. Клиентские настройки, локальный кеш, черновики для восстановления UI хранятся локально. Всё, что касается прогресса, рейтинга, валюты, хранится и проверяется исключительно на сервере. Это не усложнение, а осознанная плата за честную игру.

Практические рецепты для локальной защиты

Вернёмся к одиночным играм и соберём компактный набор инструментов, который реализуется за пару часов и снимает большинство головной боли, связанной с сохранениями.

Первый рубеж – валидация данных перед импортом. Ваш загрузчик должен прогонять DTO через функцию-нормализатор, которая проверяет диапазоны, наличие обязательных полей и логическую непротиворечивость. Если здоровье оказалось отрицательным, оно становится нулём. Если иден-

тификатор предмета ведёт на несуществующий контент, предмет удаляется. Никаких исключений, только приведение к безопасному состоянию.

Второй рубеж – атомарная запись. Выгрузили SaveData в байты, записали их во временный файл, убедились, что запись прошла без ошибок, и затем переименовали временный файл в целевой. Это гарантирует, что на диске никогда не окажется полупустого сохранения.

Третий рубеж – контрольная сумма. Перед записью вы вычисляете хеш (например, CRC32 или SHA256) от сериализованных данных и сохраняете его рядом. При загрузке первым делом проверяете хеш. Не совпал, тогда откатываемся на бэкап или грузим дефолтное состояние с сообщением об ошибке. Вот как такая проверка может выглядеть в коде:

```
public class SaveFile
{
    public int Version { get; set; } = 1;
    public PlayerSaveData Data { get; set; }
    public uint Checksum { get; set; }
}

public static class SaveIntegrity
{
    public static uint ComputeChecksum(byte[] data)
    {
        // Упрощённо: используем встроенный алгоритм.
        uint hash = 0;
        foreach (byte b in data)
            hash = (hash << 5) + hash + b;
        return hash;
    }

    public static bool Validate(SaveFile file, byte[] rawData)
    {
        uint expected = ComputeChecksum(rawData);
        return file.Checksum == expected;
    }
}
```

Четвёртый рубеж, который стоит вводить сразу, – резервная копия. Перед перезаписью основного файла переименуйте существующий в save_old.dat. Если при следующей загрузке основной файл окажется битым, а старый пройдёт проверку, вы восстановите игру из бэкапа. Игрок, возможно, потеряет десять минут прогресса, но не пятьдесят часов.

10.4. Асинхронные системы сохранения

Представьте себе напряжённый бой с боссом. Игрок уворачивается от серии ударов, ловит окно для контратаки и наносит решающий урон. Именно в этот драматический момент игра решает, что пора выполнить автоматическое сохранение. Если операция записи на диск выполняется синхронно, в том же

потоке, что и основной игровой цикл, происходит неприятное: кадр зависает на полсекунды. Этого достаточно, чтобы персонаж игрока, не дождавшись команды уклонения, пропустил фатальный удар. Игра не зависла навсегда, но мгновенный, необъяснимый фриз разрушил всё удовольствие от честной победы.

Асинхронное сохранение решает эту проблему, отделяя подготовку данных и запись на диск от критически важного цикла обновления игры. Цель архитектурного решения – гарантия того, что игровой процесс останется плавным и отзывчивым, вне зависимости от капризов файловой системы. Мы больше не заставляем главный поток ждать, пока крутится диск или заполняются буферы. Вместо этого мы организуем весь процесс как контролируемый конвейер, где каждый этап точно знает своё место.

Первая стадия этого конвейера – **захват снимка состояния** (Capture). Он всегда выполняется синхронно в основном потоке, и на то есть веская причина. Чтобы снимок был консистентным, мы должны «сфотографировать» игровой мир в конкретный, заведомо безопасный момент. Обычно это происходит в фазе `_PhysicsProcess` после завершения всех обновлений, когда все системы находятся в устоявшемся состоянии. Код, собирающий `SaveData`, читает текущие значения здоровья, координат, инвентаря и формирует DTO. На этом этапе ни в коем случае нельзя вставлять `await` или уходить в фоновый поток – мир продолжает жить, и через долю секунды данные перестанут быть согласованными.

Как только снимок собран, начинается вторая стадия – **кодирование и запись** (Encode and Write). Вот здесь асинхронность раскрывает свой потенциал. Мы берём наш чистый, не зависящий от Godot объект `PlayerSaveData`, сериализуем его в массив байтов и записываем на диск, не блокируя главный поток. В C# это достигается с помощью `Task.Run` для операций, которые могут быть потенциально длительными.

Третья стадия – **фиксация результата**. После завершения фоновой операции мы возвращаемся в основной поток, чтобы обновить UI, показать иконку успешного сохранения или, в случае ошибки, сообщить игроку о проблеме.

Крайне важно, чтобы через границу `await` переходил только подготовленный снимок данных, а не живые объекты. Если вы сохраните в замыкании ссылку на `CharacterBody2D`, а затем попытаетесь прочитать `Velocity` после завершения `Task.Delay`, ваш код упадёт, потому что узел может быть уже удалён из дерева сцены. Золотое правило асинхронной архитектуры в Godot: через `await` переносятся только чистые данные, DTO. Никаких ссылок на узлы, ресурсы или сервисы движка.

Давайте посмотрим, как этот конвейер выглядит в реальном коде. Сервис `SaveManager` принимает запрос на сохранение, захватывает состояние, а затем запускает фоновую операцию.

```
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;
using Godot;
```



```

public class SaveManager
{
    private const string SavePath = "user://save_game.dat";
    private const string TempPath = "user://save_game.tmp";
    private const string BackupPath = "user://save_game.bak";
    public async Task SaveGameAsync(PlayerSaveData saveData)
    {
        // 1. Захват снимка уже выполнен, данные готовы.
        // 2. Кодирование и запись выполняем в фоновом потоке.
        await Task.Run(async () =>
        {
            // Сериализуем данные в JSON
            var options = new JsonSerializerOptions { WriteIndented = false };
            byte[] bytes = JsonSerializer.SerializeToUtf8Bytes(saveData, options);
            // Атомарная запись на диск.
            await File.WriteAllBytesAsync(TempPath, bytes);
            // Сохраняем резервную копию и заменяем основной файл.
            if (File.Exists(SavePath))
            {
                File.Copy(SavePath, BackupPath, overwrite: true);
            }
            File.Move(TempPath, SavePath, overwrite: true);
        });
        // 3. Возвращаемся в основной поток (через сигнал или колбэк).
        GD.Print("Игра успешно сохранена.");
    }
}

```

Обратите внимание на порядок операций внутри `Task.Run`. Сначала данные пишутся во временный файл `save_game.tmp`. Затем, если существует предыдущее сохранение, оно копируется в бэкап `save_game.bak`. Только после этого временный файл переименовывается в `save_game.dat`. Это гарантирует, что даже если питание пропадёт в момент записи, на диске останется либо старый валидный файл, либо его резервная копия. Полузаписанного файла не будет никогда.

Ещё одна проблема, которую приходится решать в мире асинхронного I/O, – хаос из нескольких одновременных запросов. Представьте, что игрок нажал «Сохраниться» и сразу же выходит в главное меню. Обработчик меню тоже вызывает сохранение. Запускаются две параллельные задачи, и та, что стартовала позже, может завершиться первой, после чего старая задача пере-запишет её результат. Чтобы такого не происходило, вводится **коалесцент-ный центр** (coalescing), гарантирующий, что в каждый момент времени выполняется только одна операция записи, а поступившие новые запросы либо игнорируются, либо встают в очередь.

```

public class CoalescingSaveManager
{
    private readonly SaveManager _saveManager;
    private Task _pendingSave = Task.CompletedTask;
    private int _latestVersion = 0;

```



```

private int _completedVersion = 0;
public CoalescingSaveManager(SaveManager saveManager) => _saveManager = saveManager;
public async Task RequestSaveAsync(PlayerSaveData saveData)
{
    int currentVersion = Interlocked.Increment(ref _latestVersion);
    await _pendingSave;
    if (currentVersion <= _completedVersion) return;
    _pendingSave = _saveManager.SaveGameAsync(saveData);
    await _pendingSave;
    _completedVersion = currentVersion;
}
}

```

Такой подход с использованием монотонно растущего счётчика гарантирует, что при поступлении нескольких запросов реально выполнится только самый свежий. Все промежуточные снимки будут корректно пропущены.

Асинхронное сохранение перестаёт быть магией, от которой замирает игра, и становится просто ещё одним хорошо спроектированным модулем. Снимок собирается быстро в контролируемой фазе, тяжёлая работа с диском выносится за пределы основного потока, а атомарная запись защищает данные от потери. Самое же главное архитектурное правило (не переносить через `await` движковые объекты) защищает код от хитрых багов, связанных с гонками и временем жизни. Игрок же видит стабильный фреймрейт и наслаждается игрой, даже когда вы в фоне заботитесь о сохранности его прогресса.

11

Производительность и оптимизация

11.1. Профилирование как инженерная дисциплина

Игры отличаются от большинства прикладных программ одним фундаментальным требованием: они обязаны укладываться в жёсткий временной бюджет. Пока офисное приложение может позволить себе секундную паузу на открытие файла, игра такой паузы позволить не может. Каждые 16 миллисекунд, а на мобильных устройствах и того чаще, процессор и видеокарта должны успеть просчитать физику, обновить поведение всех сущностей, подготовить кадр и вывести его на экран. Стоит одному из этих этапов задержаться, и игрок чувствует предательский рывок, разрушающий погружение. Когда разработчик впервые сталкивается с просадками производительности, его рука инстинктивно тянется к самому «тяжёлому» на вид участку кода. Начинается охота на ведьм: кешируются всевозможные переменные, циклы переписываются в более быстрые конструкции, а вызовы методов заменяются на inline-логику. Подобная деятельность редко приносит плоды, зато щедро удобряет кодовую базу техническим долгом. Профилирование заменяет эту суету холодным, научным методом. Оно позволяет не гадать, а знать: какой именно кусок системы тратит драгоценные миллисекунды и почему он это делает.

Бюджет кадра и природа «тормозов»

Точка отсчёта в профилировании – конкретное время кадра. Если игра работает при стабильных 60 кадрах в секунду, бюджет одного кадра составляет

примерно 16.6 миллисекунды. Все ваши скрипты, физика, рендеринг, звук и даже сборка мусора должны уместиться в этот интервал. Средний показатель здесь плохой помощник. Игра, выдающая 60 FPS, но переживающая микрофриз на 50 миллисекунд раз в десять секунд, будет ощущаться как дёрганая и сломанная. Профилер в первую очередь нужен не для подсчёта среднего, а для поиска двух совершенно разных проблем: устойчиво высокой нагрузки и нерегулярных, но разрушительных спайков.

Устойчивая нагрузка лечится снижением постоянной работы. Возможно, вы обновляете AI сотни врагов каждый кадр, даже когда они находятся за тремя экранами от игрока. Спайк, напротив, почти всегда сигнализирует о разовой катастрофе. Сборщик мусора внезапно решил подчистить десять тысяч строк, созданных при форматировании текста в UI. Или система частиц в момент взрыва породила лавину новых объектов, и выделение памяти под них ударило по плавности кадра. Первый же запуск профайлера должен расставить точки над «i»: где именно заканчивается бюджет. Процессор не успевает обсчитать логику или видеокарта захлёбывается в тяжёлых шейдерах и постэффектах? Симптомы этих двух типов перегрузки различаются кардинально. Если при отключении всех визуальных украшений фреймрейт не растёт, а при уменьшении количества врагов на сцене – растёт, проблема сидит в логике. Если же пустой серый уровень летает, а красивые частицы обрушивают производительность, узким горлышком является рендеринг. Профилер не даст ответа, пока вы не поставите корректный вопрос, разделив эти две вселенные.

Поиск горячих точек и снятие гипотез

Встроенный профайлер Godot даёт карту кадра. Он раскладывает заветные 16 миллисекунд по полочкам: столько-то ушло на физику, столько-то на скрипты, столько-то на отрисовку. Увидев, что скрипты съедают 12 миллисекунд, вы выдвигаете первую гипотезу. Допустим, вы предполагаете, что корень зла – в обновлении системы патрулирования врагов. Браться сразу за переписывание кода было бы ошибкой. Вместо этого вы делаете целевой замер, который подтвердит или опровергнет догадку.

Самый простой способ – обернуть подозрительный участок ручным таймером и сравнить его вклад в общий бюджет кадра.

```
using System.Diagnostics;
public static class ProfilerUtil
{
    public static long MeasureMicros(System.Action action)
    {
        var sw = Stopwatch.StartNew();
        action();
        sw.Stop();
        return (long)(sw.Elapsed.TotalMilliseconds * 1000);
    }
}
```

Вы вызываете `ProfilerUtil.MeasureMicros(() => PatrolSystem.Update())` в `_PhysicsProcess` и видите в отладчике, что этот блок стабильно отъедает 6 миллисекунд. Гипотеза подтверждается: проблема здесь. Дальнейший разбор показывает, что система каждый кадр сканирует всё дерево сцены в поисках пути, хотя маршруты давно построены и лежат без движения. Вы меняете её логику так, чтобы пересчёт маршрута происходил раз в полсекунды или по событию входа в новую зону, и время скриптов падает до 3 миллисекунд. Вы не гадали, не оптимизировали LINQ-запросы наугад, а зафиксировали, изменили и снова зафиксировали улучшение.

Охота за спайками GC требует похожего подхода, но иного инструментария. Godot позволяет наблюдать за динамикой памяти и активностью сборщика мусора. Если каждые несколько секунд на графике виден острый пик, совпадающий с фризом, ищите код, который щедро создаёт временные объекты. Чаще всего виновниками оказываются строки. Форматирование `$"Health: {value}"` внутри `_Process` создаёт новый объект строки каждый кадр, нагружая GC. LINQ в горячем пути плодит скрытые итераторы. Ключевой вывод здесь заключается в том, что проблема не в «медленном методе», а в его скрытых последствиях. Вы смещаете фокус со времени выполнения конкретной строки на количество создаваемого ею мусора и частоту вызова.

Производительность как архитектурный аспект

Самый мощный инструмент оптимизации находится не в профайлере, а в архитектурных решениях, принятых на ранних этапах. Никакой ручной таймер не поможет, если структура кода вынуждает вас выполнять лишнюю работу. Если каждый элемент UI сам опрашивает модель данных в `_Process`, а модель при этом не меняется, вы сжигаете бюджет впустую. Если десяток систем подписаны на общую шину и на каждое, даже незначительное событие запускается каскад проверок, профилировщик покажет высокую загрузку скриптов, но истинная причина будет не в «медленных вычислениях», а в лавинообразном распространении уведомлений. Именно поэтому разделение ответственности и событийная модель, о которых мы говорили на протяжении всей книги, являются лучшей защитой от проблем с производительностью. Когда модель здоровья меняется, она публикует единственный факт `HealthChanged`. Интерфейс, и только интерфейс, реагирует на него и обновляет конкретную полосу. Ему не нужно проверять состояние каждый кадр. AI врага не обходит всю сцену в поисках цели, а получает событие от детектора `Area2D`, когда игрок входит в радиус. Каждое такое решение устраняет не просто лишнюю работу, а целый класс потенциальных спайков и постоянно растущей нагрузки.

Завершая разговор о профилировании, хочется дать один практический совет. Начинать профилирование не с запуска инструментов, а с чёткой формулировки симптома и гипотезы. «Когда я вхожу в бой с тремя магами и десятью скелетами, игра начинает рывками тормозить. Кажется, что проседает именно обсчёт AI, потому что до входа в комнату всё плавно».

Такая постановка вопроса сама по себе решает половину проблемы. Дальше остаётся подтвердить её цифрами, внести минимальное архитектурное изменение и проверить результат. Итерация за итерацией вы будете улучшать не только фреймрейт, но и саму структуру проекта, превращая оптимизацию из финального чек-листа в неотъемлемую часть разработки.

11.2. Снижение аллокаций: управление памятью в горячих путях

Плавность игры измеряется не только средней частотой кадров, но и умением выдерживать ровный, предсказуемый ритм. В предыдущем разделе мы научились замерять время кадра и находить участки, которые тратят бюджет. Но одна коварная разновидность проблем ускользает от обычных замеров процессорного времени – микрофризы, вызванные внезапной активностью сборщика мусора. Они проявляются как случайные рывки посреди плавного геймплея и способны свести на нет все усилия по оптимизации логики. Борьба с такими рывками начинается не с переписывания кода на низкоуровневые конструкции, а с понимания простой истины: в C# каждый оператор `new`, каждая конкатенация строк и каждое LINQ-выражение незаметно заполняют управляемую кучу короткоживущими объектами.

Память, выделенная под эти объекты, не освобождается мгновенно после использования. Сборщик мусора ждёт определённого порога заполнения и только затем запускает процедуру очистки. Если порог достигается в разгар боя, когда каждый кадр на счету, игрок чувствует предательскую задержку. Именно поэтому снижение аллокаций – не борьба за микросекунды, а стратегия по устранению самой причины нестабильности. Мы перестаём засорять кучу мусором, а значит, сборщик мусора может спокойно дремать, не вмешиваясь в игровой процесс.

Где прячутся главные источники мусора

Первый и самый желанный кандидат для охоты – строки. Всякий раз, когда вы пишете `healthLabel.Text = $"HP: {player.Health}/{player.MaxHealth}"`, среда исполнения создаёт новую строку, выделяет под неё память, копирует символы, а старую строку помечает как мусор. В масштабе одного кадра такая операция безобидна. Но представьте себе сцену с сотней врагов, над каждым из которых висит полоска здоровья, обновляемая каждый кадр. Сотни строковых аллокаций в каждом кадре способны пробудить сборщик мусора за считанные секунды. Решение не в том, чтобы перестать показывать здоровье, а в том, чтобы обновлять эту строку только при реальном изменении значения. Если здоровье не менялось тридцать кадров подряд, незачем вычислять один и тот же текст тридцать раз.

Второй хронический нарушитель – временные коллекции. В горячем цикле, обходящем всех видимых врагов, рука сама тянется написать `var visibleEnemies=new List<Enemy>()`, заполнить его и тут же выбросить. Для прототипа такой подход допустим, но в боевом коде, вызываемом десятки раз в секунду, каждое такое выделение превращается в поток мусора. Правильный путь – поднять коллекцию на уровень выше, сделать её полем класса и очищать через `Clear` перед повторным использованием. Этот буфер должен иметь одного явного владельца, который отвечает за его очистку и не даёт ссылке на буфер уйти за пределы своего контекста.

Третий источник – замыкания, LINQ и скрытый `boxing`. Современный C# невероятно удобен: лямбды, анонимные методы, цепочки `Where`, `Select`, `ToList` превращают громоздкий код в несколько элегантных строк. Но за каждую такую удобную конструкцию приходится платить созданием объектов-делегатов, итераторов и временных массивов. Возьмём простой пример: вы ищете ближайшего врага с помощью `enemies.OrderBy(e => Vector2.Distance(transform.position, e.position)).FirstOrDefault()`. LINQ-цепочка создаёт как минимум пару внутренних объектов, а если врагов много, ещё и временный буфер для сортировки. В горячем цикле подобные решения недопустимы. Ручной цикл с простым сравнением не только быстрее, но и полностью лишён аллокаций. Это не значит, что LINQ нужно изгнать из проекта, в загрузочных скриптах, редакторах и меню он вполне уместен. Но в методах, вызываемых каждый кадр, следует предпочитать ясные, без-аллокационные конструкции.

Аллокации могут просачиваться через самый безобидный API. Представьте метод, который возвращает список целей для атаки. Если метод создаёт новый `List<T>` при каждом вызове, он навязывает аллокацию всем своим потребителям. Намного правильнее принимать буфер извне или предоставлять данные через итерацию без создания коллекции.

Кеширование, время жизни и правильное устройство данных

Огромный пласт аллокаций уходит, когда разработчик перестаёт искать зависимости в каждом кадре. Код вроде `GetNode<Player>("/root/World/Player")` внутри `_Process` не просто требует процессорного времени на обход дерева сцены, но и часто приводит к неявным аллокациям, например к созданию `NodePath` или промежуточных управляемых объектов. Зависимость, которая нужна персонажу на протяжении всего уровня, должна быть разрешена один раз в `_Ready` или в методе инициализации и сохранена в поле. Горячий путь затем читает готовую ссылку и выполняет только полезную работу.

Подобный подход распространяется и на более сложные объекты. Долгоживущие структуры вроде контроллеров экранов, моделей здоровья или пулов снарядов создаются при старте уровня и живут до его завершения. Объекты средней продолжительности жизни, например состояния AI или

временные эффекты, создаются при входе в режим и уничтожаются при выходе. Если же какой-то объект порождается каждый кадр, это верный сигнал задуматься о его переиспользовании или переносе в более долгоживущую область.

Особого внимания заслуживают структуры и классы. Начинающие разработчики часто слышат, что `struct` живут на стеке и не создают мусора, и начинают бездумно переводить всё на значимые типы. В действительности структуры тоже могут становиться источником аллокаций – например, при упаковке в `object` или при передаче в API, ожидающий интерфейс. Кроме того, копирование больших структур может обойтись дороже, чем единственная аллокация в куче. Вменяемый подход – использовать `struct` для маленьких, не подлежащих изменению сущностей (векторов, ключей, метрик), а классы – для объектов с собственной идентичностью и жизненным циклом.

Архитектурный баланс: плавность без фанатизма

Погоня за нулевыми аллокациями способна превратить код в нечитаемое месиво из буферов, ручных циклов и запретов на любые удобные конструкции языка. Такой код столь же вреден, как и тормозящий прототип: он замедляет разработку, плодит ошибки и делает поведение системы непрозрачным. Зрелая архитектура проводит чёткую границу между горячими и холодными участками.

В горячих путях – `_PhysicsProcess`, циклы обновления сотен сущностей, обработки частых событий – действует строгая дисциплина. Никаких `LINQ`, никаких `new List<>()`, никакого форматирования строк на каждый кадр. Здесь живут переиспользуемые буферы, закешированные ссылки и простые циклы. В холодных путях царят читаемость и удобство. Там уместны `LINQ`, анонимные функции и временные коллекции, потому что они выполняются редко и не влияют на плавность геймплея.

Принимая решение об оптимизации, всегда опирайтесь на профилировщик. Прежде чем переписывать классный, выразительный код в низкоуровневую форму, убедитесь, что этот конкретный участок действительно коррелирует со спайками кадра. Возможно, проблема вовсе не в нём, а в том, что UI-панель форматирует строку здоровья на каждом кадре, хотя та не менялась уже десять секунд. Одно точечное исправление в подходящем месте даст больший эффект, чем месяц переписывания всего проекта.

В конечном счёте снижение аллокаций – не самоцель, а естественное следствие продуманной архитектуры. Когда модель публикует факты и UI обновляется только по событию, строки форматируются редко. Когда зависимости внедряются при старте сцены, исчезает необходимость в поиске по дереву. Когда коллекции переиспользуются, а горячие циклы пишутся без `LINQ`, сборщик мусора перестаёт вмешиваться в игровой процесс. Ваша задача – не объявлять войну каждому `new`, а выстроить систему, которая уважает время кадра и предсказуемо укладывается в отведённый бюджет.

11.3. Объектные пулы: переиспользование вместо создания

Представьте себе поле боя в разгар сражения. Десятки врагов одновременно атакуют игрока, и каждый залп порождает новые снаряды. Плазменные пули, ракеты, осколки – сотни крошечных объектов возникают и через секунду исчезают, врезавшись в препятствие. Если каждый такой снаряд создавать через `new` и уничтожать при попадании, в игре начнётся настоящая война со сборщиком мусора. Вы десятки раз в секунду выделяете память под новые объекты и так же быстро помечаете старые как мусор. Куча заполняется, и в самый драматичный момент, когда игрок уворачивается от последней ракеты, сборщик мусора останавливает весь процесс, чтобы навести порядок. Кадр замирает, персонаж гибнет, и игрок в бешенстве швыряет геймпад в стену.

Объектный пул устраняет эту проблему, меняя саму философию жизненного цикла объектов. Вместо того чтобы хоронить каждый снаряд и создавать новый, мы строим «склад» готовых к использованию экземпляров. Когда пушка стреляет, она не создаёт пулю, а берёт её со склада, активирует и выстреливает. Когда пуля достигает цели или вылетает за экран, она не умирает, а деактивируется и возвращается на склад. Это кардинально меняет картину потребления памяти: вместо сотен выделений и освобождений в секунду мы получаем предварительно созданные объекты, которые живут долго и нигде не исчезают. Сборщику мусора просто не с чем работать, а значит, он не проснётся посреди боя, чтобы испортить игроку впечатление. Архитектурно пул решает проблему, с которой мы столкнулись в предыдущем разделе, когда говорили о коллекциях. `List.Clear()` позволял переиспользовать одну коллекцию вместо создания новой. Пул делает то же самое, но для целых игровых объектов: снарядов, частиц, врагов в волновом режиме, временных эффектов. Объединяет их всех одна черта – массовость, кратковременность жизни и предсказуемый, повторяемый процесс инициализации.

Как устроен простейший пул и почему он требует контракта

Сердце пула – хранилище неактивных объектов и две операции: «дать» и «вернуть». Когда системе нужен новый снаряд, она вызывает `pool.Get()`, получает готовый экземпляр и активирует его. Когда снаряд отработал своё, он вызывает `pool.Return(shell)`, пул деактивирует объект и кладёт его обратно на полку. Важнейшая деталь, без которой вся конструкция превращается в источник страшных багов, – операция сброса состояния, или `reset`.

Если вы просто берёте объект из пула и используете его, а потом возвращаете, забыв очистить поля, происходит катастрофа. Снаряд, который в прошлой жизни был синим и летел влево, возвращается в пул «грязным». Через секунду его снова достают для выстрела, и он внезапно уже синий и продолжает лететь влево, хотя должен быть красным и лететь вправо. Более того, он

может до сих пор хранить ссылку на врага, в которого уже попал, и подписку на сигналы того врага. Это порождает «призрачные» взаимодействия, которые невозможно поймать отладчиком, потому что баг проявляется не в момент выстрела, а в случайном будущем.

```
using System.Collections.Generic;
public sealed class BulletPool
{
    private readonly Stack<Bullet> _free = new();
    // Отдаём пулю либо из запаса, либо создаём новую.
    public Bullet Get()
    {
        return _free.Count > 0 ? _free.Pop() : new Bullet();
    }
    // Возвращаем пулю на склад, предварительно очистив её.
    public void Return(Bullet bullet)
    {
        bullet.ResetForReuse();
        _free.Push(bullet);
    }
}
```

Сброс состояния есть архитектурный контракт. Разработчик должен чётко решить, какая из сторон отвечает за чистоту объекта: сам объект при возврате, пул при выдаче или потребитель перед началом использования. Наиболее безопасный и предсказуемый путь – сделать `ResetForReuse()` методом самого объекта, который вызывается пулом при возврате. Тогда любой, кто берёт объект из пула, может быть уверен, что получает чистый лист.

Что же именно нужно сбрасывать? В первую очередь – все поля, определяющие игровое состояние: скорость, положение, здоровье, таймеры. Во вторую – внешние связи. Объект мог подписаться на сигналы владельца или цели, зарегистрироваться в глобальном менеджере, сохранить ссылку на другой живой объект. Всё это должно быть безжалостно разорвано. Ошибка здесь стоит особенно дорого: повторно выданный снаряд, сохранивший старую подписку на смерть игрока, будет пытаться дёрнуть метод `OnTargetDead` каждый раз, когда игрок умирает, умножая количество призрачных вызовов с каждым новым кругом жизни пули.

Планирование ёмкости и стратегия роста

Когда пул внедряется в проект, сразу встаёт практический вопрос: сколько объектов создавать заранее? Этот параметр, называемый ёмкостью пула, не должен браться с потолка. Если пул слишком мал, он начнёт расти в бою, создавая новые объекты тогда, когда аллокации наиболее болезненны, – прямо во время активной перестрелки. Если пул слишком велик, он зря занимает память, что особенно критично на мобильных устройствах и консолях.

Архитектурно грамотный подход – опереться на профилировщик и продуктовые сценарии. Вы заходите в самую напряжённую боевую сцену, вклю-

чаете режим песочницы и устраиваете максимальный спам снарядами. Смотрите, сколько объектов одновременно активно в пике, и добавляете сверху процентов двадцать на случай внезапных всплесков. Это число и становится ёмкостью пула.

Создавать весь этот запас лучше в безопасные моменты – при загрузке уровня, на экране загрузки или в фазе подготовки к бою. Эта техника называется *warm-up*, и её цель – перенести неизбежную аллокацию из горячего геймплея в спокойную фазу, где игрок не заметит микрофриза. Если вы выпускаете мобильную аркаду, где битвы случаются внезапно и без загрузочных экранов, возможно, вы предпочтёте заполнять пул постепенно, небольшими партиями по мере надобности. В любом случае у пула должен быть жёсткий верхний предел. Никакой баг или читерский скрипт не должны иметь возможность заставить игру бесконечно плодить объекты, пока не кончится память.

Особого разговора заслуживает политика истощения: что делать, когда пул пуст, а новый объект нужен прямо сейчас. Для критичных сущностей вроде снарядов, от которых зависит попадание и урон, единственный выход – создать новый объект поверх лимита и, возможно, уронить кадр, но сохранить геймплей. Для вторичных, декоративных частиц можно применить деградацию: не создавать часть эффектов, пока пул не освободится. Игрок в пылу битвы вряд ли заметит, что вокруг стало на пару искр меньше, зато кадр останется плавным.

Подводные камни и архитектурная гигиена

Поскольку пул ломает привычную модель «создал–использовал–удалил», он требует строгой гигиены. Объект, возвращённый на склад, нельзя держать за ссылку. Как только вы вызвали `pool.Return(bullet)`, все ваши указатели на этот `bullet` становятся недействительными. Думать о них как об «уничтоженных» – полезная привычка. Если код продолжает обращаться к возвращённому объекту, пул превращается в генератор случайных багов, где один и тот же экземпляр используется двумя независимыми системами одновременно.

Не стоит создавать и один гигантский пул на все случаи жизни. Пул снарядов, пул частиц и пул вражеских юнитов – три разных модуля, которые живут в своих подсистемах. У них разная ёмкость, разная логика сброса и разные владельцы. Боевая система владеет пулом пуль, система спецэффектов – пулом частиц, спавнер уровней – пулом врагов. Такое разделение не позволяет ошибке в одной системе разрушить другую и делает профилирование прозрачным: если в профайлере растёт пул пуль, вы знаете, куда смотреть. Пул – мощный, но не всесильный инструмент. Прежде чем оборачивать в него очередной объект, стоит задать себе несколько вопросов. Действительно ли этот объект создаётся массово, по многу раз в секунду, и его создание вызывает заметные спайки в профайлере? Могу ли я вместо этого просто убрать лишнюю аллокацию в логике, как мы обсуждали в предыдущем разделе? И готов ли я взять на себя ответственность за ручное управление временем жизни

и сброс состояния? Если ответы положительные, пул оправдан и принесёт ровно тот эффект, ради которого его придумали, – стабильный, плавный геймплей без рывков. В конечном счёте архитектурная ценность пула не в том, что он «ускоряет игру», а в том, что он даёт контроль. Вы перестаёте быть заложником сборщика мусора и начинаете сами решать, когда и сколько памяти будет выделено. Этот переход от пассивного наблюдателя к активному управляющему ресурсами и есть главный признак зрелой инженерной культуры в геймдеве.

11.4. Динамическое управление сценами: жизненный цикл мира

Когда проект перерастает стадию прототипа, подход «загрузим всё сразу – и пусть живёт» становится главным врагом стабильности. Представьте себе огромный особняк, в котором вы зажгли свет во всех комнатах, включили воду во всех кранах и запустили отопление на полную мощность, даже если находитесь в одной спальне. Коммунальные службы такого не простят, да и счета придут астрономические. В точности так же ведёт себя игра, которая держит в памяти главное меню, уровень, интерфейс, карту мира и редактор персонажа одновременно. Память переполняется, загрузка длится вечность, а внезапная выгрузка целого графа объектов вызывает те самые микрофризы, которые мы так старательно устраняли в предыдущих разделах. Динамическое управление сценами – это дисциплина, которая возвращает вам контроль над тем, что находится в памяти в каждый конкретный момент. Сцена превращается из статического файла на диске в управляемый модуль с чётко очерченным жизненным циклом. У неё появляются явные точки рождения, активной жизни и гарантированной смерти. И что самое важное, у неё появляется хозяин, отвечающий за все эти переходы.

Стабильный корень и контекст как владельцы сцен

Наивный код, управляющий сценами, выглядит до боли знакомо. Где-то в обработчике нажатия кнопки «Начать игру» вдруг возникает магическая конструкция: `GetTree().ChangeSceneToFile(«res://levels/Level01.tscn»)`. Через час работы выясняется, что при переходе нужно ещё и валюту перенести из старого интерфейса, и данные сохранить, и музыку плавно сменить. Следом в скрипте кнопки появляются десятки строчек, спавнящих ресурсы и обнуляющих статический прогресс. Ещё через месяц, когда босс при смерти должен переключать не сцену, а загружать совершенно новый модуль, начинается адское копирование логики. Это и есть главный симптом архитектурного беспредела: игровая логика захватила управление инфраструктурой.

В хорошо организованной системе смена сцен – отдельная, инфраструктур-

турная ответственность. Доменная логика никогда не говорит «загрузи файл на диске». Она публикует факты: «уровень завершён», «игрок погиб», «пора в главное меню». Специальный модуль, `SceneManager`, слушает эти факты и решает, какой именно контент загрузить или выгрузить. Такое разделение подобно работе регулировщика на оживлённом перекрёстке. Поток машин (событий) не пытается самоорганизоваться – он подчиняется единому центру, который знает правила и следит за порядком. Чтобы эта схема работала надёжно, у менеджера должна быть надёжная опора. Представьте строительные леса, которые держатся на земле, пока вокруг них возводят и сносят разные блоки здания. В `Godot` роль таких лесов играет «стабильный корень» – небольшой набор узлов, которые живут от старта приложения до его закрытия. Сюда входят сам `SceneManager`, глобальные сервисы вроде аудио или хранилища настроек и главный контейнер `Viewport`. Именно этот стабильный корень выступает владельцем всех загружаемых модулей, гарантируя, что они будут корректно созданы и, что ещё важнее, безжалостно выгружены, когда контекст исчезнет.

```
using Godot;
public partial class GameRoot : Node
{
    private SceneManager _sceneManager;
    public override void _Ready()
    {
        // 1. Создаём инфраструктуру на стабильном корне.
        _sceneManager = new SceneManager(this);
        var audioService = new AudioService();
        AddChild(audioService);
        // 2. Загружаем первый игровой контекст – главное меню.
        _sceneManager.LoadContext(new MainMenuContext());
    }
}
```

Обратите внимание: `GameRoot` не занимается стыковкой интерфейса и логики. Он именно дирижирует появлением и удалением контекстов. Следующий уровень детализации – сам контекст, например `LevelContext`. Он получает во владение менеджер сцен, загружает нужные модули и связывает их воедино. Когда уровень завершается, контекст уничтожается, а вместе с ним – и все сцены, и все их внутренние зависимости.

```
public class LevelContext
{
    private Node _levelRoot;
    public async Task Enter(SceneManager sceneManager)
    {
        // Загружаем сцены уровня и соединяем их.
        _levelRoot = await sceneManager.LoadSceneAsync("res://levels/Level01.tscn");
        var ui = await sceneManager.LoadSceneAsync("res://ui/HUD.tscn");
        // Сборка: передача зависимостей игроку, привязка камеры...
    }
}
```

```

    public void Exit(SceneManager sceneManager)
    {
        // Гарантированно выгружаем всё, что было загружено.
        sceneManager.UnloadScene(_levelRoot);
    }
}

```

Тонкая механика выгрузки и поэтапная активация

Создание сцены – лишь половина дела. Управляемая выгрузка требует не меньшей дисциплины. Если просто вызвать `QueueFree` на корневом узле, можно породить десятки призрачных вызовов от подписок, которые не были разорваны. Правильный `SceneManager` делает выгрузку явной процедурой. Он вызывает метод `Deactivate` у модуля, чтобы тот успел отписаться от событий, остановить таймеры и разорвать связи с глобальными сервисами. Только после этого следует физическое удаление узла из дерева.

```

public interface ISceneModule
{
    void Deactivate();
}

public class SceneManager
{
    public void UnloadScene(Node sceneRoot)
    {
        // 1. Шанс на корректную деактивацию.
        if (sceneRoot is ISceneModule module)
            module.Deactivate();
        // 2. Физическое удаление.
        sceneRoot.QueueFree();
    }
}

```

Вторая важная деталь, которую упускают новички, – поэтапная активация. Вы не должны бросать игрока в только что загруженный уровень, где половина врагов ещё не до конца инициализировалась. Сцена загружается, но остаётся в состоянии «заморозки» (её `ProcessMode` выставлен в `Disabled`). Все внутренние связи, инъекции зависимостей и подписки на события происходят в этот момент молча. И только после того, как весь граф готов, сцена активируется и начинает жить полной жизнью. Этот подход полностью устраняет класс багов, связанных с порядком инициализации, когда враг пытается атаковать игрока, который ещё не появился.

```

public async Task<Node> LoadSceneAsync(string path)
{
    var scene = ResourceLoader.Load<PackedScene>(path);
    var instance = scene.Instantiate();
    // Отключаем обработку на время подготовки.
    instance.ProcessMode = ProcessModeEnum.Disabled;
    // Здесь можно выполнить инъекцию зависимостей.
}

```

```

PerformInjection(instance);
// Внедряем в активное дерево и активируем.
_worldRoot.AddChild(instance);
instance.ProcessMode = ProcessModeEnum.Inherit;
return instance;
}

```

Почему всё ломается и как это чинить

Самый коварный враг динамической архитектуры – скрытые ссылки. Модуль уже выгружен, но его объекты продолжают жить, потому что на них кто-то подписался. Или, что ещё хуже, на них подписался другой модуль, который выгрузился раньше, а теперь пытается обновить несуществующий интерфейс. Лечится это только железной дисциплиной, зашитой в интерфейсы. Если модуль реализует `ISceneModule`, его `Deactivate` обязан отписаться от всего внешнего мира. На помощь приходит и простая наблюдаемость. Хороший `SceneManager` умеет отвечать на вопросы «какой контекст сейчас активен?», «какие сцены загружены?» и «сколько памяти занимает граф объектов?». Вы можете добавить в отладочный интерфейс простые счётчики: количество активных `Area2D`, количество подписок на глобальные события, объём занятой памяти. Если после десяти циклов «войти в уровень – выйти в меню» эти цифры неуклонно ползут вверх, в коде поселился «призрак» – забытая подписка или статическая ссылка. Наконец, не стоит делать статический корень всезнайкой. Огромная ошибка – превратить `GameRoot` в склад всех возможных ссылок и глобальных переменных. Его задача – быть тонким планировщиком. Он знает о существовании `SceneManager` и, возможно, пары глобальных сервисов. Вся логика, специфичная для конкретного режима игры, живёт и умирает в рамках своего контекста. Если вы будете соблюдать это правило, динамическое управление сценами перестанет быть магией и превратится в простой, как конструктор, механизм. Ваш мир будет собираться и разбираться плавно, память останется под контролем, а игрок никогда не узнает, какая сложная инженерия скрывается за мгновенным переходом от главного меню к эпической битве.

12

Подготовка к релизу и сопровождение проекта

Игра, запускающаяся на машине разработчика, и игра, работающая у сотен игроков, – две принципиально разные программы. Первая живёт в тепличных условиях: рядом исходники, редактор, отладочные инструменты и полный доступ к файловой системе. Вторая вынуждена выживать в агрессивной среде: на чужих конфигурациях железа, с ограниченными правами, без возможности быстро поправить код и перезапустить сцену. Переход от первой ко второй и есть та самая подготовка к релизу, о которой пойдёт речь в этой главе. Но релиз – лишь верхушка айсберга. Настоящая инженерная зрелость наступает тогда, когда вы осознаёте, что продукт не умирает в момент публикации. Он начинает жить второй, гораздо более долгой жизнью: обрастает обновлениями, адаптируется под новые платформы, чинится после неожиданных багов, найденных в поле. И если архитектура не была заложена с учётом этой долгой дистанции, каждый следующий патч будет даваться всё тяжелее, пока проект не превратится в легендарный «легаси», который проще переписать с нуля. Мы разберём, как превратить прототип в продукт, как управлять его конфигурацией в разных окружениях, как выстроить процессы, позволяющие поддерживать игру долгие месяцы после первого публичного запуска, и как удерживать архитектуру от деградации под натиском бесконечных правок. Это не разовая акция, а инженерная дисциплина, которая начинается задолго до заветной кнопки «Экспортировать» и продолжается годами.

12.1. Экспорт как снятие маски с неявных зависимостей

Первый же запуск экспортированной сборки часто становится моментом отрезвления. То, что прекрасно работало в редакторе, внезапно отказывается загружать сцены, не находит ресурсы или падает с невнятной ошибкой. Разработчик в панике ищет проблему в движке, но настоящая причина гораздо прозаичнее: проект был построен на десятках неявных допущений, которые экспорт безжалостно обнажает. Запуск в редакторе, особенно из IDE, происходит в максимально комфортных условиях. Рядом лежат все исходники, временные файлы, ресурсы в распакованном виде. Консоль редактора послушно выводит любые сообщения, а отладчик позволяет в любой момент поставить точку останова. Многие вещи «случайно» доступны по абсолютным путям просто потому, что вы работаете на своей машине и структура папок никогда не менялась. Экспортированная сборка живёт в совершенно ином мире. Она запускается из другой директории, на другой машине, с другими настройками, часто – в «песочнице» с жёсткими ограничениями прав. В ней нет редактора, нет дебаг-символов (если вы их не включили явно) и часть диагностических каналов может быть отключена или менее удобна. Отсюда главный практический вывод: если ваш проект опирается на случайные свойства окружения – пути, права, наличие определённых файлов, порядок загрузки, – экспорт выявит это мгновенно. Экспорт не ломает проект; он снимает маску с неявных зависимостей, которых вы не замечали, пока работали в уютной среде разработки.

Целевые платформы как архитектурный контракт

Платформа – это не просто «Windows или Linux». Для экспорта платформа представляет собой набор ограничений и соглашений, которые напрямую влияют на архитектуру. Файловая система и правила доступа: на десктопе вы можете писать куда угодно (почти), на мобильном устройстве – только в строго отведённые песочницы. Особенности путей и разделителей: Windows использует обратный слеш, Linux и macOS – прямой, а Android вообще оперирует виртуальной файловой системой. Политика прав: на консолях приложение может быть лишено доступа к сети в фоновом режиме, а на iOS – запрашивать разрешения у пользователя. Ожидания пользователя по размещению настроек и сохранений: на ПК принято хранить сейвы в «Моих документах» или AppData, на мобильных устройствах – в защищённой папке приложения. Различия в доступности графических и аудиовозможностей: не каждая платформа тянет те же шейдеры, что и ваш мощный десктоп. Архитектурно полезно думать так: платформа диктует границы инфраструктуры, но не должна менять доменную логику. Если перенос на платформу требует переписывать правила игры, значит, границы между доменом и инфраструктурой были неустойчивыми с самого начала. Отдельный слой

платформы – форма артефакта, которую видит пользователь. Один файл или набор файлов, установка или портативный запуск, необходимость дополнительных разрешений, политика обновления – всё это определяет, где лежат данные и какие ограничения действуют. Поэтому «место для сохранений», «место для логов» и «место для кеша» – не случайный выбор автора, а часть контракта платформы, которую следует продумать заранее. Особенно важно не смешивать данные поставки и данные пользователя. Данные поставки – то, что вы экспортировали: ресурсы, конфиги по умолчанию, сцены. Данные пользователя – то, что должно жить отдельно: настройки, сохранения, кеш. Если вы храните всё «рядом с исполняемым файлом», вы невольно строите зависимость от прав и от способа установки. На одной машине это сработает, на другой – игра просто не запустится. Сильная архитектура делает эти границы явными и одинаково корректными как в редакторе, так и в экспортированной среде.

В Godot для этого предусмотрен специальный префикс `user://`, который указывает на безопасную директорию, гарантированно доступную для записи на целевой платформе. Привыкайте использовать его с первого дня. Строка `FileAccess.Open("user://savegame.json", FileAccess.ModeFlags.Write)` работает и на Windows, и на Android, и в редакторе, а попытка писать в `res://` на мобильной платформе приведёт к молчаливому отказу.

```
using Godot;
using System.IO;
public static class SavePaths
{
    public static string GetSaveFilePath(string fileName)
    {
        // user:// гарантирует запись в безопасную директорию на всех платформах.
        return ProjectSettings.GlobalizePath($"user://{fileName}");
    }
    public static string GetConfigFilePath(string configName)
    {
        return ProjectSettings.GlobalizePath($"user://config/{configName}");
    }
}
```

Воспроизводимость сборки и упаковка зависимостей

Экспортированный билд – это не только ваши сцены и скрипты. Это также зависимости, без которых проект не запустится: движок и его нативные компоненты; управляемый слой .NET (если проект на C#) и соответствующий runtime; ресурсы – текстуры, аудио, шейдеры, локализация, конфиги, которые должны быть доступны в рантайме. Архитектурная дисциплина здесь – понимать, какие зависимости являются рантайм-зависимостями, а какие – инструментальными. IDE, генераторы кода, тестовые ассеты и утилиты разработки должны оставаться в проекте, но не в поставке. Если инструменты разработки незаметно становятся частью рантайма, это означает

смешивание ответственности: продукт начинает зависеть от среды разработки. На уровне модели полезно представить экспорт как контейнеризацию проекта: вы упаковываете минимальный набор, обеспечивающий выполнение пользовательского сценария. Всё, что не влияет на выполнение сценария, должно остаться снаружи. Это относится не только к инструментам, но и к данным разработки: временные файлы, старые прототипные сцены, тестовые изображения, локальные заметки. Пока вы запускаете игру из редактора, такие вещи могут существовать рядом и не мешать. В продуктовой сборке они либо увеличивают артефакт и усложняют поддержку, либо создают риск «случайной зависимости» – например, код начинает обращаться к файлу, который есть у разработчика и отсутствует у игрока.

Для проектов на C# добавляется ещё одна грань: код в рантайме представлен не исходниками, а собранными сборками. Это меняет характер проблем. В разработке вы часто воспринимаете «скрипт» как текст и место, где можно быстро вставить лог. В релизе важно другое: какой именно набор сборок вошёл в артефакт, какие версии библиотек используются, нет ли дублирования и как отладочная информация соотносится с тем, что у вас в репозитории. Это не «внутренности компилятора», а вопрос ответственности: если вы не можете однозначно сказать, что запущено у пользователя, вы не можете устойчиво поддерживать продукт.

Полезно мысленно разделять экспорт на две стадии: «собрать код» и «упаковать продукт». Сборка кода обеспечивает, что поведение определено версиями сборок, а не состоянием исходников. Упаковка обеспечивает, что эти сборки и ресурсы попадут в артефакт в корректной форме для целевой платформы. На практике проблемы часто возникают именно на границе: когда в экспорт попадает устаревшая сборка, когда локальный запуск использует одно, а экспорт – другое или когда часть зависимостей присутствует в среде разработки и отсутствует в артефакте. Архитектурная защита здесь – привычка мыслить артефактом как единственным источником истины: «что вошло в сборку, то и будет работать», а значит, любые допущения нужно проверять именно на экспортированной версии.

Воспроизводимость сборки означает простую вещь: один и тот же исходный код и набор ресурсов должны давать один и тот же результат при повторной сборке – по крайней мере по поведению. Почему это важно: вы можете сравнивать версии продукта и точно понимать, что изменилось; команда может повторить сборку и получить тот же результат; дефекты можно воспроизводить на конкретной версии артефакта; релиз перестаёт быть «уникальным событием на машине автора». Практически воспроизводимость начинается с дисциплины: фиксированные версии движка и зависимостей, минимизация неявных шагов, понимание того, от чего зависит результат экспорта. Сборка должна иметь понятный идентификатор – версию, номер, хеш, хотя бы дату, – по которому можно связать артефакт с состоянием репозитория и параметрами сборки. Это помогает не только команде, но и самому автору: через месяц вы сможете ответить на вопрос «какая сборка стоит у тестировщика» без догадок.

Важно выработать привычку проверять сборку как независимый продукт с самого начала разработки, задолго до реального релиза. Ранний экспорт не нужен для публикации, он нужен для проверки контракта: работает ли проект как самостоятельный продукт, не опирается ли на случайные свойства машины разработчика, не протекают ли инструменты разработки в артефакт. Чем раньше вы обнаружите, что, например, сохранения пишутся «не туда» или ресурс ищется по неправильному пути, тем дешевле исправление: вы ещё не построили поверх этого десятки систем.

Регулярный экспорт также формирует дисциплину изменения. В процессе разработки вы постоянно меняете структуру сцен, имена ресурсов, конфигурационные значения, логику загрузки. Если вы не связываете эти изменения с проверкой артефакта, проект постепенно накапливает «скрытые допущения», и в конце оказывается, что релизные проблемы – это не один баг, а множество мелких несовпадений контекста. Экспорт как повторяемый шаг делает эти несовпадения видимыми и удерживает архитектуру в рабочем состоянии. В этом смысле экспорт полезно включать в собственное определение «готово». Фича считается завершённой не тогда, когда она работает в редакторе, а тогда, когда она работает в сборке, пусть даже внутренней. Такой критерий дисциплинирует архитектуру: вы вынуждены сразу думать о путях, о данных пользователя, о поведении без отладочной среды и о воспроизводимости. Со временем релиз становится повторяемой операцией, а не «особым режимом», где выявляются скрытые зависимости.

Дисциплина включения ресурсов

Ресурсы – не «данные рядом с кодом», а часть поведения продукта. Отсутствующая текстура, неверная локализация, невключённая сцена или конфиг меняют поведение так же, как баг в коде. Поэтому экспорт требует чёткой дисциплины: ресурс должен быть либо гарантированно включён в сборку, либо проект должен уметь жить без него в предсказуемом режиме; ссылки на ресурсы должны быть стабильными и не зависеть от локальной структуры машины разработчика; ресурсы должны быть организованы так, чтобы зависимость была явной – например, модуль уровня знает, какие ресурсы он требует, а не «ищет по папкам».

Это продолжает архитектурную мысль из предыдущих частей книги: границы и контракты важнее локального удобства. Если загрузка ресурсов построена на «случайно работает», экспорт делает проблему видимой.

Отдельно стоит заметить дисциплину «активов по назначению». В проекте часто есть ресурсы разных классов: продуктовые, тестовые, временные, отладочные. Пока нет явного разделения, экспорт легко упакует «всё подряд», а вы заметите это слишком поздно – по размеру дистрибутива, по времени загрузки или по неожиданным ссылкам. Архитектурное решение здесь не в «ручной чистке перед релизом», а в привычке разделять ресурсы по ответственности: то, что предназначено для исполнения, хранится и используется иначе, чем то, что предназначено для проверки или экспери-

мента. В больших проектах это становится вопросом масштабируемости: чем больше людей и сцен, тем выше вероятность, что ресурс будет случайно переиспользован не по назначению. Стабильный экспорт требует, чтобы «входит в сборку» было следствием структуры, а не внимательности автора в последнюю ночь.

Проверять экспортированный артефакт нужно системно. Полезно определить минимальный «smoke-набор» – короткий путь пользователя, который затрагивает ключевые инфраструктурные механизмы: загрузка сцены, запись настроек, сохранение и загрузка, выход и перезапуск. Такой набор не заменяет полноценное тестирование, но удерживает архитектуру от деградации и быстро выявляет, что изменилось в контексте артефакта. В идеале этот сценарий должен запускаться на каждой релизной сборке, становясь частью рутины, а не героическим усилием перед отправкой в стор.

12.2. Конфигурация как инструмент управления окружениями

Когда у вас есть одна машина и один запуск игры, все параметры можно задать прямо в коде. Хост для подключения к серверу, тайминги автосохранений, флаги включения отладки – всё это выглядит как обычные константы, которые нет смысла выносить куда-то наружу. Но как только появляется второе окружение – тестовый сервер, билд для стримера, демоверсия для выставки, – жёстко прошитые значения становятся врагом. Изменение адреса сервера или отключение аналитики требует правки исходников и перекомпиляции, а это путь к хаосу. Конфигурация – это данные, которые управляют поведением продукта без изменения кода. В контексте релиза конфигурация отвечает за «переменные окружения»: то, что должно отличаться между средами (разработка, тестирование, релиз), между платформами, между режимами эксплуатации. Архитектурно конфигурация – продолжение принципа разделения ответственности: код описывает правила и алгоритмы, конфигурация задаёт параметры и выборы, которые должны оставаться изменяемыми.

Код и конфигурация: что должно жить где

Различие между кодом и конфигурацией – не вопрос вкуса или стиля. Это вопрос стабильности продукта. Код должен содержать доменные правила, определяющие поведение игры; алгоритмы, которые должны быть одинаковыми во всех средах; инварианты и ограничения, которые защищают систему от некорректных состояний. Конфигурация должна содержать параметры, которые закономерно меняются между окружениями или версиями; адреса и идентификаторы инфраструктуры (если они вообще нужны на клиенте);

флаги включения и выключения дополнительных возможностей, таких как расширенная диагностика; значения по умолчанию, которые продукт может переопределять без перекомпиляции.

Если конфигурация отсутствует как дисциплина, проект начинает цементировать предположения: адрес сервера, пути, лимиты, тайминги, флаги диагностики, параметры баланса. Пока проект мал, это кажется безобидным. Но по мере роста любое изменение превращается в правку кода, а значит – в риск, в повторную сборку, в потерю воспроизводимости между средами. Ключевая мысль: конфигурация не должна заменять архитектуру. Если вы выносите в конфигурацию то, что на самом деле является структурной проблемой (например, зависимости между подсистемами), вы просто переносите хаос из кода в файлы. Конфигурация полезна там, где переменность естественна и контролируема, например тайм-аут подключения к серверу, а не сам факт наличия мультиплеера.

Самый распространённый вариант отсутствия конфигурации – привычка «просто написать значение в коде». Это делается из благих причин: быстрее, понятнее, «потом вынесу». Проблема в том, что «потом» редко наступает, а цена растёт. Вы теряете управляемость – чтобы изменить значение, приходится перекомпилировать и пересобрать артефакт. Вы теряете прозрачность – трудно понять, какие значения использует конкретная сборка. Вы теряете дисциплину окружений – тестовая среда начинает «случайно» совпадать с дев-настройками, потому что их нельзя разделить.

Вот как выглядит типичный «зашитый» код и как он эволюционирует к управляемой конфигурации:

```
// Прототип: значения жёстко прошиты.
public sealed class NetClient
{
    public void Connect()
    {
        var host = "127.0.0.1";
        var port = 7777;
        var timeoutMs = 2000;
        // подключение...
    }
}
```

Код компактен, но совершенно неуправляем. Адрес, порт и тайм-аут прошиты в сборку. В результате «релиз на реальный сервер» превращается в правку кода, а проверка сборки – в гадание, что именно было зашито. Исправление начинается с выделения этих значений в отдельный тип, который загружается из внешнего источника и валидируется до использования:

```
// Конфигурация как явный тип данных.
public sealed record GameConfig
{
    public string ApiUrl { get; init; } = "https://localhost:8080";
    public bool DiagnosticEnabled { get; init; } = false;
}
```

```

public int AutoSaveIntervalSeconds { get; init; } = 300;
public int TargetFrameRate { get; init; } = 60;
public static GameConfig Load(string path)
{
    if (!File.Exists(path))
        return new GameConfig(); // Безопасные значения по умолчанию.
    var json = File.ReadAllText(path);
    var config = JsonSerializer.Deserialize<GameConfig>(json) ?? new GameConfig();

    // Валидация на границе: не даём некорректным данным проникнуть в систему.
    if (string.IsNullOrEmpty(config.ApiUrl))
        config = config with { ApiUrl = "https://localhost:8080" };
    if (config.TargetFrameRate <= 0)
        config = config with { TargetFrameRate = 60 };

    return config;
}
}

```

Этот пример намеренно прост и показывает ключевые принципы: явный тип, один вход, разумные значения по умолчанию и минимальная валидация. Реальный проект обычно добавляет более строгую проверку и более аккуратную обработку ошибок, но архитектурная идея не меняется: конфигурация – контролируемый вход, а не набор магических констант.

Окружения и источники конфигурации

Окружения существуют не только в больших командах. Даже одиночный разработчик почти всегда живёт в трёх режимах. Режим разработки (dev) – максимальная наблюдаемость, быстрые проверки, экспериментирование. Режим тестирования (test) – приближение к релизным условиям, воспроизводимость, проверка артефакта. Режим релиза (release) – минимально необходимая диагностика, предсказуемость поведения, безопасность.

Архитектурно важно, что окружение – не «ветка кода с if'ами». Окружение – конфигурационный выбор, который управляет допустимыми компромиссами, но не меняет доменные правила. Если правила игры начинают зависеть от окружения, вы теряете воспроизводимость: одна и та же версия ведёт себя по-разному в разных средах, и это быстро разрушает поддержку.

Конфигурация существует не как один файл, а как цепочка источников. Даже в простом проекте встречаются минимум два: значения по умолчанию, встроенные в код (как часть контракта), и внешний источник, который можно заменить без перекомпиляции. По мере роста добавляются другие источники: пользовательские настройки, тестовые оверрайды, параметры запуска, данные платформы. Архитектурная задача – не «поддерживать всё», а определить, какие источники допустимы и в каком порядке они применяются. Иначе вы получите ситуацию, когда значение меняется «где-то», но никто не знает, кто победил в конфликте.

Практически полезно мыслить иерархией: базовые значения по умолчанию; конфигурация сборки или продукта (файл или встроенный ресурс); пользовательские настройки (то, что меняет пользователь); временные оверрайды (тестовые, диагностические). Такая модель помогает удерживать воспроизводимость: вы можете объяснить, почему конкретное значение получилось именно таким, и можете воспроизвести его на другой машине.

Для Godot особенно удобно использовать встроенные ресурсы – классы, наследующиеся от `Resource`, которые можно редактировать прямо в инспекторе редактора. Вы создаёте такой класс один раз, а затем плодите несколько файлов `.tres` для разных окружений:

```
using Godot;
public partial class GameConfig : Resource
{
    [Export] public string ApiUrl { get; set; } = "https://localhost:8080";
    [Export] public bool DiagnosticEnabled { get; set; } = false;
    [Export] public int AutoSaveIntervalSeconds { get; set; } = 300;
    [Export] public int TargetFrameRate { get; set; } = 60;
}
```

Загрузка такой конфигурации предельно проста и происходит в корне приложения до старта всех основных систем. При экспорте в сборку попадает нужный вариант, а среда разработки использует свой собственный:

```
var config = ResourceLoader.Load<GameConfig>("res://config/release_config.tres");
Engine.TargetFps = config.TargetFrameRate;
_saveManager.Configure(config.AutoSaveIntervalSeconds);
```

Секреты, версионирование и дрейф конфигурации

Под «секретами» обычно понимают токены, ключи, пароли, приватные URL. Даже если вы не делаете сетевую игру, принцип остаётся, то, что не должно быть частью публичного артефакта, нельзя хранить в коде или в открытой конфигурации. Для клиентских игр важно помнить всё, что вы отправили пользователю, в конечном счёте доступно пользователю. Поэтому «секреты в клиенте» – слабая модель безопасности. В этой книге мы не строим сетевую инфраструктуру, но архитектурный вывод полезен: доверенные границы должны быть определены заранее, а конфигурация должна отражать эти границы, а не создавать иллюзии защищённости.

В репозитории обычно полезно хранить шаблон конфигурации или конфигурацию по умолчанию; пример для разработчика (без секретов); описание ключевых параметров в любой удобной форме. Секреты и окруженческие значения, которые нельзя публиковать, должны храниться вне репозитория. Причина не только в безопасности, но и в ответственности: секреты живут в другой области владения и обновляются по другим правилам. Если вы смешиваете их с кодом, вы получаете невозможную поддержку: нужно менять секрет – нужно менять код и пересобирать продукт.

Как и формат сохранений, конфигурация со временем меняется: добавляются поля, меняются допустимые значения, появляются новые режимы. Если конфигурация живёт «как получится», проект теряет воспроизводимость, один и тот же код запускается с разными наборами параметров, и ошибки становятся «плавающими». Поэтому конфигурацию стоит воспринимать как версионизируемый контракт. Даже если вы не вводите полноценную миграцию, полезно иметь явную структуру конфигурации (тип/DTO), понятный источник (где берём и что считаем «истиной») и дисциплину обратной совместимости (как реагируем на отсутствие поля).

С ростом проекта возникает «дрейф конфигурации»: параметры появляются, но больше не используются; используются, но не документированы; имеют значения, которые перестали соответствовать смыслу; различаются между окружениями без ясной причины. Дрейф разрушает поддержку и тестирование: дефект может проявляться только в одном профиле, и выяснить это становится сложно. Чтобы ограничить дрейф, полезны две дисциплины: конфигурация должна быть валидируемой и конфигурация должна быть наблюдаемой в минимальной форме – продукт может вывести короткую сводку о том, какие ключевые параметры применены (без секретов).

Полезно не путать «валидацию на границе» и «проверки по месту использования». Проверки по месту использования неизбежны (например, защититься от null), но они не должны заменять валидированную конфигурацию. Если каждый модуль делает свою валидацию «как получится», вы получаете фрагментированную логику и разные правила в разных местах. Гораздо устойчивее иметь одну точку, где конфигурация приводится в корректное состояние: заполняются значения по умолчанию, нормализуются значения, проверяются диапазоны.

В завершение разговора о конфигурации стоит упомянуть флаги поведения (feature flags): включить или выключить возможность, выбрать алгоритм, изменить уровень диагностики. Они полезны, но легко превращаются в архитектурный долг. Проблема возникает, когда флаг начинает менять доменные правила. Тогда у вас больше нет «одной игры» – у вас несколько вариантов поведения, которые нужно тестировать, поддерживать и объяснять. Для релиза это означает риск: вы можете случайно отправить сборку с неверной комбинацией флагов или получить дефект, который проявляется только в одном сочетании. Архитектурная дисциплина простая: флаги допустимы для инфраструктурных и диагностических решений; флаги допустимы для временной совместимости в переходный период; флаги опасны, когда они меняют правила и баланс как «скрытый режим».

12.3. Debug и Release: два разных продукта

Когда разработчик говорит «у меня всё работает», он почти всегда имеет в виду дебаг-сборку, запущенную в редакторе. Там включены все проверки,

доступны отладочные интерфейсы, можно поставить точку останова. Но релиз – совсем другой артефакт. Оптимизации компилятора меняют порядок выполнения, отладочные проверки отключены, и код, который в дебаге работал стабильно, может неожиданно сломаться.

Debug и Release – не просто «переключатель скорости». Это два разных инженерных договора о компромиссах. Debug-сборка ориентирована на наблюдаемость: легче увидеть проблему, проще поставить точку останова, больше диагностических подсказок, меньше оптимизаций, больше проверок. Release-сборка ориентирована на поставку: предсказуемое поведение в целевой среде, производительность, безопасность и минимально достаточная диагностика. Одна из самых неприятных особенностей разработки заключается в том, что Debug и Release могут иметь разные профили времени и разные побочные эффекты. В debug вы чаще логируете, чаще проверяете состояния, чаще держите дополнительные ссылки для диагностики. Все эти действия меняют порядок и скорость исполнения, а иногда – и видимость ошибок. Типичная история: в дебаг-сборке анимация всегда успевает завершиться до срабатывания таймера, потому что среда разработки чуть медленнее. В релизе, на быстрой машине, таймер срабатывает раньше, персонаж переходит в другое состояние, а анимация остаётся незавершённой – баг. Или наоборот: в дебаге вы логируете каждое действие, и эти логи создают аллокации, которые маскируют настоящую утечку памяти. Релизная сборка остаётся без логов, и утечка выходит на поверхность. Архитектурный вывод: отладка должна стремиться к паритету поведения. Debug должен помогать увидеть проблему, но не должен становиться условием корректности. Это возвращает к дисциплине границ: доменная логика должна быть детерминированной настолько, насколько это возможно, а диагностика – внешним потребителем фактов, а не частью причинно-следственной цепочки.

Защита от расхождения между Debug и Release выстраивается на трёх уровнях.

Первый уровень – условная компиляция. Код, который не должен существовать в релизе, – отладочные команды, консольные читы, инструменты разработчика – оборачивается в директиву `#if DEBUG`. Это не просто флаг, который можно включить или выключить: этого кода физически нет в релизном артефакте, а значит, он не может «случайно включиться» и не занимает место.

```
using Godot;
public static class DebugOnly
{
    public static void DrawProbe(string message)
    {
#if DEBUG
        GD.Print($"[PROBE] {message}");
#endif
    }

    public static void ExecuteCheat(string command)
    {
```

```
#if DEBUG
    // Обработка отладочных команд, недоступных в релизе.
    GD.Print($"Cheat executed: {command}");
#endif
}
```

Второй уровень – рантайм-флаги, управляемые конфигурацией. Они нужны для диагностики, которая может пригодиться и в релизе, но должна быть выключена по умолчанию. Например, расширенное логирование для тестировщиков или детализированные сообщения об ошибках. Такой код существует в артефакте, но активируется только при установке соответствующего флага в конфигурации.

```
public sealed class Diagnostics
{
    private readonly bool _enabled;
    public Diagnostics(bool enabled) => _enabled = enabled;
    public void Info(string message)
    {
        if (_enabled)
            GD.Print(message);
    }
    public void Error(string message)
    {
        // Ошибки логируются всегда, даже в релизе.
        GD.PrintErr(message);
    }
}
```

Третий уровень – регулярная проверка. Релизная сборка должна запускаться как отдельный артефакт не раз в конце спринта, а как минимум раз в неделю. Только так можно заметить, что новая механика опирается на поведение, которое есть в редакторе и отсутствует в продукте, или что включение «временного» лога породило шквал аллокаций в горячем пути.

Проверку релизной сборки полезно проводить по двум направлениям: функциональная корректность и дисциплина окружения. Функциональная часть отвечает на вопрос «работают ли сценарии». Дисциплина окружения отвечает на вопрос «не протекли ли инструменты, не зависит ли поведение от дев-файлов, корректны ли пути и права, корректно ли проявляются ошибки». Эти два направления редко покрываются одним и тем же запуском, поэтому важно осознанно уделять внимание обоим.

Отдельно стоит сказать о логировании в релизе. В разработке мы привыкли щедро рассыпать `GD.Print` по коду, и это правильно: быстрая обратная связь – залог скорости. Но в релизе логирование без дисциплины становится источником проблем: большое количество сообщений и частые аллокации создают нагрузку; лог начинает содержать приватные детали, которые не должны уходить пользователю; сообщения теряют смысл, если они не отвечают на конкретный диагностический вопрос.

Архитектурный подход: логировать на границах ответственности и на переходах состояний. В релизе особенно ценны сообщения, которые помогают понять контекст: версия, режим, ключевые события переходов, ошибки загрузки и сохранения, критические нарушения контракта. Менее ценны «следы каждой мелочи» – они часто делают диагностику хуже, потому что скрывают сигнал в шуме. Полезно мыслить логированием как уровневой системой, даже если вы не вводите полноценные уровни технически: в релизе по умолчанию остаются сообщения уровня «важно», а подробные сообщения включаются только по необходимости и по контролируемому флагу.

Логирование в горячем пути – отдельная ловушка. Даже если лог выключен, форматирование сообщения может происходить до проверки уровня логирования. Поэтому логирование в горячих местах должно быть дисциплинированным: либо отключаемым, либо вынесенным на границы, либо условным.

Есть смысл заранее определить, как вы будете поддерживать продукт, когда у вас нет прямого доступа к машине игрока. В разработке можно «поставить breakpoint» и быстро понять, где ошибка. В релизе вы обычно опираетесь на следы: сообщения об ошибках, лог, контекст версии и воспроизводимость. Поэтому экспорт – момент, когда полезно минимально определить поддержку как часть артефакта: где хранится лог, как пользователь сможет его предоставить, как вы узнаете версию сборки, какие ошибки считаются критическими и как они сообщаются.

Вот минимальная иллюстрация того, как может выглядеть версионирование артефакта и вывод ключевой информации при старте:

```
using Godot;
public partial class DiagnosticsGate : Node
{
    public override void _Ready()
    {
        // Всегда выводим версию сборки и режим – это бесценно для поддержки.
        string version = ProjectSettings.GetSetting("application/config/version").
        ToString();

#if DEBUG
        GD.Print($"Game v{version} (DEBUG)");
#else
        GD.Print($"Game v{version} (RELEASE)");
#endif

        // Критические ошибки конфигурации должны быть видны сразу.
        if (!ValidateCriticalConfig())
        {
            GD.PrintErr("FATAL: Critical configuration is invalid. Check logs.");
        }
    }
    private bool ValidateCriticalConfig()
    {
        // Проверка, без которой игра не должна запускаться.
```

```
        return true;  
    }  
}
```

Это не «идеальная схема», а принцип: диагностическое поведение определяется явно, и его можно контролировать, не смешивая с доменными правилами. В долгосрочном проекте это снижает стоимость поддержки и предотвращает ситуации «у пользователя не работает, но мы не знаем, что именно он запускал».

12.4. От прототипа к архитектуре: реконструкция и рефакторинг

Почти любой игровой проект начинается как прототип. Это нормальная инженерная стратегия, вы проверяете идею, ощущение управления, темп, базовый набор механик. Прототип ценен скоростью и прямолинейностью, но он редко годится как фундамент для долгоживущего проекта. Архитектурный проект – другой тип системы, он существует не только для того, чтобы «работать сейчас», но и для того, чтобы оставаться объяснимым и изменяемым после сотен правок. Прототип – система, ориентированная на быстрый цикл обратной связи. В нём допустимы прямые зависимости «потому что быстрее», смешивание ответственности (в одном месте и ввод, и движение, и UI), код, который объясним автору здесь и сейчас, но не обязан быть устойчивым к изменениям. Архитектурный проект – система, ориентированная на контролируемый рост. В нём основная ценность – не количество написанного кода, а предсказуемость изменений. Прототипы ломаются на масштабе не потому, что «в них ошибки», а потому, что в них накапливаются неявные допущения, «этот узел всегда существует», «эта сцена всегда загружена», «UI можно обновить в любом месте», «можно в любой момент дёрнуть Input», «сохранение – просто сериализовать объект». Пока проект маленький, эти допущения не конфликтуют. Но при росте они начинают пересекаться: меняется порядок загрузки сцен, появляются паузы и меню, добавляются режимы, разные платформы, разные источники ввода, разные форматы данных. То, что работало как «быстрый путь», становится источником хрупкости. В какой-то момент вы обнаруживаете, что каждое изменение требует помнить десятки исключений и поддерживать систему «осторожностью», а не конструкцией.

Когда нужна реконструкция

Реконструкция – не «переписать проект». Это восстановить структуру вокруг текущего поведения: выделить устойчивые ответственности (ввод, движение, UI, сохранения), сделать направления зависимостей явными, удалить

скрытые каналы влияния (глобальные ссылки, хаотические сигналы), переформить состояние и переходы как контролируемый поток. Реконструкция обычно делается поверх уже найденного дизайна игры. Вы не возвращаетесь к нулю, вы переносите то, что работает, в более устойчивую форму.

Как понять, что момент для реконструкции настал? Есть несколько практических признаков. Появляются места, которые невозможно менять локально: любая правка игрока затрагивает UI, сохранения, систему сигналов, камеру. Значимые состояния выражены россыпью флагов – `isRunning`, `isJumping`, `isInMenu`, `canMove`, – и никто не может гарантировать их совместимость. Коммуникация становится неотслеживаемой: поведение зависит от того, кто когда подписался, кто кого нашёл через дерево, кто держит ссылку. Контекст исполнения «плавает»: одна и та же логика работает в одной сцене и ломается в другой из-за отличий дерева и времени жизни. И отдельный маркер – рост времени на отладку. Если на изменение в десять минут уходит два часа поиска причин, это почти всегда проблема структуры, а не синтаксиса.

Один из самых надёжных шагов при реконструкции – извлечь доменную логику из прототипного слоя интеграции с движком. В прототипе часто «правила» живут внутри узлов: кнопка напрямую меняет здоровье, игрок напрямую пишет сейв, UI напрямую решает, что считать победой. Извлечение домена выглядит как перенос «правил» в объект или модуль, который получает входы как данные, изменяет состояние предсказуемо и не зависит от конкретного узла, сцены, пути, сигнала.

Интеграционный слой на Godot-узлах остаётся необходимым: он связывает домен с движком, вводом, визуализацией, сохранениями. Но он перестаёт быть местом, где решается «что правильно», и становится местом, где решается «как подключить к рантайму». Этот шаг напрямую связан с устойчивостью под изменениями: домен можно тестировать и менять без необходимости воссоздавать конкретную сцену и порядок её загрузки.

Вот компактная иллюстрация типичного прототипного смешивания и более чистого разделения на домен и интеграцию:

```
// Прототип: ввод, движение, UI и сохранение смешаны.
public sealed class ProtoPlayer
{
    public int Hp = 100;
    public int X;
    public void Tick()
    {
        if (Input.Left) X -= 1;
        if (Input.Right) X += 1;
        if (Hp <= 0) Ui.Show("Dead");
        if (Input.Save) Save.Write("{hp:" + Hp + "}");
    }
}

// Чистый вариант: команда и эффекты разделены зависимостями.
public interface IInput
{
    bool Left { get; }
```

```

    bool Right { get; }
    bool Save { get; }
}
public interface IPlayerUi
{
    void Show(string msg);
}
public interface ISaves
{
    void Write(string payload);
}
public sealed class PlayerController
{
    private readonly IInput _in;
    private readonly IPlayerUi _ui;
    private readonly ISaves _saves;
    public int Hp { get; private set; } = 100;
    public int X { get; private set; }
    public PlayerController(IInput input, IPlayerUi ui, ISaves saves)
    {
        _in = input;
        _ui = ui;
        _saves = saves;
    }
    public void Tick()
    {
        if (_in.Left) X -= 1;
        if (_in.Right) X += 1;
        if (Hp <= 0) _ui.Show("Dead");
        if (_in.Save) _saves.Write($"{hp:{Hp}}");
    }
}

```

Во втором варианте поведение остаётся похожим, но система становится сопровождаемой: ввод можно заменить (другая схема, AI), UI можно поменять без переписывания контроллера, сохранение можно переоформить в DTO и версионирование. Это и есть смысл реконструкции: не «сделать красиво», а вернуть контроль над зависимостями.

Искусство рефакторинга

Реконструкция задаёт направление, но повседневная работа по улучшению кода – это рефакторинг. Его главное правило: сохранять поведение. Рефакторинг – не «переписывание под видом улучшения», а контролируемое изменение формы без изменения наблюдаемого результата. Сохранение поведения – не «код компилируется» и не «у меня запустилось», а сохранение наблюдаемого результата в ключевых сценариях: движение ощущается так же при тех же параметрах, переходы состояний происходят в тех же условиях, сигналы приходят в тех же фазах, сохранение и загрузка дают эквивалентное состояние.

Рефакторинг становится безопасным, когда вы заранее понимаете, какое поведение обязаны удерживать. Это связывает рефакторинг с тестируемостью и воспроизводимостью: если сценарий нельзя повторить и проверить, «сохранение поведения» превращается в догадку.

Золотое правило: работать малыми шагами. Малый шаг – изменение, которое можно объяснить одной фразой и проверить сразу. «Вынес метод», «выделил класс», «заменял прямой вызов на контракт» – каждая такая операция занимает минуты и почти гарантированно безопасна. Если шаг нельзя быстро откатить или он требует «сначала доделать ещё пять вещей», значит, он слишком крупный.

Особого внимания заслуживают скрытые зависимости – самый страшный источник багов в игровых проектах. Они возникают не из-за ошибок в алгоритмах, а из-за нарушения времени жизни объектов и порядка сигналов. Враг подписался на сигнал двери, дверь выгрузилась, враг остался с мёртвой ссылкой. Звуковой менеджер создал таймер для затухания музыки, сцена сменилась, таймер продолжает тикать в пустоте. При рефакторинге всегда задавайте себе три вопроса: что владеет временем жизни этого объекта? где симметричная отписка от события? что произойдёт, если операция завершится после того, как модуль выгружен?

В реальных проектах часто встречается промежуточная ситуация: система уже достаточно сложная, чтобы регрессии были дорогими, но тестов почти нет. Здесь помогает подход «характеризующих» проверок: вы не пытаетесь сразу доказать корректность архитектуры, вы фиксируете текущее поведение как факт. Небольшой тест чистой логики, который повторяет текущий сценарий; опорный запуск, где вы фиксируете ключевые значения; минимальная проверка артефакта для инфраструктурных границ. Это не заменяет полноценного тестирования, но делает малые шаги рефакторинга проверяемыми и снижает неопределённость изменений.

Есть соблазн начать рефакторинг с внутренностей самых «плохих» функций. Иногда это работает, но чаще даёт ограниченный эффект: вы улучшили метод, но система осталась связанной, и цена изменений почти не снизилась. Более устойчивый подход – начинать с границ ответственности: убрать прямую зависимость домена от UI, убрать прямой доступ к движковому API из чистой логики, вынести сохранения в отдельную модель данных, оформить события и состояния как договор. После таких шагов внутренние улучшения становятся дешевле и безопаснее, потому что последствия локализованы.

Вот пример рефакторинга «метода с дублированием» – частая ситуация в прототипе, когда логика повреждений обросла условиями и повторяющимися проверками:

```
// Запутанный метод: дублирование проверок и нормализации.
public int ApplyDamage(int hp, int dmg, bool shield, bool invuln)
{
    if (invuln) return hp;
    if (shield)
    {
```

```

        if (dmg > 10) hp -= (dmg - 10);
        else hp -= 0;
        if (hp < 0) hp = 0;
        return hp;
    }
    hp -= dmg;
    if (hp < 0) hp = 0;
    return hp;
}

```

Здесь дублируются проверки и нормализация значения, а смысл правила скрыт в ветвлении. Рефакторинг выделяет намерения в именованные методы:

```

// Переработанный вариант: намерение видно, дублирование устранено.
public int ApplyDamage(int hp, int dmg, bool shield, bool invuln)
{
    if (invuln) return ClampHp(hp);
    var effective = shield ? ReduceByShield(dmg) : dmg;
    return ClampHp(hp - effective);
}
private static int ReduceByShield(int dmg) => dmg > 10 ? dmg - 10 : 0;
private static int ClampHp(int hp) => hp < 0 ? 0 : hp;

```

Поведение сохранено полностью, но структура стала проще: правило получило имя, нормализация выделена и применяется в одном месте. Такой код проще изменять и проще проверять, потому что количество ветвей уменьшилось.

Важно не смешивать рефакторинг и оптимизацию. Рефакторинг меняет форму, оптимизация меняет стоимость. Иногда оптимизация требует изменения формы (например, убрать аллокации через кеширование), но тогда важно фиксировать цель – «уменьшить аллокации» – и проверять измерением, а не эстетикой. Если вы одновременно рефакторите и оптимизируете без измерений, вы теряете контроль: не знаете, что именно дало эффект, и не знаете, какое поведение было сохранено. Дисциплина простая: рефакторинг – про ясность и границы, оптимизация – про бюджет и доказательства.

12.5. Код-ревью, дрейф и долгосрочное сопровождение

Проект, который прожил полгода активной разработки, неизбежно накапливает то, что принято называть техническим долгом. Удобнее понимать долг не как моральную оценку, а как модель стоимости. Любое решение, принятое ради скорости сегодня, может увеличить стоимость изменений завтра. Это не всегда плохо: иногда долг оправдан, потому что вы покупаете скорость там,

где ещё не уверены в направлении. Проблема начинается, когда долг перестаёт быть осознанным и становится структурной нормой. Технический долг начисляет проценты: скрытые зависимости между подсистемами, дублирование логики и расхождение правил, сложные для воспроизведения дефекты (особенно на границе событий и времени жизни), рост времени на диагностику и правку, снижение уверенности в изменениях. В долгосрочном проекте вы платите не за строки кода, а за неопределённость. Архитектура снижает неопределённость, делая зависимости явными и ограничивая область изменений. Полезно различать локальный и структурный долг. Локальный долг – небольшой участок кода, который можно исправить изолированно, – неприятен, но управляем. Структурный долг – нарушение границ, размазанное по системе: UI напрямую меняет доменное состояние, события используются как скрытый поток управления, время жизни объектов не оформлено как контракт. Структурный долг опасен тем, что он снижает предсказуемость всей системы и резко увеличивает стоимость любых изменений. Архитектурный дрейф – постепенное расхождение между тем, как система задумывалась, и тем, как она реально работает. Дрейф почти никогда не происходит из-за одного плохого решения. Он происходит из-за множества маленьких уступок: «временно» добавить зависимость, «на минуту» обойти границу, «быстро» сделать доступ к данным напрямую. Каждая уступка рациональна локально, но в сумме они превращают систему в набор исключений.

Чтобы удержать систему от дрейфа, полезно иметь несколько ограждений: регулярный просмотр зависимостей на уровне модулей (что от чего зависит), явные контракты для событий и состояний (кто источник, кто потребитель, где границы), дисциплина времени жизни (кто владеет объектами, кто должен отписываться, что происходит при выгрузке), короткие, но регулярные проверки релизного артефакта (чтобы окружение не начинало «чинить» архитектуру). Эти ограждения не требуют формальных процессов – они требуют привычки задавать одинаковые вопросы при каждом изменении: «куда направлена зависимость?», «кто теперь владеет?», «не стал ли этот модуль глобальным?».

Практика ревью

Работая в одиночку, вы – и автор, и ревьюер в одном лице. Это сложнее, чем кажется. Мозг, написавший код пять минут назад, не видит в нём ошибок – он помнит, что должно быть написано, а не то, что написано на самом деле. Поэтому главный инструмент саморевью – пауза. Дописали метод, сделали коммит, отвлеклись на кофе, затем открыли diff и прочитали его так, будто увидели впервые. При ревью работайте по нескольким слоям. Первый слой – корректность. Делает ли изменение то, что заявлено? Есть ли очевидные ошибки, неожиданные побочные эффекты, ошибки порядка? На этом уровне важна ясность сценария: что должно было измениться и где виден результат.

Второй слой – архитектура. Не нарушены ли границы ответственности? Не появился ли новый глобальный канал? Не потекли ли зависимости наружу?

Этот слой особенно важен для игровых систем, потому что нарушения границ долго не проявляются, но затем делают поддержку дорогой.

Третий слой – данные и контракты. Не изменился ли формат данных (сейвы, конфиг) без версионирования? Не изменился ли контракт события (источник, частота, порядок) без явного намерения? Эти изменения часто ломают продукт «в поле», потому что зависят от старых данных и от долго-временных сценариев.

Четвёртый слой – наблюдаемость и диагностика. Можно ли понять проблему, если она возникнет у пользователя? Не добавили ли вы логирование в горячий путь? Есть ли идентификатор версии артефакта? Здесь ревью связывает изменения с релизной дисциплиной.

Пятый слой – тестируемость. Есть ли способ проверить изменение без ручного «пробежаться по игре»? Выделена ли чистая логика? Не стало ли больше скрытых зависимостей от движка? Даже минимальная проверяемость снижает риск регрессий.

Формат замечаний влияет на пропускную способность ревью и качество исправлений. Полезнее формулировать замечания как проверку контракта изменения: намерение, границы, проверяемость. «Что будет, если...» – вопрос о сценарии и риске. «Какой контракт у...» – вопрос о границе ответственности. «Можно ли сделать намерение явнее...» – вопрос о читаемости. «Как это будет проверяться...» – вопрос о воспроизводимости. Такие вопросы возвращают разговор к инженерным причинам. Отдельно стоит сказать о ревью рефакторингов. Рефакторинг ценен тем, что он управляем, а управляемость теряется, когда ревью превращается в конкурс архитектурных вкусов. Рефакторинг должен иметь заявленную цель и демонстрируемое сохранение поведения. Ревьюер не должен превращать рефакторинг в переписывание «как я бы сделал» – если предложенная альтернатива меняет цель и масштаб, её лучше вынести в отдельную задачу.

Устойчивый темп и ответственное сопровождение

Долгоживущий проект выигрывает не от максимальной скорости, а от устойчивого темпа. Устойчивый темп возможен, когда вы контролируете причины дефектов: границы ответственности, воспроизводимость сборок, дисциплину конфигурации, тестируемость логики, понятную модель времени жизни и событий. Архитектурная дисциплина делает ограничения управляемыми: вы начинаете понимать, что именно стоит дорого и почему, и можете принимать решения осознанно.

Сопровождение требует и умения ограничивать изменения. Не все улучшения одинаково полезны, и не каждую проблему стоит решать «самым правильным способом». Иногда инженерно ответственное решение – оставить систему простой, убрать лишнюю вариативность, сократить поверхность ошибок, отказаться от функции, которая не окупается поддержкой. Это не пессимизм, а способ сохранить продукт живым: устойчивость достигается не максимальным количеством возможностей, а тем, что каждая возможность

имеет ясную цену и ясную область ответственности. Один из практических критериев архитектурной устойчивости в долгой перспективе: способность пережить обновление движка. В проектах на Godot и C# обновления движка и .NET неизбежны. Если границы слабые, обновление «цепляет всё», и проект становится хрупким. Особенно опасно «скакать через пропасть»: долго не обновляться, а затем обновить движок, .NET и зависимости одновременно. Такой шаг трудно диагностировать, потому что причин становится много. Гораздо устойчивее обновлять по одному классу изменений и каждый раз проверять артефакт: запускается ли экспорт, работают ли ключевые сценарии, не изменилось ли поведение на границах.

13

Las Vegas Digital Hub: архитектура уровня продакшена на практике

Двенадцать глав мы двигались от понимания движка к пониманию системы. Разобрали, как C# взаимодействует с ядром Godot, почему статическая типизация служит архитектурным инструментом, а не украшением, и как паттерны проектирования превращают хаотичный набор скриптов в управляемую кодовую базу. Теперь наступает момент истины: мы соберём все принципы в единый проект, который не просто работает, а спроектирован выдерживать нагрузку, изменения и время. Эта глава является кульминацией книги. Мы построим архитектуру «Las Vegas Digital Hub» – расширяемой системы симуляции игрового зала с распределённой логикой. Лас-Вегас выбран не ради декорации. Казино представляет собой идеальный полигон для production-ready архитектуры. Тысячи параллельных сессий, строгие требования к RNG, событийная модель, где каждый бросок костей, вращение барабана или ставка порождают каскад последствий в разных подсистемах. Если архитектура выдерживает казино, она выдержит почти всё. Наверное, я напишу ещё раз, чтобы не было возражений по тематике примера: «Las Vegas Digital Hub» не про азартные игры, ибо это скорее полигон для проверки того, как система выдерживает хаос СОТЕН ОДНОВРЕМЕННЫХ сессий, строгие требования к генератору случайных чисел, каскад событий и бесконечные правки пра-

вил. Здесь не будет простых примеров. Каждый фрагмент кода построен на абстрактных классах, интерфейсах и дженериках. Каждое решение объяснено через принципы SOLID и через конкретные сценарии, в которых данное решение защищает систему от деградации.

13.1. Архитектурный манифест: уход от стандартных скриптов

Стандартный подход к Godot-проектам выглядит обманчиво просто: к узлу привязывается скрипт, внутри скрипта пишется логика, логика обращается к другим узлам через пути или сигналы, а состояние хранится прямо в полях скрипта. Для прототипа этого достаточно. Но для системы из полусотни игровых автоматов, глобального баланса, системы достижений, аудиоменеджера и аналитики такой путь ведёт к хаосу. Вся проблема – в природе стандартного Godot-скрипта. Он нарушает фундаментальные принципы устойчивой архитектуры один за другим. **Принцип единой ответственности (SRP)** рушится в тот момент, когда скрипт, висящий на узле игрового автомата, одновременно хранит состояние барабанов, обрабатывает ввод, обновляет UI, воспроизводит звук и начисляет выигрыш. Это не одна ответственность, а пять, и изменение логики начисления вынуждает трогать код вращения барабанов. Следом нарушается **принцип инверсии зависимостей (DIP)**, потому что скрипт напрямую ссылается на конкретные узлы сцены через `GetNode<Label>("../BalanceLabel")`, на конкретные ресурсы и реализации. При любой правке структуры сцены логика ломается. Наконец, **принцип открытости/закрытости (OCP)** становится недостижимым – добавление нового типа автомата, скажем видеопокера вместо слотов, требует изменения существующего кода, а не расширения системы. Каждое новое правило тянет за собой каскад правок в уже работающем коде.

Решение заключается в полном отказе от модели «скрипт = логика + состояние + визуализация». Мы переходим к модели, где компоненты общаются через контракты, а каждый компонент получает зависимости извне, а не создаёт их сам. В контексте Godot получает чёткое разделение на три слоя.

Доменный слой не знает о Godot. Классы, описывающие правила игры, состояния автомата, расчёт вероятностей, – чистый C# без единой ссылки на `Godot.Node`, `Godot.Signal` или `Godot.Resource`. **Инфраструктурный слой** адаптирует доменный слой к Godot. Узлы сцены реализуют интерфейсы домена, а ресурсы Godot служат контейнерами данных для доменных конфигураций. **Слой координации** связывает компоненты через `Dependency Injection`, и ни один компонент не создаёт свои зависимости напрямую.

Архитектурно можно представить как три горизонтальные полосы.

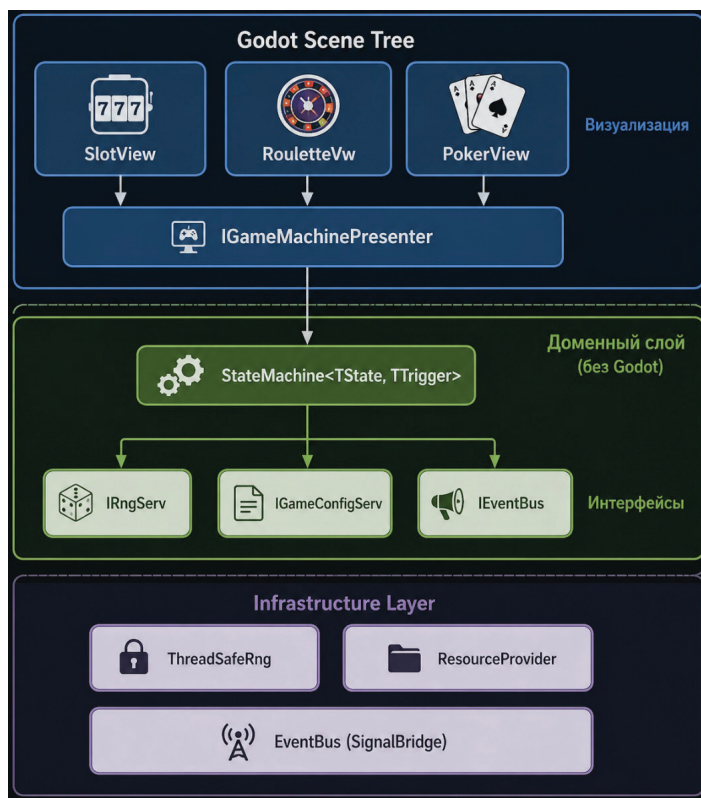


Рис. 1. Визуализация общения компонентов

Верхний уровень – сам Godot Scene Tree, где живут визуальные представления вроде `SlotView`, `RouletteView` или `PokerView`. Эти представления знают только о существовании контракта `IGameMachinePresenter`. Средний уровень – доменный слой, полностью отвязанный от Godot. В его центре находится параметризованная машина состояний `StateMachine<TState, TTrigger>`, которая управляет логикой автомата и опирается на контракты `IRngService`, `IGameConfigService` и `IEventBus`. Нижний, инфраструктурный слой содержит конкретные реализации: `ThreadSafeRng` для генерации случайных чисел, `ResourceProvider` для загрузки конфигураций и `EventBus`, построенный поверх сигналов движка. Каждый слой общается с соседями только через интерфейсы. Доменный слой не знает о существовании Godot. Инфраструктурный слой не знает о бизнес-правилах. Слой координации связывает их. Прежде чем писать конкретные классы, мы фиксируем контракты, каждый интерфейс описывает, что подсистема может делать, а что нет. Дальше большинство пояснений будут в виде комментариев и специальных блоков-пояснений перед методами, не отрываясь от кода, так сказать.

```
// Файл Interfaces/IGameMachine.cs.
```

```
// Базовый контракт игровой машины. Не знает о Godot.
```

```

// Любая машина - слот, рулетка, покер - реализует этот интерфейс.
// Выбор интерфейса, а не абстрактного класса, продиктован отсутствием общей
// реализации состояний. Разные машины имеют принципиально разные модели
// состояния. Если общая реализация появится, мы введём абстрактный класс,
// но контракт останется в интерфейсе, следуя принципу ISP -
// тонкие контракты предпочтительнее одного толстого.
namespace LasVegas.Domain.Interfaces
{
    /// <summary>
    /// Контракт игровой машины.
    /// Определяет жизненный цикл: инициализация → запуск → завершение → сброс.
    /// Перечисление MachineState делает допустимые состояния явными и
    /// проверяемыми на этапе компиляции, устраняя целый класс ошибок,
    /// связанных с неконсистентными bool-флагами.
    /// </summary>
    public interface IGameMachine
    {
        /// <summary>Уникальный идентификатор экземпляра на полу казино.</summary>
        MachineId Id { get; }
        /// <summary>Текущее состояние автомата (Idle, Spinning, Paying и так далее).</
summary>
        MachineState CurrentState { get; }
        /// <summary>Конфигурация автомата (пресет).</summary>
        IMachineConfiguration Configuration { get; }
        /// <summary>
        /// Инициализация автомата с конкретной конфигурацией.
        /// Метод, а не конструктор, используется потому, что конфигурация
        /// может меняться в рантайме - администратор меняет настройки
        /// автомата. Следуя принципу DIP, автомат получает конфигурацию
        /// извне, а не создаёт её сам.
        /// </summary>
        void Initialize(IMachineConfiguration config);
        /// <summary>
        /// Запуск сессии (вращение барабанов, бросок шарика, раздача карт).
        /// Возвращает результат асинхронно: расчёт может быть вынесен
        /// в отдельный поток (подробнее в разделе 14.5).
        /// </summary>
        Task<ISessionResult> StartSessionAsync(BetInfo bet, CancellationToken ct);
        /// <summary>Сброс автомата в начальное состояние.</summary>
        void Reset();
    }
    /// <summary>
    /// Строго типизированный идентификатор.
    /// Отдельный тип исключает возможность передать "случайную строку"
    /// вместо ID автомата - компилятор защищает от этой ошибки.
    /// </summary>
    public readonly record struct MachineId(Guid Value)
    {
        public static MachineId New() => new(Guid.NewGuid());
    }
    /// <summary>
    /// Допустимые состояния автомата.

```

```

/// Перечисление выбрано вместо int, потому что компилятор проверяет
/// полноту покрытия в switch-выражениях (начиная с C# 11).
/// Недопустимые состояния невозможно выразить в принципе.
/// </summary>
public enum MachineState
{
    Uninitialized,
    Idle,
    Running,
    Paying,
    Maintenance,
    Error
}
}

// Файл Interfaces/IMachineConfiguration.cs.
// Конфигурация автомата. Реализуется через Godot Resource в инфраструктурном слое.
// Домен видит только интерфейс, следуя принципу DIP.
// Домен определяет, какие данные ему нужны, а инфраструктура решает,
// откуда их брать – из файла, из базы данных или из редактора Godot.
namespace LasVegas.Domain.Interfaces
{
    /// <summary>
    /// Контракт конфигурации автомата.
    /// Не содержит логики – только данные.
    /// Свойства выбраны вместо методов, потому что конфигурация есть
    /// снимок значений, а не вычислитель. Методы здесь означали бы
    /// скрытую логику, которая принадлежит доменному сервису,
    /// а не контейнеру параметров.
    /// </summary>
    public interface IMachineConfiguration
    {
        /// <summary>Название типа автомата ("MegaJackpot", "ClassicRoulette").</summary>
        string MachineType { get; }
        /// <summary>Минимальная ставка.</summary>
        decimal MinBet { get; }
        /// <summary>Максимальная ставка.</summary>
        decimal MaxBet { get; }
        /// <summary>Теоретический возврат игроку (Return To Player), 0.0-1.0.</summary>
        double Rtp { get; }
        /// <summary>Волатильность (Low, Medium, High).</summary>
        VolatilityLevel Volatility { get; }
        /// <summary>Таблица выплат. Key – комбинация, Value – множитель.</summary>
        IReadOnlyDictionary<string, decimal> PayTable { get; }
    }
    public enum VolatilityLevel { Low, Medium, High }
}

// Файл Interfaces/ISessionResult.cs.
// Результат игровой сессии. Неизменяем после создания.
// Record-тип выбран потому, что компилятор автоматически генерирует
// правильные реализации Equals, GetHashCode, With-методы и ToString.
// Это значительно сокращает объем рутинного кода и усиливает гарантии

```



```
// неизменяемости. Record остаётся reference type, но ведёт себя
// как value type с семантикой сравнения по значению.
namespace LasVegas.Domain.Interfaces
{
    /// <summary>
    /// Результат завершенной сессии (одного вращения или раздачи).
    /// Неизменяем по замыслу: результат игры не должен меняться
    /// после расчёта. Это инвариант домена, защищающий от случайных
    /// или злонамеренных изменений после подведения итогов.
    /// </summary>
    public interface ISessionResult
    {
        /// <summary>Выигрыш (0, если проигрыш).</summary>
        decimal WinAmount { get; }
        /// <summary>Была ли выигрышная комбинация.</summary>
        bool IsWin { get; }
        /// <summary>Выпавшая комбинация (символы барабанов, номер на рулетке).</summary>
        string Outcome { get; }
        /// <summary>Множитель выигрыша (из таблицы выплат).</summary>
        decimal Multiplier { get; }
        /// <summary>Метка времени расчёта (для аудита и последующей аналитики).</summary>
        DateTimeOffset CalculatedAt { get; }
    }

    /// <summary>
    /// Конкретная реализация результата.
    /// Отдельный класс, а не анонимный тип, необходим, потому что
    /// результат передаётся между слоями приложения и должен иметь
    /// стабильное, документированное имя типа.
    /// </summary>
    public sealed record SessionResult(
        decimal WinAmount,
        bool IsWin,
        string Outcome,
        decimal Multiplier,
        DateTimeOffset CalculatedAt
    ) : ISessionResult;
}

// Файл Interfaces/IBetInfo.cs.
// Информация о ставке. Value object: не имеет собственной идентичности,
// сравнивается исключительно по значению. Два экземпляра с одинаковыми
// параметрами концептуально представляют одну и ту же ставку.
// Record struct выбран потому, что ставка – маленький и часто
// передаваемый объект. Struct живёт на стеке и избегает выделения памяти
// в куче, а record обеспечивает семантику сравнения по значению.
namespace LasVegas.Domain.Interfaces
{
    /// <summary>
    /// Ставка игрока. Неизменяема: после размещения ставки
    /// её параметры не должны меняться.
    /// </summary>
    public readonly record struct BetInfo(
        decimal Amount,
```

```

        string Currency,
        int Lines = 1
    );
}

```

Эти интерфейсы – фундамент, на котором стоит вся система. Они выполняют три важные задачи. Во-первых, фиксируют границы: каждый интерфейс описывает ровно одну ответственность, и `IGameMachine` понятия не имеет о визуализации, а `IMachineConfiguration` не подозревает о существовании генератора случайных чисел. Во-вторых, позволяют тестировать систему: в unit-тестах мы подставляем mock-реализации интерфейсов, не создавая Godot-сцену, и доменный слой проверяется в полной изоляции от движка. В-третьих, делают систему гибкой к заменам: если завтра потребуется другой алгоритм RNG, другая система хранения конфигурации или другой способ отображения, меняется одна реализация интерфейса, а не вся система целиком.

13.2. Система состояний: State Machine для игрового автомата

В этом разделе мы строим ядро управления состояниями для каждого игрового автомата. Представьте себе реальный игровой автомат в казино: он не может одновременно вращать барабаны и выплачивать выигрыш. У него есть чёткая последовательность действий: ожидание ставки, вращение, оценка результата, выплата. Если хоть один шаг выполняется не вовремя, вся логика ломается. State Machine – механизм, гарантирующий, что автомат всегда находится ровно в одном допустимом состоянии, а переход между состояниями происходит только по разрешённому маршруту. Разработчик, освоивший этот раздел, научится заменять хрупкие наборы bool-флагов надёжной моделью конечного автомата, применять дженерики C# для создания переиспользуемых механизмов и реализовывать guard-условия для защиты переходов. Мы также закрепим применение принципов SOLID на реальном коде: OCP позволяет добавлять новые типы автоматов через enum'ы, не затрагивая существующие классы, SRP проектирует State Machine так, чтобы та управляла только переходами, а DIP заставляет guard-условия передаваться как делегаты, а не зашиваться внутри механики. Ключевая функциональность, которую демонстрирует листинг: обобщённая `GenericStateMachine<TState, TTrigger>`, конкретная реализация `SlotMachine` с собственными состояниями и триггерами, и `SlotPresenter`, связывающий доменную логику с Godot-сценой.

Начинающие разработчики часто описывают состояние автомата набором флагов: `isSpinning`, `isPaying`, `isIdle`. Работает это до первого серьёзного бага, когда флаги могут оказаться в невозможной комбинации (например, `is-`

Spinning && isPaying), и никакой инвариант не проверяет эту ситуацию. State Machine делает невозможные состояния невыразимыми: автомат всегда находится ровно в одном состоянии из конечного множества, а переходы между ними строго контролируются.

Обобщённая (Generic) State Machine

Мы реализуем state machine, которая не привязана к конкретному типу автомата. Не абстракция ради абстракции, прямое следствие принципа ОСП: добавление нового типа автомата (покер, кено, бинго) не должно требовать переписывания механизма переходов.

```
// Файл StateMachine/GenericStateMachine.cs.  
/* Обобщённая конечная машина состояний.
```

Использование дженериков (TState, TTrigger) обусловлено тем, что разные автоматы имеют разные наборы состояний. Слот проходит через Idle, Spinning, Evaluating, Paying, а рулетка через Idle, Betting, Spinning, Result. Дженерик позволяет переиспользовать один механизм для всех машин, одновременно сохраняя строгую типизацию. Передать состояние слота в машину рулетки не даст компилятор.

Принципы SOLID, которым следует класс:

- SRP: управляет только переходами, ничего не зная о бизнес-логике.
- ОСП: новые автоматы добавляются без единого изменения в классе.
- LSP: все машины работают с одной и той же реализацией машины состояний.
- ISP: конфигурация переходов отделена от логики их выполнения.
- DIP: машина состояний не создаёт состояния, а получает их снаружи.

```
*/
```

```
using System;  
using System.Collections.Generic;  
using LasVegas.Domain.Interfaces;  
namespace LasVegas.Domain.StateMachine  
{
```

```
    /* Описание разрешённого перехода между состояниями. Конфигурация  
    переходов неизменна после создания, а сам переход является объектом-значением:  
    два перехода с одинаковыми параметрами From, To и Trigger считаются  
    одним и тем же переходом. Record-тип выбран именно для такой семантики. */
```

```
    public readonly record struct StateTransition<TState, TTrigger>(  
        TState From,  
        TState To,  
        TTrigger Trigger
```

```
) where TState : struct, Enum  
    where TTrigger : struct, Enum;
```

```
    /* Результат попытки перехода. Возврат булева значения был бы  
    недостаточен: вызывающей стороне нужно понимать причину отказа.  
    Разные причины (InvalidTrigger, GuardRejected) позволяют UI  
    корректно реагировать – показывать сообщение или просто игнорировать. */
```

```
    public enum TransitionResult  
    {
```

```
        Success,  
        InvalidTrigger,  
        GuardRejected,
```

```

MachineInError
}
/* Делегат guard-условия. Возвращает true, если переход разрешён.
Делегат, а не интерфейс, выбран потому, что guard – простая функция
без собственного состояния. Func<TState, TState, bool> получает
на вход текущее и целевое состояния. */
public delegate bool TransitionGuard<in TState>(TState from, TState to)
    where TState : struct, Enum;
/* Потокково-безопасная обобщённая конечная машина состояний.
Класс помечен как sealed, поскольку его поведение расширяется
через конфигурацию переходов, а не через наследование.
Блокировка (lock) используется из-за специфики казино:
переходы могут инициироваться из разных источников – ввод
игрока, таймер, команда администратора. Lock гарантирует
атомарность перехода. */
public sealed class GenericStateMachine<TState, TTrigger>
    where TState : struct, Enum
    where TTrigger : struct, Enum
{
    /* Текущее состояние объявлено как volatile для обеспечения
    видимости между потоками без полной блокировки на чтении. */
    private volatile TState _currentState;
    private readonly object _transitionLock = new();
    /* Конфигурация переходов в виде словаря. Ключом служит пара
    (From, Trigger), значением – кортеж из целевого состояния (To)
    и опционального guard-условия. Такой выбор структуры данных
    обеспечивает быстрый поиск. */
    private readonly Dictionary<(TState From, TTrigger Trigger),
        (TState To, TransitionGuard<TState>? Guard)> _transitions;
    /* События входа, выхода и смены состояния реализуют паттерн
    Observer без жёстких связей: машина состояний не знает,
    кто именно реагирует на эти изменения. */
    public event Action<TState, TState>? OnStateChanged;
    public event Action<TState>? OnEntry;
    public event Action<TState>? OnExit;
    public TState CurrentState => _currentState;
    /// <summary>
    /// Создаёт машину с начальным состоянием.
    /// </summary>
    public GenericStateMachine(TState initialState)
    {
        _currentState = initialState;
        _transitions = new();
    }
    /* Регистрирует разрешённый переход с опциональным guard-условием.
    Guard-проверка следует принципу DIP: машина состояний вызывает
    абстрактное условие и не имеет доступа к конкретным данным для проверки.
    Это позволяет ей, например, не знать ничего о балансе игрока,
    делегируя проверку достаточности средств внешнему guard'у. */
    public void AddTransition(
        TState from,
        TState to,

```

```

    TTrigger trigger,
    TransitionGuard<TState>? guard = null)
{
    var key = (from, trigger);
    /* Если переход уже зарегистрирован, запись перезаписывается.
       Это сделано по продуктовым причинам: конфигурация переходов
       может переопределяться администратором, и исключение здесь
       остановило бы загрузку всего зала. */
    _transitions[key] = (to, guard);
}

/* Пытается выполнить переход по триггеру и возвращает результат:
   успех или причину отказа. Возврат структуры TransitionResult
   вместо исключений выбран потому, что неудачный переход -
   нормальный ход работы. Если игрок нажал Spin во время
   обслуживания автомата, это ожидаемое поведение, а не ошибка.
   Исключения зарезервированы для действительно аварийных ситуаций. */
public TransitionResult TryTransition(TTrigger trigger)
{
    lock (_transitionLock)
    {
        var key = (_currentState, trigger);
        if (!_transitions.TryGetValue(key, out var transition))
        {
            return TransitionResult.InvalidTrigger;
        }
        /* Проверяем guard-условие, если оно задано.
           Guard отклонит переход, например, при недостаточном
           балансе для ставки и не позволит автомату
           оказаться в неконсистентном состоянии. */
        if (transition.Guard is not null &&
            !transition.Guard(_currentState, transition.To))
        {
            return TransitionResult.GuardRejected;
        }
        var previousState = _currentState;
        OnExit?.Invoke(previousState);
        _currentState = transition.To;
        OnEntry?.Invoke(_currentState);
        OnStateChanged?.Invoke(previousState, _currentState);
        return TransitionResult.Success;
    }
}

/* Принудительная установка состояния. Используется для
   восстановления после ошибок или для административных операций.
   Выделена в отдельный метод, а не в публичный сеттер, чтобы
   подчеркнуть её исключительность и обеспечить отслеживаемость. */
public void ForceState(TState state)
{
    lock (_transitionLock)
    {
        var previous = _currentState;
        OnExit?.Invoke(previous);
    }
}

```

```

        _currentState = state;
        OnEntry?.Invoke(state);
        OnStateChanged?.Invoke(previous, state);
    }
}
}
}

```

Конкретная машина: Slot Machine

Теперь применим обобщённую машину состояний к конкретному типу автомата – слоту. Мы определим его собственные состояния и триггеры, а логика вращения барабанов будет полностью отделена от визуализации.

```

// Файл Machines/SlotMachineStates.cs.
/* Состояния и триггеры слот-машины. Определены как отдельные enum'ы
вместо строк или чисел по трём причинам. Компилятор проверяет полноту
покрытия в switch-выражениях, среда разработки подсказывает допустимые
значения, а недопустимые состояния невозможно выразить синтаксически. */
namespace LasVegas.Domain.Machines.Slots
{
    /* Каждое состояние соответствует экрану, который видит игрок,
    и набору допустимых в этом состоянии действий. */
    public enum SlotState
    {
        Idle,
        Spinning,
        Evaluating,
        Paying,
        BonusRound,
        Maintenance
    }

    /* Триггеры названы в терминах домена, а не пользовательского интерфейса.
    Событие SpinRequested является доменным: кнопка, сенсор, голосовая
    команда – все они преобразуются в триггер. */
    public enum SlotTrigger
    {
        SpinRequested,
        SpinCompleted,
        EvaluationCompleted,
        PaymentCompleted,
        BonusTriggered,
        BonusCompleted,
        MaintenanceRequested,
        MaintenanceCompleted,
        ErrorDetected
    }
}

// Файл Machines/SlotMachine.cs.
/* Полная реализация слот-машины. Класс не наследует Node
и не ссылается на Godot, ибо это чистый C# с доменной логикой.
Принцип SRP соблюдается строго: класс отвечает только за правила игры.

```

```

Визуализация вынесена в отдельный класс SlotPresenter,
звук – в подсистему, слушающую IEventBus, а баланс –
в сервис IBalanceService. */
using System;
using System.Threading;
using System.Threading.Tasks;
using LasVegas.Domain.Interfaces;
using LasVegas.Domain.StateMachine;
using LasVegas.Domain.Machines.Slots;
namespace LasVegas.Domain.Machines
{
    /* Слот-машина: доменная логика без визуализации.
    Класс помечен как sealed, поскольку разные типы слотов
    (трёхбарабанные, пятибарабанные, мега-джекпот) отличаются
    исключительно конфигурацией, а не поведением. Поведение
    параметризуется через IMachineConfiguration. */
    public sealed class SlotMachine : IGameMachine
    {
        private readonly GenericStateMachine<SlotState, SlotTrigger> _stateMachine;
        private readonly IRngService _rng;
        private readonly IEventBus _eventBus;
        private IMachineConfiguration? _configuration;
        public MachineId Id { get; }
        public MachineState CurrentState => MapToMachineState(_stateMachine.CurrentState);
        public IMachineConfiguration Configuration =>
            _configuration ?? throw new InvalidOperationException(
                "Machine not initialized. Call Initialize() first.");
        /* Зависимости передаются через конструктор, а не добываются
        через сервис-локатор или Godot Autoload. Такой подход делает
        все зависимости видимыми в сигнатуре, позволяет компилятору
        проверить их наличие и упрощает тестирование с заменой
        реальных сервисов на mock-объекты. */
        public SlotMachine(
            MachineId id,
            IRngService rng,
            IEventBus eventBus)
        {
            Id = id;
            _rng = rng ?? throw new ArgumentNullException(nameof(rng));
            _eventBus = eventBus ?? throw new ArgumentNullException(nameof(eventBus));
            _stateMachine = new GenericStateMachine<SlotState, SlotTrigger>(
                SlotState.Idle);
            ConfigureTransitions();
            _stateMachine.OnStateChanged += OnStateChanged;
        }
        /* Конфигурация допустимых переходов машины состояний.
        Каждый вызов AddTransition добавляет одно правило вида
        "из состояния А в состояние В по триггеру Т".
        Переходы определены в коде, а не в data-driven конфигурации,
        потому что являются инвариантом безопасности: игрок не должен
        оказаться в состоянии, которое не поддерживается автоматом. */
        private void ConfigureTransitions()

```

```

{
    _stateMachine.AddTransition(
        SlotState.Idle, SlotState.Spining,
        SlotTrigger.SpinRequested,
        guard: (from, to) => _configuration is not null);
    _stateMachine.AddTransition(
        SlotState.Spining, SlotState.Evaluating,
        SlotTrigger.SpinCompleted);
    _stateMachine.AddTransition(
        SlotState.Evaluating, SlotState.Paying,
        SlotTrigger.EvaluationCompleted);
    _stateMachine.AddTransition(
        SlotState.Evaluating, SlotState.Idle,
        SlotTrigger.EvaluationCompleted);
    _stateMachine.AddTransition(
        SlotState.Paying, SlotState.Idle,
        SlotTrigger.PaymentCompleted);
    _stateMachine.AddTransition(
        SlotState.Paying, SlotState.BonusRound,
        SlotTrigger.BonusTriggered);
    _stateMachine.AddTransition(
        SlotState.BonusRound, SlotState.Idle,
        SlotTrigger.BonusCompleted);
    foreach (SlotState from in Enum.GetValues<SlotState>())
    {
        if (from != SlotState.Maintenance)
        {
            _stateMachine.AddTransition(from, SlotState.Maintenance,
                SlotTrigger.MaintenanceRequested);
        }
    }
    _stateMachine.AddTransition(
        SlotState.Maintenance, SlotState.Idle,
        SlotTrigger.MaintenanceCompleted);
}

public void Initialize(IMachineConfiguration config)
{
    ArgumentNullException.ThrowIfNull(config);
    _configuration = config;
    _eventBus.Publish(new MachineInitializedEvent(Id, config.MachineType));
}

/* Асинхронный запуск игровой сессии. Метод принимает
Cancellation token, что позволяет отменить длительный расчёт,
и возвращает Task - исключения не теряются, в отличие от async void.
Сам расчёт результата вынесен в Task.Run
(раздел 14.5 описывает многопоточную модель подробнее). */
public async Task<ISessionResult> StartSessionAsync(
    BetInfo bet, CancellationToken ct)
{
    if (_configuration is null)
        throw new InvalidOperationException("Machine not initialized.");
    if (bet.Amount < _configuration.MinBet ||

```



```

        bet.Amount > _configuration.MaxBet)
    {
        throw new ArgumentOutOfRangeException(
            nameof(bet),
            $"Bet must be between {_configuration.MinBet} " +
            $"and {_configuration.MaxBet}.");
    }
    var transitionResult = _stateMachine.TryTransition(
        SlotTrigger.SpinRequested);
    if (transitionResult != TransitionResult.Success)
    {
        throw new InvalidOperationException(
            $"Cannot start session: {transitionResult}. " +
            $"Current state: {_stateMachine.CurrentState}");
    }
    _eventBus.Publish(new BetPlacedEvent(Id, bet));
    try
    {
        var result = await Task.Run(
            () => CalculateOutcome(bet), ct);
        _stateMachine.TryTransition(SlotTrigger.SpinCompleted);
        _stateMachine.TryTransition(SlotTrigger.EvaluationCompleted);
        if (result.IsWin)
        {
            _stateMachine.TryTransition(
                SlotTrigger.PaymentCompleted);
            _eventBus.Publish(new WinEvent(
                Id, result.WinAmount, result.Outcome));
        }
        return result;
    }
    catch (OperationCanceledException)
    {
        _stateMachine.ForceState(SlotState.Idle);
        throw;
    }
    catch (Exception)
    {
        _stateMachine.ForceState(SlotState.Maintenance);
        throw;
    }
}

/* Расчёт результата вращения. Чистая функция,
зависящая только от генератора случайных чисел и конфигурации. */
private ISessionResult CalculateOutcome(BetInfo bet)
{
    var reelCount = 5;
    var symbolCount = _configuration.PayTable.Count;
    var symbols = new string[reelCount];
    for (int i = 0; i < reelCount; i++)
    {
        var idx = _rng.Next(symbolCount);

```

```

        symbols[i] = _configuration.PayTable.Keys.ElementAt(idx);
    }
    var outcome = string.Join("|", symbols);
    decimal multiplier = 0m;
    if (symbols.Distinct().Count() == 1)
    {
        _configuration.PayTable.TryGetValue(symbols[0], out multiplier);
    }
    else if (symbols.Distinct().Count() == 2)
    {
        var grouped = symbols.GroupBy(s => s)
            .OrderByDescending(g => g.Count());
        var bestSymbol = grouped.First().Key;
        _configuration.PayTable.TryGetValue(bestSymbol, out multiplier);
        multiplier = (decimal)((double)multiplier * 0.5);
    }
    var winAmount = bet.Amount * multiplier;
    return new SessionResult(
        WinAmount: winAmount,
        IsWin: winAmount > 0,
        Outcome: outcome,
        Multiplier: multiplier,
        CalculatedAt: DateTimeOffset.UtcNow);
}

public void Reset()
{
    _stateMachine.ForceState(SlotState.Idle);
    _eventBus.Publish(new MachineResetEvent(Id));
}

/* Отображение внутреннего состояния (SlotState) в публичное
(MachineState). Внутренние состояния могут быть богаче публичных:
например, SlotState.BonusRound транслируется в MachineState.Running. */
private static MachineState MapToMachineState(SlotState state) => state switch
{
    SlotState.Idle => MachineState.Idle,
    SlotState.Spining => MachineState.Running,
    SlotState.Evaluating => MachineState.Running,
    SlotState.Paying => MachineState.Paying,
    SlotState.BonusRound => MachineState.Running,
    SlotState.Maintenance => MachineState.Maintenance,
    _ => MachineState.Error
};

/* Обработчик изменения состояния. Публикует событие в шину
вместо прямого вызова методов UI, звука или аналитики.
Машина состояний не должна знать о потребителях этих событий,
она лишь фиксирует факт перехода. */
private void OnStateChanged(SlotState from, SlotState to)
{
    _eventBus.Publish(new MachineStateChangedEvent(
        Id, from.ToString(), to.ToString()));
}
}
}

```

Визуализация: SlotPresenter (связь с Godot)

Нижеследующий код показывает, как доменная логика SlotMachine соединяется с визуальным представлением в Godot. Класс SlotPresenter служит мостом между миром чистого C#, где живут правила игры, и миром Godot, где живут узлы сцены, кнопки и анимации. Presenter не содержит бизнес-логики: он подписывается на доменные события и обновляет представление. Этот паттерн называется Passive View – визуализация полностью пассивна и лишь отражает изменения, произошедшие в домене.

```
// Файл Presenters/SlotPresenter.cs.
/* Presenter для слот-машины, связывающий доменную логику с Godot-сценой.
Выделение логики в отдельный класс, а не размещение её прямо в Node,
продиктовано принципом SRP: Node отвечает за жизненный цикл и дерево сцены,
Presenter – за отображение доменных событий, а SlotMachine – за правила игры.
При замене движка Godot на другой переписывается только Presenter, тогда как
домен остаётся нетронутым. */
using Godot;
using LasVegas.Domain.Interfaces;
using LasVegas.Domain.Machines;
using LasVegas.Domain.Machines.Slots;
namespace LasVegas.Infrastructure.Presenters
{
    /* Ключевое слово partial требуется Godot 4.x для C#-скриптов,
привязанных к узлам. Генератор исходного кода создаёт вторую часть
класса с атрибутами [Export] и [Signal]. */
    public sealed partial class SlotPresenter : Node
    {
        private SlotMachine? _machine;
        private IEventBus? _eventBus;
        /* Ссылки на узлы сцены объявлены с атрибутом [Export],
что позволяет назначать их в редакторе и избегать строковых
путей типа GetNode("Path/To/Node"). Рефакторинг сцены
при таком подходе не ломает код. */
        [Export] public Label WinLabel { get; set; } = null!;
        [Export] public Label BalanceLabel { get; set; } = null!;
        [Export] public Button SpinButton { get; set; } = null!;
        [Export] public AnimationPlayer AnimPlayer { get; set; } = null!;
        /* Явная инициализация методом Initialize вместо использования
_RReady. Метод _Ready вызывается Godot при входе в дерево,
но порядок разрешения зависимостей в нём не гарантирован.
Ручная инициализация даёт полный контроль над порядком. */
        public void Initialize(
            SlotMachine machine,
            IEventBus eventBus)
        {
            _machine = machine;
            _eventBus = eventBus;
            _eventBus.Subscribe<MachineStateChangedEvent>(OnMachineStateChanged);
            _eventBus.Subscribe<WinEvent>(OnWin);
            SpinButton.Pressed += OnSpinPressed;
        }
    }
}
```

```

        UpdateVisualState();
    }
    /* Обработчик нажатия кнопки Spin содержит минимум логики:
    он лишь запускает процесс, а вся бизнес-логика остаётся
    внутри SlotMachine. */
    private async void OnSpinPressed()
    {
        if (_machine is null) return;
        SpinButton.Disabled = true;
        try
        {
            var bet = new BetInfo(
                Amount: 10.0m,
                Currency: "USD",
                Lines: 5);
            var result = await _machine.StartSessionAsync(
                bet, default);
            WinLabel.Text = result.IsWin
                ? $"WIN: {result.WinAmount:F2}"
                : "No win";
            if (result.IsWin)
            {
                AnimPlayer.Play("win_celebration");
            }
        }
        catch (Exception ex)
        {
            GD.PrintErr($"Slot error: {ex.Message}");
        }
        finally
        {
            SpinButton.Disabled = false;
        }
    }
    private void OnMachineStateChanged(MachineStateChangedEvent e)
    {
        if (e.MachineId != _machine?.Id) return;
        UpdateVisualState();
    }
    private void OnWin(WinEvent e)
    {
        if (e.MachineId != _machine?.Id) return;
        AnimPlayer.Play("coins_falling");
    }
    private void UpdateVisualState()
    {
        if (_machine is null) return;
        SpinButton.Disabled = _machine.CurrentState != MachineState.Idle;
    }
    /* Очистка подписок при выходе из дерева сцены строго обязательна.
    Удаление узла без отписки от EventBus приводит к вызовам методов
    уже несуществующего объекта, что является утечкой памяти
    и источником NullReferenceException. */

```

```

public override void _ExitTree()
{
    if (_eventBus is not null)
    {
        _eventBus.Unsubscribe<MachineStateChangedEvent>(
            OnMachineStateChanged);
        _eventBus.Unsubscribe<WinEvent>(OnWin);
    }
    SpinButton.Pressed -= OnSpinPressed;
    base._ExitTree();
}
}
}

```

13.3. Data-Driven подход: Resource-пресеты

Этот раздел показывает, как сделать архитектуру гибкой за счёт вынесения параметров из кода в конфигурационные данные. Вы увидите, как Godot Resource используется в качестве контейнера, который реализует доменный интерфейс `IMachineConfiguration`. Один и тот же класс `SlotMachine` может стать классическим слотом, мега-джекпотом или рулеткой просто потому, что получил другой пресет конфигурации. Никакого нового кода, никаких новых классов – только данные. Понимание этого раздела даст практику применения принципа DIP на границе «домен–инфраструктура» (домен определяет интерфейс конфигурации, а Godot Resource его реализует), навык создания фабрики пресетов как централизованного места для стандартных конфигураций, понимание паттерна загрузки с fallback (сначала кеш, затем файл, потом фабрика) и умение валидировать данные на границе слоёв до того, как они попадут в домен. Ключевая идея заключается в том, что тип автомата определяется данными, а не кодом. Добавление нового типа автомата = создание нового .tres-файла или нового метода в фабрике, а не написание нового класса.

Проблема хардкода конфигурации

В типичном прототипе параметры автомата зашиты прямо в код: количество барабанов, символы, множители, минимальная ставка. При добавлении нового автомата приходится менять код, перекомпилировать и перевыкладывать билд. В казино, где администратор может менять RTP автомата в реальном времени, такой подход неприемлем.

Решение: Godot Resource как контейнер конфигурации

Godot Resource – сериализуемый объект данных, который можно создавать и редактировать в инспекторе, сохранять как .tres-файл и загружать в ран-

тайме. Мы используем его как мост между редактором и доменным интерфейсом `IMachineConfiguration`.

```
// Файл Resources/MachineConfigurationResource.cs.
/* Godot Resource, реализующий доменный интерфейс IMachineConfiguration.
Интеграция с редактором Godot выбрана потому, что дизайнер может создавать
и редактировать пресеты без участия программиста. Resource также поддерживает
ссылки на другие ресурсы (например, текстуры символов) и сериализуется
автоматически. Реализация доменного интерфейса следует принципу DIP: домен
определяет, какие данные конфигурации ему нужны, а инфраструктура
предоставляет их через Resource. Домен не знает о Godot Resource,
а Resource знает о домене, и направление зависимости сохраняется
от инфраструктуры к домену. */
using System.Collections.Generic;
using Godot;
namespace LasVegas.Infrastructure.Resources
{
    /* Атрибут [GlobalClass] делает ресурс доступным в инспекторе Godot
    для создания новых экземпляров. Без этого атрибута Godot не покажет
    данный тип в диалоге "New Resource". */
    [GlobalClass]
    public sealed partial class MachineConfigurationResource
        : Resource, IMachineConfiguration
    {
        /* Каждое поле, помеченное [Export], становится доступным
        для редактирования в инспекторе Godot. Дизайнер видит эти поля
        как настраиваемые параметры конфигурации. */
        [Export] public string MachineType { get; set; } = "ClassicSlot";
        [Export] public double MinBet { get; set; } = 0.10;
        [Export] public double MaxBet { get; set; } = 1000.0;
        /* Return To Player – теоретический процент возврата.
        Диапазон 0.0-1.0 выбран вместо процентов потому, что внутри
        системы используются доли, а умножение на сто является
        заботой слоя представления. Атрибут PropertyHint.Range
        добавляет ползунок в инспекторе Godot. */
        [Export(PropertyHint.Range, "0.0,1.0")]
        public double Rtp { get; set; } = 0.95;
        [Export] public VolatilityLevel Volatility { get; set; }
            = VolatilityLevel.Medium;
        /* Таблица выплат. Ключом служит название символа, значением –
        множитель ставки. Использование Godot.Collections.Dictionary
        продиктовано тем, что Godot Resource не умеет напрямую
        сериализовать C#-словарь Dictionary<string, decimal>.
        Для сериализации применяется Godot-тип, а при обращении
        через интерфейс происходит конвертация. */
        [Export]
        public Godot.Collections.Dictionary PayTableRaw { get; set; } = new();
        /* Реализация доменного интерфейса. Конвертация Godot-типов
        в C#-типы происходит на границе слоёв. */
        decimal IMachineConfiguration.MinBet => (decimal)MinBet;
        decimal IMachineConfiguration.MaxBet => (decimal)MaxBet;
        IReadOnlyDictionary<string, decimal> IMachineConfiguration.PayTable
```

```

    {
        get
        {
            var result = new Dictionary<string, decimal>();
            foreach (var key in PayTableRaw.Keys)
            {
                if (key.VariantType == Variant.Type.String &&
                    PayTableRaw[key].VariantType == Variant.Type.Float)
                {
                    result[(string)key] = (decimal)(double)PayTableRaw[key];
                }
            }
            return result;
        }
    }
}

```

Пресеты для разных типов автоматов

Data-Driven подход означает, что каждый тип автомата представляет собой набор данных, а не отдельный класс. Мы создаём Resource-пресеты.

```

// Файл Resources/SlotMachinePresets.cs.
/* Фабрика пресетов для стандартных типов слотов. Инкапсуляция создания
в фабрике объясняется двумя причинами: здесь собраны все правила
валидации пресета в одном месте, и это единая точка для изменения логики
создания. В реальном проекте пресеты создаются в редакторе и сохраняются
как .tres-файлы, а данный класс служит программной альтернативой
для тестов и прототипирования. */
using Godot;
using LasVegas.Domain.Interfaces;
namespace LasVegas.Infrastructure.Resources
{
    /* Статический класс оправдан тем, что пресеты являются чистыми
    данными и не требуют хранения состояния экземпляра. */
    public static class SlotMachinePresets
    {
        /* Создаёт пресет классического трёхбарабанного слота
        с высокой частотой небольших выигрышей. */
        public static MachineConfigurationResource ClassicSlot()
        {
            var resource = new MachineConfigurationResource
            {
                MachineType = "ClassicSlot",
                MinBet = 0.25,
                MaxBet = 100.0,
                Rtp = 0.95,
                Volatility = VolatilityLevel.Low,
                PayTableRaw = new Godot.Collections.Dictionary
                {
                    {"cherry", 5.0},

```

```

        {"lemon", 8.0},
        {"orange", 10.0},
        {"plum", 15.0},
        {"bell", 20.0},
        {"bar", 50.0},
        {"seven", 100.0},
        {"diamond", 500.0}
    }
};
return resource;
}

/* Создаёт пресет высоковолатильного пятибарабанного слота
с редкими, но крупными выигрышами. */
public static MachineConfigurationResource MegaJackpot()
{
    var resource = new MachineConfigurationResource
    {
        MachineType = "MegaJackpot",
        MinBet = 1.0,
        MaxBet = 500.0,
        Rtp = 0.92,
        Volatility = VolatilityLevel.High,
        PayTableRaw = new Godot.Collections.Dictionary
        {
            {"ten", 2.0},
            {"jack", 3.0},
            {"queen", 5.0},
            {"king", 8.0},
            {"ace", 12.0},
            {"wild", 25.0},
            {"scatter", 50.0},
            {"mega", 1000.0}
        }
    };
    return resource;
}

/* Создаёт пресет для европейской рулетки. Рулетка использует
ту же архитектуру (IGameMachine), но получает другую конфигурацию. */
public static MachineConfigurationResource EuropeanRoulette()
{
    var resource = new MachineConfigurationResource
    {
        MachineType = "EuropeanRoulette",
        MinBet = 1.0,
        MaxBet = 10000.0,
        /* RTP 0.973 соответствует вероятности 36/37
        для европейской рулетки с одним zero. */
        Rtp = 0.973,
        Volatility = VolatilityLevel.Medium,
        PayTableRaw = new Godot.Collections.Dictionary
        {
            {"straight", 36.0},

```



```

        {"split", 18.0},
        {"street", 12.0},
        {"corner", 9.0},
        {"red_black", 2.0},
        {"odd_even", 2.0},
        {"dozen", 3.0}
    }
};
return resource;
}
/* Создаёт пресет для видеопокера. В таблице выплат используются
комбинации покерных рук вместо символов, характерных для слотов. */
public static MachineConfigurationResource VideoPoker()
{
    var resource = new MachineConfigurationResource
    {
        MachineType = "VideoPoker",
        MinBet = 0.50,
        MaxBet = 50.0,
        /* RTP 0.984 соответствует оптимальной стратегии
        для разновидности Jacks or Better. */
        Rtp = 0.984,
        Volatility = VolatilityLevel.Low,
        PayTableRaw = new Godot.Collections.Dictionary
        {
            {"royal_flush", 800.0},
            {"straight_flush", 50.0},
            {"four_of_kind", 25.0},
            {"full_house", 9.0},
            {"flush", 6.0},
            {"straight", 4.0},
            {"three_of_kind", 3.0},
            {"two_pair", 2.0},
            {"jacks_or_better", 1.0}
        }
    };
    return resource;
}
}
}
}

```

Загрузчик конфигураций: ResourceProvider

// Файл Services/ResourceConfigurationProvider.cs.
/* Сервис загрузки конфигураций из Godot Resources. Вынесение логики загрузки в отдельный класс, а не использование GD.Load внутри каждого автомата, продиктовано принципом SRP. Загрузка ресурсов – самостоятельная ответственность. Кроме того, этот класс кеширует каждый пресет, загружая его только один раз, и выполняет валидацию конфигурации перед передачей в домен. Следуя принципу DIP, домен получает IMachineConfiguration и не знает, что источником послужил Godot Resource. */
using System;

```

using System.Collections.Generic;
using Godot;
namespace LasVegas.Infrastructure.Services
{
    /* Кеширование конфигураций критически важно в сценарии казино,
    где на полу может стоять более сотни автоматов с одинаковым пресетом.
    Загружать .tres-файл сто раз не имеет смысла. */
    public sealed class ResourceConfigurationProvider : IDisposable
    {
        private readonly Dictionary<string, IMachineConfiguration> _cache = new();
        private bool _disposed;

        /* Загружает конфигурацию по имени пресета. Метод поддерживает
        два пути загрузки: из файла (для продакшена) и из фабрики пресетов
        (для прототипирования). Алгоритм проверяет кеш, затем пытается
        загрузить файл, в случае неудачи обращается к фабрике. */
        public IMachineConfiguration Load(string machineType)
        {
            if (_cache.TryGetValue(machineType, out var cached))
            {
                return cached;
            }
            var path = $"res://configs/machines/{machineType}.tres";
            if (ResourceLoader.Exists(path))
            {
                var resource = ResourceLoader.Load<MachineConfigurationResource>(path);
                if (resource is not null)
                {
                    var config = ValidateAndWrap(resource);
                    _cache[machineType] = config;
                    return config;
                }
            }
            var preset = machineType switch
            {
                "ClassicSlot" => SlotMachinePresets.ClassicSlot(),
                "MegaJackpot" => SlotMachinePresets.MegaJackpot(),
                "EuropeanRoulette" => SlotMachinePresets.EuropeanRoulette(),
                "VideoPoker" => SlotMachinePresets.VideoPoker(),
                _ => throw new ArgumentException(
                    $"Unknown machine type: {machineType}")
            };
            var wrapped = ValidateAndWrap(preset);
            _cache[machineType] = wrapped;
            return wrapped;
        }

        /* Валидация конфигурации. Невалидная конфигурация является
        багом данных, а не багом логики, поэтому проверка происходит
        на границе слоёв, до того как данные попадут в домен.
        Инварианты, проверяемые здесь: MinBet должен быть положительным,
        MaxBet не может быть меньше MinBet, а RTP обязан находиться
        в диапазоне от нуля до единицы. */
        private static IMachineConfiguration ValidateAndWrap(

```

```

MachineConfigurationResource resource)
{
    if (resource.MinBet <= 0)
        throw new InvalidOperationException(
            "MinBet must be positive.");
    if (resource.MaxBet < resource.MinBet)
        throw new InvalidOperationException(
            "MaxBet must be >= MinBet.");
    if (resource.Rtp is < 0 or > 1)
        throw new InvalidOperationException(
            "RTP must be between 0 and 1.");
    /* Resource напрямую реализует IMachineConfiguration,
    поэтому возвращается без дополнительной обёртки. */
    return resource;
}

public void Dispose()
{
    if (!_disposed)
    {
        _cache.Clear();
        _disposed = true;
    }
}
}
}
}

```

13.4. Событийная модель: Event Bus для децентрализованной синхронизации

Раздел посвящён одной из центральных проблем любой сложной системы: как сделать так, чтобы подсистемы обменивались информацией, не зная друг о друге напрямую. В казино, когда игрок выигрывает, об этом должны узнать одновременно подсистема баланса (чтобы начислить деньги), звуковая подсистема (чтобы воспроизвести фанфары), UI (чтобы показать анимацию), аналитика (чтобы записать событие) и система достижений (чтобы проверить рекорд). Если каждый автомат будет напрямую вызывать методы пяти разных сервисов, изменение любого из них сломает всю систему. Event Bus решает эту проблему через паттерн Publisher/Subscriber. Отправитель события не знает, кто его получит. Получатель не знает, кто его отправил. Они связаны только через тип события. Такая архитектура устраняет жёсткие связи и делает систему открытой для расширения: новая подсистема просто подписывается на нужные события, не трогая существующий код.

Понимание этого раздела даст читателю практическую реализацию типизированной шины событий с потокобезопасностью, навык проектирования доменных событий как неизменяемых record-типов, понимание моста

между доменным Event Bus и Godot Signals (SignalBridge), а также практику обработки исключений в обработчиках таким образом, чтобы ошибка в аналитике не блокировала начисление выигрыша. Ключевыми компонентами архитектуры выступают маркерный интерфейс IDomainEvent, реализация шины EventBus и мост в Godot – SignalBridge.

Проблема жёстких связей между подсистемами

Когда игровой автомат напрямую вызывает методы баланса, звука и UI, появляется жёсткая связанность. Изменение API баланса ломает все автоматы, а добавление новой подсистемы, например аналитики, требует изменения кода каждого автомата. Это прямое нарушение принципа ОСР, создающее хрупкую систему.

Решение: Event Bus как посредник

Event Bus реализует паттерн Publisher/Subscriber, где отправитель и получатель ничего не знают друг о друге. Их связывает исключительно тип события.

```
// Файл Events/DomainEvents.cs.
/* Доменные события, представленные как неизменяемые record-типы.
Record выбран потому, что события являются снимками данных и после
публикации их содержимое не должно меняться. Record автоматически
обеспечивает неизменяемость и семантику сравнения по значению.
Размещение всех событий в отдельном пространстве имён превращает их
в публичный контракт, упрощая навигацию для любой подсистемы. */
using System;
using LasVegas.Domain.Interfaces;
namespace LasVegas.Domain.Events
{
    /* Базовый интерфейс для всех доменных событий. Пустой маркерный
    интерфейс позволяет EventBus фильтровать подписки по типу события
    через дженерики. */
    public interface IDomainEvent { }

    /* Автомат инициализирован и готов к работе.
    На это событие подписываются CasinoFloor (для учёта)
    и AnalyticsService (для статистики). */
    public sealed record MachineInitializedEvent(
        MachineId MachineId,
        string MachineType
    ) : IDomainEvent;

    /* Игрок разместил ставку.
    На это событие подписываются BalanceService (списать средства),
    SoundService (воспроизвести звук монет) и AnalyticsService
    (записать транзакцию). */
    public sealed record BetPlacedEvent(
        MachineId MachineId,
        BetInfo Bet
    ) : IDomainEvent;

    /* Игрок выиграл.
```

```

На это событие подписываются BalanceService (начислить выигрыш),
SoundService (звук выигрыша), UIService (анимация)
и AchievementService (проверить достижение). */
public sealed record WinEvent(
    MachineId MachineId,
    decimal Amount,
    string Outcome
) : IDomainEvent;
/* Состояние автомата изменилось.
На это событие подписываются Presenter (обновить UI)
и MaintenanceService (отслеживать ошибки). */
public sealed record MachineStateChangedEvent(
    MachineId MachineId,
    string PreviousState,
    string NewState
) : IDomainEvent;
/* Автомат сброшен в начальное состояние. */
public sealed record MachineResetEvent(
    MachineId MachineId
) : IDomainEvent;
}
// Файл Events/IEventBus.cs.
/* Контракт шины событий. Выбор интерфейса продиктован требованиями
тестируемости (в unit-тестах подставляется mock) и заменяемости
(можно перейти на другую реализацию без изменения домена). Кроме того,
интерфейс позволяет управлять жизненным циклом шины, делая её,
например, scored для конкретной сессии. */
using System;
namespace LasVegas.Domain.Interfaces
{
    /* Шина событий, реализующая типизированный publish/subscribe.
    Использование обобщённых методов гарантирует type safety:
    опубликовать WinEvent в подписку на BetPlacedEvent не позволит
    компилятор. Такой подход надёжнее строчковых каналов. */
    public interface IEventBus
    {
        /* Подписка на события указанного типа.
        Возврат IDisposable позволяет применять using и автоматически
        отписываться при выходе из области видимости, предотвращая
        утечки подписчиков. */
        IDisposable Subscribe<TEvent>(Action<TEvent> handler)
            where TEvent : IDomainEvent;
        /* Явная отписка конкретного обработчика. Дополняет механизм
        IDisposable и служит страховочной сетью для Godot, где метод
        _ExitTree не всегда вызывается предсказуемо. */
        void Unsubscribe<TEvent>(Action<TEvent> handler)
            where TEvent : IDomainEvent;
        /* Синхронная публикация события. Все обработчики вызываются
        в текущем потоке, что гарантирует предсказуемый порядок
        обработки и сохранение исключений. В контексте Godot основной
        поток является единственным местом для обновления UI,
        а для асинхронной обработки сам подписчик оборачивает

```

```

        свою логику в Task.Run. */
        void Publish<TEvent>(TEvent evt)
            where TEvent : IDomainEvent;
    }
}

// Файл Events/EventBus.cs.
/* Реализация шины событий. Потокобезопасность достигается
за счёт использования ConcurrentDictionary для хранения подписок
и блокировки lock при итерации по списку обработчиков.
Обработка исключений построена так, что ошибка в одном обработчике
не блокирует остальные: в казино выигрыш не должен пропасть
из-за сбоя в аналитике. */
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace LasVegas.Infrastructure.Events
{
    /* Глобальная шина событий, помеченная как sealed.
    Потокобезопасность реализована через ConcurrentDictionary
    для добавления и удаления подписок из разных потоков,
    а список обработчиков защищён lock для атомарности при итерации. */
    public sealed class EventBus : IEventBus
    {
        private readonly ConcurrentDictionary<Type, List<Delegate>> _subscriptions = new();
        /* Подписка на события указанного типа. Возвращает токен подписки,
        чтобы вызывающая сторона могла автоматически отписаться при выходе
        из области видимости через using. Такое поведение критически важно
        в Godot, где узлы могут быть удалены из дерева в любой момент. */
        public IDisposable Subscribe<TEvent>(Action<TEvent> handler)
            where TEvent : Domain.Interfaces.IDomainEvent
        {
            ArgumentNullException.ThrowIfNull(handler);
            var handlers = _subscriptions.GetOrAdd(
                typeof(TEvent),
                _ => new List<Delegate>());
            lock (handlers)
            {
                handlers.Add(handler);
            }
            return new Subscription<TEvent>(this, handler);
        }
        public void Unsubscribe<TEvent>(Action<TEvent> handler)
            where TEvent : Domain.Interfaces.IDomainEvent
        {
            if (_subscriptions.TryGetValue(typeof(TEvent), out var handlers))
            {
                lock (handlers)
                {
                    handlers.Remove(handler);
                }
            }
        }
    }
}

```

```

/* Публикация события. Каждый обработчик защищён try/catch,
чтобы исключение в одном из них не прерывало цепочку вызовов.
Ошибка в аналитике не должна блокировать начисление выигрыша.
Логирование ошибки выполняется, а сама ошибка не подавляется
молча. Список обработчиков копируется перед итерацией, поскольку
обработчик может отписаться внутри своего вызова, что привело бы
к модификации коллекции во время итерации. */
public void Publish<TEvent>(TEvent evt)
    where TEvent : Domain.Interfaces.IDomainEvent
{
    ArgumentNullException.ThrowIfNull(evt);
    if (!_subscriptions.TryGetValue(typeof(TEvent), out var handlers))
        return;
    List<Delegate> snapshot;
    lock (handlers)
    {
        snapshot = new List<Delegate>(handlers);
    }
    foreach (var handler in snapshot)
    {
        try
        {
            ((Action<TEvent>)handler)(evt);
        }
        catch (Exception ex)
        {
            GD.PrintErr(
                $"Event handler error for {typeof(TEvent).Name}: " +
                $"{ex.Message}");
        }
    }
}

/* Токен подписки. При Dispose отписывает обработчик.
Interlocked гарантирует, что Dispose выполнится только один раз,
даже при параллельных вызовах. */
private sealed class Subscription<TEvent> : IDisposable
    where TEvent : Domain.Interfaces.IDomainEvent
{
    private readonly EventBus _bus;
    private readonly Action<TEvent> _handler;
    private int _disposed;
    public Subscription(EventBus bus, Action<TEvent> handler)
    {
        _bus = bus;
        _handler = handler;
    }
    public void Dispose()
    {
        if (Interlocked.Exchange(ref _disposed, 1) == 0)
        {
            _bus.Unsubscribe(_handler);
        }
    }
}

```

```

    }
}
}
/* Заглушка для GD.PrintErr в доменном слое.
В реальном проекте заменяется на интерфейс ILogger. */
internal static class GD
{
    public static void PrintErr(string msg) =>
        System.Diagnostics.Debug.WriteLine($"[ERR] {msg}");
}
}

```

Мост между Event Bus и Godot Signals

Godot Signals представляют собой нативный механизм сцены. Event Bus является доменным механизмом. Они не конкурируют, а дополняют друг друга. Мост SignalBridge транслирует события из одного мира в другой.

```

// Файл Bridges/SignalBridge.cs.
/* Мост между доменным Event Bus и Godot Signals. Доменный Event Bus
не знает о Godot, а Godot-сцены не знают о доменном Event Bus.
Мост соединяет их, не нарушая разделение слоёв и следуя принципу ISP,
так как подписывается только на те события, которые действительно
нужны для трансляции в Godot Signals. */
using Godot;
using LasVegas.Domain.Events;
using LasVegas.Domain.Interfaces;
namespace LasVegas.Infrastructure.Bridges
{
    /* Транслирует доменные события в Godot Signals.
    Подключается как Autoload, что является Godot-способом создания
    синглтона, живущего всё время жизни приложения и доступного
    из любой сцены с учётом жизненного цикла движка. */
    public sealed partial class SignalBridge : Node
    {
        private IEventBus? _eventBus;
        private readonly System.Collections.Generic.List<IDisposable>
            _subscriptions = new();
        /* Godot Signals. Атрибут [Signal] генерирует Godot-обёртку.
        Полезная нагрузка передаётся в виде строк, поскольку Godot
        Signals поддерживают только Variant-совместимые типы,
        а доменные типы необходимо предварительно сериализовать. */
        [Signal]
        public delegate void BalanceChangedSignalHandler(long playerId, float newBalance);
        [Signal]
        public delegate void WinOccurredSignalHandler(string machineId, float amount);
        [Signal]
        public delegate void MachineStateChangedSignalHandler(
            string machineId, string newState);
        /* Инициализация моста. Вызывается из GameController. */
        public void Initialize(IEventBus eventBus)
        {

```



```

        _eventBus = eventBus;
        _subscriptions.Add(
            _eventBus.Subscribe<WinEvent>(OnWinEvent));
        _subscriptions.Add(
            _eventBus.Subscribe<MachineStateChangedEvent>(
                OnStateChanged));
    }
    /* Транслирует доменное событие WinEvent в Godot Signal.
    Благодаря этому любой узел сцены может подписаться
    через Godot-механизм, не зная о существовании Event Bus. */
    private void OnWinEvent(WinEvent e)
    {
        EmitSignal(SignalName.WinOccurred,
            e.MachineId.Value.ToString(),
            (float)e.Amount);
    }
    private void OnStateChanged(MachineStateChangedEvent e)
    {
        EmitSignal(SignalName.MachineStateChanged,
            e.MachineId.Value.ToString(),
            e.NewState);
    }
    public override void _ExitTree()
    {
        foreach (var sub in _subscriptions)
        {
            sub.Dispose();
        }
        _subscriptions.Clear();
        base._ExitTree();
    }
}

```

13.5. Многопоточность и RNG: безопасные вычисления вероятностей

Раздел посвящён одной из самых актуальных по важности проблем игровой архитектуры: как выполнять тяжёлые вычисления, не блокируя основной поток рендеринга. Представьте казино с сотней автоматов, каждый из которых при вращении барабанов рассчитывает вероятности прямо в главном потоке. Игра замрёт на время расчётов, анимация станет рваной, а в худшем случае произойдёт полное зависание. Для симуляции, где на каждый кадр отводится около шестнадцати миллисекунд, такая блокировка неприемлема.

Чистые вычисления – генерация случайных чисел, оценка комбинаций – выносятся в фоновые потоки через `Task.Run`, а результат возвращается в главный поток через `await`. Для корректной отмены операций применяется `Cancella-`

tionToken. Читатель, освоивший этот раздел, научится проектировать потокобезопасный генератор случайных чисел на базе ThreadLocal<Random>, поймёт, почему System.Random не подходит для многопоточной среды, и создаст криптографически стойкую альтернативу через RandomNumberGenerator. Кроме того, вы узнаете, как ограничивать параллелизм через SemaphoreSlim, чтобы тысячи одновременных задач не перегрузили пул потоков, и как использовать Task.WhenAll для массовой параллельной обработки игровых сессий.

Проблема: однопоточная природа Godot

Godot, как и большинство игровых движков, не является потокобезопасным. Все операции с узлами, ресурсами и API движка обязаны выполняться в главном потоке. Однако расчёт вероятностей, оценка комбинаций и генерация случайных чисел для сотни автоматов относятся к категории CPU-интенсивной работы, которая не должна блокировать рендер.

Решение: Task.Run для вычислений и CancellationToken для отмены

Мы выносим чистые вычисления в фоновые потоки через Task.Run, а результат возвращаем в главный поток через await. CancellationToken позволяет отменить вычисления при выходе из сцены или при закрытии приложения.

```
// Файл Services/IRngService.cs.
/* Контракт сервиса генерации случайных чисел. Критическая важность
RNG для казино диктует два требования: он должен быть криптографически
стойким и аудируемым. Вынесение за интерфейс позволяет заменить генератор
для тестирования, подставляя заранее предопределённые результаты,
и делает потокобезопасность частью публичного контракта, а не деталью
реализации. */
/* Два метода выбраны по принципу разделения интерфейсов. Разным
подсистемам нужны случайные величины в разной форме: целые в диапазоне
и числа с плавающей точкой. Потребность в байтовых массивах на уровне
домена отсутствует, поэтому соответствующий метод не включён в контракт,
чтобы не раздувать его. Если такая потребность появится – соответствующий
интерфейс будет добавлен и расширен через композицию, а не через
наследование. */
using System;
namespace LasVegas.Domain.Interfaces
{
    public interface IRngService
    {
        /// <summary>
        /// Возвращает случайное целое в диапазоне [0, maxValue).
        /// Потокобезопасен: можно вызывать из любого потока.
        /// </summary>
        int Next(int maxValue);
        /// <summary>
        /// Возвращает случайное число двойной точности в диапазоне [0.0, 1.0).

```

```

    /// Потокбезопасен.
    /// </summary>
    double NextDouble();
}
}

// Файл Services/ThreadSafeRngService.cs.
/* Потокбезопасная реализация RNG. Использование встроенного
System.Random в многопоточной среде, например при одновременном
вызове из сотни фоновых задач, небезопасно. Concurrent-вызовы
способны повредить внутреннее состояние генератора или привести
к повторению одинаковых значений. Блокировка lock при большом
числе потоков создаст серьёзную конкуренцию. Решением служит
ThreadLocal<Random>, который предоставляет каждому потоку
собственный экземпляр генератора с уникальным начальным зерном,
что полностью устраняет конкуренцию. */
/* ВАЖНО: данный класс не предоставляет криптографической стойкости.
Для реального казино необходимо использовать RandomNumberGenerator
из пространства имён System.Security.Cryptography.
Здесь приведена упрощённая версия для иллюстрации архитектуры. */
using System;
using System.Threading;
namespace LasVegas.Infrastructure.Services
{
    /* Класс помечен как sealed, поскольку наследование не предусмотрено.
    Если потребуется другая стратегия – будет создана другая реализация
    IRngService. */
    public sealed class ThreadSafeRngService : IRngService, IDisposable
    {
        /* ThreadLocal гарантирует, что каждый поток получит собственный
        экземпляр Random. Начальное зерно формируется на основе сочетания
        текущего системного времени и хеша идентификатора потока.
        Произведение Environment.TickCount на 31 плюс ManagedThreadId
        сводит к минимуму вероятность совпадения зерна у разных потоков. */
        private readonly ThreadLocal<Random> _localRandom;
        private bool _disposed;
        public ThreadSafeRngService()
        {
            _localRandom = new ThreadLocal<Random>(() =>
            {
                var seed = unchecked(
                    Environment.TickCount * 31 +
                    Thread.CurrentThread.ManagedThreadId);
                return new Random(seed);
            });
        }
        public int Next(int maxValue)
        {
            ObjectDisposedException.ThrowIf(_disposed, this);
            if (maxValue <= 0)
                throw new ArgumentOutOfRangeException(nameof(maxValue));
            return _localRandom.Value!.Next(maxValue);
        }
        public double NextDouble()

```

```

    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        return _localRandom.Value!.NextDouble();
    }
    public void Dispose()
    {
        if (!_disposed)
        {
            _localRandom.Dispose();
            _disposed = true;
        }
    }
}

```

/* Криптографически стойкий RNG для production.

Использует RandomNumberGenerator из System.Security.Cryptography.

Два класса реализуют один и тот же интерфейс: ThreadSafeRngService

служит для разработки и тестирования (быстрый, детерминированный

при подмене), а CryptoRngService – для продакшена (стойкий,

но более медленный). Замена одной реализации на другую

выполняется одной строкой в контейнере внедрения зависимостей. */

```
public sealed class CryptoRngService : IRngService, IDisposable
```

```

{
    private readonly System.Security.Cryptography.RandomNumberGenerator _rng =
        System.Security.Cryptography.RandomNumberGenerator.Create();
    private bool _disposed;
    public int Next(int maxValue)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        if (maxValue <= 0)
            throw new ArgumentOutOfRangeException(nameof(maxValue));
        var bytes = new byte[4];
        _rng.GetBytes(bytes);
        var value = BitConverter.ToUInt32(bytes, 0);
        return (int)(value % (uint)maxValue);
    }
    public double NextDouble()
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        var bytes = new byte[8];
        _rng.GetBytes(bytes);
        var value = BitConverter.ToUInt64(bytes, 0);
        return value / (double)ulong.MaxValue;
    }
    public void Dispose()
    {
        if (!_disposed)
        {
            _rng.Dispose();
            _disposed = true;
        }
    }
}
}

```

Массовый расчёт: CasinoSessionProcessor

Когда на полу находятся сотни автоматов, каждый из которых способен вращать барабаны одновременно, требуется координатор, управляющий параллельными вычислениями.

```
// Файл Services/CasinoSessionProcessor.cs.
/* Координатор параллельных игровых сессий. Обрабатывает запросы
от десятков или сотен автоматов одновременно, не блокируя главный
поток Godot. Централизованное управление даёт единую точку
для CancellationToken, ограничения параллелизма через SemaphoreSlim,
стандартную обработку ошибок и мониторинг текущей загрузки.
Потокобезопасность достигается за счёт ConcurrentDictionary
для учёта незавершённых задач и SemaphoreSlim для контроля
одновременного выполнения. */
/* Отдельный класс, а не рассыпанные по автоматам вызовы Task.WhenAll,
оправдан централизацией следующих обязанностей: управление общим
токеном отмены, глобальное ограничение параллелизма, единая обработка
ошибок и прозрачный мониторинг количества сессий, находящихся
в обработке. */
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using LasVegas.Domain.Interfaces;
namespace LasVegas.Infrastructure.Services
{
    public sealed record BatchSessionResult(
        int TotalRequested,
        int Completed,
        int Failed,
        IReadOnlyList<ISessionResult> Results
    );
    /* Координатор параллельных игровых сессий. Ограничение
    параллелизма по умолчанию равно количеству процессорных ядер,
    потому что Task.Run использует пул потоков. Одновременный запуск
    тысячи задач приведёт к созданию множества потоков и интенсивному
    переключению контекста, что замедлит общую работу. Оптимальным
    числом для CPU-нагруженных операций является именно
    Environment.ProcessorCount. */
    public sealed class CasinoSessionProcessor : IDisposable
    {
        private readonly SemaphoreSlim _concurrencyLimiter;
        private readonly ConcurrentDictionary<MachineId, Task<ISessionResult>> _pending;
        private CancellationTokenSource? _cts;
        private bool _disposed;
        public CasinoSessionProcessor(int? maxConcurrency = null)
        {
            var concurrency = maxConcurrency ?? Environment.ProcessorCount;
            _concurrencyLimiter = new SemaphoreSlim(concurrency, concurrency);
            _pending = new ConcurrentDictionary<MachineId, Task<ISessionResult>>();
        }
    }
}
```

```

        _cts = new CancellationTokenSource();
    }
    /* Запускает сессию на автомате в фоновом потоке, не блокируя
    вызывающий код. SemaphoreSlim.WaitAsync гарантирует, что
    количество одновременно выполняемых задач не превысит заданный
    лимит, предотвращая перегрузку пула потоков и обеспечивая
    предсказуемую производительность. */
    public async Task<ISessionResult> ProcessSessionAsync(
        IGameMachine machine,
        BetInfo bet)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        var ct = _cts?.Token ?? CancellationToken.None;
        await _concurrencyLimiter.WaitAsync(ct);
        try
        {
            var task = machine.StartSessionAsync(bet, ct);
            _pending[machine.Id] = task;
            var result = await task;
            return result;
        }
        finally
        {
            _pending.TryRemove(machine.Id, out _);
            _concurrencyLimiter.Release();
        }
    }
    /* Массовая обработка сессий для нескольких автоматов.
    Task.WhenAll запускает все задачи параллельно в рамках
    заданного ограничения и ожидает их завершения. В отличие
    от последовательного цикла, который обработал бы N автоматов
    за время O(N * время_сессии), WhenAll выполняет всю группу
    за время, близкое к времени одной сессии, при условии
    достаточного параллелизма. */
    public async Task<BatchSessionResult> ProcessBatchAsync(
        IReadOnlyList<IGameMachine Machine, BetInfo Bet> sessions)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        var tasks = new List<Task<ISessionResult>>();
        var results = new List<ISessionResult>();
        var failed = 0;
        foreach (var (machine, bet) in sessions)
        {
            tasks.Add(ProcessSessionAsync(machine, bet));
        }
        try
        {
            var completed = await Task.WhenAll(tasks);
            results.AddRange(completed);
        }
        catch (AggregateException)
        {

```

```

        foreach (var task in tasks)
        {
            if (task.IsCompletedSuccessfully)
                results.Add(task.Result);
            else if (task.IsFaulted)
                failed++;
        }
    }
    return new BatchSessionResult(
        TotalRequested: sessions.Count,
        Completed: results.Count,
        Failed: failed,
        Results: results);
}

/* Отменяет все незавершённые сессии. Вызов производится
при закрытии приложения или в случае возникновения ошибки.
Использование кооперативной отмены через CancellationToken
предпочтительнее деструктивных методов вроде Thread.Abort,
которые могут оставить данные в неконсистентном состоянии,
что особенно опасно в контексте финансовых операций казино. */
public void CancelAll()
{
    _cts?.Cancel();
    _cts?.Dispose();
    _cts = new CancellationTokenSource();
}

public int PendingCount => _pending.Count;
public void Dispose()
{
    if (!_disposed)
    {
        _cts?.Cancel();
        _cts?.Dispose();
        _concurrencyLimiter.Dispose();
        _pending.Clear();
        _disposed = true;
    }
}
}
}

```

13.6. Оптимизация в условиях Лас-Вегаса: проверенные практики

Раздел представляет собой сборник проверенных решений для систем, где на экране одновременно работают тысячи объектов, а задержка в шестнадцать миллисекунд отделяет плавный игровой опыт от рваного. Мы рассмотрим

три конкретные проблемы: устранение GC-пауз при массовом создании визуальных объектов с помощью object pooling, минимизацию аллокаций в горячих путях через Span<T> и stackalloc и общую структуру проекта, делающую зависимости явными. Каждый паттерн демонстрируется через конкретный пример из контекста казино: пул монет для анимации выигрыша, allocation-free расчёт комбинаций, Composition Root как единая точка сборки системы. После изучения раздела разработчик будет понимать, почему оператор new в горячем пути превращается в бомбу замедленного действия и как Object Pool решает эту проблему, как Span<T> и stackalloc устраняют heap-аллокации без потери читаемости и как Composition Root собирает все компоненты воедино, не допуская появления new для сервисов за своими пределами.

Object Pooling для визуальных элементов

Когда сотня автоматов одновременно показывают анимацию падающих монет, создавать и уничтожать объекты для каждой монеты означает прямой путь к GC-паузам. Object Pool переиспользует объекты, устраняя аллокации.

```
// Файл Pooling/GenericObjectPool.cs.
/* Обобщённый пул объектов. Использование дженериков позволяет
хранить в пуле любые ссылочные типы – монеты, частицы, символы
барабанов – с единой механикой переиспользования. Для хранения
выбран ConcurrentBag, оптимизированный под частые операции
добавления и извлечения из разных потоков без блокировки.
В отличие от Stack<T> с lock, здесь отсутствует contention. */
using System;
using System.Collections.Concurrent;
namespace LasVegas.Infrastructure.Pooling
{
    /* Делегат фабрики объектов. Выбор делегата вместо интерфейса
    обусловлен тем, что создание объекта – это функция, а не объект
    с самостоятельным состоянием. */
    public delegate T PoolObjectFactory<out T>() where T : class;
    /* Обобщённый пул объектов с предварительным созданием.
    Предварительное заполнение пула оправдано тем, что в момент
    проигрыша анимации выигрыша объекты должны быть уже готовы
    к использованию. Создание "на лету" вызывает нежелательный
    всплеск аллокаций, который сборщик мусора обнаружит
    в следующем кадре. При уничтожении пула все объекты,
    реализующие IDisposable, корректно очищаются. */
    public sealed class GenericObjectPool<T> : IDisposable
        where T : class
    {
        private readonly ConcurrentBag<T> _objects;
        private readonly PoolObjectFactory<T> _factory;
        private readonly Action<T>? _resetAction;
        private readonly int _maxCapacity;
        private bool _disposed;
        /* Создаёт пул с заданными параметрами. Параметр initialSize
        определяет количество предсозданных объектов. Параметр
```


maxCapacity служит верхним пределом для предотвращения неограниченного роста при временных пиках нагрузки. Действие resetAction выполняется при возврате объекта в пул, например для скрытия узла или сброса позиции. */

```
public GenericObjectPool(
    PoolObjectFactory<T> factory,
    int initialSize = 16,
    int maxCapacity = 256,
    Action<T>? resetAction = null)
{
    _factory = factory ?? throw new ArgumentNullException(nameof(factory));
    _resetAction = resetAction;
    _maxCapacity = maxCapacity;
    _objects = new ConcurrentBag<T>();
    for (int i = 0; i < initialSize; i++)
    {
        _objects.Add(_factory());
    }
}

/* Берёт объект из пула. Если пул пуст – создаёт новый.
В игровом контексте ожидание освобождения наносит больший
урон, чем единичная аллокация, поэтому пропуск кадра
недопустим. */
public T Rent()
{
    ObjectDisposedException.ThrowIf(_disposed, this);
    if (_objects.TryTake(out var obj))
    {
        return obj;
    }
    return _factory();
}

/* Возвращает объект в пул. Предварительно применяет
действие сброса состояния, если оно было задано.
Если пул уже заполнен до максимальной ёмкости, объект
просто удаляется сборщиком мусора, что предпочтительнее
неограниченного роста занимаемой памяти. */
public void Return(T obj)
{
    ArgumentNullException.ThrowIfNull(obj);
    if (_disposed) return;
    _resetAction?.Invoke(obj);
    if (_objects.Count < _maxCapacity)
    {
        _objects.Add(obj);
    }
}

public void Dispose()
{
    if (!_disposed)
    {
        while (_objects.TryTake(out var obj))
        {

```

```

        (obj as IDisposable)?.Dispose();
    }
    _disposed = true;
}
}
}
}
}
}
}
}
}
}

```

Минимизация аллокаций в горячих путях

В горячих путях не должно быть heap-аллокаций. Аллокации допустимы при инициализации, загрузке уровня, создании или уничтожении сессий, то есть в точках, где пауза GC незаметна.

```

// Файл BestPractices/AllocationFreePatterns.cs.
/* Паттерны для минимизации GC-давления в казино.
Демонстрация контраста между наивным LINQ-подходом
и оптимизированной версией без аллокаций в куче. */
using System;
using System.Collections.Generic;
namespace LasVegas.Infrastructure.BestPractices
{
    public static class AllocationFreePatterns
    {
        /* Демонстрирует антипаттерн: LINQ в горячем пути.
        LINQ создаёт IEnumerable, closure-объекты и промежуточные
        коллекции. Для разового вызова это незаметно. Но когда
        сотня автоматов оценивают пять барабанов шестьдесят раз
        в секунду, речь идёт о тысячах аллокаций каждую секунду. */
        public static decimal CalculateWinLinq(
            IReadOnlyList<string> symbols,
            IReadOnlyDictionary<string, decimal> payTable)
        {
            var groups = symbols.GroupBy(s => s)
                .OrderByDescending(g => g.Count());
            return 0m;
        }

        /* Правильный подход: цикл с переменными на стеке.
        Span<T> размещается на стеке и не создаёт нагрузки на GC,
        но его нельзя хранить в полях класса, поэтому Span
        используется только для локальных переменных.
        Для массивов, размер которых известен во время выполнения,
        применяется ArrayPool. Метод ArrayPool.Shared.Rent берёт
        массив из пула, а не выделяет новый, а Return возвращает
        его обратно. */
        public static decimal CalculateWinOptimized(
            ReadOnlySpan<string> symbols,
            IReadOnlyDictionary<string, decimal> payTable)
        {
            /* stackalloc выделяет память на стеке – zero GC pressure.
            Длины в три-пять элементов достаточно для барабанов. */
            Span<int> counts = stackalloc int[symbols.Length];

```

```

Span<string> uniqueSymbols = stackalloc string[symbols.Length];
int uniqueCount = 0;
for (int i = 0; i < symbols.Length; i++)
{
    bool found = false;
    for (int j = 0; j < uniqueCount; j++)
    {
        if (uniqueSymbols[j] == symbols[i])
        {
            counts[j]++;
            found = true;
            break;
        }
    }
    if (!found)
    {
        uniqueSymbols[uniqueCount] = symbols[i];
        counts[uniqueCount] = 1;
        uniqueCount++;
    }
}
int maxCount = 0;
string bestSymbol = "";
for (int i = 0; i < uniqueCount; i++)
{
    if (counts[i] > maxCount)
    {
        maxCount = counts[i];
        bestSymbol = uniqueSymbols[i];
    }
}
if (maxCount >= 2 &&
    payTable.TryGetValue(bestSymbol, out var multiplier))
{
    return multiplier * maxCount / symbols.Length;
}
return 0m;
}
}
}

```

Структура проекта и резюме Best Practices

Представленная архитектура не является просто набором разрозненных классов, её физическое воплощение в структуре папок и сборок отражает границы слоёв и направление зависимостей. Корень решения содержит три основные группы: домен (*LasVegas.Domain*), где живут интерфейсы и чистые правила, не зависящие от Godot; инфраструктуру (*LasVegas.Infrastructure*), где находятся реализации этих интерфейсов, ресурсы Godot и сервисы; и корень композиции (*LasVegas.CompositionRoot*), который является единственной точкой, знающей о конкретных реализациях и связывающей всё воедино.

Такая организация превращает абстрактные принципы в наглядную карту проекта: открыв папку Domain/Interfaces, разработчик сразу видит все контракты системы, а заглянув в Infrastructure/Services, находит их техническую реализацию. Нижележащая диаграмма иллюстрирует это разделение и ключевые связи между компонентами, о которых шла речь в этой главе.

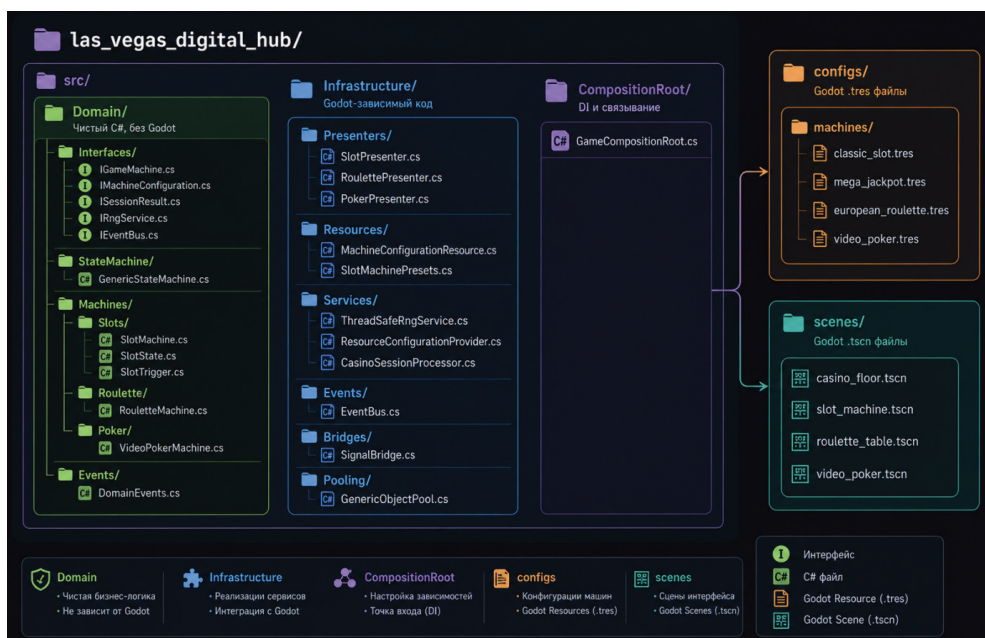


Рис. 2. Изображение структуры проекта

Резюмируя лучшие практики, стоит подчеркнуть три главные идеи. Во-первых, массовое создание короткоживущих объектов в горячих путях, таких как выпадение монет, всегда должно обслуживаться объектными пулами – это не просто оптимизация, а способ предотвратить непредсказуемые паузы сборщика мусора. Во-вторых, любой код, выполняющийся каждый кадр, обязан избегать аллокаций в куче, используя `Span<T>` и `stackalloc` вместо временных коллекций и LINQ-запросов. В-третьих, все высокоуровневые компоненты должны собираться в единственном корне композиции, который делает зависимости явными и не позволяет конструкции `new` для сервисов расползтись по всей программе. Эти три практики, применённые вместе, и создают ту самую предсказуемую производительность, которая отличает прототип от продукта, готового к нагрузкам реального казино.

Composition Root: связываем всё вместе

Это узловая точка архитектуры. Она единственная знает о конкретных реализациях, а все остальные компоненты знают только об интерфейсах.

```
// Файл CompositionRoot/GameCompositionRoot.cs.
/* Единственная точка, где создаются все объекты и связываются
зависимости. Вне этого класса не должно быть оператора new
для сервисов. Такой подход делает все зависимости видимыми
в одном месте, гарантирует контроль над порядком создания,
упрощает замену любой реализации и добавление сквозной
функциональности (AOP) вроде логирования или метрик. */
using Godot;
using LasVegas.Domain.Interfaces;
using LasVegas.Domain.Machines;
using LasVegas.Infrastructure.Events;
using LasVegas.Infrastructure.Services;
using LasVegas.Infrastructure.Resources;
using LasVegas.Infrastructure.Bridges;
using System.Collections.Generic;
namespace LasVegas.CompositionRoot
{
    /* Корень композиции, реализованный как Autoload-узел в Godot.
    Он живёт всё время жизни приложения и доступен из любой сцены.
    Такой подход выбран вместо Service Locator, потому что
    Service Locator скрывает зависимости, тогда как Composition Root
    делает их явными через конструкторы. */
    public sealed partial class GameCompositionRoot : Node
    {
        public static GameCompositionRoot? Instance { get; private set; }
        public IEventBus EventBus { get; private set; } = null!;
        public IRngService RngService { get; private set; } = null!;
        public CasinoSessionProcessor SessionProcessor { get; private set; } = null!;
        public ResourceConfigurationProvider ConfigProvider { get; private set; } = null!;
        private readonly Dictionary<MachineId, IGameMachine> _machines = new();
        public override void _Ready()
        {
            Instance = this;
            /* Шаг 1: создаём инфраструктурные сервисы.
            Порядок создания важен: EventBus нужен для машин,
            RngService нужен для машин, ConfigProvider нужен
            для загрузки конфигураций. */
            EventBus = new EventBus();
            /* Выбор реализации RNG осуществляется одной строкой.
            ThreadSafeRngService служит для разработки,
            CryptoRngService – для production. Вся система
            остаётся неизменной, следуя принципу DIP. */
            RngService = new ThreadSafeRngService();
            SessionProcessor = new CasinoSessionProcessor(
                maxConcurrency: System.Environment.ProcessorCount);
            ConfigProvider = new ResourceConfigurationProvider();
            /* Шаг 2: Создаём мосты между слоями. */
            var signalBridge = new SignalBridge();
            AddChild(signalBridge);
            signalBridge.Initialize(EventBus);
            /* Шаг 3: создаём автоматы на полу казино. */
            InitializeCasinoFloor();
        }
    }
}
```

```

        GD.Print("Las Vegas Digital Hub initialized.");
        GD.Print($"  Machines: {_machines.Count}");
        GD.Print($"  RNG: {RngService.GetType().Name}");
        GD.Print($"  Max concurrency: " +
            $"{System.Environment.ProcessorCount}");
    }
    /* Создаёт автоматы на полу казино.
    В реальном проекте список загружается из конфигурации. */
    private void InitializeCasinoFloor()
    {
        for (int i = 0; i < 10; i++)
        {
            var machine = CreateMachine("ClassicSlot");
            _machines[machine.Id] = machine;
        }
        for (int i = 0; i < 5; i++)
        {
            var machine = CreateMachine("MegaJackpot");
            _machines[machine.Id] = machine;
        }
        for (int i = 0; i < 3; i++)
        {
            var machine = CreateMachine("EuropeanRoulette");
            _machines[machine.Id] = machine;
        }
        for (int i = 0; i < 2; i++)
        {
            var machine = CreateMachine("VideoPoker");
            _machines[machine.Id] = machine;
        }
    }
    /* Фабричный метод: создаёт автомат по типу.
    Оператор switch выбран вместо reflection, чтобы сохранить
    проверку на этапе компиляции, избежать накладных расходов
    во время исполнения и сделать список поддерживаемых типов
    явным и видимым. */
    private IGameMachine CreateMachine(string machineType)
    {
        var config = ConfigProvider.Load(machineType);
        /* Конструктору SlotMachine передаются зависимости,
        но класс не знает, какие конкретно реализации RNG
        и EventBus ему предоставлены. */
        var machine = new SlotMachine(
            id: MachineId.New(),
            rng: RngService,
            eventBus: EventBus);
        machine.Initialize(config);
        return machine;
    }
    /* Завершение приложения. CancellationToken необходимо
    отменить, пулы очистить, подписки удалить, иначе останутся
    утечки ресурсов и зависшие потоки. */

```

```

public override void _Notification(int what)
{
    if (what == NotificationWMCloseRequest)
    {
        SessionProcessor.CancelAll();
        foreach (var machine in _machines.Values)
        {
            machine.Reset();
        }
        SessionProcessor.Dispose();
        RngService.Dispose();
        ConfigProvider.Dispose();
    }
}
}
}

```

Заключение

Мы вместе прошли большой путь. От самых первых строк на C#, в которых персонаж просто печатал сообщение в консоль, мы добрались до полноценной архитектуры продакшен-уровня, способной выдержать нагрузку целого игрового зала в самом сердце Лас-Вегаса. По дороге мы не просто учились писать код, мы учились смотреть на код как на живую систему, в которой у каждого класса есть своя зона ответственности, у каждой зависимости – понятное направление, а у каждого изменения – безопасный радиус влияния. Надеюсь, вы запомнили главное: архитектура не в паттернах, которыми мы так элегантно овладели, и не в инструментах, которые освоили. Она в привычке задавать вопросы. Почему этот модуль знает о том, о чём не должен знать? В какой момент жизни находится этот объект и кто за него отвечает? Что сломается, если я перепишу этот кусок? Эти вопросы – способ сохранить радость творчества в проекте, который растёт и усложняется.

Вы научились разделять слои и направления зависимостей, проектировать от интерфейсов, а не от готовых решений, и собирать доменную логику в чистые, тестируемые сущности. Вы знаете, как делать игры, которые не рассыпаются от внезапного обновления движка или от того, что геймдизайнер в последний момент попросил добавить «ещё одну маленькую фичу». Это знание – ваш главный актив. Оно останется с вами независимо от того, на каком движке вы будете работать завтра. Помните, что даже самая продуманная архитектура не самоцель. Она служит единственной цели – дарить игроку тот самый, ни с чем не сравнимый, волшебный опыт. За всем этим железобетонным фундаментом из интерфейсов, сервисов и неизменяемых DTO стоит человек, который нажмёт на кнопку «Новая игра». Пусть ваш код будет достаточно живым, чтобы удивлять, и достаточно надёжным, чтобы не подвести.

Спасибо, что прошли этот путь вместе с нами. Теперь дело за вами. Открывайте Godot, создавайте свои миры и делайте их такими, какими вы их видите. У вас для этого есть всё необходимое. Удачи, и до встречи в ваших собственных проектах.

Предметный указатель

Godot

- дерево сцен, 48

- узлы

 - AnimationPlayer, 42

 - Area2D, 156

 - CharacterBody2D, 41

 - CollisionShape2D, 41

 - RigidBody2D, 136

- физика

 - маска коллизионная, 159

 - слой коллизионный, 156

UI и представление

- поток данных

- однонаправленный, 210

- слой представления, 98

Архитектура

- отношения

 - композиция, 205

 - наследование, 40

- связанность, 78

- сильная связанность, 203

- слабая связанность, 78

- принципы

 - детерминизм, 133

 - инвариант, 72

 - инкапсуляция, 71

 - контракт, 81

 - разделение ответственности, 11

- рефакторинг, преждевременная

- абстракция, 94

Данные и состояние

- сериализация, общее понятие, 221

- снимок состояния, 149

Паттерны

- диспетчер событий, шина, 202

- конечный автомат,

- иерархический, 184

- машина состояний, 188

Книги издательства «ДМК Пресс»
можно купить оптом и в розницу на складе издательства по адресу:
Москва, ул. Электродная, д. 2, стр. 12, офис 7, тел. +7 (499) 322-19-38,
а также заказать на сайте **www.dmkpress.com**
с доставкой в любой регион РФ

Валерий Валерьевич Казанцев,
Александр Владимирович Гнайберг

С# и Godot.
Разработка чистых и масштабируемых игровых систем

Главный редактор	<i>Яценков В. С.</i> editor@dmkpress.com
Корректор	<i>Синяева Г. И.</i>
Верстка	<i>Чаннова А. А.</i>
Дизайн обложки	<i>Трофимова С. В.</i>

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 25,51. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**

Эта книга закладывает архитектурный фундамент для создания игр на Godot и C#, не давая проекту рассыпаться по мере роста. Она формирует системное понимание того, как писать код, способный выдержать изменения, расширение и годы сопровождения. Главное отличие от большинства аналогичных книг по разработке игр кроется в фокусе на инженерных принципах: разделение ответственности, управление зависимостями и построение тестируемых, масштабируемых систем.

Ключевые темы книги:

- разделение кода на доменный, инфраструктурный и презентационный слои;
- проектирование от интерфейсов с опорой на явные контракты;
- построение машин состояний, событийных шин и систем взаимодействия;
- управление временем жизни объектов, сохранениями и асинхронностью;
- создание архитектуры, пригодной для модульного и интеграционного тестирования;
- практики оптимизации для production: пулы, снижение аллокаций, управление сценами.

Издание предназначено начинающим и опытным разработчикам игр, желающим перейти от стихийного кода к осознанному проектированию. Требуется знание языка C# на начальном уровне.



www.dmk.pf



ISBN 978-5-93700-483-3

